

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

**GVHD: ThS. Nguyễn Mạnh Tuấn,
TS. Nguyễn Quốc Tuấn**

Chủ đề thực hiện:

XÂY DỰNG ỨNG DỤNG “ĐI CHỢ TIỆN LỢI”

Mã lớp: **162304** Nhóm: **11**

Môn học: **Phát triển ứng dụng đa nền tảng** Mã môn học: **IT4788**

Các thành viên trong nhóm:

STT	Họ và tên	MSSV	Email
1	Nguyễn Thế Anh	20224921	anh.nt224921@sis.hust.edu.vn
2	Quản Tuấn Huy	20225014	huy.qt225014@sis.hust.edu.vn
3	Phạm Trung Kiên	20225026	kien.pt225026@sis.hust.edu.vn
4	Nhữ Ngọc Minh	20225046	minh.nn225046@sis.hust.edu.vn
5	Trần Bảo Long	20225038	long.tb225038@sis.hust.edu.vn

Hà Nội, tháng 12 năm 2025

Lời nói đầu

Trong bối cảnh cuộc sống hiện đại ngày càng bận rộn, việc quản lý công việc gia đình, đặc biệt là khâu mua sắm và chuẩn bị bữa ăn hàng ngày, trở thành một thách thức không nhỏ. Việc đi chợ hay siêu thị thường tốn nhiều thời gian và dễ xảy ra tình trạng mua thừa hoặc thiếu thực phẩm. Thêm vào đó, việc không theo dõi sát sao thực phẩm trong tủ lạnh dẫn đến tình trạng lãng phí do hết hạn sử dụng. Nhận thấy nhu cầu thực tế này, chúng tôi đã quyết định xây dựng dự án “Xây dựng ứng dụng di động đa nền tảng “Đi chợ tiện lợi” ”. Ứng dụng này ra đời với mong muốn cung cấp một công cụ hữu ích, giúp người dùng đơn giản hóa và tối ưu hóa các công việc liên quan đến mua sắm và quản lý thực phẩm.

Đề tài “Đi chợ tiện lợi” mang ý nghĩa thiết thực, giải quyết trực tiếp những vấn đề hàng ngày mà nhiều cá nhân và gia đình gặp phải. Ứng dụng không chỉ giúp tiết kiệm thời gian và công sức trong việc mua sắm và nấu nướng mà còn đóng vai trò quan trọng trong việc quản lý chi tiêu và dinh dưỡng hiệu quả hơn. Bằng cách cung cấp các công cụ để theo dõi thực phẩm, lên kế hoạch bữa ăn và nhắc nhở thông minh về hạn sử dụng, ứng dụng góp phần giảm thiểu lãng phí thực phẩm – một vấn đề đang được xã hội quan tâm. Hơn nữa, với tính năng chia sẻ danh sách mua sắm trong gia đình, ứng dụng còn tăng cường sự phối hợp giữa các thành viên, giúp việc mua sắm trở nên khoa học và hiệu quả, từ đó nâng cao chất lượng cuộc sống.

Nhóm 11

Mục lục

1	Khảo sát bài toán	14
1.1	Mô tả yêu cầu bài toán	14
1.2	Xác định thông tin cơ bản cho nghiệp vụ của bài toán	15
2	Đặc tả yêu cầu bài toán	16
2.1	Giới thiệu chung	16
2.2	Biểu đồ use case	17
2.2.1	Biểu đồ use case tổng quan	17
2.2.2	Biểu đồ use case phân rã mức 2	17
2.3	Các yêu cầu phi chức năng	46
2.3.1	Tính dễ dùng	46
2.3.2	Hiệu năng	47
2.3.3	Tính tin cậy	47
2.3.4	Tính dễ bảo trì	47
2.3.5	Tính khả chuyển	47
2.3.6	Yêu cầu an toàn bảo mật	47
2.3.7	Yêu cầu về giao diện	48
3	Thiết kế giao diện	49
3.1	Thiết kế giao diện low-fidelity / mid-fidelity prototype	49
3.1.1	UC1: Quản lý danh sách mua sắm	49
3.1.2	UC2: Quản lý dự định bữa ăn	50
3.1.3	UC3: Quản lý thực phẩm trong bếp	50
3.1.4	UC4: Quản lý công thức nấu ăn	51
3.1.5	UC5: Quản lý danh sách thực phẩm	51
3.1.6	UC6: Quản lý nhóm	53
3.1.7	UC7: Thống kê và báo cáo	54
3.1.8	UC8: Quản trị người dùng	54
3.1.9	UC9: Xác thực	55
3.2	Xây dựng các màn hình giao diện của ứng dụng (high-fidelity)	55
3.2.1	Hệ thống Theme và Styles tổng thể	56
3.2.2	Kỹ thuật điều hướng (Navigation)	56
3.2.3	Sơ đồ điều hướng	57
3.2.4	Chi tiết thiết kế các màn hình theo Use Case	58
4	Thiết kế Backend	93
4.1	Thiết kế cơ sở dữ liệu cho Backend	93
4.1.1	Sơ đồ thực thể quan hệ (ERD)	93
4.1.2	Lược đồ cơ sở dữ liệu	94
4.1.3	Đặc tả chi tiết các bảng dữ liệu	95
4.2	Thiết kế API Backend	108
4.2.1	Danh sách API	108
4.2.2	Đặc tả chi tiết API	115
4.2.3	Phần 1: API Quản Lý Thực Phẩm Nhóm	166
4.2.4	Phần 2: API Danh Mục và Đơn Vị	172
4.2.5	Phần 3: API Quản Trị Thực Phẩm Hệ Thống (Admin)	174
5	Xây dựng ứng dụng đa nền tảng	219
5.1	Kiến trúc ứng dụng	219

5.1.1	Lựa chọn kiến trúc phần mềm và giải thích sơ bộ	219
5.1.2	Các cải tiến so với lý thuyết	220
5.2	Tổ chức thư mục của dự án	221
5.2.1	Cấu trúc mã nguồn Backend (Spring Boot)	221
5.2.2	Cấu trúc mã nguồn Frontend (React Native)	221
5.3	Thiết kế chi tiết các gói	222
5.3.1	Thiết kế chi tiết cho UC1: Quản lý danh sách mua sắm	222
5.3.2	Thiết kế chi tiết cho UC2: Quản lý dự định bữa ăn	223
5.3.3	Thiết kế chi tiết cho UC3: Quản lý thực phẩm trong bếp	224
5.3.4	Thiết kế chi tiết cho UC4: Quản lý công thức nấu ăn	225
5.3.5	Thiết kế chi tiết cho UC5: Quản lý danh sách thực phẩm	226
5.3.6	Thiết kế chi tiết cho UC6: Quản lý nhóm	227
5.3.7	Thiết kế chi tiết cho UC7: Thống kê và báo cáo	228
5.3.8	Thiết kế chi tiết cho UC8: Quản trị người dùng	229
5.4	Thiết kế chi tiết lớp	230
5.4.1	Các Lớp Entity (Domain Layer)	230
5.4.2	Các Lớp Repository (Domain Layer)	233
5.4.3	Các Lớp UseCase (Application Layer)	234
5.4.4	Các Lớp Controller (Presentation Layer)	238
5.5	Các giải pháp khác đã xây dựng trong chương trình	242
5.5.1	Cơ chế giao tiếp với Back-end và xử lý truy vấn API	243
5.5.2	Chiến lược quản lý trạng thái ứng dụng (State Management)	243
5.5.3	Giải pháp lưu trữ dữ liệu cục bộ và bảo mật	243
5.5.4	Hệ thống đa ngôn ngữ và chuẩn hóa hiển thị	244
5.5.5	Tích hợp dịch vụ nền tảng (Native Features)	244
6	Kiểm thử chương trình	245
6.1	Kiểm thử các chức năng đã thực hiện	245
6.1.1	Kiểm thử cho chức năng: Đăng ký tài khoản (UC-09.1)	245
6.1.2	Kiểm thử cho chức năng: Theo dõi danh sách mua sắm (UC-01.3)	246
6.1.3	Kiểm thử cho chức năng: Quản lý thực phẩm trong bếp (UC-03.1 & UC-03.4)	248
6.1.4	Kiểm thử cho chức năng: Tạo nhóm và Thêm thành viên (UC-06.1 & UC-06.2)	249
6.1.5	Kiểm thử cho chức năng: Quản lý Công thức nấu ăn (UC-04.1 & UC-04.3)	250
6.1.6	Kiểm thử cho chức năng: Lên thực đơn (UC-02.1)	252
6.2	Kiểm thử yêu cầu phi chức năng	252
	Tài liệu tham khảo	256
	Phụ lục	257

Danh mục hình ảnh

1	Biểu đồ Use Case tổng quan của hệ thống	17
2	Biểu đồ Usecase quản lý danh sách mua sắm	18
3	Biểu đồ Usecase quản lý dự định bữa ăn	23
4	Biểu đồ Use Case quản lý thực phẩm trong bếp	25
5	Biểu đồ Use Case quản lý công thức nấu ăn	28
6	Biểu đồ Use Case quản lý danh sách thực phẩm	31
7	Biểu đồ Use Case quản lý nhóm	34

8	Biểu đồ Use Case quản lý người dùng	41
9	Thiết kế giao diện cho UC1: Xem các danh sách mua sắm	49
10	Thiết kế giao diện cho UC1: Quản lý danh sách mua sắm	49
11	Thiết kế giao diện cho UC2: Quản lý dự định bữa ăn	50
12	Thiết kế giao diện cho UC3: Quản lý thực phẩm trong bếp	50
13	Thiết kế giao diện cho UC4: Quản lý công thức nấu ăn	51
14	Thiết kế giao diện cho UC5: Quản lý danh sách thực phẩm	51
15	Thiết kế giao diện cho UC5: Chi tiết thực phẩm	52
16	Thiết kế giao diện cho UC5: Thêm thực phẩm mới	52
17	Thiết kế giao diện cho UC6: Quản lý nhóm	53
18	Thiết kế giao diện cho UC6: Quản lý nhóm - chi tiết nhóm	53
19	Thiết kế giao diện cho UC7: Thống kê và báo cáo	54
20	Thiết kế giao diện cho UC8: Quản trị người dùng	54
21	Thiết kế giao diện cho UC9: Đăng ký/đăng nhập	55
22	Thiết kế giao diện cho UC9: Quản lý hồ sơ cá nhân	55
23	Sơ đồ điều hướng của ứng dụng	57
24	Thiết kế giao diện trang chủ	58
25	Thiết kế trang quản lý danh sách mua sắm - 1	59
26	Thiết kế giao diện chi tiết danh sách mua sắm - 2	59
27	Thiết kế giao diện chi tiết danh sách mua sắm - 3	60
28	Thiết kế giao diện quản lý dự định bữa ăn - 1	61
29	Thiết kế giao diện quản lý dự định bữa ăn - 2	61
30	Thiết kế giao diện quản lý thực phẩm trong bếp - 1	62
31	Thiết kế giao diện quản lý thực phẩm trong bếp - 2	63
32	Thiết kế giao diện quản lý công thức nấu ăn - 1	65
33	Thiết kế giao diện quản lý công thức nấu ăn - 2	66
34	Thiết kế giao diện quản lý công thức nấu ăn - 3	66
35	Thiết kế giao diện quản lý thực phẩm - 1	70
36	Thiết kế giao diện quản lý thực phẩm - 2	70
37	Thiết kế giao diện quản lý thực phẩm - 3	71
38	Thiết kế giao diện quản lý nhóm - 1	75
39	Thiết kế giao diện quản lý nhóm - 2	76
40	Thiết kế giao diện quản lý nhóm - 3	76
41	Thiết kế giao diện thống kê và báo cáo - 1	80
42	Thiết kế giao diện thống kê và báo cáo - 2	80
43	Thiết kế giao diện thống kê và báo cáo - 3	81
44	Thiết kế giao diện quản trị người dùng	87
45	Thiết kế giao diện đăng nhập	92
46	Thiết kế giao diện đăng ký	92
47	Sơ đồ thực thể quan hệ (ERD) của hệ thống	93
48	Lược đồ cơ sở dữ liệu của hệ thống	94
49	Sơ đồ phân lớp các gói (Package Diagram) theo Clean Architecture	219
50	Biểu đồ gói chi tiết cho UC1: Quản lý danh sách mua sắm	222
51	Biểu đồ gói chi tiết cho UC2: Quản lý dự định bữa ăn	223
52	Biểu đồ gói chi tiết cho UC3: Quản lý thực phẩm trong bếp	224
53	Biểu đồ gói chi tiết cho UC4: Quản lý công thức nấu ăn	225
54	Biểu đồ gói chi tiết cho UC5: Quản lý danh sách thực phẩm	226
55	Biểu đồ gói chi tiết cho UC6: Quản lý nhóm	227
56	Biểu đồ gói chi tiết cho UC7: Thống kê và báo cáo	228
57	Biểu đồ gói chi tiết cho UC8: Quản trị người dùng	229

Danh mục bảng biểu

1	Bảng phân công nhiệm vụ và đánh giá thành viên	13
2	Chi tiết luồng xử lý các nghiệp vụ chính của ứng dụng	15
3	Bảng liệt kê và mô tả các tác nhân trong hệ thống	16
4	Bảng tổng hợp danh mục Use Case của hệ thống	16
5	Đặc tả ca sử dụng UC001.1: Tạo danh sách mua sắm	19
6	Đặc tả ca sử dụng UC001.2: Chỉnh sửa danh sách mua sắm	20
7	Đặc tả ca sử dụng UC001.3: Theo dõi danh sách mua sắm	21
8	Đặc tả ca sử dụng UC001.4: Gợi ý khi bổ sung thực phẩm vào danh sách mua sắm	22
9	Đặc tả ca sử dụng UC002.1: Tạo thực đơn mới	23
10	Đặc tả ca sử dụng UC002.3: Gợi ý món ăn từ thực phẩm trong bếp	24
11	Đặc tả ca sử dụng UC003.1: Thêm thực phẩm vào bếp	25
12	Đặc tả ca sử dụng UC003.2: Chỉnh sửa thực phẩm trong bếp	26
13	Đặc tả ca sử dụng UC003.3: Xóa thực phẩm khỏi bếp	26
14	Đặc tả ca sử dụng UC003.4: Nhắc nhở thực phẩm sắp hết hạn	27
15	Đặc tả ca sử dụng UC003.5: Xem danh sách thực phẩm trong bếp	27
16	Đặc tả ca sử dụng UC004.1: Tạo công thức nấu ăn chung	28
17	Đặc tả ca sử dụng UC004.2: Chỉnh sửa công thức nấu ăn	29
18	Đặc tả ca sử dụng UC004.3: Tạo công thức nấu ăn cá nhân	29
19	Đặc tả ca sử dụng UC004.4: Xóa công thức nấu ăn	30
20	Đặc tả ca sử dụng UC004.5: Tìm kiếm công thức món ăn	30
21	Đặc tả ca sử dụng UC005.1: Tạo thực phẩm cá nhân	31
22	Đặc tả ca sử dụng UC005.2: Tạo thực phẩm chung	32
23	Đặc tả ca sử dụng UC005.3: Chỉnh sửa thực phẩm	32
24	Đặc tả ca sử dụng UC005.4: Xóa thực phẩm	33
25	Đặc tả ca sử dụng UC005.5: Tìm kiếm thực phẩm	33
26	Đặc tả ca sử dụng UC006.1: Tạo nhóm	34
27	Đặc tả ca sử dụng UC006.2: Thêm thành viên vào nhóm	35
28	Đặc tả ca sử dụng UC006.3: Xóa thành viên khỏi nhóm	36
29	Đặc tả ca sử dụng UC006.4: Rời khỏi nhóm	37
30	Đặc tả ca sử dụng UC006.5: Chuyển quyền trưởng nhóm	37
31	Đặc tả ca sử dụng UC006.6: Xem thông tin nhóm	38
32	Đặc tả ca sử dụng UC006.7: Xóa nhóm	38
33	Đặc tả ca sử dụng UC007.1: Xem báo cáo mua sắm	39
34	Đặc tả ca sử dụng UC007.2: Xem báo cáo tiêu thụ thực phẩm	39
35	Đặc tả ca sử dụng UC007.3: Xem báo cáo thực đơn	40
36	Đặc tả ca sử dụng UC008.1: Xem danh sách người dùng	41
37	Đặc tả ca sử dụng UC008.2: Khóa/Mở khóa tài khoản	42
38	Đặc tả ca sử dụng UC008.3: Xóa tài khoản	42
39	Đặc tả ca sử dụng UC009.1: Đăng ký tài khoản	43
40	Đặc tả ca sử dụng UC009.2: Đăng nhập	44
41	Đặc tả ca sử dụng UC009.3: Đăng xuất	44
42	Đặc tả ca sử dụng UC009.4: Quên mật khẩu	45
43	Đặc tả ca sử dụng UC009.5: Cập nhật thông tin cá nhân	45
44	Đặc tả ca sử dụng UC009.6: Đổi mật khẩu	46
45	Cấu trúc bảng USER	95
46	Cấu trúc bảng GROUP	96
47	Cấu trúc bảng GROUP_MEMBER	96
48	Cấu trúc bảng CATEGORY	97

49	Cấu trúc bảng FOOD	98
50	Cấu trúc bảng UNIT	99
51	Cấu trúc bảng KITCHEN_ITEM	100
52	Cấu trúc bảng RECIPE	101
53	Cấu trúc bảng RECIPE_INGREDIENT	102
54	Cấu trúc bảng MEAL_PLAN	103
55	Cấu trúc bảng SHOPPING_LIST	104
56	Cấu trúc bảng SHOPPING_LIST_ITEM	105
57	Cấu trúc bảng NOTIFICATION	106
58	Cấu trúc bảng EMAIL_VERIFICATION	107
59	Cấu trúc bảng SYSTEM_LOG	107
60	Mã thành công	109
61	Lỗi xác thực và phân quyền	109
62	Lỗi người dùng	110
63	Lỗi xác thực email	110
64	Lỗi nhóm	110
65	Lỗi thực phẩm	111
66	Lỗi bếp	111
67	Lỗi mua sắm	111
68	Lỗi kế hoạch bữa ăn	112
69	Lỗi công thức nấu ăn	112
70	Lỗi danh mục và đơn vị	112
71	Lỗi thông báo	112
72	Lỗi validation	112
73	Lỗi hệ thống	113
74	Danh sách API của hệ thống	115
75	Tham số Request Body - API Tạo Danh Sách	115
76	Test Cases - API 1.1 Tạo Danh Sách	116
77	Query Parameters - API Lấy Danh Sách	117
78	Test Cases - API 1.2 Lấy Danh Sách	118
79	Path Parameters - API Chi Tiết Danh Sách	118
80	Test Cases - API 1.3 Chi Tiết Danh Sách	119
81	Test Cases - API 1.4 Danh Sách Được Giao	120
82	Path Parameters - API Cập Nhật Danh Sách	121
83	Request Body - API Cập Nhật Danh Sách	121
84	Test Cases - API 1.5 Cập Nhật Danh Sách	122
85	Path Parameters - API Xóa Danh Sách	122
86	Test Cases - API 1.6 Xóa Danh Sách	123
87	Path Parameters - API Hoàn Thành Danh Sách	123
88	Test Cases - API 1.7 Hoàn Thành Danh Sách	124
89	Path Parameters - API Thêm Món	124
90	Request Body Parameters - API Thêm Món	125
91	Test Cases - API 1.8 Thêm Món	126
92	Path Parameters - API Cập Nhật Món	126
93	Request Body - API Cập Nhật Món	126
94	Test Cases - API 1.9 Cập Nhật Món	127
95	Path Parameters - API Xóa Món	127
96	Test Cases - API 1.10 Xóa Món	128
97	Path Parameters - API Đánh Dấu Món	128
98	Request Body - API Đánh Dấu Món	129

99	Test Cases - API 1.11 Đánh Dấu Món	129
100	Tham số Request Body - API Tạo Kế Hoạch Bữa Ăn	130
101	Test Cases - API 2.1 Tạo Kế Hoạch Bữa Ăn	131
102	Test Cases - API 2.2 Lấy Danh Sách Kế Hoạch	132
103	Path Parameters - API Chi Tiết Kế Hoạch	133
104	Test Cases - API 2.3 Chi Tiết Kế Hoạch	134
105	Path Parameters - API Cập Nhật Kế Hoạch	134
106	Tham số Request Body - API Cập Nhật Kế Hoạch	134
107	Test Cases - API 2.4 Cập Nhật Kế Hoạch	135
108	Path Parameters - API Xóa Kế Hoạch	136
109	Test Cases - API 2.5 Xóa Kế Hoạch	136
110	Query Parameters - API Lấy Kế Hoạch Theo Ngày	137
111	Test Cases - API 2.6 Lấy Kế Hoạch Theo Ngày	138
112	Query Parameters - API Lấy Kế Hoạch Theo Khoảng Thời Gian	138
113	Test Cases - API 2.7 Lấy Kế Hoạch Theo Khoảng Thời Gian	140
114	Query Parameters - API Lấy Kế Hoạch Sắp Tới	140
115	Test Cases - API 2.8 Lấy Kế Hoạch Sắp Tới	141
116	Tham số Request Body - API Thêm Thực Phẩm Vào Bếp	142
117	Test Cases - API 3.1 Thêm Thực Phẩm	143
118	Query Parameters - API Lấy Danh Sách Thực Phẩm	144
119	Test Cases - API 3.2 Lấy Danh Sách Thực Phẩm	145
120	Path Parameters - API Chi Tiết Thực Phẩm	145
121	Test Cases - API 3.3 Chi Tiết Thực Phẩm	146
122	Query Parameters - API Lấy Thực Phẩm Sắp Hết Hạn	147
123	Test Cases - API 3.4 Lấy Thực Phẩm Sắp Hết Hạn	148
124	Path Parameters - API Cập Nhật Thực Phẩm	148
125	Tham số Request Body - API Cập Nhật Thực Phẩm	149
126	Test Cases - API 3.5 Cập Nhật Thực Phẩm	150
127	Path Parameters - API Xóa Thực Phẩm	150
128	Test Cases - API 3.6 Xóa Thực Phẩm	151
129	Tham số Request Body - API Tạo Công Thức Nấu Ăn	152
130	Test Cases - API 4.1 Tạo Công Thức	153
131	Query Parameters - API Lấy Danh Sách Công Thức	154
132	Test Cases - API 4.2 Lấy Danh Sách Công Thức	155
133	Path Parameters - API Chi Tiết Công Thức	155
134	Test Cases - API 4.3 Chi Tiết Công Thức	156
135	Path Parameters - API Cập Nhật Công Thức	157
136	Tham số Request Body - API Cập Nhật Công Thức	157
137	Test Cases - API 4.4 Cập Nhật Công Thức	158
138	Path Parameters - API Xóa Công Thức	158
139	Test Cases - API 4.5 Xóa Công Thức	159
140	Path Parameters - API Thêm Nguyên Liệu	159
141	Tham số Request Body - API Thêm Nguyên Liệu	160
142	Test Cases - API 4.6 Thêm Nguyên Liệu	161
143	Path Parameters - API Cập Nhật Nguyên Liệu	161
144	Tham số Request Body - API Cập Nhật Nguyên Liệu	162
145	Test Cases - API 4.7 Cập Nhật Nguyên Liệu	163
146	Path Parameters - API Xóa Nguyên Liệu	163
147	Test Cases - API 4.8 Xóa Nguyên Liệu	164
148	Path Parameters - API Tìm Công Thức Theo Nguyên Liệu	164

149	Test Cases - API 4.9 Tìm Công Thức Theo Nguyên Liệu	165
150	Tham số Request Body - API Thêm Thực Phẩm	166
151	Test Cases - API 5.1 Thêm Thực Phẩm	167
152	Query Parameters - API Lấy Danh Sách Thực Phẩm	167
153	Test Cases - API 5.2 Lấy Danh Sách Thực Phẩm	168
154	Path Parameters - API Xem Chi Tiết Thực Phẩm	169
155	Test Cases - API 5.3 Xem Chi Tiết Thực Phẩm	170
156	Path Parameters - API Cập Nhật Thực Phẩm	170
157	Request Body - API Cập Nhật Thực Phẩm	170
158	Test Cases - API 5.4 Cập Nhật Thực Phẩm	171
159	Path Parameters - API Xóa Thực Phẩm	171
160	Test Cases - API 5.5 Xóa Thực Phẩm	172
161	Test Cases - API 5.6 Lấy Danh Sách Danh Mục	173
162	Test Cases - API 5.7 Lấy Danh Sách Đơn Vị	174
163	Request Body - API Tạo Thực Phẩm Hệ Thống	175
164	Test Cases - API 5.8 Tạo Thực Phẩm Hệ Thống	175
165	Query Parameters - API Lấy Danh Sách Thực Phẩm Hệ Thống	176
166	Test Cases - API 5.9 Lấy Danh Sách Thực Phẩm Hệ Thống	177
167	Path Parameters - API Xem Chi Tiết Thực Phẩm Hệ Thống	177
168	Test Cases - API 5.10 Xem Chi Tiết Thực Phẩm Hệ Thống	178
169	Path Parameters - API Cập Nhật Thực Phẩm Hệ Thống	178
170	Request Body - API Cập Nhật Thực Phẩm Hệ Thống	178
171	Test Cases - API 5.11 Cập Nhật Thực Phẩm Hệ Thống	179
172	Path Parameters - API Xóa Thực Phẩm Hệ Thống	180
173	Test Cases - API 5.12 Xóa Thực Phẩm Hệ Thống	180
174	Tham số Request Body - API Tạo Nhóm	181
175	Test Cases - API 6.1 Tạo Nhóm	182
176	Test Cases - API 6.2 Xem Thông Tin Nhóm	183
177	Tham số Request Body - API Thêm Thành Viên	183
178	Test Cases - API 6.3 Thêm Thành Viên	185
179	Tham số Request Body - API Xóa Thành Viên	185
180	Test Cases - API 6.4 Xóa Thành Viên	186
181	Test Cases - API 6.5 Rời Khỏi Nhóm	187
182	Query Parameters - API Báo Cáo Tổng Quan	187
183	Test Cases - API 7.1 Báo Cáo Tổng Quan	189
184	Query Parameters - API Báo Cáo Tủ Lạnh	189
185	Test Cases - API 7.2 Báo Cáo Tủ Lạnh	190
186	Query Parameters - API Báo Cáo Mua Sắm	191
187	Test Cases - API 7.3 Báo Cáo Mua Sắm	192
188	Query Parameters - API Báo Cáo Bữa Ăn	192
189	Test Cases - API 7.4 Báo Cáo Bữa Ăn	194
190	Query Parameters - API Báo Cáo Đầy Đủ	194
191	Test Cases - API 7.5 Báo Cáo Đầy Đủ	195
192	Query Parameters - API Lấy Danh Sách Người Dùng	196
193	Test Cases - API 8.1 Lấy Danh Sách Người Dùng	197
194	Test Cases - API 8.2 Lấy Thống Kê Người Dùng	198
195	Path Parameters - API Lấy Chi Tiết Người Dùng	198
196	Test Cases - API 8.3 Lấy Chi Tiết Người Dùng	199
197	Path Parameters - API Cập Nhật Người Dùng	199
198	Request Body - API Cập Nhật Người Dùng	200

199	Test Cases - API 8.4 Cập Nhật Người Dùng	201
200	Path Parameters - API Bật/Tắt Trạng Thái	201
201	Test Cases - API 8.5 Bật/Tắt Trạng Thái	202
202	Path Parameters - API Xóa Người Dùng	202
203	Test Cases - API 8.6 Xóa Người Dùng	203
204	Request Body - API Đăng Ký	203
205	Test Cases - API 9.1 Đăng Ký	205
206	Request Body - API Đăng Nhập	205
207	Test Cases - API 9.2 Đăng Nhập	206
208	Request Body - API Xác Thực Email	206
209	Test Cases - API 9.3 Xác Thực Email	207
210	Test Cases - API 9.4 Gửi Lại Mã Xác Thực	208
211	Request Body - API Đổi Mật Khẩu	209
212	Test Cases - API 9.5 Đổi Mật Khẩu	209
230	CRC Card - User	230
231	Thuộc tính - User	230
232	CRC Card - Group	231
233	CRC Card - KitchenItem	231
234	Thuộc tính - KitchenItem	231
235	CRC Card - Recipe	232
236	Thuộc tính - Recipe	232
237	CRC Card - ShoppingList	232
238	Thuộc tính - ShoppingList	233
239	CRC Card - IUserRepository	233
240	Phương thức - IUserRepository	233
241	CRC Card - IKitchenItemRepository	234
242	Phương thức - IKitchenItemRepository	234
243	CRC Card - IRecipeRepository	234
244	Phương thức - IRecipeRepository	234
245	CRC Card - UserUseCase	235
246	Phương thức chính - UserUseCase	235
247	CRC Card - GroupUseCase	235
248	Phương thức chính - GroupUseCase	235
249	CRC Card - KitchenUseCase	236
250	Phương thức chính - KitchenUseCase	236
251	CRC Card - RecipeUseCase	236
252	Phương thức chính - RecipeUseCase	236
253	CRC Card - ShoppingUseCase	237
254	Phương thức chính - ShoppingUseCase	237
255	CRC Card - MealPlanUseCase	237
256	Phương thức chính - MealPlanUseCase	237
257	CRC Card - ReportUseCase	238
258	Phương thức chính - ReportUseCase	238
259	CRC Card - AdminUserUseCase	238
260	Phương thức chính - AdminUserUseCase	238
261	CRC Card - AuthController	239
262	Các endpoint - AuthController	239
263	CRC Card - GroupController	239
264	Các endpoint - GroupController	239
265	CRC Card - KitchenController	240

266	Các endpoint - KitchenController	240
267	CRC Card - RecipeController	240
268	Các endpoint - RecipeController	240
269	CRC Card - ShoppingController	241
270	Các endpoint - ShoppingController	241
271	CRC Card - MealPlanController	241
272	Các endpoint - MealPlanController	241
273	CRC Card - ReportController	242
274	Các endpoint - ReportController	242
275	CRC Card - AdminUserController	242
276	Các endpoint - AdminUserController	242
277	Phân tầng lưu trữ dữ liệu cục bộ	244
278	Thông tin Test Case TC_AUTH_01	245
279	Các bước thực hiện Test Case TC_AUTH_01	246
280	Bảng tổng hợp kết quả kiểm thử chức năng Đăng ký tài khoản	246
281	Thông tin Test Case TC_SHOP_01	247
282	Các bước thực hiện Test Case TC_SHOP_01	247
283	Bảng tổng hợp kết quả kiểm thử chức năng Theo dõi danh sách mua sắm	248
284	Thông tin Test Case TC_KITCHEN_01	248
285	Test Case TC_KITCHEN_01 - Các bước thực hiện	249
286	Bảng tổng hợp kết quả kiểm thử chức năng Quản lý thực phẩm trong bếp	249
287	Thông tin Test Case TC_GROUP_01	249
288	Các bước thực hiện Test Case TC_GROUP_01	250
289	Bảng tổng hợp kết quả kiểm thử chức năng Tạo nhóm và Thêm thành viên	250
290	Thông tin Test Case TC_RECIPES_01	251
291	Các bước thực hiện Test Case TC_RECIPES_01	251
292	Bảng tổng hợp kết quả kiểm thử chức năng Quản lý Công thức nấu ăn	252
293	Bảng tổng hợp kết quả kiểm thử chức năng Lên thực đơn	252
294	Bảng đánh giá yêu cầu phi chức năng	253

Danh mục mã nguồn

1	Response thành công API 1.1	115
2	Response thành công API 1.2	117
3	Response thành công API 1.3	118
4	Response thành công API 1.4	120
5	Response thành công API 1.5	121
6	Response thành công API 1.6	122
7	Response thành công API 1.7	123
8	Request Body API 1.8	124
9	Response thành công API 1.8	125
10	Response thành công API 1.9	126
11	Response thành công API 1.10	127
12	Response thành công API 1.11	129
13	Response thành công API 2.1	130
14	Response thành công API 2.2	131
15	Response thành công API 2.3	133
16	Response thành công API 2.4	134
17	Response thành công API 2.5	136

18	Response thanh cong API 2.6	137
19	Response thanh cong API 2.7	138
20	Response thanh cong API 2.8	140
21	Response thanh cong API 3.1	142
22	Response thanh cong API 3.2	144
23	Response thanh cong API 3.3	146
24	Response thanh cong API 3.4	147
25	Response thanh cong API 3.5	149
26	Response thanh cong API 3.6	150
27	Response thanh cong API 4.1	152
28	Response thanh cong API 4.2	154
29	Response thanh cong API 4.3	155
30	Response thanh cong API 4.4	157
31	Response thanh cong API 4.5	159
32	Response thanh cong API 4.6	160
33	Response thanh cong API 4.7	162
34	Response thanh cong API 4.8	163
35	Response thanh cong API 4.9	164
36	Response thanh cong API 5.1	166
37	Response thanh cong API 5.2	167
38	Response thanh cong API 5.3	169
39	Response thanh cong API 5.4	170
40	Response thanh cong API 5.5	171
41	Response thanh cong API 5.6	172
42	Response thanh cong API 5.7	173
43	Response thanh cong API 5.8	175
44	Response thanh cong API 5.9	176
45	Response thanh cong API 5.10	177
46	Response thanh cong API 5.11	179
47	Response thanh cong API 5.12	180
48	Response thanh cong API 6.1	181
49	Response thanh cong API 6.2	182
50	Response thanh cong API 6.3	184
51	Response thanh cong API 6.4	185
52	Response thanh cong API 6.5	186
53	Response thanh cong API 7.1	187
54	Response thanh cong API 7.2	189
55	Response thanh cong API 7.3	191
56	Response thanh cong API 7.4	192
57	Response thanh cong API 7.5	194
58	Response thanh cong API 8.1	196
59	Response thanh cong API 8.2	197
60	Response thanh cong API 8.3	198
61	Response thanh cong API 8.4	200
62	Response thanh cong API 8.5	201
63	Response thanh cong API 8.6	202
64	Response thanh cong API 9.1	204
65	Response thanh cong API 9.2	205
66	Response thanh cong API 9.3	206
67	Response thanh cong API 9.4	207

68 Response thanh cong API 9.5 209

Phân công thành viên trong nhóm

Họ và tên	Email	SĐT	Tổng hợp công việc thực hiện	Đánh giá
Nguyễn Thế Anh	anh.nt224921@sis.hust.edu.vn	0912345xxx	- Phát triển Frontend. - Viết báo cáo.	100%
Quản Tuấn Huy	huy.qt225014@sis.hust.edu.vn	0987654xxx	Phát triển Frontend.	100%
Phạm Trung Kiên	kien.pt225026@sis.hust.edu.vn	0901122xxx	- Phân tích yêu cầu. - Viết tài liệu.	100%
Nhữ Ngọc Minh	minh.nn225046@sis.hust.edu.vn	0933445xxx	Phát triển Backend.	100%
Trần Bảo Long	long.tb225038@sis.hust.edu.vn	0977889xxx	Phát triển Backend.	100%

Bảng 1: Bảng phân công nhiệm vụ và đánh giá thành viên

1 Khảo sát bài toán

1.1 Mô tả yêu cầu bài toán

Trong cuộc sống hiện đại, người dùng thường gặp khó khăn trong việc quản lý danh sách các mặt hàng cần mua, theo dõi thực phẩm đã có trong tủ lạnh, cũng như lên kế hoạch cho các bữa ăn hàng ngày sao cho tiết kiệm và khoa học. Ứng dụng “Đi chợ tiện lợi” được xây dựng nhằm giải quyết các vấn đề đó bằng cách hỗ trợ người dùng quản lý toàn bộ quy trình mua sắm và tiêu thụ thực phẩm một cách thông minh và hiệu quả.

Cụ thể, ứng dụng cho phép người dùng tạo và chia sẻ danh sách mua sắm với các thành viên trong gia đình, giúp phối hợp việc mua hàng và tránh trùng lặp. Người dùng có thể lưu trữ thông tin thực phẩm trong tủ lạnh, theo dõi ngày hết hạn và nhận thông báo nhắc nhở khi thực phẩm sắp hỏng. Hệ thống hỗ trợ lên kế hoạch bữa ăn theo ngày hoặc tuần, đồng thời đề xuất món ăn dựa trên các nguyên liệu sẵn có, giúp người dùng tận dụng thực phẩm hiệu quả hơn.

Ngoài ra, ứng dụng còn cho phép quản lý công thức nấu ăn yêu thích, tìm kiếm và phân loại thực phẩm, cũng như cung cấp báo cáo thống kê về hoạt động mua sắm và tiêu thụ, giúp người dùng theo dõi thói quen chi tiêu và cải thiện chế độ dinh dưỡng.

Để đảm bảo hoạt động ổn định và đầy đủ chức năng, hệ thống yêu cầu phát triển server lưu trữ dữ liệu, dịch vụ web xử lý yêu cầu từ ứng dụng di động, chức năng quản trị dành cho admin, cũng như giao diện người dùng thân thiện và push notification thông minh. Toàn bộ ứng dụng được phát triển theo hướng đa nền tảng, đảm bảo khả năng sử dụng linh hoạt trên nhiều thiết bị di động khác nhau.

1.2 Xác định thông tin cơ bản cho nghiệp vụ của bài toán

Nghệp vụ	Input	Process	Output
1. Tạo danh sách mua sắm	- Thực phẩm (tên, số lượng, đơn vị) - Người đảm nhận (trong group)	- Kiểm tra thực đơn gợi ý món - Lưu danh sách vào CSDL - Gửi thông báo	- Danh sách được tạo - Thông báo gửi đi
2. Tạo thực đơn cho bữa ăn	- Ngày/bữa ăn - Danh sách món ăn - Thực phẩm trong bếp	- Gợi ý món phù hợp thực phẩm sẵn có - Lưu thực đơn vào CSDL	- Thực đơn mới - Gợi ý món ăn
3. Thêm thực phẩm vào bếp	- Tên, số lượng, đơn vị - Ngày hết hạn	- Kiểm tra trùng lặp - Nếu trùng → chỉnh sửa - Nếu mới → lưu CSDL	Thực phẩm mới được thêm vào danh sách bếp
4. Tạo công thức nấu ăn	- Tên món ăn - Nguyên liệu - Người tạo	- Kiểm tra thông tin hợp lệ - Lưu công thức vào CSDL	- Công thức mới được tạo - Sẵn sàng tìm kiếm
5. Tạo thực phẩm	- Tên thực phẩm - Danh mục	- Kiểm tra trùng tên - Lưu vào CSDL cá nhân	Thực phẩm cá nhân được tạo và thêm vào nhóm

Bảng 2: Chi tiết luồng xử lý các nghiệp vụ chính của ứng dụng

2 Đặc tả yêu cầu bài toán

2.1 Giới thiệu chung

Xác định các tác nhân của hệ thống

STT	Tên tác nhân	Mô tả tác nhân
1	Người dùng	Người sử dụng ứng dụng.
2	Quản trị viên	Quản trị viên quản lý hệ thống.

Bảng 3: Bảng liệt kê và mô tả các tác nhân trong hệ thống

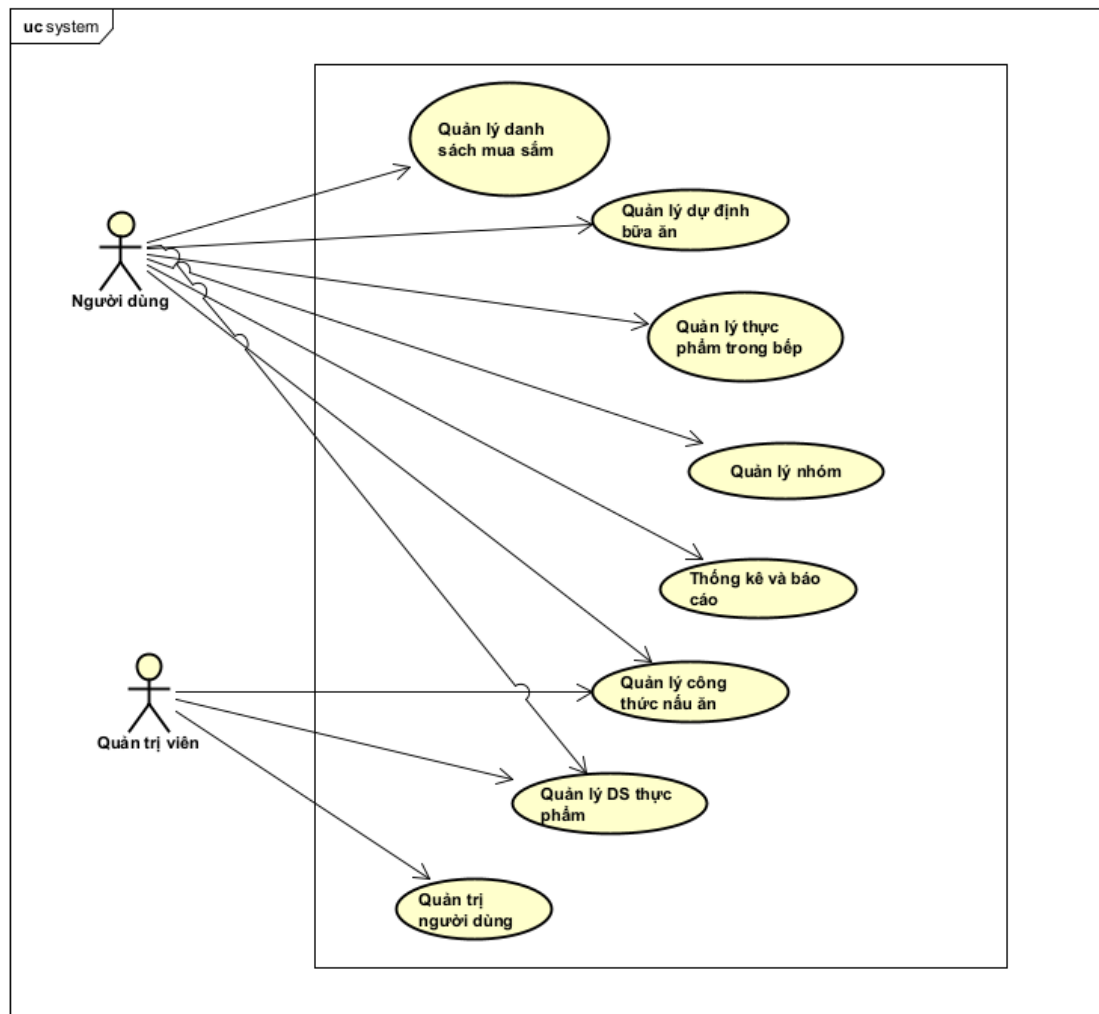
Xác định các ca sử dụng

STT	Mã UC	Tên Use Case	Mô tả Use Case	Tác nhân	Độ phức tạp
1	UC001	Quản lý danh sách mua sắm	Người dùng tạo, chỉnh sửa, xóa, theo dõi danh sách mặt hàng cần mua. Giao cho người dùng đi chợ.	Người dùng	Cao
2	UC002	Quản lý dự định bữa ăn	Người dùng lên kế hoạch bữa ăn theo ngày hoặc tuần.	Người dùng	Cao
3	UC003	Quản lý thực phẩm trong bếp	Người dùng nhập, chỉnh sửa, theo dõi thông tin thực phẩm tích trữ (tủ lạnh, bếp).	Người dùng	Thấp
4	UC004	Quản lý công thức nấu ăn	Người dùng lưu trữ và quản lý các công thức nấu ăn.	Người dùng, Quản trị viên	Trung bình
5	UC005	Quản lý danh sách thực phẩm	Người dùng quản lý danh sách các loại thực phẩm.	Người dùng, Quản trị viên	Trung bình
6	UC006	Quản lý nhóm	Người dùng tạo/xóa nhóm, thêm/xóa thành viên trong nhóm.	Người dùng	Thấp
7	UC007	Thống kê và báo cáo	Hệ thống tạo báo cáo về mua sắm và tiêu thụ thực phẩm.	Người dùng	Trung bình
8	UC008	Quản trị người dùng	Quản trị viên quản lý tài khoản người dùng.	Quản trị viên	Thấp
9	UC009	Xác thực và phân quyền	Người dùng đăng ký, đăng nhập, đăng xuất, quên mật khẩu, cập nhật thông tin cá nhân và đổi mật khẩu.	Người dùng	Trung bình

Bảng 4: Bảng tổng hợp danh mục Use Case của hệ thống

2.2 Biểu đồ use case

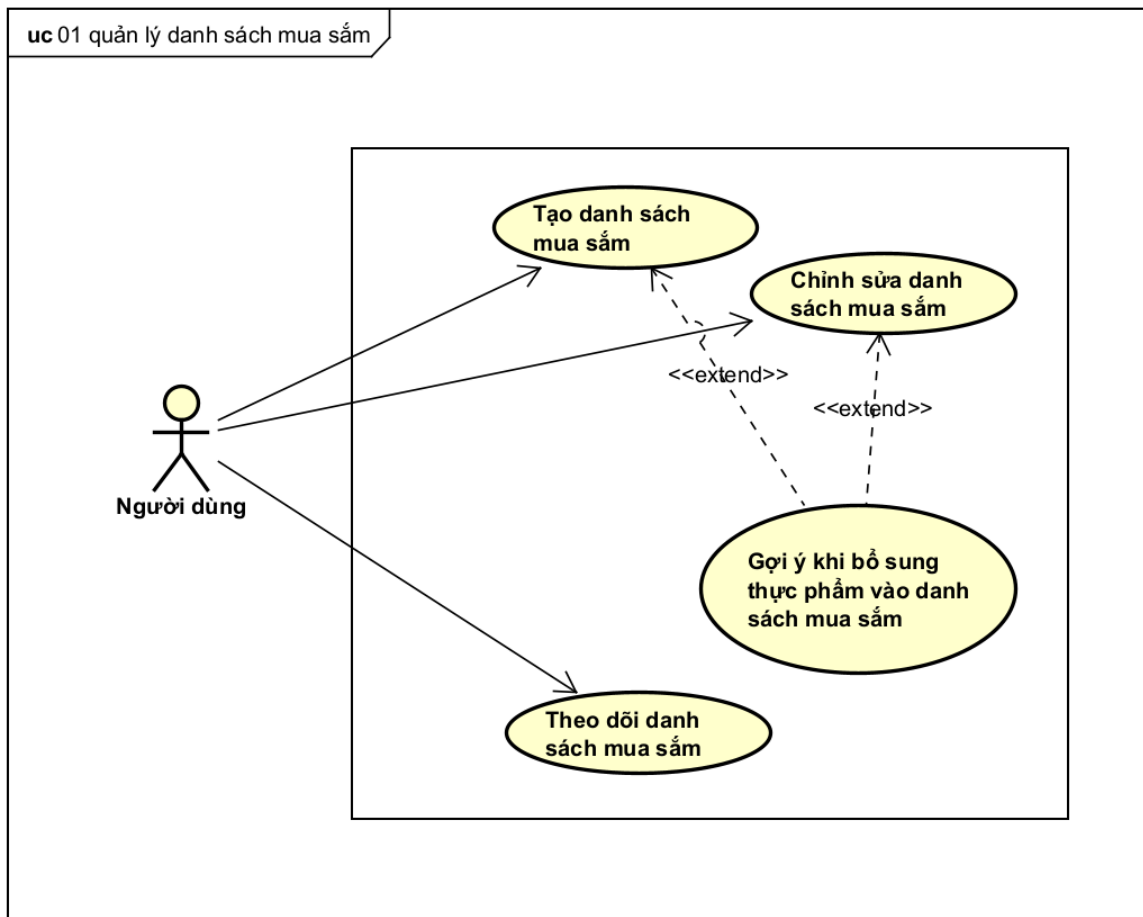
2.2.1 Biểu đồ use case tổng quan



Hình 1: Biểu đồ Use Case tổng quan của hệ thống

2.2.2 Biểu đồ use case phân rã mức 2

Quản lý danh sách mua sắm



Hình 2: Biểu đồ Usecase quản lý danh sách mua sắm

Mã UC	UC001.1	Tên UC: Tạo danh sách mua sắm
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia vào một group	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Điền thực phẩm cần mua
2	Người dùng	Chọn người đảm nhận
3	Người dùng	Hoàn thành tạo danh sách
4	Hệ thống	Lưu vào database
5	Hệ thống	Gửi thông báo cho người đảm nhận
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
1a	Hệ thống	Điều kiện: Trong group có thực đơn lên sẵn Hành động: Thực hiện Usecase UC001.4 (Gợi ý bổ sung thực phẩm)
Điều kiện sau	Danh sách mua sắm được tạo ra, người đảm nhiệm nhận được thông báo	

Bảng 5: Đặc tả ca sử dụng UC001.1: Tạo danh sách mua sắm

Mã UC	UC001.2	Tên UC: Chỉnh sửa danh sách mua sắm
Tác nhân	Người dùng	
Điều kiện trước	Group đã có danh sách mua sắm tạo trước đó	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Nếu cần thêm thực phẩm mới, điền thực phẩm cần mua.
2	Người dùng	Nếu cần xóa thực phẩm có sẵn, chọn xóa thực phẩm.
3	Người dùng	Nếu cần chỉnh sửa thực phẩm có sẵn, sửa lại thông tin thực phẩm.
4	Người dùng	Nếu cần chỉnh sửa người đảm nhận, sửa lại trường người đảm nhận.
5	Người dùng	Hoàn thành chỉnh sửa danh sách.
6	Hệ thống	Update trong database.
7	Hệ thống	Gửi thông báo cho người đảm nhận và cả người đảm nhận cũ (nếu có).
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
1a	Hệ thống	Điều kiện: Trong group có thực đơn lên sẵn Hành động: Thực hiện Usecase UC001.4 (Gợi ý bổ sung thực phẩm)
Điều kiện sau	Danh sách mua sắm được chỉnh sửa, người đảm nhiệm nhận được thông báo	

Bảng 6: Đặc tả ca sử dụng UC001.2: Chỉnh sửa danh sách mua sắm

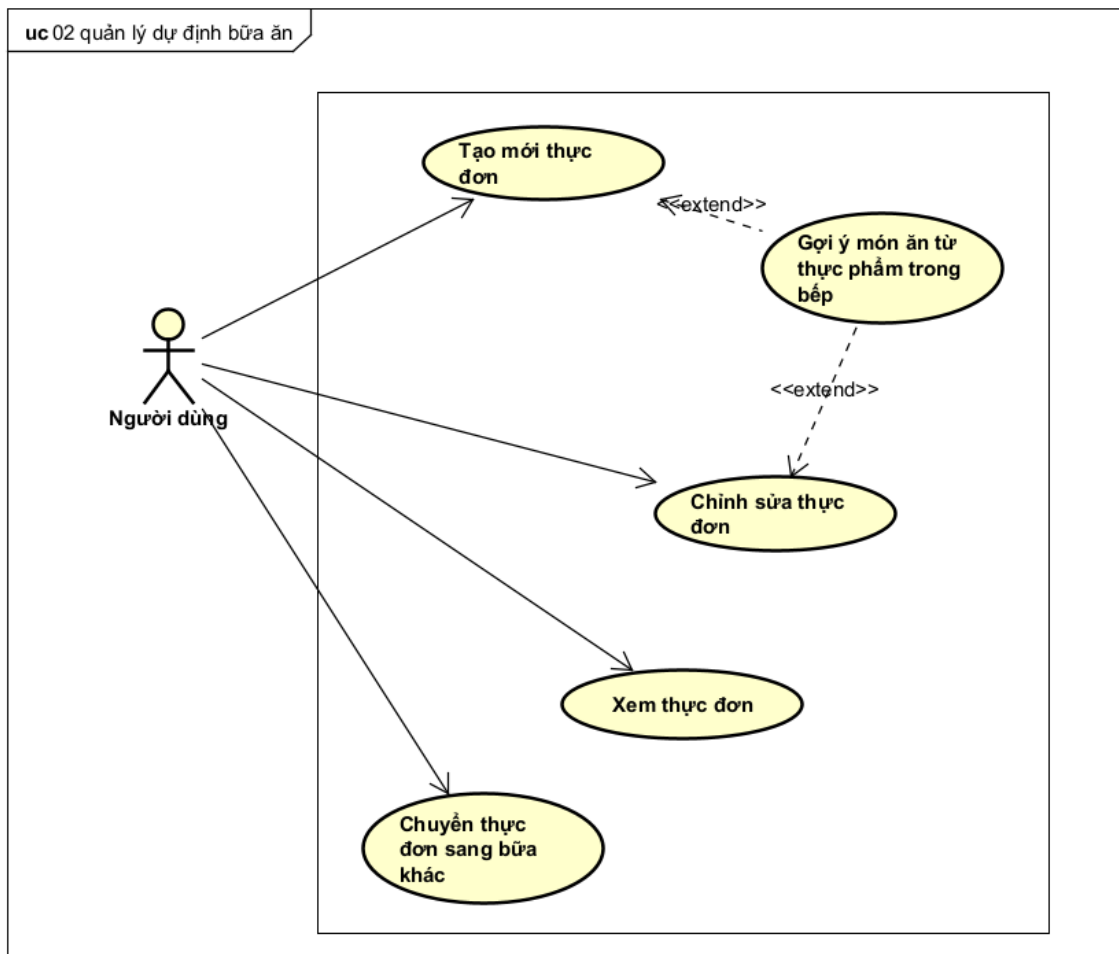
Mã UC	UC001.3	Tên UC: Theo dõi danh sách mua sắm
Tác nhân	Người dùng	
Điều kiện trước	Group đã có danh sách mua sắm tạo trước đó	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở danh sách mua sắm (hiển thị dưới dạng check-list).
2	Người dùng	Tick chọn một mặt hàng khi đã mua.
3	Hệ thống	Hiển thị hộp thoại nhập số lượng thực tế mua được.
4	Người dùng	Nhập số lượng thực tế và xác nhận.
5	Hệ thống	Ghi nhận số lượng thực tế đã mua: - Nếu số lượng thực tế = số lượng dự định → đánh dấu “đã mua”. - Nếu số lượng thực tế < số lượng dự định → đánh dấu “đã mua” và tạo thêm 1 mục cần mua là số lượng thực phẩm đó còn thiếu.
6	Người dùng	Lặp lại cho các mặt hàng khác.
7	Người dùng	Chọn hoàn thành đi chợ.
8	Người dùng	Gửi thông báo đến toàn group.
Điều kiện sau	Thông báo hoàn thành đi chợ được gửi đến toàn group	

Bảng 7: Đặc tả ca sử dụng UC001.3: Theo dõi danh sách mua sắm

Mã UC	UC001.4	Tên UC: Gợi ý bổ sung thực phẩm
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đang cần thêm một mục vào danh sách mua sắm; Trong group của người dùng có “thực đơn lên sẵn”	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Hệ thống	Kiểm tra dữ liệu: - Thực đơn đã lên lịch: Kiểm tra thực đơn cho những bữa sắp tới. - Công thức nấu ăn: Kiểm tra nguyên liệu cần dùng trong thực đơn đó. - Bếp: Kiểm tra nguyên liệu đó có trong bếp không.
2	Hệ thống	Kiểm tra điều kiện xem nguyên liệu đó có số lượng ít hơn hay khác đơn vị so với công thức hay không.
3	Hệ thống	Gợi ý thực phẩm nên bổ sung.
4	Người dùng	Chọn đồng ý để thêm thực phẩm vào danh sách mua sắm.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
2a	Hệ thống	Điều kiện: Nếu thực phẩm đó đã đủ hoặc thừa. Hành động: Quay lại bước 1 để tìm nguyên liệu tiếp theo.
4a	Hệ thống	Điều kiện: Người dùng không đồng ý thêm thực phẩm đó. Hành động: Quay lại bước 1 để tìm nguyên liệu tiếp theo.
Điều kiện sau	Thực phẩm cần mua được gợi ý cho người dùng	

Bảng 8: Đặc tả ca sử dụng UC001.4: Gợi ý khi bổ sung thực phẩm vào danh sách mua sắm

Quản lý dự định bữa ăn



Hình 3: Biểu đồ Usecase quản lý dự định bữa ăn

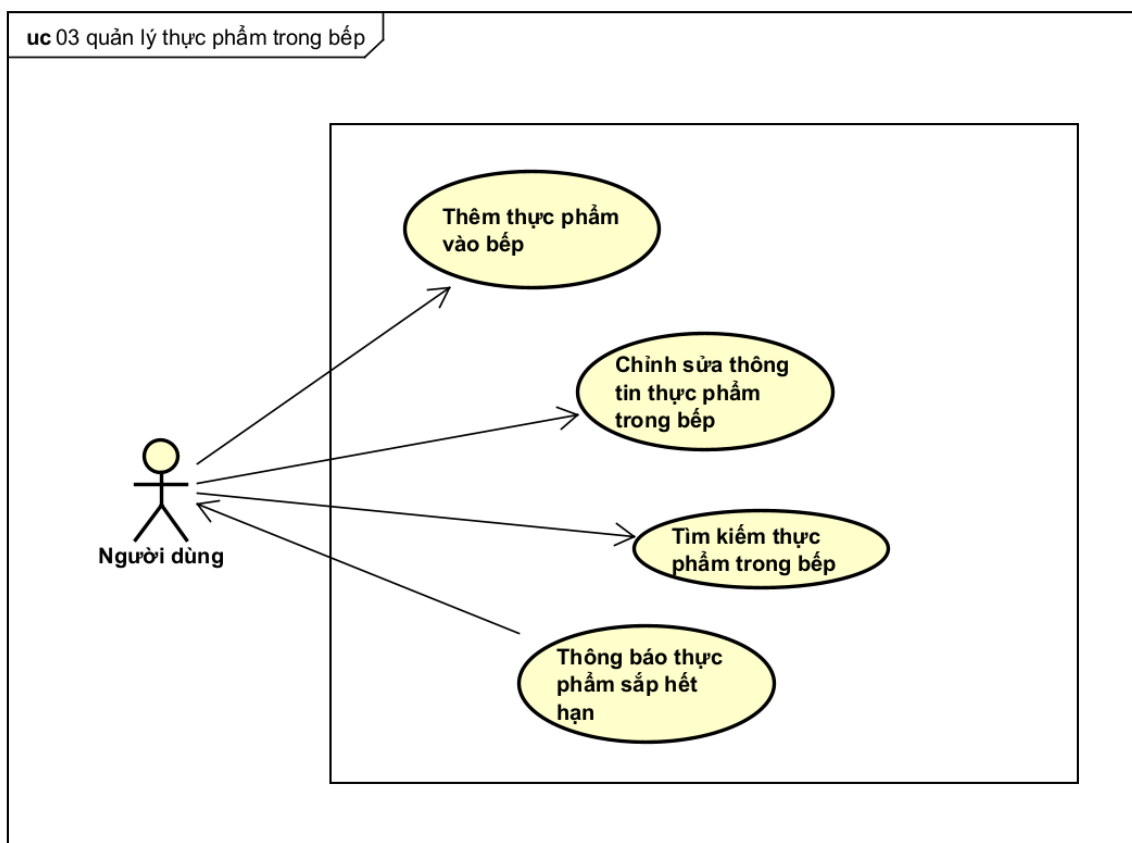
Mã UC	UC002.1	Tên UC: Tạo thực đơn mới
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia vào một group	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn bữa muốn tạo thực đơn.
2	Người dùng	Thêm từng món cho thực đơn.
3	Người dùng	Hoàn tất lên thực đơn.
4	Hệ thống	Lưu dữ liệu vào database.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
2a	Hệ thống	Điều kiện: Nếu trong bếp có thực phẩm. Hành động: Kích hoạt Usecase UC002.3.
Điều kiện sau	Thực đơn mới được tạo ra	

Bảng 9: Đặc tả ca sử dụng UC002.1: Tạo thực đơn mới

Mã UC	UC002.3	Tên UC: Gợi ý món ăn từ thực phẩm trong bếp
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đang thêm món vào thực đơn, trong bếp đã có sẵn thực phẩm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Hệ thống	Kiểm tra: - Bếp: Kiểm tra các thực phẩm có sẵn trong bếp. - Thực đơn: Kiểm tra các món ăn sử dụng thực phẩm đó làm nguyên liệu chính.
2	Hệ thống	Gợi ý món ăn đó cho người dùng.
3	Người dùng	Đồng ý thêm món vào thực đơn.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
2a	Hệ thống	Điều kiện: Chọn không đồng ý thêm món ăn. Hành động: Quay lại bước 1 để gợi ý món ăn tiếp theo.
Điều kiện sau	Thực đơn mới được tạo ra	

Bảng 10: Đặc tả ca sử dụng UC002.3: Gợi ý món ăn từ thực phẩm trong bếp

Quản lý thực phẩm trong bếp



Hình 4: Biểu đồ Use Case quản lý thực phẩm trong bếp

Mã UC	UC003.1	Tên UC: Thêm thực phẩm vào bếp
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia vào một group	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn thêm thực phẩm.
2	Người dùng	Điền thông tin thực phẩm.
3	Hệ thống	Lưu dữ liệu vào database.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
2a	Hệ thống	Điều kiện: Nếu trong bếp có thực phẩm. Hành động: Kích hoạt Usecase UC002.3.
Điều kiện sau	Thực phẩm mới được thêm vào trong bếp	

Bảng 11: Đặc tả ca sử dụng UC003.1: Thêm thực phẩm vào bếp

Mã UC	UC003.2	Tên UC: Chỉnh sửa thực phẩm trong bếp
Tác nhân	Người dùng	
Điều kiện trước	Trong tủ lạnh đã có sẵn sản phẩm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn thực phẩm cần chỉnh sửa.
2	Người dùng	Chỉnh sửa thông tin thực phẩm (Chỉ chỉnh sửa số lượng và đơn vị).
3	Hệ thống	Lưu dữ liệu vào database.
Điều kiện sau	Thực phẩm được chỉnh sửa ở danh sách thực phẩm trong bếp	

Bảng 12: Đặc tả ca sử dụng UC003.2: Chỉnh sửa thực phẩm trong bếp

Mã UC	UC003.3	Tên UC: Xóa thực phẩm khỏi bếp
Tác nhân	Người dùng	
Điều kiện trước	Trong bếp đã có sẵn thực phẩm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn thực phẩm cần xóa.
2	Hệ thống	Hiển thị xác nhận xóa.
3	Người dùng	Xác nhận xóa.
4	Hệ thống	Xóa thực phẩm khỏi database.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
3a	Hệ thống	Điều kiện: Người dùng hủy bỏ Hành động: Quay lại danh sách thực phẩm
Điều kiện sau	Thực phẩm được xóa khỏi danh sách thực phẩm trong bếp	

Bảng 13: Đặc tả ca sử dụng UC003.3: Xóa thực phẩm khỏi bếp

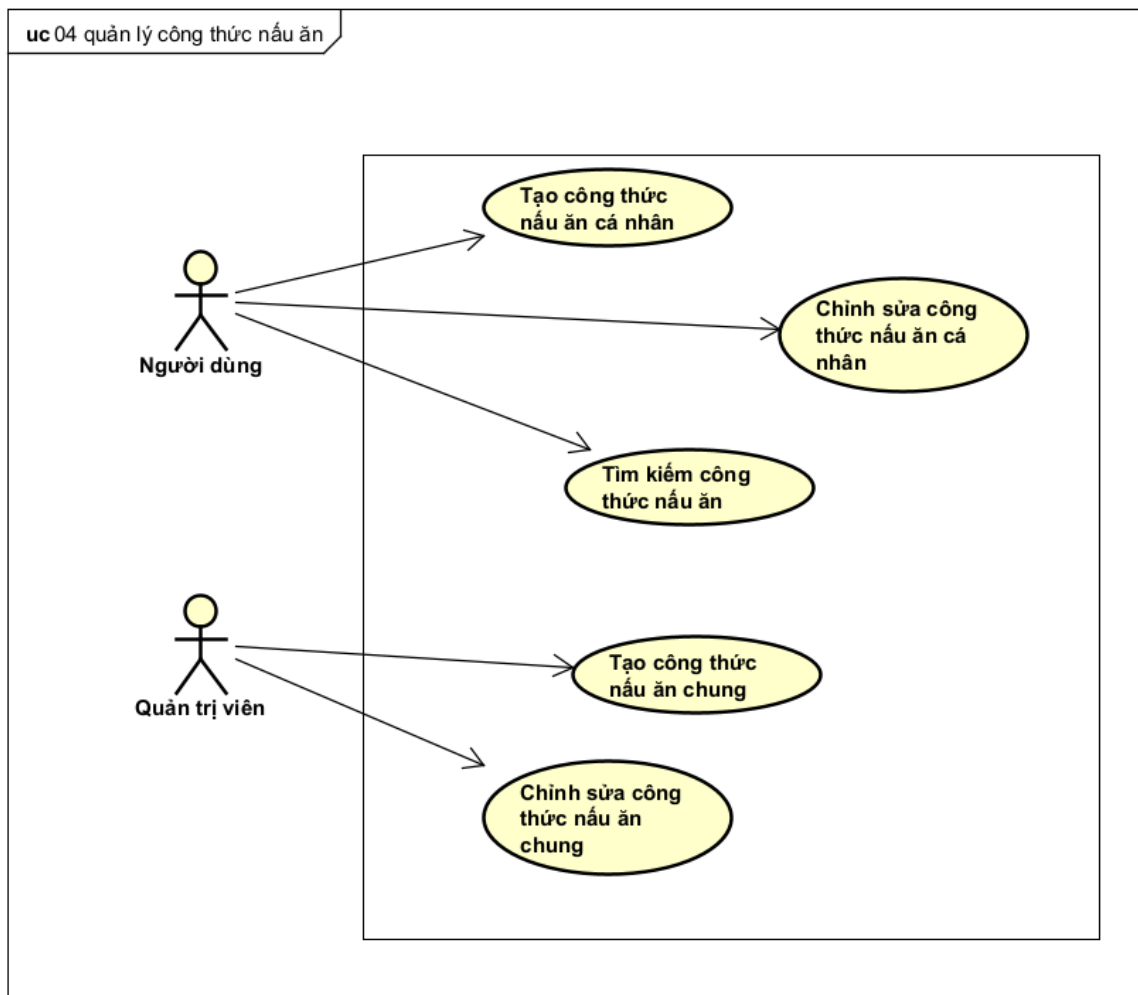
Mã UC	UC003.4	Tên UC: Nhắc nhở thực phẩm sắp hết hạn
Tác nhân	Hệ thống (tự động)	
Điều kiện trước	Trong bếp có thực phẩm với ngày hết hạn được thiết lập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Hệ thống	Kiểm tra định kỳ (mỗi ngày) danh sách thực phẩm trong bếp.
2	Hệ thống	Xác định các thực phẩm có ngày hết hạn trong vòng 3 ngày tới.
3	Hệ thống	Tạo thông báo đẩy (push notification) cho tất cả thành viên trong nhóm.
4	Hệ thống	Hiển thị nội dung thông báo theo template.
Điều kiện sau	Người dùng nhận được thông báo về thực phẩm sắp hết hạn	

Bảng 14: Đặc tả ca sử dụng UC003.4: Nhắc nhở thực phẩm sắp hết hạn

Mã UC	UC003.5	Tên UC: Xem danh sách thực phẩm trong bếp
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập và tham gia nhóm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở màn hình quản lý bếp.
2	Hệ thống	Hiển thị danh sách thực phẩm (tên, số lượng, đơn vị, ngày hết hạn, trạng thái).
3	Người dùng	Có thể lọc theo danh mục, sắp xếp theo ngày hết hạn.
Điều kiện sau	Danh sách thực phẩm được hiển thị	

Bảng 15: Đặc tả ca sử dụng UC003.5: Xem danh sách thực phẩm trong bếp

Quản lý công thức nấu ăn



Hình 5: Biểu đồ Use Case quản lý công thức nấu ăn

Mã UC	UC004.1	Tên UC: Tạo công thức nấu ăn chung
Tác nhân	Quản trị viên	
Điều kiện trước	Quản trị viên đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Quản trị viên	Chọn tạo công thức nấu ăn chung.
2	Quản trị viên	Điền thông tin cho công thức nấu ăn mới.
3	Quản trị viên	Hoàn thành điền thông tin.
4	Hệ thống	Lưu dữ liệu vào database.
Điều kiện sau	Công thức nấu ăn chung mới được tạo ra	

Bảng 16: Đặc tả ca sử dụng UC004.1: Tạo công thức nấu ăn chung

Mã UC	UC004.2	Tên UC: Chỉnh sửa công thức nấu ăn
Tác nhân	Người dùng (công thức cá nhân), QTV (công thức chung)	
Điều kiện trước	Công thức nấu ăn đã tồn tại	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng/QTV	Chọn công thức nấu ăn cần chỉnh sửa.
2	Hệ thống	Hiển thị thông tin công thức hiện tại.
3	Người dùng/QTV	Chỉnh sửa các trường thông tin công thức.
4	Người dùng/QTV	Hoàn thành chỉnh sửa.
5	Hệ thống	Cập nhật dữ liệu vào database.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
1a	Hệ thống	Điều kiện: Người dùng không có quyền chỉnh sửa công thức (công thức chung hoặc công thức của nhóm khác) Hành động: Hiển thị thông báo lỗi
Điều kiện sau	Công thức nấu ăn được cập nhật	

Bảng 17: Đặc tả ca sử dụng UC004.2: Chỉnh sửa công thức nấu ăn

Mã UC	UC004.3	Tên UC: Tạo công thức nấu ăn cá nhân
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia vào nhóm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn tạo công thức nấu ăn cho nhóm.
2	Người dùng	Điền thông tin cho công thức nấu ăn mới.
3	Người dùng	Hoàn thành điền thông tin.
4	Hệ thống	Lưu dữ liệu vào database.
Điều kiện sau	Công thức nấu ăn cá nhân mới được tạo ra	

Bảng 18: Đặc tả ca sử dụng UC004.3: Tạo công thức nấu ăn cá nhân

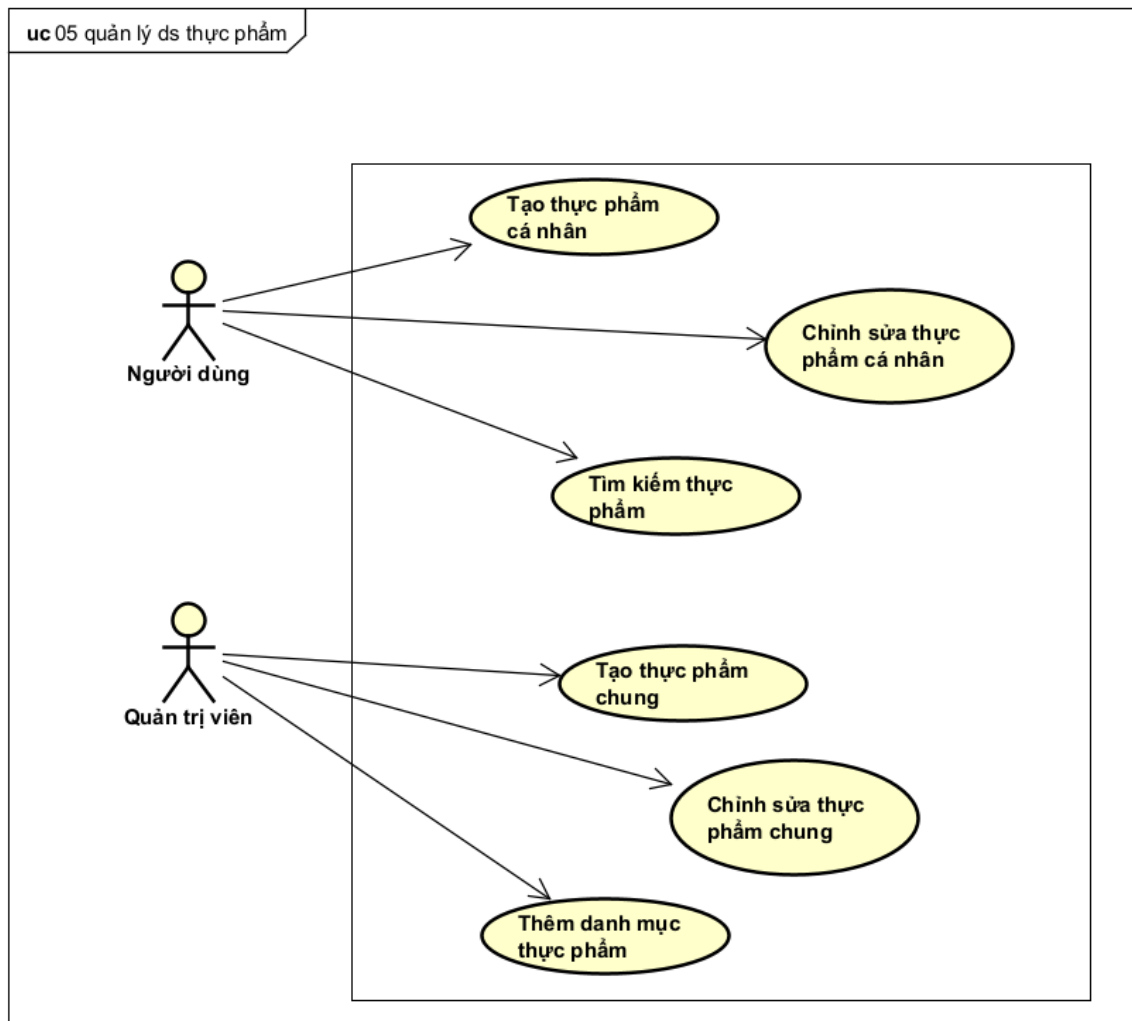
Mã UC	UC004.4	Tên UC: Xóa công thức nấu ăn
Tác nhân	Người dùng (công thức cá nhân), QTV (công thức chung)	
Điều kiện trước	Công thức nấu ăn đã tồn tại	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng/QTV	Chọn công thức nấu ăn cần xóa.
2	Hệ thống	Hiển thị xác nhận xóa.
3	Người dùng/QTV	Xác nhận xóa.
4	Hệ thống	Xóa công thức khỏi database.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
1a	Hệ thống	Điều kiện: Người dùng không có quyền xóa công thức Hành động: Hiển thị thông báo lỗi
3a	Hệ thống	Điều kiện: Người dùng/QTV hủy bỏ Hành động: Quay lại danh sách công thức
Điều kiện sau	Công thức nấu ăn được xóa khỏi hệ thống	

Bảng 19: Đặc tả ca sử dụng UC004.4: Xóa công thức nấu ăn

Mã UC	UC004.5	Tên UC: Tìm kiếm công thức món ăn
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Nhập vào thanh tìm kiếm tên món ăn hoặc nguyên liệu.
2	Hệ thống	Trả về danh sách món ăn.
Điều kiện sau	Trả về danh sách kết quả các món ăn	

Bảng 20: Đặc tả ca sử dụng UC004.5: Tìm kiếm công thức món ăn

Quản lý danh sách thực phẩm



Hình 6: Biểu đồ Use Case quản lý danh sách thực phẩm

Mã UC	UC005.1	Tên UC: Tạo thực phẩm cá nhân
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia vào nhóm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn thực phẩm mới cho nhóm.
2	Người dùng	Điền thông tin cho thực phẩm mới.
3	Hệ thống	Kiểm tra tên thực phẩm có trùng không.
4	Người dùng	Hoàn thành tạo thực phẩm mới.
5	Hệ thống	Lưu dữ liệu vào database.
Điều kiện sau	Thực phẩm cá nhân mới được tạo ra	

Bảng 21: Đặc tả ca sử dụng UC005.1: Tạo thực phẩm cá nhân

Mã UC	UC005.2	Tên UC: Tạo thực phẩm chung
Tác nhân	Quản trị viên	
Điều kiện trước	QTV đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	QTV	Chọn tạo thực phẩm chung.
2	QTV	Điền đầy đủ thông tin thực phẩm.
3	Hệ thống	Kiểm tra tên thực phẩm có trùng không.
4	QTV	Hoàn thành tạo thực phẩm mới.
5	Hệ thống	Lưu dữ liệu vào database.
Điều kiện sau	Thực phẩm chung được tạo ra, có thể sử dụng bởi tất cả người dùng	

Bảng 22: Đặc tả ca sử dụng UC005.2: Tạo thực phẩm chung

Mã UC	UC005.3	Tên UC: Chỉnh sửa thực phẩm
Tác nhân	Người dùng (thực phẩm cá nhân), QTV (thực phẩm chung)	
Điều kiện trước	Thực phẩm đã tồn tại	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng/QTV	Chọn thực phẩm cần chỉnh sửa.
2	Hệ thống	Hiển thị thông tin hiện tại.
3	Người dùng/QTV	Chỉnh sửa các trường thông tin.
4	Hệ thống	Kiểm tra tên thực phẩm có trùng không.
5	Người dùng/QTV	Hoàn thành chỉnh sửa.
6	Hệ thống	Cập nhật dữ liệu vào database.
Điều kiện sau	Thông tin thực phẩm được cập nhật	

Bảng 23: Đặc tả ca sử dụng UC005.3: Chỉnh sửa thực phẩm

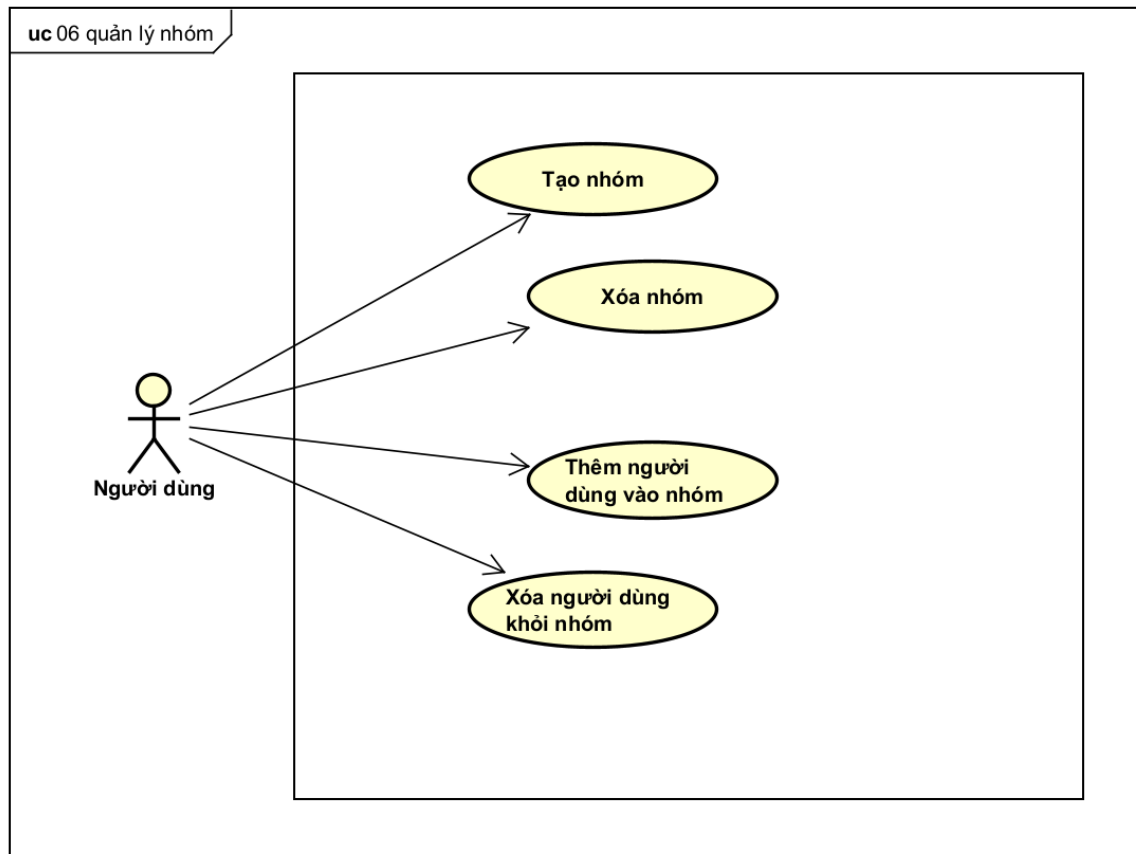
Mã UC	UC005.4	Tên UC: Xóa thực phẩm
Tác nhân	Người dùng (thực phẩm cá nhân), QTV (thực phẩm chung)	
Điều kiện trước	Thực phẩm đã tồn tại và không được sử dụng trong bất kỳ công thức hoặc bếp nào	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng/QTV	Chọn thực phẩm cần xóa.
2	Hệ thống	Kiểm tra xem thực phẩm có đang được sử dụng không.
3	Hệ thống	Hiển thị xác nhận xóa.
4	Người dùng/QTV	Xác nhận xóa.
5	Hệ thống	Xóa thực phẩm khỏi database.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
2a	Hệ thống	Điều kiện: Thực phẩm đang được sử dụng trong công thức hoặc bếp Hành động: Hiển thị thông báo không thể xóa
4a	Hệ thống	Điều kiện: Người dùng/QTV hủy bỏ Hành động: Quay lại danh sách thực phẩm
Điều kiện sau	Thực phẩm được xóa khỏi hệ thống	

Bảng 24: Đặc tả ca sử dụng UC005.4: Xóa thực phẩm

Mã UC	UC005.5	Tên UC: Tìm kiếm thực phẩm
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Nhập từ khóa tìm kiếm (tên thực phẩm hoặc danh mục).
2	Hệ thống	Tìm kiếm trong thực phẩm chung và thực phẩm của nhóm.
3	Hệ thống	Hiển thị danh sách kết quả.
Điều kiện sau	Danh sách thực phẩm phù hợp được hiển thị	

Bảng 25: Đặc tả ca sử dụng UC005.5: Tìm kiếm thực phẩm

Quản lý nhóm



Hình 7: Biểu đồ Use Case quản lý nhóm

Mã UC	UC006.1	Tên UC: Tạo nhóm
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn tạo nhóm mới.
2	Người dùng	Điền thông tin nhóm (tên nhóm, mô tả).
3	Người dùng	Hoàn thành tạo nhóm.
4	Hệ thống	Tạo nhóm và gán người dùng làm trưởng nhóm.
5	Hệ thống	Lưu dữ liệu vào database.
Điều kiện sau	Nhóm mới được tạo ra, người dùng trở thành trưởng nhóm	

Bảng 26: Đặc tả ca sử dụng UC006.1: Tạo nhóm

Mã UC	UC006.2	Tên UC: Thêm thành viên vào nhóm
Tác nhân	Trưởng nhóm hoặc thành viên có quyền	
Điều kiện trước	Nhóm đã tồn tại, người dùng là trưởng nhóm hoặc có quyền thêm thành viên	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn thêm thành viên.
2	Người dùng	Nhập email/số điện thoại của thành viên mới.
3	Hệ thống	Tìm kiếm tài khoản trong hệ thống.
4	Hệ thống	Hiển thị thông tin tài khoản tìm được.
5	Người dùng	Xác nhận thêm thành viên.
6	Hệ thống	Gửi lời mời tham gia nhóm.
7	Thành viên mới	Nhận thông báo và chấp nhận lời mời.
8	Hệ thống	Thêm thành viên vào nhóm.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
3a	Hệ thống	Điều kiện: Không tìm thấy tài khoản Hành động: Hiển thị thông báo “Không tìm thấy người dùng”
7a	Hệ thống	Điều kiện: Thành viên từ chối lời mời Hành động: Thông báo cho người gửi lời mời
Điều kiện sau	Thành viên mới được thêm vào nhóm, nhận được thông báo	

Bảng 27: Đặc tả ca sử dụng UC006.2: Thêm thành viên vào nhóm

Mã UC	UC006.3	Tên UC: Xóa thành viên khỏi nhóm
Tác nhân	Trưởng nhóm	
Điều kiện trước	Nhóm có ít nhất 2 thành viên	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Trưởng nhóm	Chọn thành viên cần xóa.
2	Hệ thống	Hiển thị xác nhận xóa.
3	Trưởng nhóm	Xác nhận xóa.
4	Hệ thống	Xóa thành viên khỏi nhóm.
5	Hệ thống	Gửi thông báo cho thành viên bị xóa.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
1a	Hệ thống	Điều kiện: Không thể xóa trưởng nhóm Hành động: Hiển thị thông báo lỗi
Điều kiện sau	Thành viên bị xóa khỏi nhóm, nhận được thông báo	

Bảng 28: Đặc tả ca sử dụng UC006.3: Xóa thành viên khỏi nhóm

Mã UC	UC006.4	Tên UC: Rời khỏi nhóm
Tác nhân	Người dùng (thành viên nhóm)	
Điều kiện trước	Người dùng là thành viên của nhóm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn rời khỏi nhóm.
2	Hệ thống	Hiển thị xác nhận.
3	Người dùng	Xác nhận rời nhóm.
4	Hệ thống	Xóa người dùng khỏi nhóm.
5	Hệ thống	Gửi thông báo cho các thành viên còn lại.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
3a	Hệ thống	Điều kiện: Người dùng là trưởng nhóm và nhóm có thành viên khác Hành động: Yêu cầu chuyển quyền trưởng nhóm trước
3b	Hệ thống	Điều kiện: Người dùng là thành viên duy nhất Hành động: Nhóm sẽ bị xóa
Điều kiện sau	Người dùng rời khỏi nhóm, các thành viên còn lại nhận được thông báo	

Bảng 29: Đặc tả ca sử dụng UC006.4: Rời khỏi nhóm

Mã UC	UC006.5	Tên UC: Chuyển quyền trưởng nhóm
Tác nhân	Trưởng nhóm	
Điều kiện trước	Nhóm có ít nhất 2 thành viên	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Trưởng nhóm	Chọn chuyển quyền.
2	Trưởng nhóm	Chọn thành viên mới làm trưởng nhóm.
3	Hệ thống	Hiển thị xác nhận.
4	Trưởng nhóm	Xác nhận.
5	Hệ thống	Cập nhật vai trò.
6	Hệ thống	Gửi thông báo cho tất cả thành viên.
Điều kiện sau	Quyền trưởng nhóm được chuyển cho thành viên khác	

Bảng 30: Đặc tả ca sử dụng UC006.5: Chuyển quyền trưởng nhóm

Mã UC	UC006.6	Tên UC: Xem thông tin nhóm
Tác nhân	Người dùng (thành viên nhóm)	
Điều kiện trước	Người dùng là thành viên của nhóm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở màn hình thông tin nhóm.
2	Hệ thống	Hiển thị thông tin (tên nhóm, trưởng nhóm, số thành viên, danh sách thành viên, ngày tạo).
Điều kiện sau	Thông tin nhóm được hiển thị	

Bảng 31: Đặc tả ca sử dụng UC006.6: Xem thông tin nhóm

Mã UC	UC006.7	Tên UC: Xóa nhóm
Tác nhân	Trưởng nhóm	
Điều kiện trước	Người dùng là trưởng nhóm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Trưởng nhóm	Chọn xóa nhóm.
2	Hệ thống	Hiển thị cảnh báo về việc xóa tất cả dữ liệu.
3	Trưởng nhóm	Xác nhận xóa.
4	Hệ thống	Xóa nhóm và dữ liệu liên quan.
5	Hệ thống	Gửi thông báo cho tất cả thành viên.
Điều kiện sau	Nhóm và tất cả dữ liệu liên quan bị xóa, thành viên nhận thông báo	

Bảng 32: Đặc tả ca sử dụng UC006.7: Xóa nhóm

Thống kê và báo cáo

Mã UC	UC007.1	Tên UC: Xem báo cáo mua sắm
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia nhóm và có dữ liệu mua sắm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở màn hình báo cáo.
2	Người dùng	Chọn loại báo cáo “Mua sắm”.
3	Người dùng	Chọn khoảng thời gian (tuần/tháng/năm).
4	Hệ thống	Tính toán và hiển thị báo cáo (tổng số lần đi chợ, top 10 thực phẩm mua nhiều nhất, biểu đồ xu hướng, thành viên đi chợ nhiều nhất).
Điều kiện sau	Báo cáo mua sắm được hiển thị	

Bảng 33: Đặc tả ca sử dụng UC007.1: Xem báo cáo mua sắm

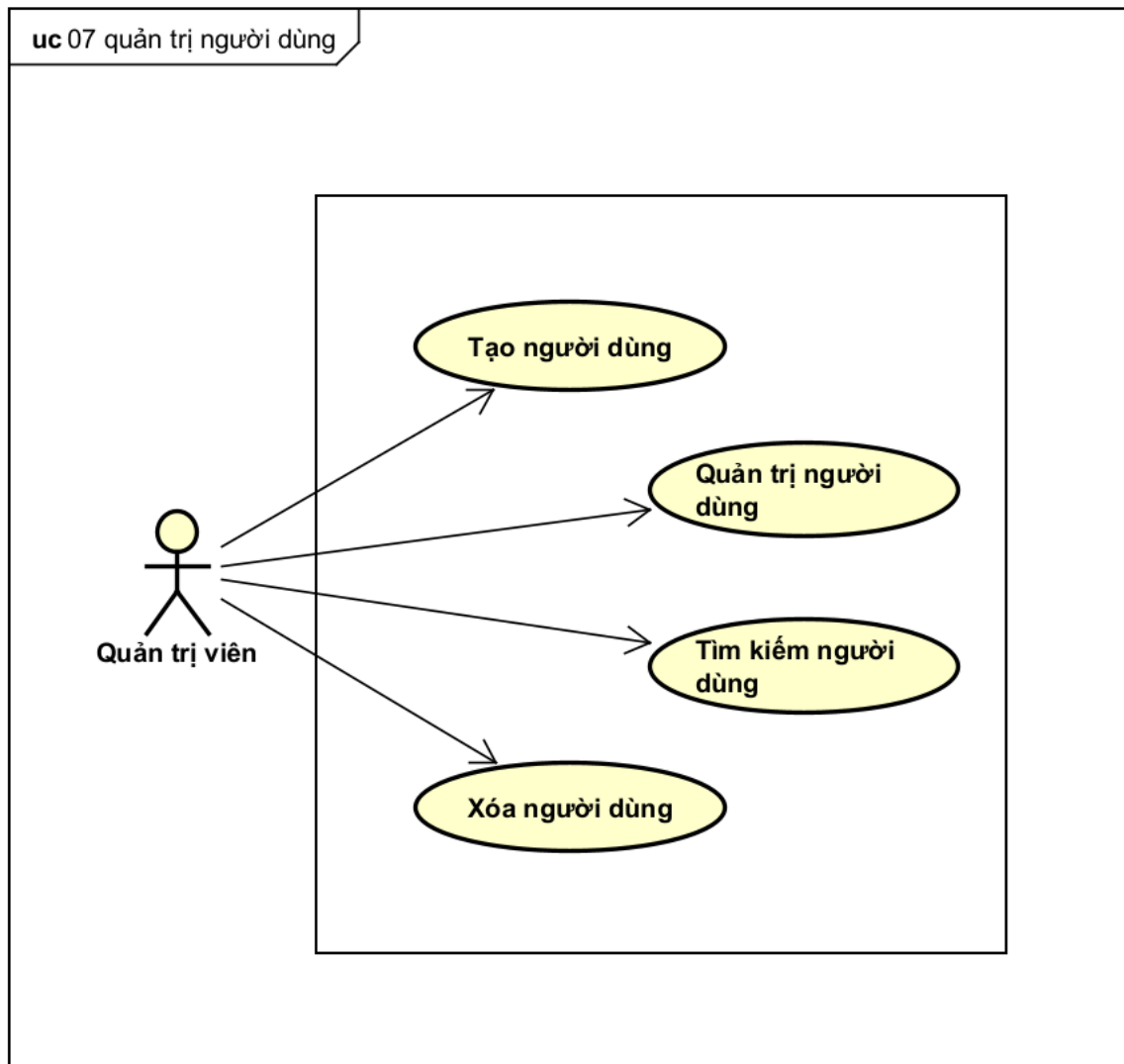
Mã UC	UC007.2	Tên UC: Xem báo cáo tiêu thụ thực phẩm
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia nhóm và có dữ liệu thực phẩm	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở màn hình báo cáo.
2	Người dùng	Chọn loại báo cáo “Tiêu thụ”.
3	Người dùng	Chọn khoảng thời gian (tuần/tháng/năm).
4	Hệ thống	Tính toán và hiển thị báo cáo (tổng số thực phẩm đã tiêu thụ, tỷ lệ lãng phí, top món ăn được nấu nhiều nhất, biểu đồ phân bổ danh mục).
Điều kiện sau	Báo cáo tiêu thụ được hiển thị	

Bảng 34: Đặc tả ca sử dụng UC007.2: Xem báo cáo tiêu thụ thực phẩm

Mã UC	UC007.3	Tên UC: Xem báo cáo thực đơn
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã tham gia nhóm và có dữ liệu thực đơn	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở màn hình báo cáo.
2	Người dùng	Chọn loại báo cáo “Thực đơn”.
3	Người dùng	Chọn khoảng thời gian (tuần/tháng).
4	Hệ thống	Hiển thị thống kê (tổng số bữa ăn đã lên lịch, tỷ lệ hoàn thành, món ăn phổ biến, thống kê theo bữa).
Điều kiện sau	Báo cáo thực đơn được hiển thị	

Bảng 35: Đặc tả ca sử dụng UC007.3: Xem báo cáo thực đơn

Quản trị người dùng



Hình 8: Biểu đồ Use Case quản lý người dùng

Mã UC	UC008.1	Tên UC: Xem danh sách người dùng
Tác nhân	Quản trị viên	
Điều kiện trước	QTV đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	QTV	Mở màn hình quản lý người dùng.
2	Hệ thống	Hiển thị danh sách người dùng (ID, tên, email, số điện thoại, trạng thái, ngày đăng ký).
3	QTV	Có thể tìm kiếm, lọc theo trạng thái.
Điều kiện sau	Danh sách người dùng được hiển thị	

Bảng 36: Đặc tả ca sử dụng UC008.1: Xem danh sách người dùng

Mã UC	UC008.2	Tên UC: Khóa/Mở khóa tài khoản
Tác nhân	Quản trị viên	
Điều kiện trước	Tài khoản người dùng tồn tại	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	QTV	Chọn tài khoản cần khóa/mở khóa.
2	QTV	Chọn hành động (khóa/mở khóa).
3	Hệ thống	Hiển thị xác nhận.
4	QTV	Xác nhận.
5	Hệ thống	Cập nhật trạng thái tài khoản.
6	Hệ thống	Gửi email thông báo cho người dùng.
Điều kiện sau	Trạng thái tài khoản được thay đổi	

Bảng 37: Đặc tả ca sử dụng UC008.2: Khóa/Mở khóa tài khoản

Mã UC	UC008.3	Tên UC: Xóa tài khoản
Tác nhân	Quản trị viên	
Điều kiện trước	Tài khoản người dùng tồn tại	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	QTV	Chọn tài khoản cần xóa.
2	Hệ thống	Hiển thị cảnh báo về việc xóa dữ liệu.
3	QTV	Xác nhận xóa.
4	Hệ thống	Xóa tài khoản và dữ liệu liên quan.
5	Hệ thống	Ghi log hành động.
Điều kiện sau	Tài khoản và dữ liệu liên quan bị xóa	

Bảng 38: Đặc tả ca sử dụng UC008.3: Xóa tài khoản

Xác thực và phân quyền

Mã UC	UC009.1	Tên UC: Đăng ký tài khoản
Tác nhân	Người dùng chưa có tài khoản	
Điều kiện trước	Không có	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn đăng ký tài khoản.
2	Người dùng	Điền thông tin (họ tên, email, số điện thoại, mật khẩu, xác nhận mật khẩu).
3	Hệ thống	Kiểm tra tính hợp lệ của dữ liệu.
4	Hệ thống	Kiểm tra email/số điện thoại đã tồn tại chưa.
5	Hệ thống	Gửi mã xác thực qua email/SMS.
6	Người dùng	Nhập mã xác thực.
7	Hệ thống	Xác thực mã và tạo tài khoản.
8	Hệ thống	Gửi email chào mừng.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
4a	Hệ thống	Điều kiện: Email/số điện thoại đã tồn tại Hành động: Hiển thị thông báo lỗi
6a	Hệ thống	Điều kiện: Mã không đúng hoặc hết hạn Hành động: Cho phép gửi lại mã
Điều kiện sau	Tài khoản mới được tạo, người dùng có thể đăng nhập	

Bảng 39: Đặc tả ca sử dụng UC009.1: Đăng ký tài khoản

Mã UC	UC009.2	Tên UC: Đăng nhập
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã có tài khoản	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở ứng dụng.
2	Người dùng	Nhập email và mật khẩu.
3	Hệ thống	Xác thực thông tin.
4	Hệ thống	Tạo token và lưu session.
5	Hệ thống	Chuyển người dùng vào màn hình chính.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
3a	Hệ thống	Điều kiện: Thông tin không đúng Hành động: Hiển thị thông báo lỗi
3b	Hệ thống	Điều kiện: Tài khoản bị khóa Hành động: Hiển thị thông báo tài khoản bị khóa
Điều kiện sau	Người dùng được xác thực và truy cập vào hệ thống	

Bảng 40: Đặc tả ca sử dụng UC009.2: Đăng nhập

Mã UC	UC009.3	Tên UC: Đăng xuất
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn đăng xuất.
2	Hệ thống	Hiển thị xác nhận.
3	Người dùng	Xác nhận.
4	Hệ thống	Xóa token và session.
5	Hệ thống	Chuyển về màn hình đăng nhập.
Điều kiện sau	Session bị xóa, người dùng về màn hình đăng nhập	

Bảng 41: Đặc tả ca sử dụng UC009.3: Đăng xuất

Mã UC	UC009.4	Tên UC: Quên mật khẩu
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã có tài khoản	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn “Quên mật khẩu”.
2	Người dùng	Nhập email đăng ký.
3	Hệ thống	Kiểm tra email có tồn tại không.
4	Hệ thống	Gửi mã xác thực qua email.
5	Người dùng	Nhập mã xác thực.
6	Hệ thống	Xác thực mã.
7	Người dùng	Nhập mật khẩu mới.
8	Hệ thống	Cập nhật mật khẩu.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
3a	Hệ thống	Điều kiện: Email không tồn tại Hành động: Hiển thị thông báo
5a	Hệ thống	Điều kiện: Mã không đúng Hành động: Cho phép thử lại (tối đa 3 lần)
Điều kiện sau	Mật khẩu được đặt lại	

Bảng 42: Đặc tả ca sử dụng UC009.4: Quên mật khẩu

Mã UC	UC009.5	Tên UC: Cập nhật thông tin cá nhân
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Mở màn hình thông tin cá nhân.
2	Hệ thống	Hiển thị thông tin hiện tại.
3	Người dùng	Chỉnh sửa thông tin (tên, số điện thoại, avatar).
4	Người dùng	Xác nhận cập nhật.
5	Hệ thống	Lưu thông tin mới.
Điều kiện sau	Thông tin cá nhân được cập nhật	

Bảng 43: Đặc tả ca sử dụng UC009.5: Cập nhật thông tin cá nhân

Mã UC	UC009.6	Tên UC: Đổi mật khẩu
Tác nhân	Người dùng	
Điều kiện trước	Người dùng đã đăng nhập	
Luồng thực thi chính		
STT	Thực hiện	Hành động
1	Người dùng	Chọn đổi mật khẩu.
2	Người dùng	Nhập mật khẩu hiện tại.
3	Hệ thống	Xác thực mật khẩu hiện tại.
4	Người dùng	Nhập mật khẩu mới và xác nhận.
5	Hệ thống	Kiểm tra mật khẩu mới hợp lệ.
6	Hệ thống	Cập nhật mật khẩu.
7	Hệ thống	Gửi email thông báo đổi mật khẩu.
Luồng thực thi mở rộng		
STT	Thực hiện	Hành động và Điều kiện
3a	Hệ thống	Điều kiện: Mật khẩu hiện tại không đúng Hành động: Hiển thị lỗi
5a	Hệ thống	Điều kiện: Mật khẩu mới không hợp lệ Hành động: Hiển thị yêu cầu
Điều kiện sau	Mật khẩu được thay đổi	

Bảng 44: Đặc tả ca sử dụng UC009.6: Đổi mật khẩu

2.3 Các yêu cầu phi chức năng

2.3.1 Tính dễ dùng

- Giao diện người dùng được thiết kế thân thiện, dễ hiểu, sử dụng được cho cả người dùng ít kinh nghiệm với công nghệ.
- Các thao tác như tạo, chỉnh sửa, xóa dữ liệu cần rõ ràng và có xác nhận trước khi thực hiện.
- Khi người dùng nhập sai dữ liệu, hệ thống phải hiển thị thông báo lỗi cụ thể, dễ hiểu, gồm:
 - Mô tả lỗi (“Ngày hết hạn không hợp lệ”).
 - Gợi ý cách khắc phục (“Vui lòng nhập ngày hợp lệ theo định dạng DD/MM/YYYY”).
- Các biểu tượng và nút bấm có chú thích, gợi ý (tooltip, placeholder).
- Màu sắc, kích thước chữ, bố cục cần đảm bảo tính dễ đọc và nhất quán trên nhiều thiết bị (điện thoại, máy tính bảng).

2.3.2 Hiệu năng

- Ứng dụng cần phản hồi các thao tác người dùng trong vòng 2 giây cho các chức năng cơ bản (tạo, sửa, xem danh sách, tìm kiếm).
- Tốc độ tải dữ liệu từ server phải đảm bảo mượt mà, không gây giật lag trong quá trình sử dụng.
- Cơ chế cache cục bộ được sử dụng để truy cập nhanh dữ liệu gần nhất khi thiết bị mất kết nối Internet.
- Hệ thống backend (server và CSDL) cần hỗ trợ tối thiểu 1000 người dùng đồng thời trong giai đoạn triển khai thử nghiệm.

2.3.3 Tính tin cậy

- Hệ thống phải lưu trữ dữ liệu an toàn, không bị mất khi người dùng tắt ứng dụng đột ngột hoặc mất mạng.
- Dữ liệu người dùng được đồng bộ lên server ngay khi có kết nối mạng trở lại.
- Cơ chế sao lưu (backup) dữ liệu định kỳ mỗi ngày.
- Tự động phục hồi (auto-reconnect) khi mất kết nối ngắn hạn.

2.3.4 Tính dễ bảo trì

- Mã nguồn được chia thành các module rõ ràng: frontend (giao diện), backend (API, xử lý logic), database (lưu trữ).
- Có tài liệu hướng dẫn cài đặt, cấu hình, và hướng dẫn lập trình viên mở rộng chức năng.
- Hệ thống được phát triển theo mô hình RESTful API, dễ dàng thay đổi hoặc mở rộng sang các nền tảng khác (web, desktop).
- Sử dụng công nghệ phổ biến, mã nguồn mở để dễ dàng bảo trì, cập nhật phiên bản.

2.3.5 Tính khả chuyển

- Ứng dụng được phát triển bằng công nghệ đa nền tảng (ví dụ: Flutter hoặc React Native), có thể chạy trên cả Android và iOS.
- Cấu hình hệ thống linh hoạt, dễ dàng triển khai trên nhiều môi trường (local, cloud).
- Dữ liệu được lưu trữ trên cơ sở dữ liệu đám mây (ví dụ: Firebase, MongoDB Atlas hoặc MySQL Cloud).

2.3.6 Yêu cầu an toàn bảo mật

- Dữ liệu người dùng được mã hóa khi truyền giữa client và server (sử dụng HTTPS).
- Mật khẩu người dùng phải được lưu trữ dưới dạng mã hóa (hash + salt).
- Mỗi tài khoản người dùng chỉ được phép truy cập và chỉnh sửa dữ liệu cá nhân hoặc dữ liệu nhóm mà họ thuộc về.

- Cơ chế phân quyền rõ ràng: người dùng, trưởng nhóm gia đình, quản trị viên.
- Hạn chế truy cập trái phép qua API (sử dụng JWT token hoặc OAuth2).

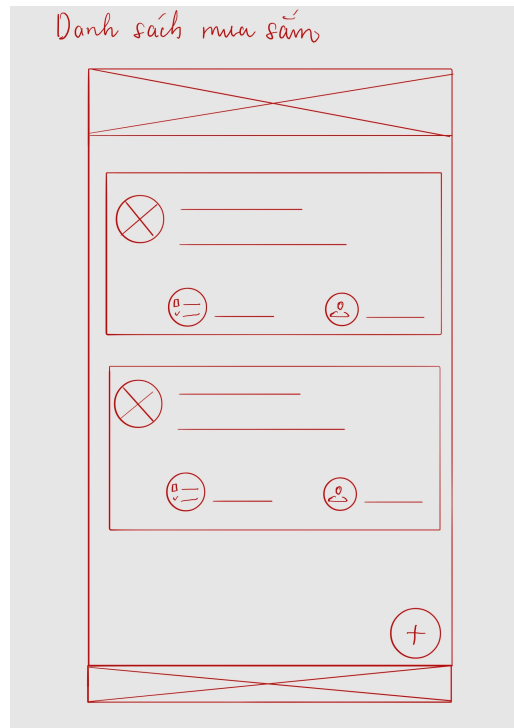
2.3.7 Yêu cầu về giao diện

- Giao diện người dùng (UI) được thiết kế theo phong cách tối giản, hiện đại, hỗ trợ chế độ Dark/Light mode.
- Có màn hình hướng dẫn sử dụng (onboarding) cho người dùng mới.
- Toàn bộ giao diện phải hiển thị tốt trên màn hình từ 5–10 inch (smartphone, tablet).
- Các biểu tượng, màu sắc và font chữ thống nhất với bộ nhận diện thương hiệu của ứng dụng.

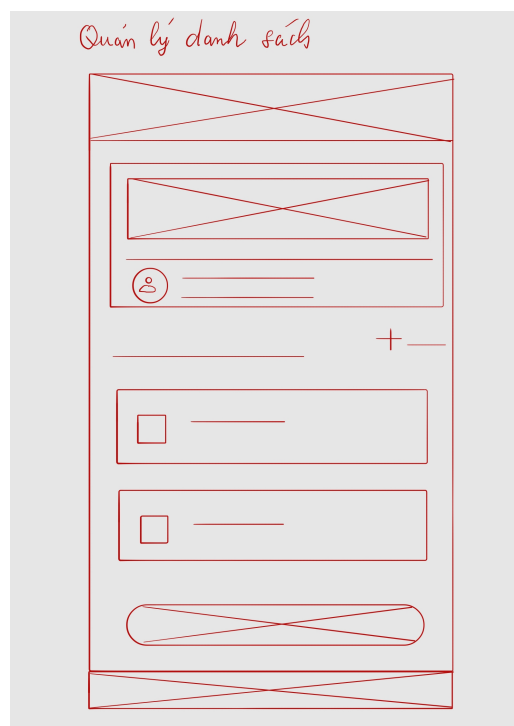
3 Thiết kế giao diện

3.1 Thiết kế giao diện low-fidelity / mid-fidelity prototype

3.1.1 UC1: Quản lý danh sách mua sắm

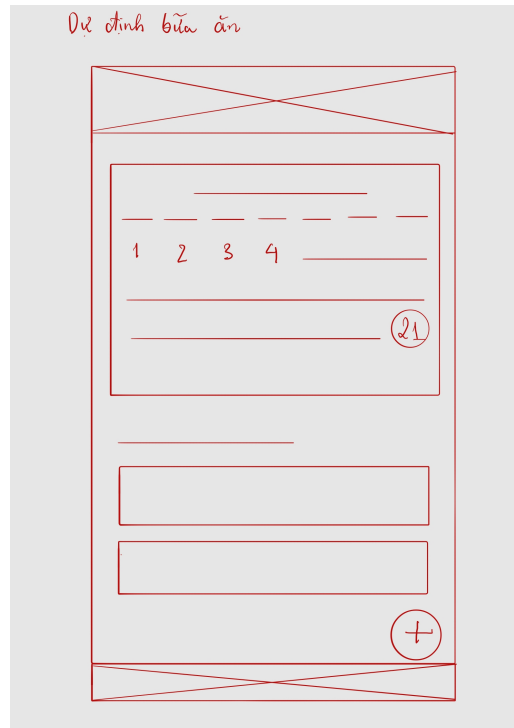


Hình 9: Thiết kế giao diện cho UC1: Xem các danh sách mua sắm



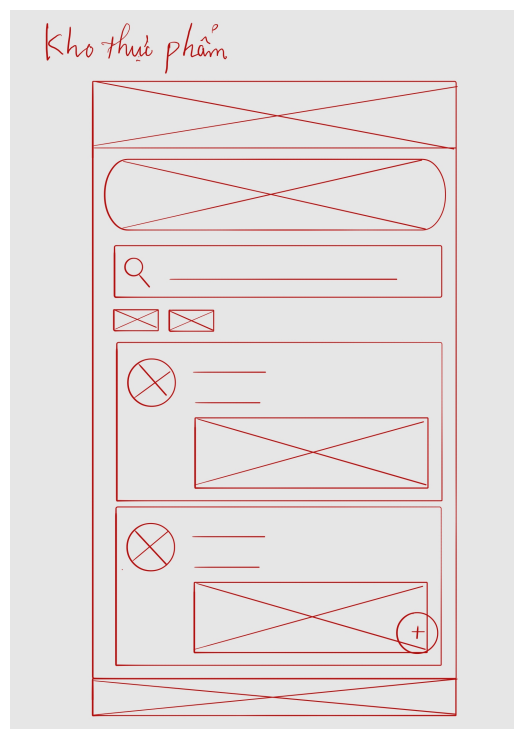
Hình 10: Thiết kế giao diện cho UC1: Quản lý danh sách mua sắm

3.1.2 UC2: Quản lý dự định bữa ăn



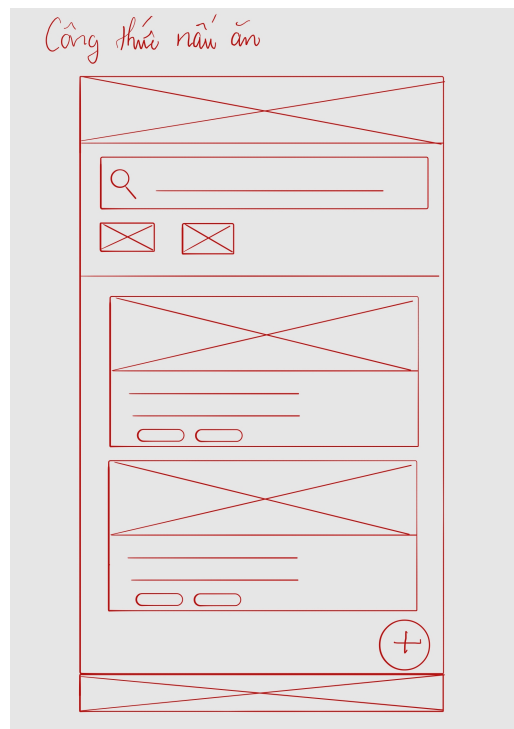
Hình 11: Thiết kế giao diện cho UC2: Quản lý dự định bữa ăn

3.1.3 UC3: Quản lý thực phẩm trong bếp



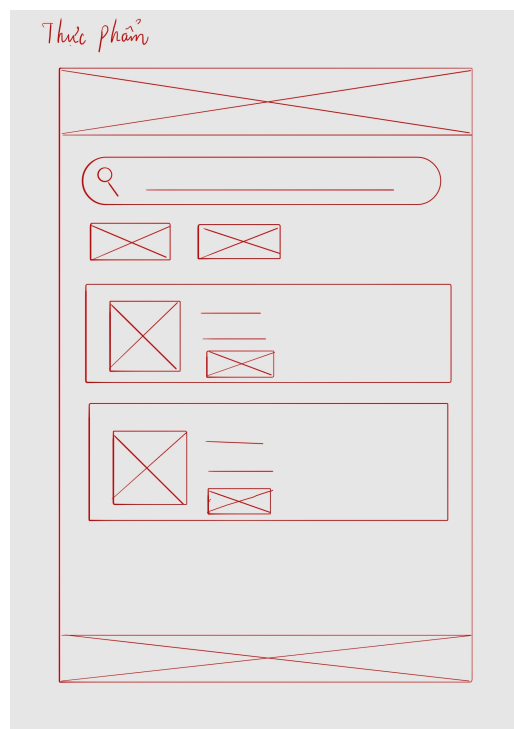
Hình 12: Thiết kế giao diện cho UC3: Quản lý thực phẩm trong bếp

3.1.4 UC4: Quản lý công thức nấu ăn

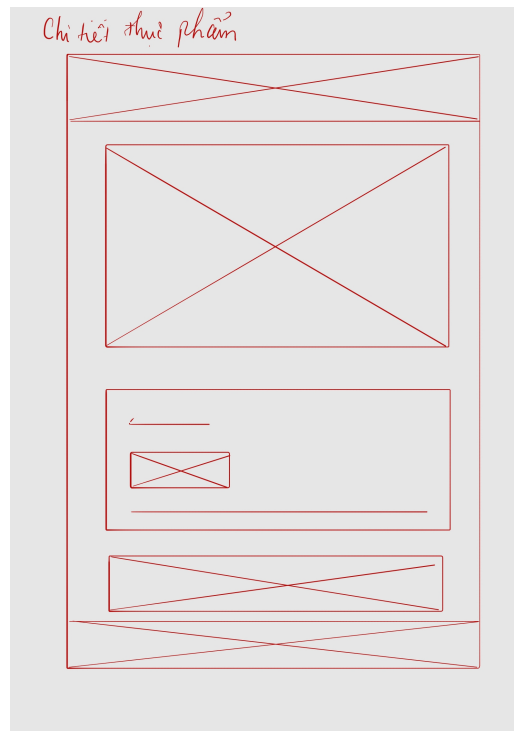


Hình 13: Thiết kế giao diện cho UC4: Quản lý công thức nấu ăn

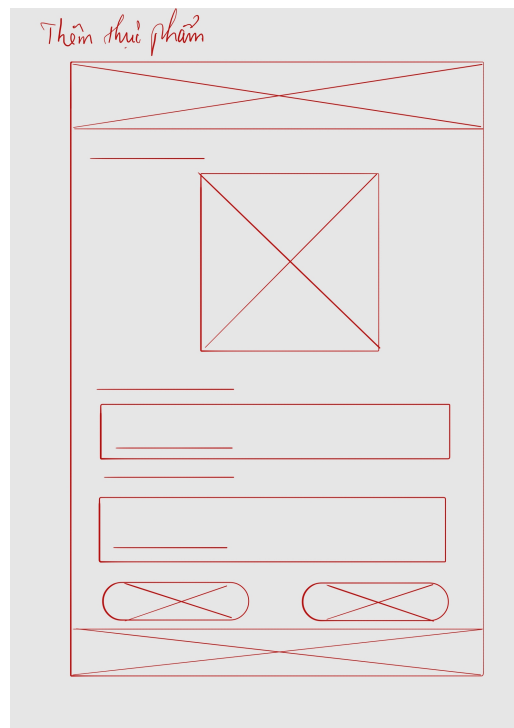
3.1.5 UC5: Quản lý danh sách thực phẩm



Hình 14: Thiết kế giao diện cho UC5: Quản lý danh sách thực phẩm

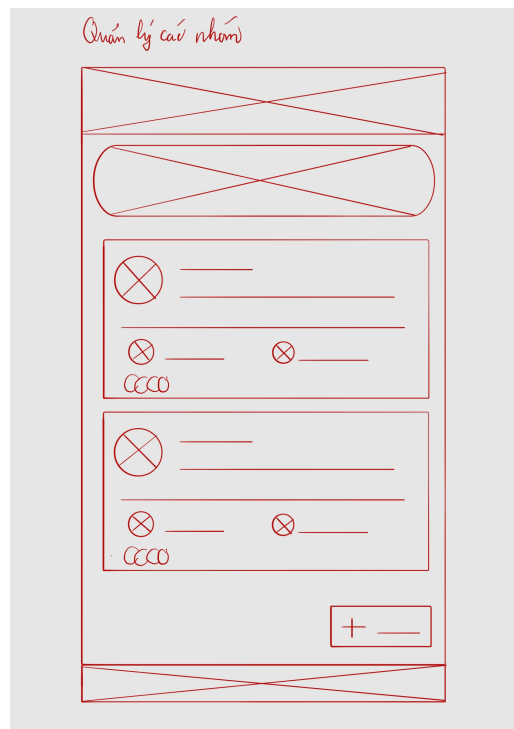


Hình 15: Thiết kế giao diện cho UC5: Chi tiết thực phẩm

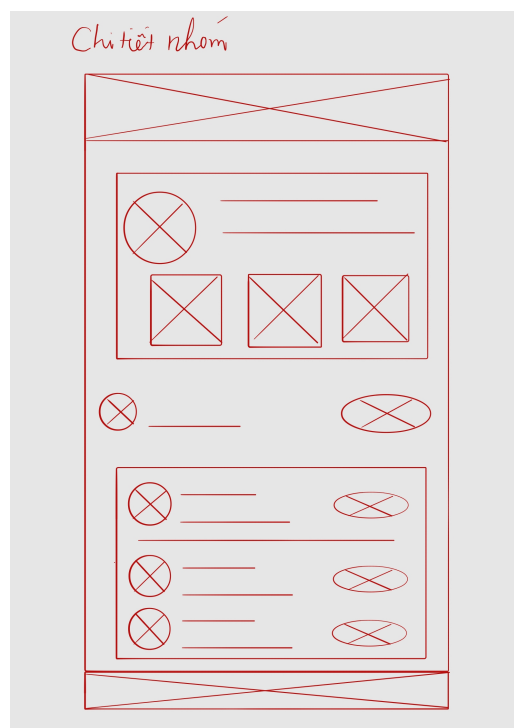


Hình 16: Thiết kế giao diện cho UC5: Thêm thực phẩm mới

3.1.6 UC6: Quản lý nhóm

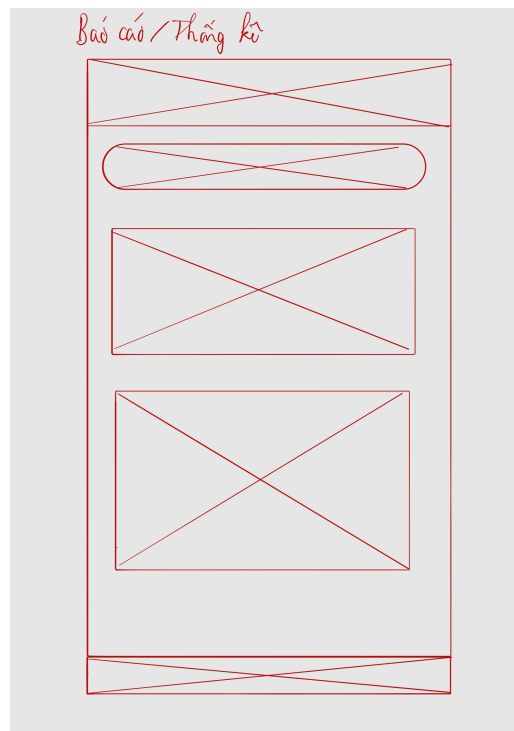


Hình 17: Thiết kế giao diện cho UC6: Quản lý nhóm



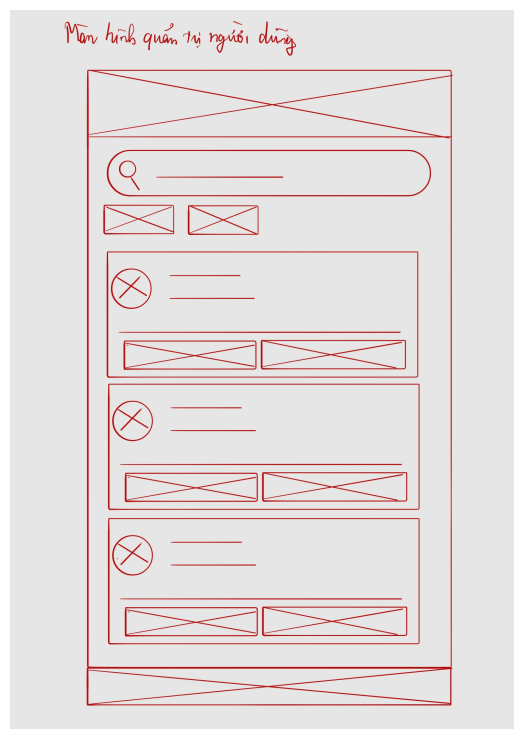
Hình 18: Thiết kế giao diện cho UC6: Quản lý nhóm - chi tiết nhóm

3.1.7 UC7: Thống kê và báo cáo



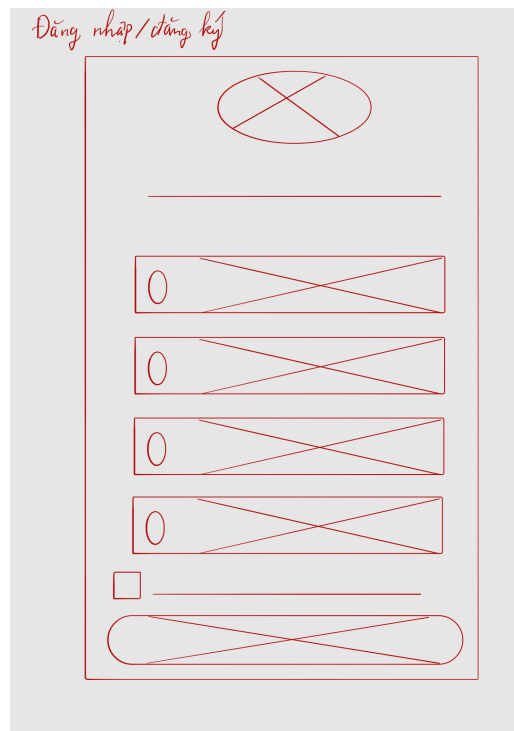
Hình 19: Thiết kế giao diện cho UC7: Thống kê và báo cáo

3.1.8 UC8: Quản trị người dùng

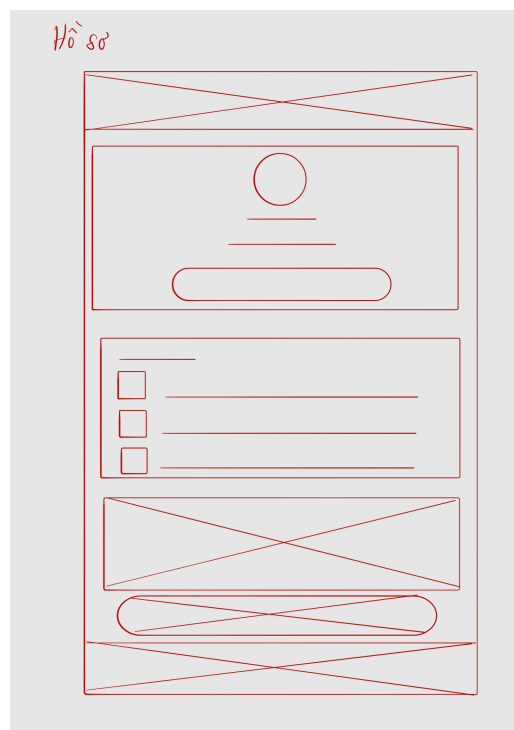


Hình 20: Thiết kế giao diện cho UC8: Quản trị người dùng

3.1.9 UC9: Xác thực



Hình 21: Thiết kế giao diện cho UC9: Đăng ký/đăng nhập



Hình 22: Thiết kế giao diện cho UC9: Quản lý hồ sơ cá nhân

3.2 Xây dựng các màn hình giao diện của ứng dụng (high-fidelity)

Trong giai đoạn này, nhóm đã tiến hành chuyển đổi từ các bản thiết kế low-fidelity sang giao diện hoàn chỉnh (high-fidelity) bằng framework React Native. Toàn bộ giao diện tuân thủ

quy chuẩn Material Design 3, tập trung vào trải nghiệm người dùng mượt mà và trực quan.

3.2.1 Hệ thống Theme và Styles tổng thể

Trước khi đi vào chi tiết các màn hình, hệ thống Styles được định nghĩa nhất quán để đảm bảo tính thẩm mỹ của thương hiệu:

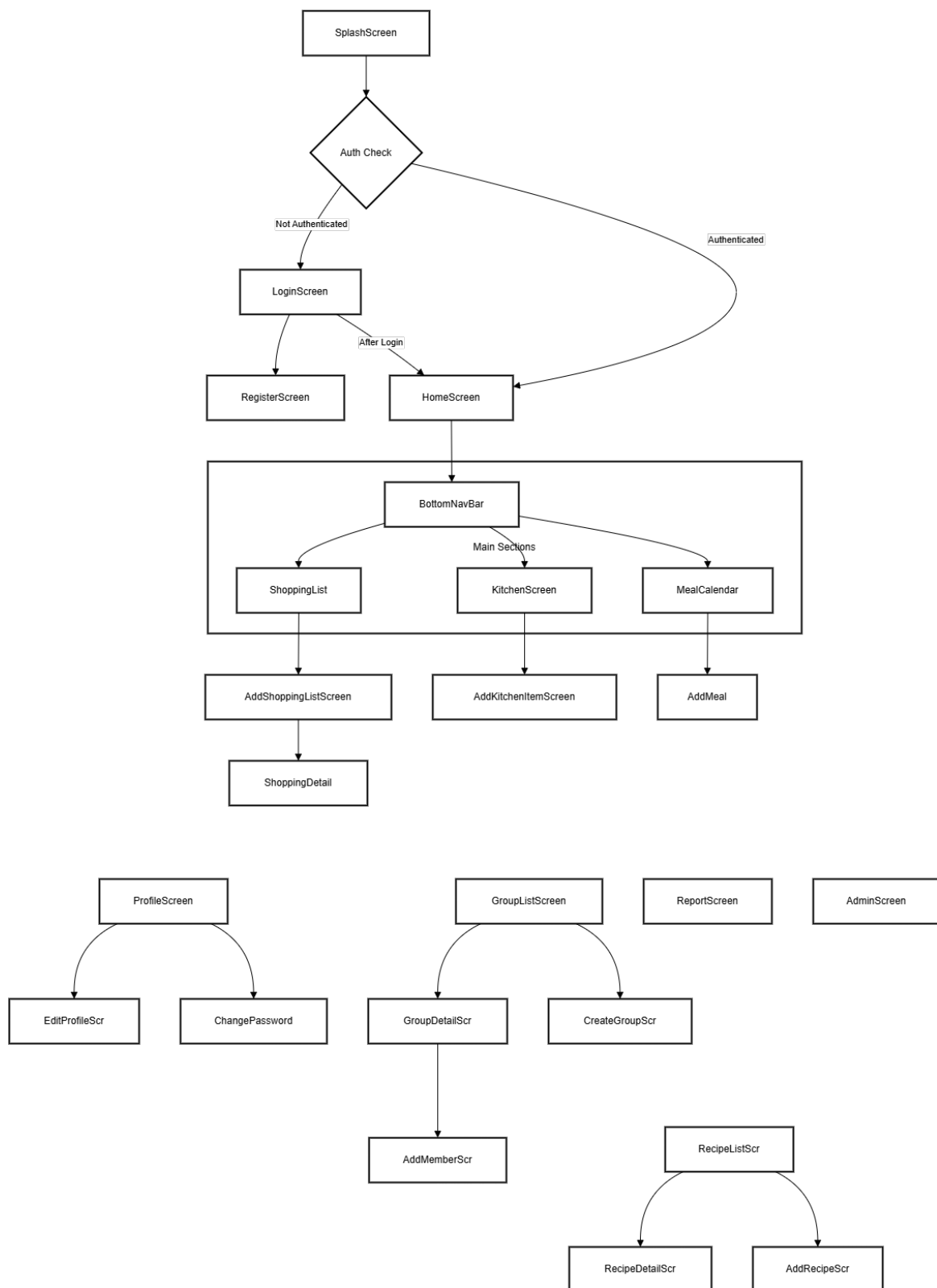
- **Màu sắc (Color System):** Sử dụng bảng màu Material 3 với màu chủ đạo là Đỏ táo (Primary: #C93C3C) tượng trưng cho thực phẩm tươi và sự ấm cúng, kết hợp với các màu bổ trợ như Tím nho (Secondary: #5C3D99) và Xanh lá (Accent: #4CAF50) cho các thông báo trạng thái tươi mới.
- **Typography:** Sử dụng bộ font **Nunito** với các biến thể từ Regular đến ExtraBold, phù hợp với ứng dụng dạng Lifestyle và Food.
- **Spacing & Shadow:** Sử dụng đơn vị cơ sở 4px và hệ thống Elevation của Material Design để tạo chiều sâu cho các thành phần Card và Surface.

3.2.2 Kỹ thuật điều hướng (Navigation)

Ứng dụng không sử dụng thư viện điều hướng thông thường mà triển khai giải pháp **Custom Navigation Hook** kết hợp với *History Stack*:

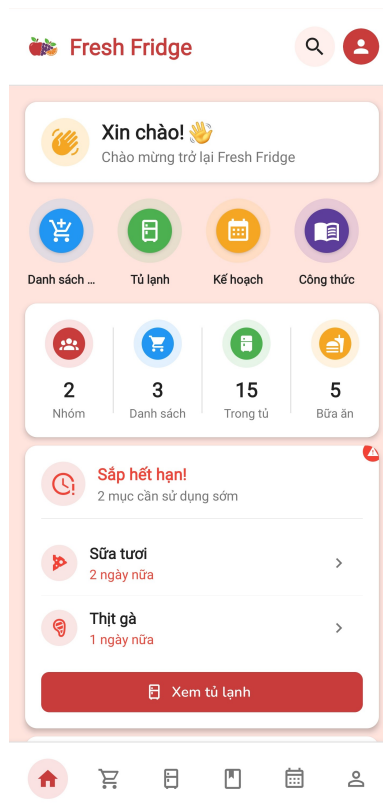
- **Cơ chế:** Quản lý mảng lịch sử (History Array) để thực hiện các thao tác `navigate`, `goBack` và `reset`.
- **Hiệu ứng:** Sử dụng *Animated API* để tạo hiệu ứng Fade và Scale (0.95x) khi chuyển màn hình với thời gian trễ 300ms.

3.2.3 Sơ đồ điều hướng



Hình 23: Sơ đồ điều hướng của ứng dụng

3.2.4 Chi tiết thiết kế các màn hình theo Use Case



Hình 24: Thiết kế giao diện trang chủ

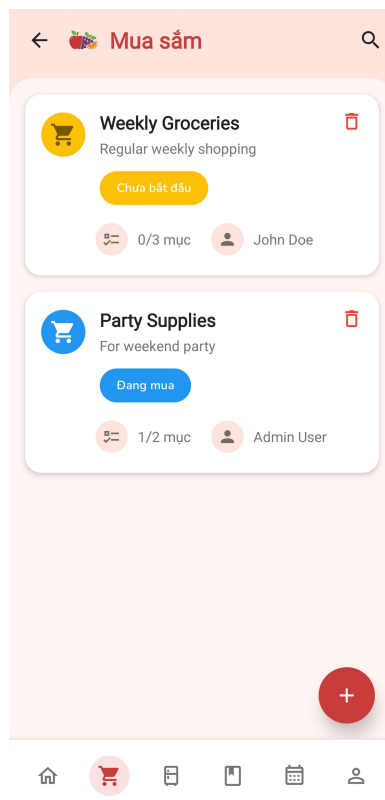
1. Nhóm UC-01: Quản lý danh sách mua sắm Màn hình này cho phép người dùng theo dõi và thực hiện các kế hoạch mua sắm của gia đình.

- **Cấu trúc cây Component:**

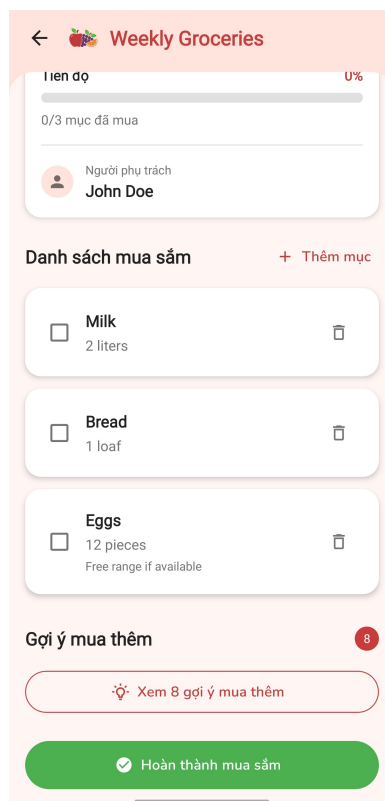
- SafeAreaView → Custom Header (Logo + Title)
- ScrollView → Danh sách các ShoppingCart
- Trong mỗi ShoppingCart: Avatar (Người đảm nhận) + ProgressBar (Tiến độ mua sắm) + Chip (Trạng thái)
- FAB (Floating Action Button): Nút tạo mới danh sách

- **Xử lý cử chỉ và Input:**

- **Gestures:** Sử dụng *Pull-to-refresh* để đồng bộ dữ liệu mua sắm từ server backend.
- **Interactions:** Khi người dùng tick vào vật phẩm, hệ thống kích hoạt hộp thoại nhập số lượng thực tế. Nếu số lượng mua ít hơn dự định, hệ thống tự động tách mục mới cho phần thiếu.



Hình 25: Thiết kế trang quản lý danh sách mua sắm - 1



Hình 26: Thiết kế giao diện chi tiết danh sách mua sắm - 2

Hình 27: Thiết kế giao diện chi tiết danh sách mua sắm - 3

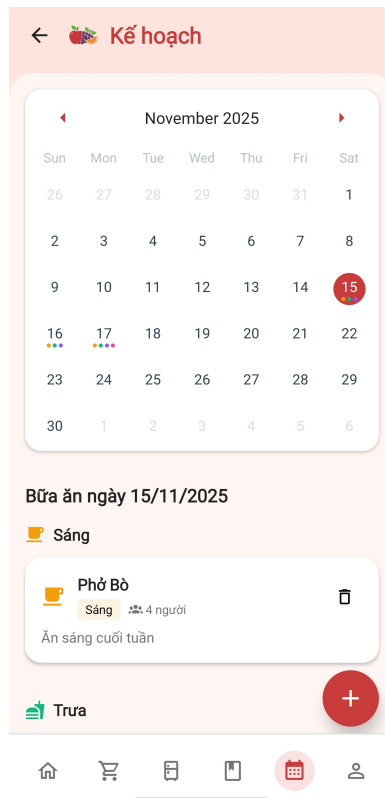
3. Nhóm UC-02: Quản lý dự định bữa ăn Giao diện lịch biểu giúp người dùng lên kế hoạch ăn uống khoa học.

- **Cấu trúc cây Component:**

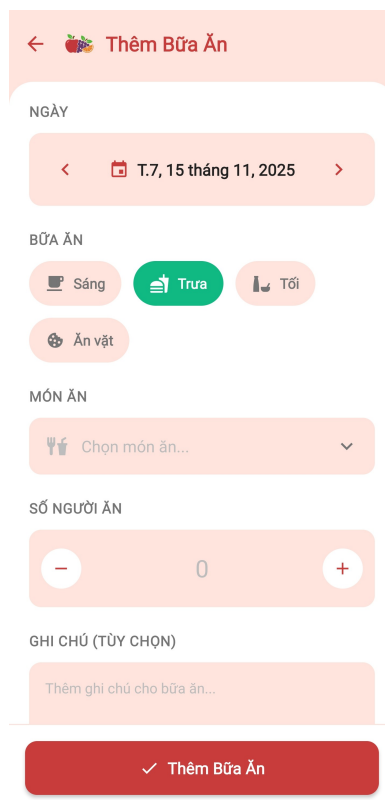
- **Calendar** (Thư viện *react-native-calendars*): Hiển thị lịch tháng với các dấu chấm (*Dots*) đánh dấu bữa ăn.
- **SelectedDateSection**: Hiển thị chi tiết bữa sáng, trưa, tối của ngày được chọn.
- **MealCard**: Chứa icon loại bữa ăn, tên món ăn và nút xóa.

- **Xử lý cử chỉ và Input:**

- **Gestures**: Vuốt ngang trên Calendar để chuyển tháng; Chạm vào một ngày để cập nhật danh sách bữa ăn bên dưới thông qua sự kiện `onDayPress`.
- **Interactions**: Nhấn vào FAB "+" để mở màn hình chọn công thức nấu ăn thêm vào thực đơn ngày đang chọn.



Hình 28: Thiết kế giao diện quản lý dự định bữa ăn - 1



Hình 29: Thiết kế giao diện quản lý dự định bữa ăn - 2

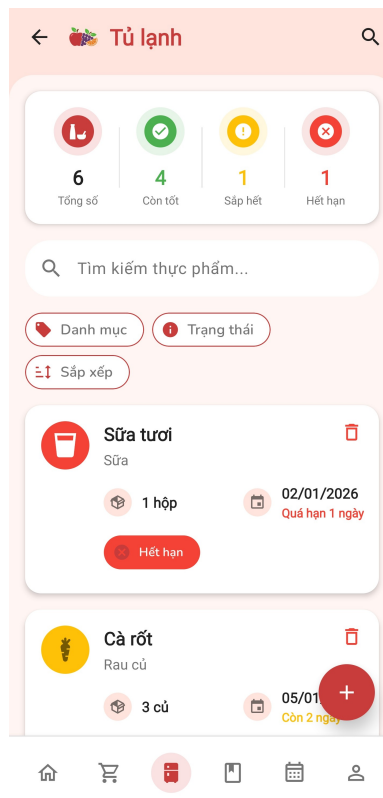
2. Nhóm UC-03: Quản lý thực phẩm trong bếp Giao diện quản lý kho lương thực hiện có với cảnh báo hạn sử dụng thông minh.

- **Cấu trúc cây Component:**

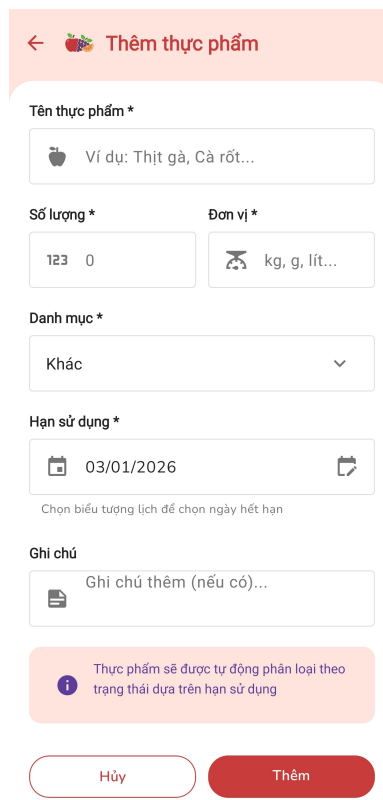
- **View.filterSection:** Gồm Searchbar và hàng Menu lọc (Danh mục, Trạng thái, Sắp xếp).
- **ScrollView → GridContainer:** Hiển thị các ItemCard theo dạng lưới 2 cột.
- Trong mỗi ItemCard: Hình ảnh thực phẩm + ExpiryRow (Icon lịch + Text ngày hết hạn).

- **Xử lý cử chỉ và Input:**

- **User Input:** Searchbar xử lý sự kiện `onChangeText` để lọc danh sách thực phẩm theo thời gian thực.
- **Gestures:** Chạm giữ (Long press) vào một thực phẩm để mở menu thao tác nhanh (Sửa/Xóa).
- **Visual Style:** Màu sắc của nhãn trạng thái (*Chip*) thay đổi linh hoạt: Xanh (> 3 ngày), Cam (Sắp hết hạn), Đỏ (Đã hết hạn).



Hình 30: Thiết kế giao diện quản lý thực phẩm trong bếp - 1



← Thêm thực phẩm

Tên thực phẩm *

Ví dụ: Thịt gà, Cà rốt...

Số lượng * Đơn vị *

123 0 kg, g, lít...

Danh mục *

Khác

Hạn sử dụng *

03/01/2026

Chọn biểu tượng lịch để chọn ngày hết hạn

Ghi chú

Ghi chú thêm (nếu có)...

Thực phẩm sẽ được tự động phân loại theo trạng thái dựa trên hạn sử dụng

Hủy Thêm

Hình 31: Thiết kế giao diện quản lý thực phẩm trong bếp - 2

4. Nhóm UC-04: Quản lý công thức nấu ăn Giao diện quản lý kho công thức nấu ăn.

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Nút Back + Logo + Tiêu đề “Công thức”.
- `View.searchContainer`: `Searchbar` (react-native-paper) cho phép tìm kiếm công thức theo tên.
- `View.filterContainer`: Hàng các Menu lọc gồm:
 - * Chip “Loại” (COMMON/PERSONAL)
 - * Chip “Độ khó” (Dễ/Trung bình/Khó)
 - * Chip “Sắp xếp” (Mới nhất/Đánh giá/Thời gian/Tên)
- `FlatList` → `GridContainer`: Hiển thị các `RecipeCard` theo dạng lưới 2 cột với `infinite scroll`.
- Trong mỗi `RecipeCard`:
 - * `Image`: Hình ảnh món ăn (hoặc placeholder icon `silverware-fork-knife`).
 - * `Text.recipeName`: Tên công thức.
 - * `StatsRow`: Icon + thời gian tổng | Icon sao + rating | Icon người + servings.
 - * `ChipsRow`: `DifficultyChip` + `PersonalChip` (nếu là công thức cá nhân).
 - * `IconButton.delete`: Nút xóa (chỉ hiện với PERSONAL).
- FAB: Floating Action Button màu primary để thêm công thức mới.
- `BottomNavBar`: Tab “recipe” đang active.

- **Xử lý cử chỉ và Input:**

- **User Input:** Searchbar xử lý `onChangeText` để lọc công thức theo thời gian thực.
- **Filter Menus:** Nhấn vào Chip mở Menu dropdown với các tùy chọn lọc.
- **Infinite Scroll:** `FlatList` gọi `fetchNextPage()` khi scroll đến cuối danh sách.
- **Navigation:** Nhấn vào `RecipeCard` gọi `onRecipePress(recipeId)` để xem chi tiết.
- **Delete Action:** Nhấn nút xóa hiển thị `Alert.alert` xác nhận trước khi gọi `deleteRecipeMutat`
- **Pull-to-refresh:** `RefreshControl` cho phép kéo xuống để reload danh sách.

- **Visual Style:**

- **Màu sắc độ khó (`DifficultyChip`):**
 - * Xanh lá (`#66BB6A`) - Dễ (EASY)
 - * Cam (`#FFA726`) - Trung bình (MEDIUM)
 - * Đỏ (`#EF5350`) - Khó (HARD)
- **Personal Badge:** Chip viền outline với text "Cá nhân" cho công thức riêng của nhóm.
- **Rating Display:** Icon sao màu vàng + điểm đánh giá (0-5) + số lượt đánh giá.
- **Time Format:** Hiển thị tổng thời gian (`prepTime` + `cookTime`) theo định dạng "X giờ Y phút" hoặc "X phút".

- **State Management:**

- `searchQuery`: Chuỗi tìm kiếm công thức.
- `selectedType`: Lọc theo loại (`COMMON`/`PERSONAL`/`undefined`).
- `selectedDifficulty`: Lọc theo độ khó (`EASY`/`MEDIUM`/`HARD`/`undefined`).
- `sortBy`: Tiêu chí sắp xếp (`'createdAt'`, `'rating'`, `'prepTime'`, `'name'`).
- `useInfiniteRecipes`: React Query hook với infinite scroll pagination.

Màn hình Chi tiết công thức

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Nút Back + “Chi tiết công thức” + nút Edit/Delete (chỉ `PERSONAL`).
- `ScrollView`:
 - * `Image`: Hình ảnh lớn của món ăn.
 - * `Card.InfoCard`: Tên + Mô tả + Tags + `StatsRow` (thời gian, servings, rating, difficulty).
 - * `Card.TimeDetails`: Hiển thị chi tiết thời gian chuẩn bị và thời gian nấu riêng biệt.
 - * `Card.MainIngredients`: Danh sách nguyên liệu chính với icon `food-variant`.
 - * `Card.SecondaryIngredients`: Danh sách nguyên liệu phụ với icon `food-apple`.
 - * `Card.Instructions`: Các bước làm được đánh số với icon `chef-hat`.

- **Data Display:**

- **Ingredients:** Phân loại thành `MAIN` và `SECONDARY`, hiển thị với bullet point màu sắc khác nhau.
- **Instructions:** Danh sách các bước được đánh số thứ tự.
- **Tags:** Hiển thị dạng `Chip compact` trong `tagsContainer`.

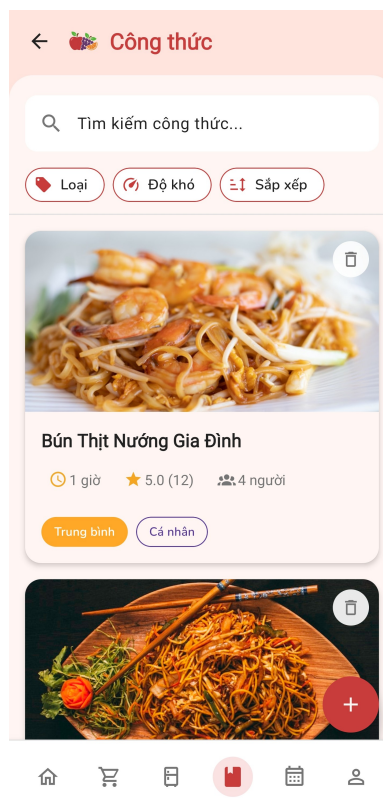
Màn hình Thêm/Sửa công thức (AddRecipeScreen)

- **Cấu trúc cây Component:**

- SafeAreaView → KeyboardAvoidingView: Đảm bảo form không bị che khi mở bàn phím.
- ScrollView chứa các section form:
 - * TextInput.name: Tên công thức (required).
 - * TextInput.description: Mô tả (multiline).
 - * TextInput.imageUrl: URL hình ảnh.
 - * Menu.type: Chọn loại (COMMON/PERSONAL).
 - * Menu.difficulty: Chọn độ khó (Dễ/Trung bình/Khó).
 - * Row: TextInput.prepTime + TextInput.cookTime + TextInput.servings.
 - * Section.mainIngredients: Danh sách nguyên liệu chính với nút thêm/xóa.
 - * Section.secondaryIngredients: Danh sách nguyên liệu phụ với nút thêm/xóa.
 - * Section.instructions: Các bước làm với nút thêm/xóa bước.
 - * Section.tags: Input thêm tag + danh sách tags hiện có.
 - * Button.save: Nút lưu công thức.

- **Xử lý Form:**

- **Dynamic Lists:** addIngredient(), removeIngredient(), addInstruction(), removeInstruction()
- **Validation:** Kiểm tra tất cả required fields trước khi submit.
- **Edit Mode:** Nếu có recipeId, load dữ liệu từ useRecipe() và populate form.
- **Submit:** Gọi createMutation hoặc updateMutation tùy theo mode.



Hình 32: Thiết kế giao diện quản lý công thức nấu ăn - 1

Thêm công thức

Thông tin cơ bản

Tên công thức *

Ví dụ: Phở bò, Bún chả...

Mô tả *

Mô tả ngắn về công thức...

URL hình ảnh *

https://example.com/image.jpg

Loại công thức

Cá nhân

▼

Độ khó

Dễ

▼

Chuẩn bị (phút) *

0

Nấu (phút) *

0

Số người ăn *

0

Hình 33: Thiết kế giao diện quản lý công thức nấu ăn - 2

02:19

 96

Chi tiết công thức

Bún Thịt Nướng Gia Đình

Cá nhân

Công thức bún thịt nướng đặc biệt của gia đình tôi

Bún

Nướng

Gia đình

🕒 1 giờ

👤 4 người

★ 5.0 (12)

Trung bình

Chuẩn bị

Nấu

30 phút

30 phút

📋 Nguyên liệu chính

- Thịt vai: 500 g
- Bún: 400 g

Hình 34: Thiết kế giao diện quản lý công thức nấu ăn - 3

66

Fresh Fridge

5. Nhóm UC-05: Quản lý danh sách thực phẩm Giao diện quản lý thực phẩm giúp người dùng tạo, xem, chỉnh sửa và xóa thực phẩm trong hệ thống.

Màn hình Danh sách thực phẩm (FoodListScreen)

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Nút Back + Logo + Tiêu đề “Thực phẩm” + Nút Plus (thêm).
- `Surface.searchContainer`: Thanh tìm kiếm:
 - * `Avatar.Icon` “magnify”: Icon tìm kiếm.
 - * `TextInput`: Nhập từ khóa tìm kiếm.
 - * `Avatar.Icon` “close-circle”: Nút xóa từ khóa (hiển thị khi có nội dung).
- `ScrollView` horizontal → Chip filters: Bộ lọc danh mục ngang:
 - * Chip “Tất cả”: Hiển thị tất cả thực phẩm.
 - * `Chip[categoryId]`: Các danh mục từ API (Rau củ, Thịt, Hải sản, v.v.).
- `ScrollView` vertical → `FoodList`: Danh sách các `FoodCard`.
- Trong mỗi `FoodCard`:
 - * `Image/Avatar.Icon`: Hình ảnh thực phẩm hoặc placeholder icon “food-apple”.
 - * `Text.foodName`: Tên thực phẩm.
 - * `Text.foodCategory`: Tên danh mục (với icon “tag”).
 - * `Chip.systemBadge`: Badge “Hệ thống” nếu `isSystemFood = true`.
 - * `IconButton.chevron-right`: Mũi tên điều hướng chi tiết.

- **Xử lý cử chỉ và Input:**

- **Search:** Nhập văn bản vào `TextInput` gọi `setSearchQuery()` để lọc theo tên.
- **Clear Search:** Nhấn icon “close-circle” để xóa trắng từ khóa tìm kiếm.
- **Category Filter:** Nhấn vào Chip gọi `setSelectedCategoryId()` để lọc theo danh mục.
- **Navigation:** Nhấn vào `FoodCard` gọi `onFoodPress(foodId)` để xem chi tiết.
- **Create Action:** Nhấn nút Plus trong header gọi `onAddFood()` để tạo thực phẩm mới.

- **Visual Style:**

- **Màu sắc Chip:**
 - * Trắng (`colors.white`) - Trạng thái chưa chọn, viền `colors.border`.
 - * Primary (`colors.primary`) - Trạng thái đã chọn, text trắng, elevation 2.
- **System Badge:** Nền xanh (`colors.info`), text trắng, icon “shield-check”.
- **Empty State:** Hiển thị icon “food-apple-outline” kích thước lớn, text “Chưa có thực phẩm” và nút hành động.
- **Placeholder Image:** Nền `surfaceVariant`, icon “food-apple” kích thước 40px.

- **State Management:**

- `searchQuery`: Từ khóa tìm kiếm từ `useState()`.

- `selectedCategoryId`: ID danh mục đang lọc từ `useState()`.
- `foods`: Danh sách thực phẩm từ `useFoods()` hook với tham số `{name, categoryId}`.
- `categories`: Danh sách danh mục từ `useCategories()` hook.

Màn hình Chi tiết thực phẩm (FoodDetailScreen)

• Cấu trúc cây Component:

- `SafeAreaView` → `Header`: Nút Back + Logo + “Chi tiết thực phẩm” + Nút Edit (chỉ hiện nếu không phải thực phẩm hệ thống).
- `ScrollView`:
 - * `Surface.imageSection`: Khu vực hình ảnh:
 - `Image`: Ảnh thực phẩm (chiều cao 250px).
 - `Avatar.Icon`: Placeholder icon “food-apple” nếu thiếu ảnh.
 - * `Surface.infoSection`: Thông tin thực phẩm:
 - `Text.foodName`: Tên thực phẩm (24px, bold).
 - `View.metaRow`: `Avatar.Icon` “tag” + Text danh mục.
 - `Chip.systemBadge`: Badge hiển thị nếu là thực phẩm hệ thống.
 - `View.dateInfo`: Ngày tạo định dạng vi-VN.
 - * `View.actions`: Nút “Xóa thực phẩm” (chỉ hiện nếu không phải thực phẩm hệ thống).
- `Portal.Dialog`: Dialog xác nhận trước khi xóa thực phẩm.

• Xử lý cử chỉ và Input:

- **Edit Action**: Nhấn nút “pencil” trong header gọi `onEdit(foodId)`.
- **Delete Action**: Nhấn nút “Xóa thực phẩm” → Dialog xác nhận → `deleteMutation.mutateAs`
- **Navigation**: Nút Back gọi `onBack()` để quay lại danh sách.

• Visual Style:

- **Image Section**: Nền trắng, bo góc 16px, elevation 2.
- **Info Section**: Padding 20px, bo góc 16px.
- **Delete Button**: Nền trắng, viền đỏ (`colors.error`), text đỏ, icon “delete”.
- **System Badge**: Nền xanh (`colors.info`), height 32px, icon “shield-check”.

• State Management:

- `food`: Thông tin chi tiết thực phẩm từ `useFood(foodId)` hook.
- `deleteDialogVisible`: Trạng thái ẩn/hiển dialog xác nhận xóa.
- `deleteMutation`: Mutation hook từ `useDeleteFood()`.

Màn hình Thêm/Sửa thực phẩm (AddFoodScreen)

- **Cấu trúc cây Component:**

- SafeAreaView → Header: Nút Back + Logo + Tiêu đề động (Thêm/Sửa thực phẩm).
- ScrollView:
 - * View.section (Hình ảnh):
 - View.imageContainer: Image.preview hoặc TouchableOpacity.placeholder.
 - TouchableOpacity.removeImageButton: Icon “close-circle” trên nền đỏ.
 - * View.section (Thông tin):
 - TextInput: Nhập tên thực phẩm (outlined mode, maxLength 100).
 - Menu → TouchableOpacity.selector: Chọn danh mục.
 - * View.buttonContainer: Nút “Hủy” (outlined) + Nút “Lưu” (contained).

- **Xử lý Form:**

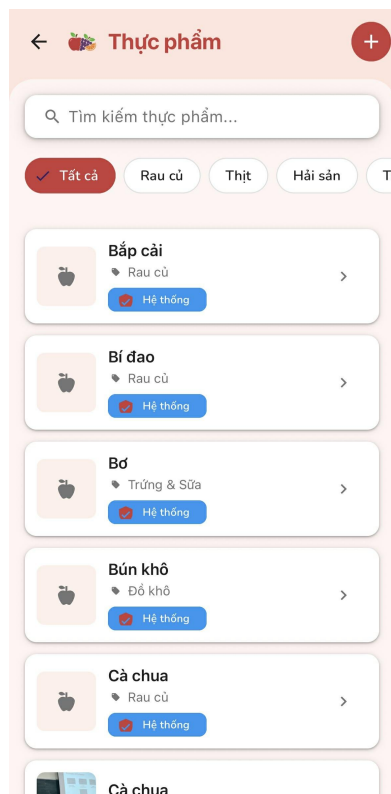
- **Image Picker:** Gọi `ImagePicker.launchImageLibraryAsync()` với tỉ lệ 1:1.
- **Validation:** Kiểm tra bắt buộc nhập tên và chọn danh mục trước khi submit.
- **Submit Action:**
 - * Create: Gọi `createMutation.mutateAsync()`.
 - * Edit: Gọi `updateMutation.mutateAsync()`.

- **Visual Style:**

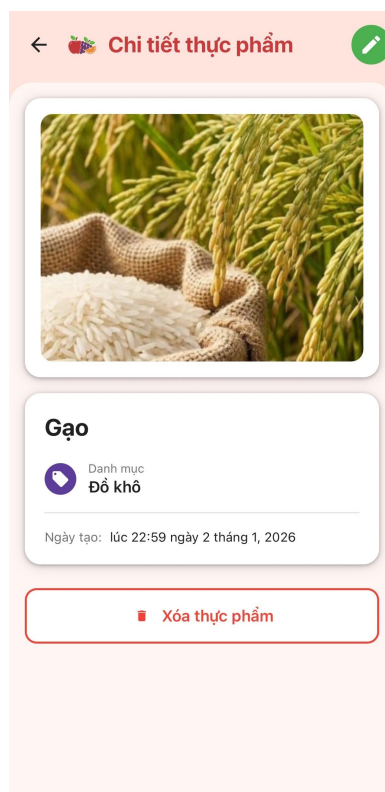
- **Image Preview:** Bo góc 12px, kích thước 200x200px.
- **Image Placeholder:** Viền đứt nét (*dashed*) 2px, nền `surfaceVariant`.
- **Input Style:** Active outline màu `primary`, hiển thị counter độ dài ký tự.
- **Selector:** Nền trắng, viền `colors.border`, biểu tượng “chevron-down” bên phải.

- **State Management:**

- `isEdit`: Boolean xác định chế độ Thêm mới hay Chỉnh sửa.
- `imageUri` & `imageFile`: Quản lý dữ liệu hình ảnh được chọn.
- `createMutation/updateMutation`: Các mutation hook xử lý gửi dữ liệu lên Server.



Hình 35: Thiết kế giao diện quản lý thực phẩm - 1



Hình 36: Thiết kế giao diện quản lý thực phẩm - 2

Hình 37: Thiết kế giao diện quản lý thực phẩm - 3

5. Nhóm UC-06: Quản lý nhóm Giao diện quản lý nhóm gia đình/bạn bè để cùng chia sẻ tủ lạnh và thực đơn.

Màn hình Danh sách nhóm (GroupListScreen)

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Logo + Tiêu đề “Nhóm của tôi” + Nút Search.
- `Surface.summaryCard`: Thống kê tổng quan:
 - * `Avatar.Icon` + Tổng số nhóm
 - * `Divider` dọc
 - * `Avatar.Icon` + Tổng số thành viên (tổng hợp từ tất cả nhóm)
- `View.content` → `FlatList`: Danh sách các `GroupCard`.
- Trong mỗi `GroupCard`:
 - * `Avatar.Icon` “account-group”: Icon nhóm màu primary.
 - * `Text.groupName`: Tên nhóm.
 - * `Text.groupDescription`: Mô tả (tối đa 2 dòng).
 - * `IconButton.chevron-right`: Mũi tên điều hướng.
 - * `StatsRow`: Icon + số thành viên | Icon + ngày tạo.
 - * `MembersPreview`: Dãy avatar thành viên (tối đa 4 + badge “+N”).
- FAB: Floating Action Button “Tạo nhóm” màu primary.
- `BottomNavBar`: Tab “home” đang active.

- **Xử lý cử chỉ và Input:**

- **Navigation:** Nhấn vào GroupCard gọi `selectGroup()` và `onGroupPress()` để xem chi tiết nhóm.
- **Create Action:** Nhấn FAB gọi `onCreatePress()` để chuyển đến màn hình tạo nhóm.
- **Pull-to-refresh:** `RefreshControl` cho phép kéo xuống gọi `refetch()` để reload danh sách.
- **Visual Style:**
 - **Màu sắc theo số thành viên:**
 - * Xanh lá (`colors.success`) - Từ 5 thành viên trở lên
 - * Xanh dương (`colors.info`) - Từ 3-4 thành viên
 - * Tím (`colors.tertiary`) - Dưới 3 thành viên
 - **Empty State:** Hiển thị icon lớn, text “Chưa có nhóm nào” và nút tạo nhóm khi danh sách rỗng.
 - **Date Format:** Ngày tạo hiển thị theo định dạng “dd Tháng” (vi-VN).
- **State Management:**
 - `groups`: Danh sách nhóm từ `useGroups()` hook.
 - `refreshing`: Trạng thái đang refresh.
 - `selectGroup`: Action từ `useGroupState()` để chọn nhóm hiện tại.

Màn hình Chi tiết nhóm (GroupDetailScreen)

- **Cấu trúc cây Component:**
 - `SafeAreaView` → Header: Nút Back + Logo + “Chi tiết nhóm” + Nút Settings.
 - `ScrollView`:
 - * `Surface.infoCard`: Thông tin nhóm:
 - `Avatar.Icon`: Icon nhóm lớn (70px).
 - `Text.groupName`: Tên nhóm.
 - `Text.description`: Mô tả nhóm.
 - `StatsContainer`: 3 thẻ thống kê (Số thành viên | Ngày tạo | Vai trò).
 - * `Section.members`: Danh sách thành viên:
 - `SectionHeader`: Tiêu đề + Nút “Thêm” (nếu có quyền).
 - `Card.membersCard`: Danh sách `MemberItem`.
 - * `MemberItem` bao gồm:
 - `Avatar.Image/Icon`: Avatar thành viên.
 - `Text.memberName`: Tên thành viên.
 - `Text.memberEmail`: Email thành viên.
 - `RoleBadge`: Badge hiển thị vai trò.
 - `Menu`: Menu tùy chọn (đổi vai trò, xóa - chỉ hiện với quyền quản lý).
 - * `Actions`: Nút “Xóa nhóm” (Owner) hoặc “Rời khỏi nhóm” (Member).
 - `Portal.Dialog`: Dialog xác nhận xóa/rời nhóm.
- **Xử lý cử chỉ và Input:**

- **Add Member:** Nhấn nút “Thêm” gọi `onAddMember()` (yêu cầu quyền OWNER/ADMIN).
- **Member Menu:** Nhấn icon 3 chấm mở Menu với các tùy chọn:
 - * “Thay đổi vai trò”
 - * “Chuyển quyền chủ nhóm” (chỉ OWNER)
 - * “Xóa khỏi nhóm”
- **Delete Group:** Nhấn nút “Xóa nhóm” → Dialog xác nhận → `deleteGroupMutation`.
- **Leave Group:** Nhấn nút “Rời khỏi nhóm” → Dialog xác nhận → `leaveGroupMutation`.

- **Visual Style:**

- **Màu sắc vai trò (RoleBadge):**
 - * Primary (`colors.primary`) - Chủ nhóm (OWNER)
 - * Tím (`colors.tertiary`) - Quản trị (ADMIN)
 - * Cam (`colors.accent`) - Thành viên (MEMBER)
- **Icon vai trò:**
 - * crown - Chủ nhóm
 - * shield-account - Quản trị viên
 - * account - Thành viên
- **Nút hành động:**
 - * “Xóa nhóm”: Nền đỏ (`colors.error`), text trắng.
 - * “Rời khỏi nhóm”: Nền trắng, viền đỏ, text đỏ.

- **State Management:**

- `selectedGroup`: Nhóm đang xem từ `useGroupState()`.
- `menuVisible`: ID thành viên đang mở menu (null nếu đóng).
- `deleteDialogVisible`: Hiển thị dialog xóa nhóm.
- `leaveDialogVisible`: Hiển thị dialog rời nhóm.
- `currentMember`: Thông tin thành viên hiện tại để kiểm tra quyền.

Màn hình Tạo nhóm (CreateGroupScreen)

- **Cấu trúc cây Component:**

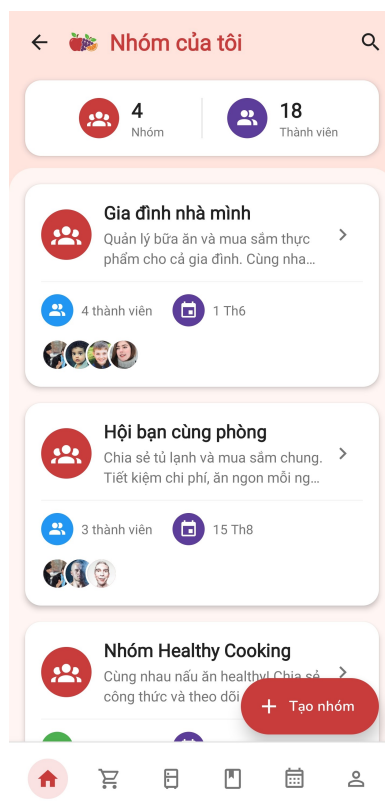
- `SafeAreaView` → Header: Nút Back + Logo + “Tạo nhóm mới”.
- `KeyboardAvoidingView` → `ScrollView`:
 - * `Surface.heroCard`: Phần giới thiệu:
 - `Avatar.Icon`: Icon nhóm lớn (80px).
 - `Text.heroTitle`: “Tạo nhóm của bạn”.
 - `Text.heroSubtitle`: Mô tả ngắn về tính năng.
 - * `Card.formCard`: Form nhập liệu:
 - `FormInput.name`: Tên nhóm (required).
 - `FormInput.description`: Mô tả (multiline, optional).
 - * `View.buttonContainer`: Hàng nút hành động:
 - `TouchableOpacity.cancelButton`: Nút “Hủy”.

- `TouchableOpacity.createButton`: Nút “Tạo nhóm” với icon plus.
- * `Card.tipsCard`: Card mẹo hữu ích với bullet list.
- **Xử lý Form:**
 - **Validation:** Sử dụng `react-hook-form` với `zodResolver` và schema `createGroupSchema`.
 - **Submit:** Gọi `createGroupMutation.mutateAsync()` với dữ liệu `{name, description}`.
 - **Error Handling:** Hiển thị `errorContainer` khi mutation thất bại.
 - **Loading State:** Disable form và đổi text nút thành “Đang tạo...” khi đang submit.
- **Visual Style:**
 - **Input Label:** Icon avatar nhỏ + text label, icon màu primary cho tên, secondary cho mô tả.
 - **Tips Card:** Nền `tertiaryContainer`, icon lightbulb, danh sách mẹo với bullet point.
 - **Button Style:**
 - * “Hủy”: Nền `surfaceVariant`, text `textSecondary`.
 - * “Tạo nhóm”: Nền `primary`, text trắng, có icon plus.
- **State Management:**
 - `control`: Controller từ `useForm()`.
 - `isSubmitting`: Trạng thái đang submit form.
 - `createGroupMutation`: Mutation hook từ `useCreateGroup()`.

Màn hình Thêm thành viên (AddMemberScreen)

- **Cấu trúc cây Component:**
 - `SafeAreaView` → Header: Nút Back + Logo + “Thêm thành viên”.
 - `KeyboardAvoidingView` → `ScrollView`:
 - * `Surface.groupInfoCard`: Thông tin nhóm đang thêm vào:
 - `Avatar.Icon`: Icon nhóm.
 - `Text.groupInfoLabel`: “Thêm vào nhóm”.
 - `Text.groupInfoName`: Tên nhóm.
 - * `Card.formCard`: Form nhập liệu:
 - `FormInput.email`: Email thành viên (required).
 - `Section.roleSelection`: Chọn vai trò với `RadioButton.Group` gồm `TouchableOpacity` cho Thành viên (icon account) và Quản trị viên (icon shield-account).
 - * `View.buttonContainer`: Nút “Hủy” + “Thêm thành viên”.
 - * `Card.infoCard`: Card giải thích quyền hạn vai trò.
- **Xử lý Form:**
 - **Validation:** Schema `addMemberSchema` kiểm tra email hợp lệ.
 - **Role Selection:** `RadioButton.Group` với state `selectedRole`.

- **Submit:** Gọi `addMemberMutation.mutateAsync({groupId, data: {email, role}})`.
- **Success:** Hiển thị `Alert.alert` và gọi `onSuccess()`.
- **Visual Style:**
 - **Role Option:**
 - * Nền `surfaceVariant`, viền trong suốt khi chưa chọn.
 - * Nền `primaryContainer`, viền `primary` khi được chọn.
 - **Màu sắc vai trò:**
 - * Tím (`colors.tertiary`) - Quản trị viên
 - * Cam (`colors.accent`) - Thành viên
 - **Info Card:** Nền `infoLight`, giải thích 3 loại vai trò với bullet màu tương ứng.
- **State Management:**
 - `selectedGroup`: Nhóm đang thêm thành viên từ `useGroupState()`.
 - `selectedRole`: Vai trò đã chọn (mặc định `MemberRole.MEMBER`).
 - `control`: Controller từ `useForm()`.
 - `addMemberMutation`: Mutation hook từ `useAddMember()`.



Hình 38: Thiết kế giao diện quản lý nhóm - 1

← 🍎 Tạo nhóm mới

Tạo nhóm của bạn
Quản lý tủ lạnh cùng gia đình, bạn bè hoặc đồng nghiệp

Tên nhóm *
Nhập tên nhóm (vd: Gia đình tôi)

Mô tả (không bắt buộc)
Mô tả ngắn về nhóm...

Hủy + Tạo nhóm

Mẹo hữu ích

- Đặt tên nhóm dễ nhớ và mô tả rõ ràng
- Mời thành viên sau khi tạo nhóm xong

Hình 39: Thiết kế giao diện quản lý nhóm - 2

← 🍎 Chi tiết nhóm ⚙️

Gia đình nhà mình
Quản lý bữa ăn và mua sắm thực phẩm cho cả gia đình. Cùng nhau lên kế hoạch bữa ăn hàng tuần!

4 THÀNH VIÊN 1 Th6 NGÀY TẠO Chủ nhóm VAI TRÒ

Thành viên + Thêm

John Doe
user@freshfridge.com CHỦ NHÓM

Admin User
admin@freshfridge.com QUẢN TRỊ ⋮

Nguyễn Văn Minh
minh@freshfridge.com THÀNH VIÊN ⋮

Trần Thị Linh
linh@freshfridge.com THÀNH VIÊN ⋮

Hình 40: Thiết kế giao diện quản lý nhóm - 3

6. Nhóm UC-07: Thống kê và báo cáo Giao diện xem thống kê và báo cáo về tủ lạnh, mua sắm và bữa ăn.

Màn hình Báo cáo (ReportScreen)

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Nút Back + Logo + Tiêu đề “Báo cáo” + Nút Download.
- `View.tabContainer` → `ScrollView` (horizontal):
 - * `SegmentedButtons`: 4 tab chuyển đổi:
 - “Tổng quan” (icon: `view-dashboard`)
 - “Tủ lạnh” (icon: `fridge`)
 - “Mua sắm” (icon: `cart`)
 - “Bữa ăn” (icon: `silverware`)
- `ScrollView.content`: Nội dung thay đổi theo tab đang active.
- `BottomNavBar`: Tab “home” đang active.

- **Tab Tổng quan (Overview):**

- `View.summaryGrid`: Lưới 2x2 các `Surface.summaryCard`:
 - * Card “Trong tủ lạnh”: Icon `fridge`, nền `primaryContainer`.
 - * Card “Sắp hết hạn”: Icon `alert-circle`, nền `warningLight`.
 - * Card “Hết hạn”: Icon `close-circle`, nền `errorLight`.
 - * Card “Lãng phí”: Icon `currency-usd`, nền `secondaryContainer`.
- `Card.categoryDistribution`: Phân loại thực phẩm:
 - * Header: Icon `chart-pie` + “Phân loại thực phẩm”.
 - * Danh sách `CategoryItem`: Dot màu + Tên danh mục + Số lượng + `ProgressBar`.
- `Card.topItems`: Thực phẩm phổ biến:
 - * Header: Icon `star` + “Thực phẩm phổ biến”.
 - * Danh sách `TopItem`: Rank badge + Tên + Category + Số lần.

- **Tab Tủ lạnh (Kitchen):**

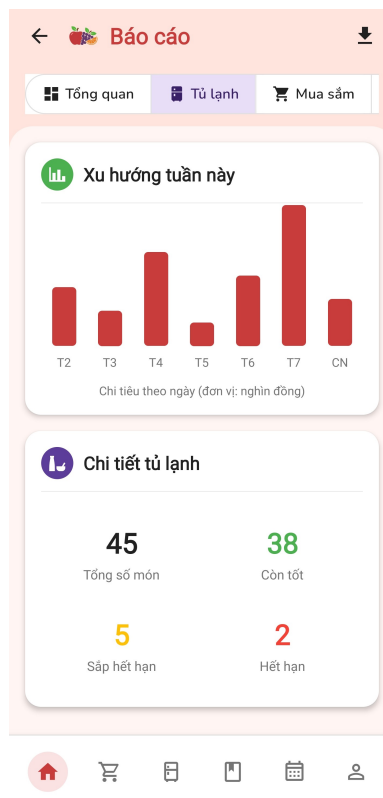
- `Card.weeklyTrend`: Xu hướng tuần này:
 - * Header: Icon `chart-bar` + “Xu hướng tuần này”.
 - * `View.chartContainer`: Biểu đồ cột 7 ngày (T2-CN).
 - * `ChartBarFill`: Cột màu `primary` với chiều cao tỷ lệ theo giá trị.
 - * `ChartLegend`: Chú thích “Chỉ tiêu theo ngày”.
- `Card.kitchenDetails`: Chi tiết tủ lạnh:
 - * Header: Icon `food-variant` + “Chi tiết tủ lạnh”.
 - * `View.statsGrid`: Lưới 2x2 hiển thị:
 - Tổng số món (màu mặc định)
 - Còn tốt (màu `success`)
 - Sắp hết hạn (màu `warning`)
 - Hết hạn (màu `error`)

- **Tab Mua sắm (Shopping):**

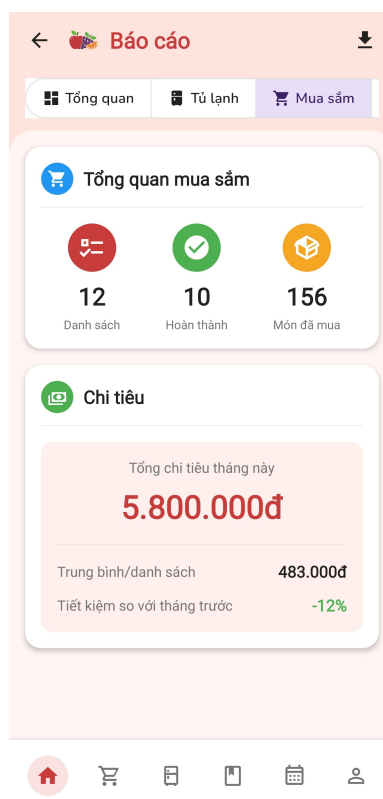
- `Card.shoppingOverview`: Tổng quan mua sắm:

- * Header: Icon `cart` + “Tổng quan mua sắm”.
- * `View.shoppingStats`: 3 thẻ thống kê:
 - “Danh sách”: Icon `format-list-checks` (primary).
 - “Hoàn thành”: Icon `check-circle` (success).
 - “Món đã mua”: Icon `package-variant` (tertiary).
- `Card.spending`: Chi tiêu:
 - * Header: Icon `cash-multiple` + “Chi tiêu”.
 - * `View.spendingContainer`: Nền `surfaceVariant`:
 - `SpendingMain`: Tổng chi tiêu tháng (font lớn, màu primary).
 - `SpendingDetail`: Trung bình/danh sách + Tiết kiệm so với tháng trước.
- **Tab Bữa ăn (Meal):**
 - `Card.mealOverview`: Thống kê bữa ăn:
 - * Header: Icon `silverware-fork-knife` + “Thống kê bữa ăn”.
 - * `View.mealStatCircle`: Hình tròn lớn hiển thị tổng số bữa ăn.
 - * `View.mealStatDetails`: 3 thẻ chi tiết:
 - “Bữa sáng”: Icon `weather-sunny` (tertiary).
 - “Bữa trưa”: Icon `white-balance-sunny` (warning).
 - “Bữa tối”: Icon `weather-night` (secondary).
 - `Card.recipeUsage`: Sử dụng công thức:
 - * Header: Icon `book-open-variant` + “Sử dụng công thức”.
 - * `View.recipeStats`: 2 cột với divider:
 - Số công thức đã dùng.
 - Tỷ lệ áp dụng (%).
- **Xử lý cử chỉ và Input:**
 - **Tab Navigation**: `SegmentedButtons.onValueChange` cập nhật `activeTab` để render nội dung tương ứng.
 - **Pull-to-refresh**: `RefreshControl` cho phép kéo xuống để reload dữ liệu báo cáo.
 - **Download Action**: Nút download trên header (chức năng đang phát triển).
 - **Horizontal Scroll**: Tab selector có thể cuộn ngang trên màn hình nhỏ.
- **Visual Style:**
 - **Summary Cards**:
 - * Nền gradient nhẹ theo ngữ cảnh (primary/warning/error/secondary).
 - * Icon tròn với màu nền tương ứng.
 - * Số liệu font lớn (24px), label font nhỏ (12px).
 - **Progress Bars**: Height 8px, border-radius 4px, màu theo danh mục.
 - **Bar Chart**:
 - * Chiều cao tối đa 150px.
 - * Cột width 24px, border-radius 4px, màu primary.
 - * Label ngày ở dưới mỗi cột.

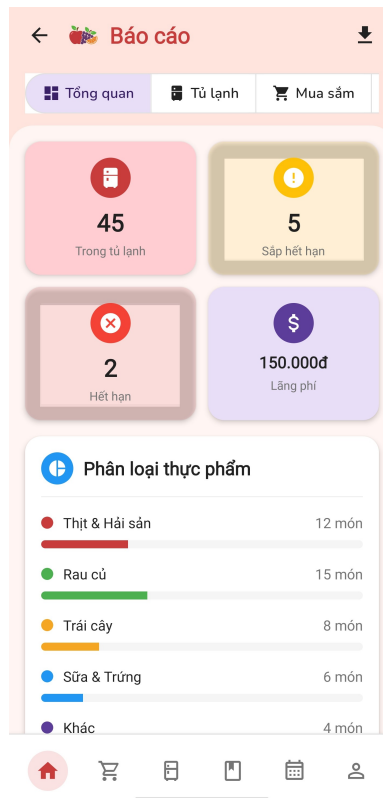
- **Meal Circle:**
 - * Kích thước 120x120px, border-radius 60px.
 - * Nền `primaryContainer`, số liệu font 36px màu `primary`.
- **Currency Format:** Định dạng VND với `toLocaleString('vi-VN')`.
- **State Management:**
 - `activeTab`: Tab đang hiển thị ('overview' | 'kitchen' | 'shopping' | 'meal').
 - `refreshing`: Trạng thái đang refresh dữ liệu.
 - `mockReportData`: Dữ liệu báo cáo (mock data, sẽ thay bằng API call).
 - `maxWeeklySpent`: Giá trị max của tuần để tính tỷ lệ biểu đồ cột (`useMemo`).
- **Dữ liệu hiển thị:**
 - **Overview:**
 - * Tổng số thực phẩm trong tủ lạnh.
 - * Số thực phẩm sắp hết hạn / đã hết hạn.
 - * Giá trị lãng phí (VND).
 - * Phân bố theo danh mục với tỷ lệ phần trăm.
 - * Top 5 thực phẩm được mua nhiều nhất.
 - **Kitchen:**
 - * Biểu đồ xu hướng chi tiêu 7 ngày.
 - * Chi tiết trạng thái tủ lạnh (còn tốt/sắp hết hạn/hết hạn).
 - **Shopping:**
 - * Số danh sách mua sắm / đã hoàn thành.
 - * Tổng số món đã mua.
 - * Tổng chi tiêu tháng.
 - * Trung bình chi tiêu mỗi danh sách.
 - * Phần trăm tiết kiệm so với tháng trước.
 - **Meal:**
 - * Tổng số bữa ăn đã lên lịch.
 - * Phân bố theo bữa sáng/trưa/tối.
 - * Số công thức đã sử dụng.
 - * Tỷ lệ áp dụng công thức.



Hình 41: Thiết kế giao diện thống kê và báo cáo - 1



Hình 42: Thiết kế giao diện thống kê và báo cáo - 2



Hình 43: Thiết kế giao diện thống kê và báo cáo - 3

8. Nhóm UC-08: Quản trị người dùng

Màn hình Danh sách người dùng (UserManagementScreen)

• Cấu trúc cây Component:

- SafeAreaView → Header: Logo + Tiêu đề “Quản trị người dùng” + Icon Notification.
- View.searchContainer: Searchbar (react-native-paper) cho phép tìm kiếm theo tên, email hoặc số điện thoại.
- View.filterContainer: Hàng các Chip lọc gồm:
 - * Chip “Trạng thái” (Tất cả/Hoạt động/Bị khóa)
 - * Chip “Sắp xếp” (Mới nhất/Tên A-Z/Tên Z-A)
 - * IconButton “Làm mới” để reload danh sách
- View.statsContainer: Hiển thị thống kê tổng quan:
 - * Card.StatCard: Tổng số người dùng với icon account-group
 - * Card.StatCard: Người dùng hoạt động với icon account-check
 - * Card.StatCard: Tài khoản bị khóa với icon account-lock
- FlatList → UserCard: Hiển thị danh sách người dùng với infinite scroll.
- Trong mỗi UserCard:
 - * Avatar: Ảnh đại diện người dùng với fallback icon account.
 - * View.userInfo:
 - Text.userName: Họ tên người dùng (bold).

- `Text.userEmail`: Email với icon **email**.
- `Text.userPhone`: Số điện thoại với icon **phone**.
- `Text.registeredDate`: Ngày đăng ký với icon **calendar**.
- * `Chip.statusChip`: Badge trạng thái (Hoạt động/Bị khóa):
 - Màu xanh lá (**#4CAF50**) - Hoạt động
 - Màu đỏ (**#F44336**) - Bị khóa
- * `View.actionsRow`: Hàng các nút hành động:
 - `IconButton.view`: Icon eye để xem chi tiết.
 - `IconButton.lock`: Icon lock/lock-open để khóa/mở khóa.
 - `IconButton.delete`: Icon delete để xóa tài khoản.
- `Portal` → `Dialog.confirmDialog`: Dialog xác nhận cho các hành động quan trọng.
- `BottomNavBar`: Tab “admin” đang active (chỉ hiện với QTV).

- **Xử lý cử chỉ và Input:**

- **User Input**: `Searchbar` xử lý `onChangeText` với `debounce 300ms` để tìm kiếm.
- **Filter Actions**: Nhấn vào `Chip` trạng thái để lọc danh sách theo trạng thái.
- **Sort Actions**: Nhấn vào `Chip` sắp xếp mở `Menu` với các tùy chọn sắp xếp.
- **Infinite Scroll**: `FlatList` gọi `fetchNextPage()` khi scroll đến cuối danh sách.
- **View Details**: Nhấn vào `UserCard` hoặc nút `eye` gọi `onUserPress(userId)` để xem chi tiết.
- **Lock/Unlock**: Nhấn nút khóa hiển thị `Dialog.confirm` xác nhận trước khi gọi `toggleUserStatusMutation`.
- **Delete Action**: Nhấn nút xóa hiển thị `Dialog.confirm` với cảnh báo nghiêm trọng trước khi gọi `deleteUserMutation`.
- **Pull-to-refresh**: `RefreshControl` cho phép kéo xuống để reload danh sách.
- **Empty State**: Hiển thị `EmptyState` khi không có kết quả tìm kiếm hoặc chưa có người dùng.

- **Visual Style:**

- **Card Layout**: Mỗi `UserCard` có `elevation` nhẹ, `padding 16`, `borderRadius 8`, `margin vertical 8`.
- **Status Colors**:
 - * Xanh lá (**#4CAF50**) - Tài khoản hoạt động (ACTIVE)
 - * Đỏ (**#F44336**) - Tài khoản bị khóa (LOCKED)
- **Avatar**: Size 56x56 với fallback background màu `primary`.
- **Action Buttons**: `IconButton` size 24 với màu theo ngữ cảnh:
 - * View: màu `primary` (**#007AFF**)
 - * Lock/Unlock: màu `warning` (**#FF9800**)
 - * Delete: màu `error` (**#F44336**)
- **Typography**: `userName` size 16 bold, `email/phone` size 14 regular, `date` size 12 muted.
- **Stats Cards**: Grid 3 cột với background gradient nhẹ, icon size 32, số liệu size 24 bold.

- **State Management:**

- `searchQuery`: Chuỗi tìm kiếm người dùng (debounced).
- `statusFilter`: Lọc theo trạng thái ('ALL'/'ACTIVE'/'LOCKED').
- `sortBy`: Tiêu chí sắp xếp ('createdAt', 'name__asc', 'name__desc').
- `useInfiniteUsers`: React Query hook với infinite scroll pagination.
- `selectedUser`: User đang được chọn cho action (lock/delete).
- `dialogVisible`: Boolean điều khiển hiển thị dialog xác nhận.
- `dialogType`: Loại action đang xác nhận ('lock', 'unlock', 'delete').

- **API Integration:**

- `useInfiniteQuery`: Fetch danh sách người dùng với pagination.
- `useMutation`: Toggle status (UC-08.2 - Khóa/Mở khóa tài khoản).
- `useMutation`: Delete user (UC-08.3 - Xóa tài khoản).
- **Invalidation**: Sau mỗi mutation thành công, invalidate query 'users' để refresh danh sách.

Màn hình Chi tiết người dùng (UserDetailsScreen)

- **Cấu trúc cây Component:**

- `SafeAreaView` → `Header`: Nút Back + “Chi tiết người dùng” + Menu actions.
- `ScrollView`:
 - * `View.profileHeader`: Section header với gradient background:
 - `Avatar.large`: Ảnh đại diện size 120x120.
 - `Text.userName`: Họ tên size 24 bold.
 - `Chip.statusChip`: Badge trạng thái lớn với icon.
 - * `Card.basicInfo`: Thông tin cơ bản:
 - `List.Item`: Email với icon `email` (copyable).
 - `List.Item`: Số điện thoại với icon `phone` (copyable).
 - `List.Item`: ID người dùng với icon `identifier`.
 - `List.Item`: Ngày đăng ký với icon `calendar`.
 - * `Card.groupsInfo`: Thông tin nhóm:
 - `Text.sectionTitle`: “Nhóm tham gia” với icon `account-group`.
 - `FlatList`: Danh sách các nhóm người dùng tham gia, mỗi nhóm hiển thị `Chip.groupChip` (tên nhóm + vai trò Leader/Member) và `Text.groupMembers` (số thành viên).
 - * `Card.activityStats`: Thống kê hoạt động:
 - `View.statRow`: Tổng số bữa ăn đã lên lịch với icon `food`.
 - `View.statRow`: Số công thức cá nhân đã tạo với icon `chef-hat`.
 - `View.statRow`: Số lần đi chợ đã thực hiện với icon `cart`.
 - `View.statRow`: Lần hoạt động cuối với icon `clock`.
 - * `Card.accountHistory`: Lịch sử tài khoản:
 - `Timeline`: Các sự kiện quan trọng bao gồm ngày tạo tài khoản, các lần bị khóa/mở khóa (nếu có), và các lần thay đổi thông tin quan trọng.

- * **View.actionsContainer:** Các nút hành động chính:
 - **Button.toggleStatus:** Nút khóa/mở khóa tài khoản (màu warning).
 - **Button.sendNotification:** Nút gửi thông báo cho người dùng (màu info).
 - **Button.delete:** Nút xóa tài khoản (màu error, outlined).
- **Xử lý cử chỉ và Input:**
 - **Copy Actions:** Nhấn giữ vào email/phone để copy vào clipboard với **Snackbar** xác nhận.
 - **Navigation to Groups:** Nhấn vào **groupChip** để navigate đến chi tiết nhóm.
 - **Toggle Status:** Nhấn nút khóa/mở khóa hiển thị **Dialog** xác nhận với lý do (**TextInput multiline**).
 - **Send Notification:** Nhấn nút gửi thông báo mở **Dialog** nhập tiêu đề và nội dung thông báo.
 - **Delete Action:** Nhấn nút xóa hiển thị **Dialog** với cảnh báo nghiêm trọng và yêu cầu nhập mật khẩu admin xác nhận.
 - **Menu Actions:** Menu phía trên phải với options:
 - * “Gửi email reset mật khẩu”
 - * “Xem log hoạt động”
 - * “Export dữ liệu người dùng”
- **Visual Style:**
 - **Profile Header:** Gradient background từ primary đến primary-dark, text màu trắng.
 - **Info Cards:** Các Card riêng biệt với elevation 2, margin vertical 8, padding 16.
 - **List Items:** Icon size 24 bên trái, text size 16, description size 14 muted.
 - **Timeline:** Đường dọc màu primary, circle nodes, text timestamp size 12.
 - **Action Buttons:** Full width, height 48, borderRadius 8, spacing 12 giữa các nút.
 - **Group Chips:** Màu khác nhau cho role:
 - * Leader: Gold (#FFD700)
 - * Member: Blue (#2196F3)
- **State Management:**
 - **userId:** ID người dùng được truyền từ navigation params.
 - **useUser(userId):** React Query hook fetch chi tiết người dùng.
 - **useUserGroups(userId):** Fetch danh sách nhóm của user.
 - **useUserActivity(userId):** Fetch thống kê hoạt động.
 - **useUserHistory(userId):** Fetch lịch sử tài khoản.
 - **dialogState:** Object chứa state của các dialog (visible, type, data).
 - **reason:** String lưu lý do khóa/mở khóa tài khoản.
- **Data Display:**
 - **Date Formatting:** Hiển thị ngày theo format "dd/MM/yyyy HH:mm".
 - **Status Badge:** Icon + text + background color tùy theo trạng thái.
 - **Empty States:** Hiển thị message thích hợp khi user chưa có nhóm/hoạt động.
 - **Loading States:** Skeleton loaders cho mỗi section trong khi fetch data.

Dialog xác nhận Khóa/Mở khóa tài khoản

- **Cấu trúc cây Component:**

- Portal → Dialog.lockDialog:
 - * Dialog.Title: “Khóa tài khoản” hoặc “Mở khóa tài khoản”.
 - * Dialog.Content:
 - Text: Thông báo xác nhận hành động.
 - Text.userName: Tên người dùng bị ảnh hưởng (bold).
 - TextInput.reason: Input lý do khóa/mở khóa (multiline, required).
 - Checkbox: “Gửi email thông báo cho người dùng” (checked mặc định).
 - * Dialog.Actions:
 - Button.cancel: Nút “Hủy”.
 - Button.confirm: Nút “Xác nhận” (disabled nếu reason rỗng).

- **Validation:**

- Lý do phải có ít nhất 10 ký tự.
- Hiện thị error message bên dưới TextInput nếu không hợp lệ.

- **Submit Action:**

- Gọi toggleUserStatusMutation.mutate({userId, status, reason, sendEmail}).
- Hiện thị loading spinner trong khi xử lý.
- Sau khi thành công: đóng dialog, hiện thị SnackBar thông báo, invalidate queries.
- Nếu lỗi: hiện thị error message trong dialog.

Dialog xác nhận Xóa tài khoản

- **Cấu trúc cây Component:**

- Portal → Dialog.deleteDialog:
 - * Dialog.Icon: Icon cảnh báo alert-circle màu đỏ.
 - * Dialog.Title: “Xóa tài khoản người dùng” (màu error).
 - * Dialog.Content:
 - Text.warning: Cảnh báo nghiêm trọng về việc xóa tài khoản (màu error, bold).
 - List: Danh sách dữ liệu sẽ bị xóa gồm tất cả thông tin cá nhân, tất cả nhóm mà user là leader (sẽ bị giải tán), tất cả công thức cá nhân đã tạo, và tất cả lịch sử hoạt động.
 - Text.confirmation: “Để xác nhận, vui lòng nhập mật khẩu admin:”.
 - TextInput.adminPassword: Input mật khẩu (secureTextEntry).
 - TextInput.confirmText: Input text xác nhận “XÓA TÀI KHOẢN” (all caps).
 - * Dialog.Actions:
 - Button.cancel: Nút “Hủy”.
 - Button.confirm: Nút “Xóa vĩnh viễn” (màu error, disabled until validated).

- **Validation:**

- Mật khẩu admin không được rỗng.
- Text xác nhận phải chính xác là “XÓA TÀI KHOẢN” (case-sensitive).
- Nút xác nhận chỉ enable khi cả hai điều kiện đều thỏa mãn.

- **Submit Action:**

- Gọi `deleteUserMutation.mutate({userId, adminPassword})`.
- Hiển thị loading spinner trong khi xử lý.
- Sau khi thành công:
 - * Đóng dialog
 - * Hiển thị **Snackbar** xác nhận xóa thành công
 - * Navigate về màn hình danh sách
 - * Invalidate queries 'users'
- Nếu lỗi (mật khẩu sai, lỗi server): hiển thị error message trong dialog.

- **Visual Style:**

- Dialog có `maxWidth 400`, `borderRadius 12`.
- Icon cảnh báo size 64, màu error, ở giữa.
- Warning text với background màu error nhạt, `padding 12`, `borderRadius 8`.
- List items với bullet màu error, `spacing 8`.
- `TextInput` có `helperText` hướng dẫn.
- `Confirm button` có variant 'contained', `backgroundColor error`.

Dialog Gửi thông báo cho người dùng

- **Cấu trúc cây Component:**

- Portal → `Dialog.notificationDialog`:
 - * `Dialog.Title`: “Gửi thông báo” với icon bell.
 - * `Dialog.Content`:
 - `TextInput.title`: Tiêu đề thông báo (required, `maxLength 100`).
 - `TextInput.body`: Nội dung thông báo (multiline, required, `maxLength 500`).
 - `Checkbox`: “Gửi qua email” (optional).
 - `Checkbox`: “Gửi push notification” (checked mặc định).
 - `Text.charCount`: Hiển thị số ký tự đã nhập/tối đa.
 - * `Dialog.Actions`:
 - `Button.cancel`: Nút “Hủy”.
 - `Button.send`: Nút “Gửi” (disabled nếu title hoặc body rỗng).

- **Validation:**

- Tiêu đề không được rỗng, tối đa 100 ký tự.
- Nội dung không được rỗng, tối đa 500 ký tự.
- Ít nhất một phương thức gửi (email hoặc push) phải được chọn.

- **Submit Action:**

- Gọi `sendNotificationMutation.mutate({userId, title, body, sendEmail, sendPush})`.
- Hiện thị loading trong khi xử lý.
- Sau khi thành công: đóng dialog, hiện thị `Snackbar` “Đã gửi thông báo”.



Hình 44: Thiết kế giao diện quản trị người dùng

4. Nhóm UC-09: Xác thực (Login/Register)

Màn hình Đăng nhập

- **Cấu trúc cây Component:**

- `SafeAreaView` → `KeyboardAvoidingView`: Tránh bàn phím che form.
- `ScrollView`:
 - * `View.logoContainer`: Logo app + Text slogan.
 - * `TextInput.email`: Input email với icon email.
 - * `TextInput.password`: Input mật khẩu với icon lock (`secureTextEntry`).
 - * `TouchableOpacity.forgotPassword`: Link “Quên mật khẩu?”.
 - * `Button.login`: Nút “Đăng nhập” (primary).
 - * `View.divider`: Đường kẻ với text “hoặc”.
 - * `Button.register`: Nút “Đăng ký tài khoản mới” (outlined).

- **Xử lý Input:**

- Validation email format và mật khẩu không rỗng.

- Nhấn “Đăng nhập” gọi `loginMutation`.
- Nhấn “Quên mật khẩu” navigate đến `ForgotPasswordScreen`.
- Nhấn “Đăng ký” navigate đến `RegisterScreen`.
- Sau login thành công: lưu token, navigate đến `HomeScreen`.
- Hiển thị error message nếu sai email/password.

- **State:** email, password, showPassword, loginMutation.

Màn hình Đăng ký

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Nút Back + “Đăng ký”.
- `ScrollView`:
 - * `TextInput.fullName`: Họ tên (required).
 - * `TextInput.email`: Email (required, email format).
 - * `TextInput.phone`: Số điện thoại (required, 10 số).
 - * `TextInput.password`: Mật khẩu (required, min 8 ký tự, có chữ hoa + số).
 - * `TextInput.confirmPassword`: Xác nhận mật khẩu (phải giống password).
 - * `View.passwordRules`: Hiển thị yêu cầu mật khẩu với icon check/uncheck:
 - Ít nhất 8 ký tự
 - Có chữ hoa
 - Có chữ số
 - * `Checkbox.terms`: “Tôi đồng ý với Điều khoản sử dụng” (required).
 - * `Button.register`: Nút “Đăng ký” (disabled khi không hợp lệ).
 - * `View.loginLink`: Text “Đã có tài khoản?” + Link “Đăng nhập”.

- **Validation:**

- Họ tên không rỗng.
- Email đúng format và chưa tồn tại.
- SĐT 10 số và chưa tồn tại.
- Password min 8 ký tự, có chữ hoa, số.
- Confirm password khớp với password.
- Terms phải được check.
- Hiển thị error realtime dưới mỗi field.

- **Xử lý Input:**

- Nhấn “Đăng ký” gọi `registerMutation`.
- Sau register thành công: gửi email xác thực, navigate đến `VerificationScreen`.
- Nếu email/SĐT đã tồn tại: hiển thị error message.

- **State:** formData, errors, termsAccepted, registerMutation.

Màn hình Xác thực OTP

- **Cấu trúc cây Component:**

- SafeAreaView → Header: Nút Back + “Xác thực tài khoản”.
- View.content:
 - * Icon: Icon email-check lớn.
 - * Text.instruction: “Nhập mã xác thực đã gửi đến {email}”.
 - * View.otpInputs: 6 ô input OTP (1 số/ô, auto-focus).
 - * View.timer: Text “Mã có hiệu lực trong {countdown}s”.
 - * Button.verify: Nút “Xác thực” (disabled khi chưa đủ 6 số).
 - * TouchableOpacity.resend: Link “Gửi lại mã” (disabled khi countdown > 0).

- **Xử lý Input:**

- Auto-focus ô tiếp theo khi nhập số.
- Nhấn “Xác thực” gọi `verifyOtpMutation`.
- Sau verify thành công: navigate đến HomeScreen.
- Nếu mã sai: hiển thị error, xóa input.
- Nhấn “Gửi lại”: reset countdown 60s, gọi `resendOtpMutation`.

- **State:** `otpCode`, `countdown`, `canResend`, `verifyMutation`.

Màn hình Quên mật khẩu

- **Cấu trúc cây Component:**

- SafeAreaView → Header: Nút Back + “Quên mật khẩu”.
- View.content:
 - * Icon: Icon lock-reset lớn.
 - * Text.instruction: “Nhập email đăng ký để nhận mã khôi phục”.
 - * TextInput.email: Input email.
 - * Button.send: Nút “Gửi mã” (disabled khi email không hợp lệ).
 - * TouchableOpacity.backToLogin: Link “Quay lại đăng nhập”.

- **Xử lý Input:**

- Validate email format.
- Nhấn “Gửi mã” gọi `sendResetCodeMutation`.
- Sau gửi thành công: navigate đến ResetPasswordScreen với email.
- Nếu email không tồn tại: hiển thị error.

- **State:** `email`, `emailError`, `sendCodeMutation`.

Màn hình Đặt lại mật khẩu

- **Cấu trúc cây Component:**
 - SafeAreaView → Header: Nút Back + “Đặt lại mật khẩu”.
 - View.content:
 - * Text.instruction: “Nhập mã xác thực và mật khẩu mới”.
 - * TextInput.code: Mã xác thực (6 số).
 - * TextInput.newPassword: Mật khẩu mới (min 8 ký tự).
 - * TextInput.confirmPassword: Xác nhận mật khẩu.
 - * View.passwordRules: Yêu cầu mật khẩu với icon check/uncheck.
 - * Button.reset: Nút “Đặt lại mật khẩu”.
- **Validation:**
 - Mã xác thực 6 số.
 - Password min 8 ký tự, có chữ hoa, số.
 - Confirm password khớp.
- **Xử lý Input:**
 - Nhấn “Đặt lại” gọi `resetPasswordMutation`.
 - Sau reset thành công: hiển thị success message, navigate đến LoginScreen.
 - Nếu mã sai/hết hạn: hiển thị error.
- **State:** `code`, `newPassword`, `confirmPassword`, `resetMutation`.

Màn hình Đổi mật khẩu (trong app)

- **Cấu trúc cây Component:**
 - SafeAreaView → Header: Nút Back + “Đổi mật khẩu”.
 - ScrollView:
 - * TextInput.currentPassword: Mật khẩu hiện tại (secureTextEntry).
 - * TextInput.newPassword: Mật khẩu mới (min 8 ký tự).
 - * TextInput.confirmPassword: Xác nhận mật khẩu mới.
 - * View.passwordRules: Yêu cầu mật khẩu.
 - * Button.change: Nút “Đổi mật khẩu”.
- **Validation:**
 - Current password không rỗng.
 - New password khác current password.
 - New password min 8 ký tự, có chữ hoa, số.
 - Confirm password khớp với new password.
- **Xử lý Input:**
 - Nhấn “Đổi mật khẩu” gọi `changePasswordMutation`.
 - Sau thành công: gửi email thông báo, hiển thị success, navigate back.
 - Nếu current password sai: hiển thị error.
- **State:** `passwords`, `errors`, `changePasswordMutation`.

Màn hình Cập nhật thông tin cá nhân

- **Cấu trúc cây Component:**

- `SafeAreaView` → Header: Nút Back + “Thông tin cá nhân” + Nút Save.
- `ScrollView`:
 - * `View.avatarSection`:
 - `Avatar.large`: Ảnh đại diện size 120x120.
 - `IconButton.edit`: Nút edit overlay trên avatar.
 - * `Card.personalInfo`:
 - `TextInput.fullName`: Họ tên (editable).
 - `TextInput.email`: Email (disabled).
 - `TextInput.phone`: SĐT (editable).
 - * `Card.security`:
 - `List.Item`: “Đổi mật khẩu” với icon `chevron-right`.
 - `List.Item`: “Xác thực 2 bước” với `Switch`.
 - * `Card.preferences`:
 - `List.Item`: “Thông báo push” với `Switch`.
 - `List.Item`: “Thông báo email” với `Switch`.
 - * `Button.logout`: Nút “Đăng xuất” (outlined, màu error).

- **Xử lý Input:**

- Nhấn avatar edit mở `ImagePicker` (camera/gallery).
- Nhấn Save gọi `updateProfileMutation`.
- Nhấn “Đổi mật khẩu” navigate đến `ChangePasswordScreen`.
- Toggle switches gọi `updateSettingsMutation`.
- Nhấn “Đăng xuất” hiển thị `Dialog` xác nhận, sau đó logout và navigate về `LoginScreen`.

- **State:** `userData`, `settings`, `hasChanges`, `updateMutation`.



FRESH FRIDGE

Welcome back! Please login to continue.

Email

Password

☐ Remember me [Forgot password?](#)

Login

Don't have an account? [Register](#)

Test Credentials:
 user@freshfridge.com / password123
 admin@freshfridge.com / admin123

Hình 45: Thiết kế giao diện đăng nhập



FRESH FRIDGE

Join Fresh Fridge and start managing your kitchen smartly!

Full Name

Email

Phone Number (Optional)

Password

Confirm Password

☐ I agree to the Terms and Conditions

Register

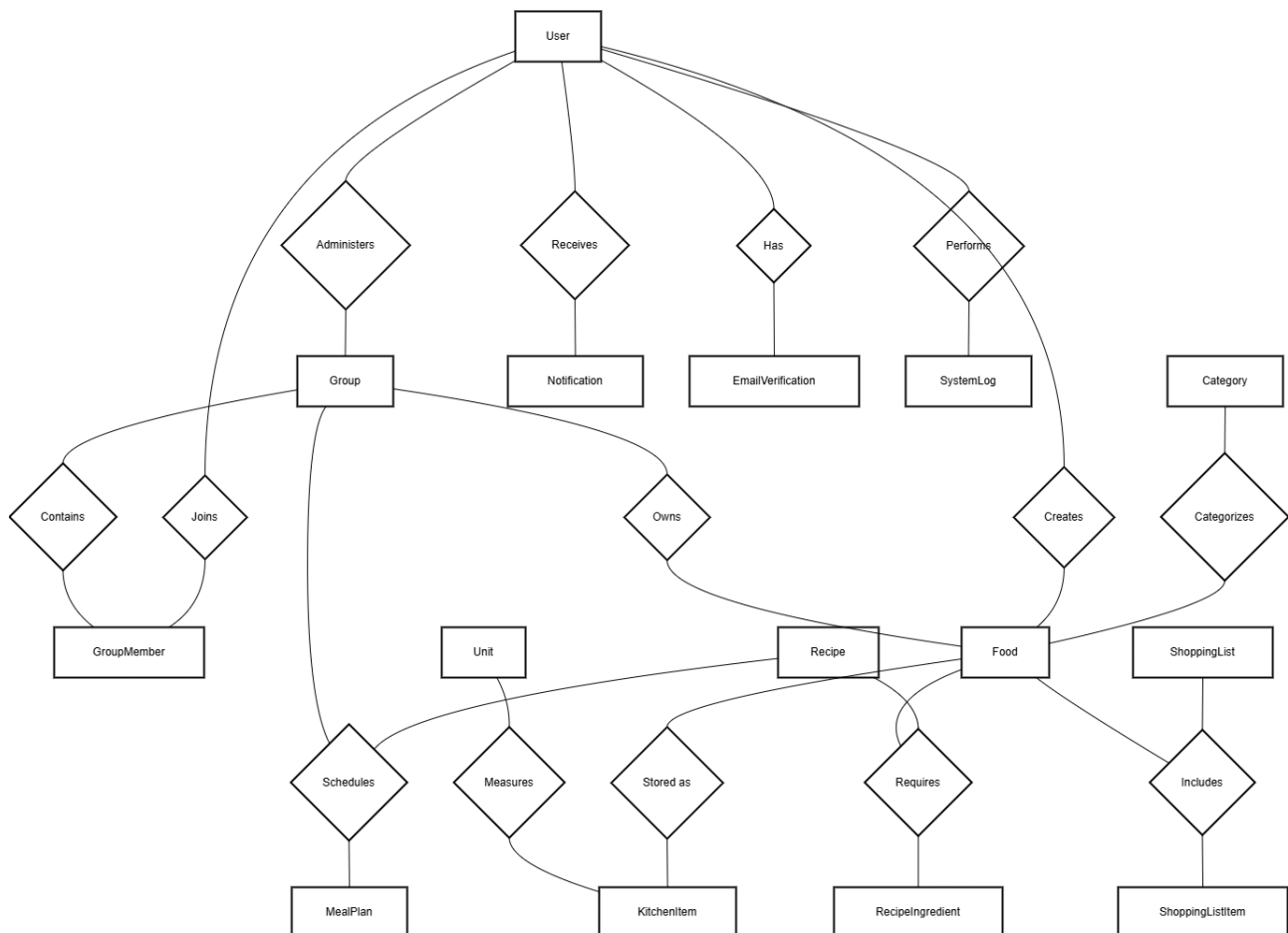
Already have an account? [Login](#)

Hình 46: Thiết kế giao diện đăng ký

4 Thiết kế Backend

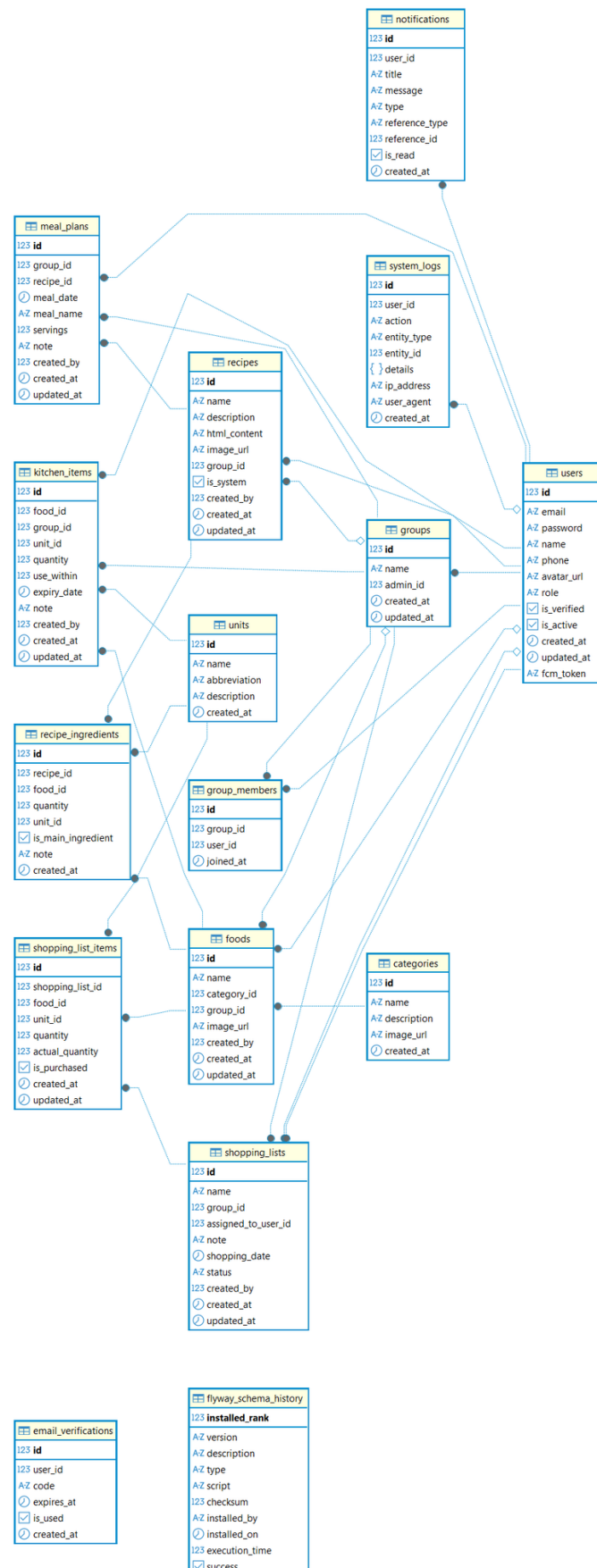
4.1 Thiết kế cơ sở dữ liệu cho Backend

4.1.1 Sơ đồ thực thể quan hệ (ERD)



Hình 47: Sơ đồ thực thể quan hệ (ERD) của hệ thống

4.1.2 Lược đồ cơ sở dữ liệu



Hình 48: Lược đồ cơ sở dữ liệu của hệ thống

4.1.3 Đặc tả chi tiết các bảng dữ liệu

Bảng USER - Người dùng Mục đích: Lưu trữ thông tin tài khoản và hồ sơ người dùng hệ thống.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất của người dùng
email	VARCHAR(255)	UNIQUE, NOT NULL	Email đăng nhập, phải là duy nhất trong hệ thống
password	VARCHAR(255)	NOT NULL	Mật khẩu đã được mã hóa (BCrypt)
name	VARCHAR(100)	NOT NULL	Họ và tên đầy đủ của người dùng
phone	VARCHAR(20)	UNIQUE	Số điện thoại liên hệ
avatar_url	TEXT	NULL	URL ảnh đại diện của người dùng
role	VARCHAR(20)	NOT NULL, DEFAULT 'USER'	Vai trò: USER, ADMIN
is_verified	BOOLEAN	NOT NULL, DEFAULT FALSE	Trạng thái xác thực email
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Trạng thái hoạt động của tài khoản
fcm_token	VARCHAR(500)	NULL	Token Firebase Cloud Messaging cho push notification
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo tài khoản
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Thời gian cập nhật gần nhất

Bảng 45: Cấu trúc bảng USER

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: email
- INDEX: phone

Bảng GROUP - Nhóm gia đình Mục đích: Quản lý các nhóm người dùng (gia đình, bạn bè) cùng sử dụng tủ lạnh chung.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất của nhóm
name	VARCHAR(100)	NOT NULL	Tên nhóm
admin_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID của người quản trị nhóm
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo nhóm
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 46: Cấu trúc bảng GROUP

Foreign Keys:

- admin_id REFERENCES user(id) ON DELETE CASCADE

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: admin_id
- INDEX: admin_id

Bảng GROUP_MEMBER - Thành viên nhóm **Mục đích:** Bảng trung gian quản lý quan hệ nhiều-nhiều giữa User và Group.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
group_id	BIGINT	NOT NULL, FOREIGN KEY → group(id)	ID của nhóm
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID của người dùng
joined_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tham gia nhóm

Bảng 47: Cấu trúc bảng GROUP_MEMBER

Foreign Keys:

- group_id REFERENCES group(id) ON DELETE CASCADE

- user_id REFERENCES user(id) ON DELETE CASCADE

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: user_id (một user chỉ thuộc MỘT nhóm)
- INDEX: group_id
- INDEX: user_id

Bảng CATEGORY - Danh mục thực phẩm Mục đích: Phân loại thực phẩm theo các nhóm (rau củ, thịt cá, gia vị...).

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
name	VARCHAR(100)	NOT NULL, UNIQUE	Tên danh mục
description	TEXT	NULL	Mô tả chi tiết về danh mục
image_url	VARCHAR(500)	NULL	URL ảnh minh họa cho danh mục
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo

Bảng 48: Cấu trúc bảng CATEGORY

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: name

Dữ liệu mẫu:

- Rau củ quả
- Thịt và gia cầm
- Cá và hải sản
- Trứng và sữa
- Gia vị và nguyên liệu khô
- Đồ uống
- Bánh kẹo

Bảng FOOD - Thực phẩm Mục đích: Lưu trữ thông tin các loại thực phẩm (hệ thống và do người dùng tạo).

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
name	VARCHAR(200)	NOT NULL	Tên thực phẩm
category_id	BIGINT	NOT NULL, FOREIGN KEY → category(id)	ID danh mục
group_id	BIGINT	NULL, FOREIGN KEY → group(id)	ID nhóm (NULL = thực phẩm hệ thống)
image_url	VARCHAR(500)	NULL	URL ảnh thực phẩm
created_by	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người tạo
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 49: Cấu trúc bảng FOOD

Foreign Keys:

- category_id REFERENCES category(id) ON DELETE RESTRICT
- group_id REFERENCES group(id) ON DELETE CASCADE
- created_by REFERENCES user(id) ON DELETE SET NULL

Indexes:

- PRIMARY KEY: id
- INDEX: category_id
- INDEX: group_id
- INDEX: created_by
- INDEX: (name, group_id)

Bảng UNIT - Đơn vị đo lường Mục đích: Định nghĩa các đơn vị đo lường sử dụng trong hệ thống.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
name	VARCHAR(50)	NOT NULL, UNIQUE	Tên đơn vị (kilogram, lít, muỗng canh...)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
abbreviation	VARCHAR(10)	NOT NULL, UNIQUE	Ký hiệu viết tắt (kg, l, tbsp...)
description	TEXT	NULL	Mô tả chi tiết về đơn vị
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo

Bảng 50: Cấu trúc bảng UNIT

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: name
- UNIQUE INDEX: abbreviation

Dữ liệu mẫu:

- kilogram (kg), gram (g)
- lít (l), mililit (ml)
- muỗng canh (tbsp), muỗng cà phê (tsp)
- cái, quả, củ, con

Bảng KITCHEN_ITEM - Nguyên liệu trong tủ lạnh Mục đích: Quản lý thực phẩm có sẵn trong tủ lạnh của từng nhóm.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
food_id	BIGINT	NOT NULL, FOREIGN KEY → food(id)	ID thực phẩm
group_id	BIGINT	NOT NULL, FOREIGN KEY → group(id)	ID nhóm sở hữu
unit_id	BIGINT	NOT NULL, FOREIGN KEY → unit(id)	ID đơn vị đo
quantity	DECIMAL(10,2)	NOT NULL	Số lượng hiện có
use_within	INT	NULL	Số ngày nên sử dụng hết
expiry_date	DATE	NULL	Ngày hết hạn
note	TEXT	NULL	Ghi chú thêm

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
created_by	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người thêm vào
created_at	TIMESTAMP	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP	Thời gian thêm
updated_at	TIMESTAMP	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 51: Cấu trúc bảng KITCHEN_ITEM

Foreign Keys:

- food_id REFERENCES food(id) ON DELETE CASCADE
- group_id REFERENCES group(id) ON DELETE CASCADE
- unit_id REFERENCES unit(id) ON DELETE RESTRICT
- created_by REFERENCES user(id) ON DELETE SET NULL

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: (food_id, group_id)
- INDEX: food_id
- INDEX: group_id
- INDEX: expiry_date
- INDEX: created_by

Bảng RECIPE - Công thức nấu ăn Mục đích: Lưu trữ các công thức nấu ăn (hệ thống và do người dùng tạo).

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
name	VARCHAR(200)	NOT NULL	Tên món ăn
description	TEXT	NULL	Mô tả ngắn về món ăn
html_content	TEXT	NOT NULL	Hướng dẫn nấu ăn chi tiết (HTML)
image_url	VARCHAR(500)	NULL	URL ảnh món ăn
group_id	BIGINT	NULL, FOREIGN KEY → group(id)	ID nhóm (NULL = công thức hệ thống)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
is_system	BOOLEAN	NOT NULL, DEFAULT FALSE	Đánh dấu công thức hệ thống
created_by	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người tạo
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 52: Cấu trúc bảng RECIPE

Foreign Keys:

- group_id REFERENCES group(id) ON DELETE CASCADE
- created_by REFERENCES user(id) ON DELETE SET NULL

Indexes:

- PRIMARY KEY: id
- INDEX: group_id
- INDEX: is_system
- INDEX: created_by
- FULLTEXT INDEX: name, description

Bảng RECIPE_INGREDIENT - Nguyên liệu của công thức **Mục đích:** Bảng trung gian liên kết Recipe và Food, xác định nguyên liệu cần thiết.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
recipe_id	BIGINT	NOT NULL, FOREIGN KEY → recipe(id)	ID công thức
food_id	BIGINT	NOT NULL, FOREIGN KEY → food(id)	ID thực phẩm
quantity	DECIMAL(10,2)	NOT NULL	Số lượng cần thiết
unit_id	BIGINT	NOT NULL, FOREIGN KEY → unit(id)	ID đơn vị đo

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
is_main_ingredient	BOOLEAN	NOT NULL, DEFAULT FALSE	Đánh dấu nguyên liệu chính
note	TEXT	NULL	Ghi chú về nguyên liệu (tùy chọn, bắt buộc...)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo

Bảng 53: Cấu trúc bảng RECIPE_INGREDIENT

Foreign Keys:

- recipe_id REFERENCES recipe(id) ON DELETE CASCADE
- food_id REFERENCES food(id) ON DELETE CASCADE
- unit_id REFERENCES unit(id) ON DELETE RESTRICT

Indexes:

- PRIMARY KEY: id
- INDEX: recipe_id
- INDEX: food_id
- PARTIAL INDEX: is_main_ingredient WHERE is_main_ingredient = TRUE (tối ưu query nguyên liệu chính)

Bảng MEAL_PLAN - Kế hoạch bữa ăn Mục đích: Lập lịch các món ăn cho nhóm theo ngày và bữa.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
group_id	BIGINT	NOT NULL, FOREIGN KEY → group(id)	ID nhóm
recipe_id	BIGINT	NOT NULL, FOREIGN KEY → recipe(id)	ID công thức
meal_date	DATE	NOT NULL	Ngày ăn
meal_name	VARCHAR(50)	NOT NULL	Tên bữa ăn (Sáng, Trưa, Tối, Phụ)
servings	INT	NOT NULL, DEFAULT 1	Số người ăn (số suất)
note	TEXT	NULL	Ghi chú thêm

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
created_by	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người lên lịch
created_at	TIMESTAMP	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP	Thời gian tạo
updated_at	TIMESTAMP	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 54: Cấu trúc bảng MEAL_PLAN

Foreign Keys:

- group_id REFERENCES group(id) ON DELETE CASCADE
- recipe_id REFERENCES recipe(id) ON DELETE CASCADE
- created_by REFERENCES user(id) ON DELETE SET NULL

Indexes:

- PRIMARY KEY: id
- INDEX: (group_id, meal_date) - composite index cho query theo nhóm và ngày
- INDEX: recipe_id

Constraints:

- CHECK constraint: meal_name IN ('SANG', 'TRUA', 'TOI')

Bảng SHOPPING_LIST - Danh sách mua sắm Mục đích: Quản lý danh sách đi chợ/mua sắm của nhóm.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
name	VARCHAR(200)	NOT NULL	Tên danh sách
group_id	BIGINT	NOT NULL, FOREIGN KEY → group(id)	ID nhóm
assigned_to_user_id	BIGINT	NULL, FOREIGN KEY → user(id)	ID người được phân công đi mua
note	TEXT	NULL	Ghi chú
shopping_date	DATE	NULL	Ngày dự kiến đi mua

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	Trạng thái: PENDING, IN_PROGRESS, COMPLETED
created_by	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người tạo danh sách
created_at	TIMESTAMP	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP	Thời gian tạo
updated_at	TIMESTAMP	NOT NULL, DEFAULT CUR- RENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 55: Cấu trúc bảng SHOPPING_LIST

Foreign Keys:

- group_id REFERENCES group(id) ON DELETE CASCADE
- assigned_to_user_id REFERENCES user(id) ON DELETE SET NULL
- created_by REFERENCES user(id) ON DELETE SET NULL

Indexes:

- PRIMARY KEY: id
- INDEX: group_id
- INDEX: assigned_to_user_id
- INDEX: status
- INDEX: shopping_date

Bảng SHOPPING_LIST_ITEM - Mục trong danh sách mua sắm Mục đích: Chi tiết các món cần mua trong danh sách mua sắm.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
shopping_list_id	BIGINT	NOT NULL, FOREIGN KEY → shopping_list(id)	ID danh sách mua sắm
food_id	BIGINT	NOT NULL, FOREIGN KEY → food(id)	ID thực phẩm cần mua

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
unit_id	BIGINT	NOT NULL, FOREIGN KEY → unit(id)	ID đơn vị đo
quantity	DECIMAL(10,2)	NOT NULL	Số lượng dự kiến mua
actual_quantity	DECIMAL(10,2)	NULL	Số lượng thực tế đã mua
is_purchased	BOOLEAN	NOT NULL, DEFAULT FALSE	Đã mua hay chưa
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Thời gian cập nhật

Bảng 56: Cấu trúc bảng SHOPPING_LIST_ITEM

Foreign Keys:

- shopping_list_id REFERENCES shopping_list(id) ON DELETE CASCADE
- food_id REFERENCES food(id) ON DELETE CASCADE
- unit_id REFERENCES unit(id) ON DELETE RESTRICT

Indexes:

- PRIMARY KEY: id
- UNIQUE INDEX: (shopping_list_id, food_id)
- INDEX: shopping_list_id
- INDEX: food_id
- INDEX: is_purchased

Bảng NOTIFICATION - Thông báo Mục đích: Lưu trữ thông báo gửi đến người dùng.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người nhận
title	VARCHAR(200)	NOT NULL	Tiêu đề thông báo
message	TEXT	NOT NULL	Nội dung thông báo
type	VARCHAR(50)	NOT NULL	Loại thông báo

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
reference_type	VARCHAR(50)	NULL	Loại đối tượng liên quan (RECIPE, MEAL_PLAN...)
reference_id	BIGINT	NULL	ID của đối tượng liên quan
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	Đã đọc hay chưa
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo

Bảng 57: Cấu trúc bảng NOTIFICATION

Foreign Keys:

- user_id REFERENCES user(id) ON DELETE CASCADE

Indexes:

- PRIMARY KEY: id
- INDEX: user_id
- INDEX: is_read
- INDEX: (user_id, is_read, created_at)
- INDEX: type

Các loại thông báo (type):

- EXPIRY_WARNING: Cảnh báo thực phẩm sắp hết hạn
- MEAL_REMINDER: Nhắc nhở bữa ăn theo lịch
- SHOPPING_ASSIGNED: Được phân công đi mua
- GROUP_INVITE: Lời mời tham gia nhóm
- RECIPE_SHARED: Có công thức mới được chia sẻ
- SYSTEM: Thông báo hệ thống

Bảng EMAIL_VERIFICATION - Xác thực email Mục đích: Lưu mã OTP để xác thực email khi đăng ký hoặc đổi mật khẩu.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	ID người dùng
code	VARCHAR(10)	NOT NULL	Mã OTP (6 số)

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
expires_at	TIMESTAMP	NOT NULL	Thời điểm hết hạn
is_used	BOOLEAN	NOT NULL, DEFAULT FALSE	Đã sử dụng hay chưa
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian tạo

Bảng 58: Cấu trúc bảng EMAIL_VERIFICATION

Foreign Keys:

- user_id REFERENCES user(id) ON DELETE CASCADE

Indexes:

- PRIMARY KEY: id
- INDEX: user_id
- INDEX: (user_id, code, is_used)
- INDEX: expires_at

Bảng SYSTEM_LOG - Nhật ký hệ thống Mục đích: Ghi lại các hành động quan trọng trong hệ thống để audit và debug.

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh duy nhất
user_id	BIGINT	NULL, FOREIGN KEY → user(id)	ID người thực hiện hành động
action	VARCHAR(100)	NOT NULL	Tên hành động
entity_type	VARCHAR(50)	NOT NULL	Loại đối tượng bị tác động
entity_id	BIGINT	NULL	ID của đối tượng bị tác động
details	JSONB	NULL	Thông tin chi tiết về hành động (JSONB cho indexing tốt hơn)
ip_address	VARCHAR(45)	NULL	Địa chỉ IP của người thực hiện
user_agent	TEXT	NULL	Thông tin trình duyệt/thiết bị
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời gian xảy ra

Bảng 59: Cấu trúc bảng SYSTEM_LOG

Foreign Keys:

- user_id REFERENCES user(id) ON DELETE SET NULL

Indexes:

- PRIMARY KEY: id
- INDEX: user_id
- INDEX: action
- INDEX: entity_type
- INDEX: created_at
- INDEX: (entity_type, entity_id)

Các hành động được log:

- LOGIN, LOGOUT
- CREATE_GROUP, UPDATE_GROUP, DELETE_GROUP
- ADD_KITCHEN_ITEM, UPDATE_KITCHEN_ITEM, DELETE_KITCHEN_ITEM
- CREATE_RECIPE, UPDATE_RECIPE, DELETE_RECIPE
- CREATE_MEAL_PLAN, UPDATE_MEAL_PLAN, DELETE_MEAL_PLAN
- CREATE_SHOPPING_LIST, COMPLETE_SHOPPING_LIST
- FAILED_LOGIN_ATTEMPT
- CHANGE_PASSWORD
- EXPORT_DATA

4.2 Thiết kế API Backend

4.2.1 Danh sách API

Thông tin kết nối

Danh sách mã phản hồi (response code) Quy ước mã:

- **00000**: Mã thành công
- **001xx**: Lỗi xác thực và phân quyền (Authentication)
- **002xx**: Lỗi người dùng (User)
- **003xx**: Lỗi xác thực email (Email Verification)
- **004xx**: Lỗi nhóm (Group)
- **005xx**: Lỗi thực phẩm (Food)
- **006xx**: Lỗi bếp (Kitchen)
- **007xx**: Lỗi mua sắm (Shopping)
- **008xx**: Lỗi kế hoạch bữa ăn (Meal)
- **009xx**: Lỗi công thức nấu ăn (Recipe)

- **010xx**: Lỗi danh mục và đơn vị (Category & Unit)
- **011xx**: Lỗi thông báo (Notification)
- **012xx**: Lỗi validation (Validation)
- **099xx**: Lỗi hệ thống (System)

Danh sách mã chi tiết:

Mã thành công:

Mã	Thông điệp	HTTP Status	Ghi chú
00000	Thành công	200 OK	Yêu cầu được xử lý thành công

Bảng 60: Mã thành công

Lỗi xác thực và phân quyền (001xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00100	Email hoặc mật khẩu không đúng	401 UNAUTHORIZED	Thông tin đăng nhập không chính xác
00101	Token đã hết hạn	401 UNAUTHORIZED	JWT token đã hết thời gian hiệu lực
00102	Token không hợp lệ	401 UNAUTHORIZED	JWT token bị sai định dạng hoặc không xác thực được
00103	Bạn chưa đăng nhập	401 UNAUTHORIZED	Yêu cầu chưa được xác thực
00104	Bạn không có quyền truy cập	403 FORBIDDEN	Người dùng không có quyền thực hiện hành động
00105	Email chưa được xác thực	403 FORBIDDEN	Tài khoản chưa xác thực email
00106	Tài khoản đã bị khóa	403 FORBIDDEN	Tài khoản bị khóa bởi admin

Bảng 61: Lỗi xác thực và phân quyền

Lỗi người dùng (002xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00200	Không tìm thấy người dùng	404 NOT_FOUND	User ID không tồn tại trong hệ thống
00201	Email đã được sử dụng	409 CONFLICT	Email đã được đăng ký bởi người dùng khác
00202	Số điện thoại đã được sử dụng	409 CONFLICT	Số điện thoại đã tồn tại trong hệ thống
00203	Mật khẩu cũ không đúng	400 BAD_REQUEST	Mật khẩu hiện tại nhập sai khi đổi mật khẩu

Mã	Thông điệp	HTTP Status	Ghi chú
00204	Mật khẩu xác nhận không khớp	400 BAD_REQUEST	Mật khẩu và xác nhận mật khẩu không giống nhau
00205	Mật khẩu phải có ít nhất 6 ký tự	400 BAD_REQUEST	Định dạng mật khẩu không hợp lệ

Bảng 62: Lỗi người dùng

Lỗi xác thực email (003xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00300	Mã xác thực đã hết hạn	400 BAD_REQUEST	OTP/Verification code đã quá thời gian sử dụng
00301	Mã xác thực không đúng	400 BAD_REQUEST	OTP/Verification code nhập sai
00302	Email đã được xác thực	400 BAD_REQUEST	Tài khoản đã được xác thực trước đó
00303	Gửi email xác thực thất bại	500 INTER- NAL_SERVER_ERROR	Lỗi hệ thống khi gửi email

Bảng 63: Lỗi xác thực email

Lỗi nhóm (004xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00400	Không tìm thấy nhóm	404 NOT_FOUND	Group ID không tồn tại
00401	Bạn đã ở trong một nhóm	409 CONFLICT	Người dùng đã tham gia nhóm khác
00402	Bạn chưa tham gia nhóm nào	400 BAD_REQUEST	Người dùng chưa thuộc nhóm nào
00403	Chỉ chủ nhóm mới có quyền này	403 FORBIDDEN	Hành động chỉ dành cho chủ nhóm
00404	Không thể xóa chủ nhóm	400 BAD_REQUEST	Không thể xóa người dùng là chủ nhóm
00405	Người dùng không thuộc nhóm này	400 BAD_REQUEST	User không phải thành viên của nhóm
00406	Không tìm thấy lời mời	404 NOT_FOUND	Invitation ID không tồn tại
00407	Lời mời đã hết hạn	400 BAD_REQUEST	Invitation đã quá thời gian hiệu lực
00408	Đã gửi lời mời cho người này	409 CONFLICT	Đã có invitation chưa xử lý cho user

Bảng 64: Lỗi nhóm

Lỗi thực phẩm (005xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00500	Không tìm thấy thực phẩm	404 NOT_FOUND	Food ID không tồn tại trong database
00501	Thực phẩm đã tồn tại trong nhóm	409 CONFLICT	Thực phẩm này đã có trong nhóm

Bảng 65: Lỗi thực phẩm

Lỗi bếp (006xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00600	Không tìm thấy món trong bếp	404 NOT_FOUND	Kitchen item ID không tồn tại
00601	Thực phẩm này đã có trong bếp	409 CONFLICT	Thực phẩm đã được thêm vào bếp
00602	Số lượng không đủ	400 BAD_REQUEST	Số lượng thực phẩm không đủ để sử dụng
00603	Ngày hết hạn không hợp lệ	400 BAD_REQUEST	Expiry date sai định dạng hoặc đã quá khứ

Bảng 66: Lỗi bếp

Lỗi mua sắm (007xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00700	Không tìm thấy danh sách mua sắm	404 NOT_FOUND	Shopping list ID không tồn tại
00701	Không tìm thấy mục trong danh sách	404 NOT_FOUND	Shopping item ID không tồn tại
00702	Thực phẩm đã có trong danh sách	409 CONFLICT	Item đã được thêm vào shopping list
00703	Mục này đã được mua	400 BAD_REQUEST	Item đã có trạng thái "đã mua"
00704	Trạng thái danh sách không hợp lệ	400 BAD_REQUEST	Shopping list status không đúng

Bảng 67: Lỗi mua sắm

Lỗi kế hoạch bữa ăn (008xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00800	Không tìm thấy kế hoạch bữa ăn	404 NOT_FOUND	Meal plan ID không tồn tại

Mã	Thông điệp	HTTP Status	Ghi chú
00801	Ngày kế hoạch không hợp lệ	400 BAD_REQUEST	Meal date sai định dạng

Bảng 68: Lỗi kế hoạch bữa ăn

Lỗi công thức nấu ăn (009xx):

Mã	Thông điệp	HTTP Status	Ghi chú
00900	Không tìm thấy công thức	404 NOT_FOUND	Recipe ID không tồn tại
00901	Bạn không có quyền xem công thức này	403 FORBIDDEN	Recipe là riêng tư hoặc không thuộc nhóm

Bảng 69: Lỗi công thức nấu ăn

Lỗi danh mục và đơn vị (010xx):

Mã	Thông điệp	HTTP Status	Ghi chú
01000	Không tìm thấy danh mục	404 NOT_FOUND	Category ID không tồn tại
01001	Không tìm thấy đơn vị	404 NOT_FOUND	Unit ID không tồn tại

Bảng 70: Lỗi danh mục và đơn vị

Lỗi thông báo (011xx):

Mã	Thông điệp	HTTP Status	Ghi chú
01100	Không tìm thấy thông báo	404 NOT_FOUND	Notification ID không tồn tại

Bảng 71: Lỗi thông báo

Lỗi validation (012xx):

Mã	Thông điệp	HTTP Status	Ghi chú
01200	Dữ liệu không hợp lệ	400 BAD_REQUEST	Request body không đúng format
01201	Thiếu trường bắt buộc	400 BAD_REQUEST	Thiếu required fields trong request
01202	Định dạng không hợp lệ	400 BAD_REQUEST	Dữ liệu không đúng kiểu hoặc format

Bảng 72: Lỗi validation

Lỗi hệ thống (099xx):

Mã	Thông điệp	HTTP Status	Ghi chú
09900	Lỗi hệ thống	500 INTERNAL_SERVER_ERROR	Lỗi không xác định từ server
09901	Dịch vụ tạm thời không khả dụng	503 SERVICE_UNAVAILABLE	Server hoặc service tạm thời không hoạt động
09902	Tải file thất bại	500 INTERNAL_SERVER_ERROR	Lỗi khi upload file lên Cloudinary

Bảng 73: Lỗi hệ thống

Danh sách API

Mã UC	Use Case	Danh Sách API	Mô Tả
UC001	Quản lý danh sách mua sắm	/shopping/lists (POST) /shopping/lists (GET) /shopping/lists/{listId} (GET) /shopping/lists/my-assigned /shopping/lists/{listId} (PUT) /shopping/lists/{listId} (DELETE) /shopping/lists/{listId}/complete /shopping/lists/{listId}/items (POST) /shopping/items/{itemId} (PUT) /shopping/items/{itemId} (DELETE) /shopping/items/{itemId}/purchased	Người dùng tạo, chỉnh sửa, xóa, theo dõi danh sách mua sắm. Giao cho người dùng đi chợ
UC002	Quản lý dự định bữa ăn	/meal-plans (POST) /meal-plans (GET) /meal-plans/{id} (GET) /meal-plans/{id} (PUT) /meal-plans/{id} (DELETE) /meal-plans/daily /meal-plans/range /meal-plans/upcoming	Người dùng lên kế hoạch bữa ăn theo ngày hoặc tuần
UC003	Quản lý thực phẩm trong bếp	/kitchen (POST) /kitchen (GET) /kitchen/{itemId} (GET) /kitchen/expiring /kitchen/{itemId} (PUT) /kitchen/{itemId} (DELETE)	Người dùng nhập, chỉnh sửa, theo dõi thông tin các thực phẩm trong bếp

Mã UC	Use Case	Danh Sách API	Mô Tả
UC004	Quản lý công thức nấu ăn	/recipes (POST) /recipes (GET) /recipes/{recipeId} (GET) /recipes/{recipeId} (PUT) /recipes/{recipeId} (DELETE) /recipes/{recipeId}/ingredients (POST) /recipes/ingredients/{ingredientId} (PUT) /recipes/ingredients/{ingredientId} (DELETE) /recipes/by-ingredient/{foodId}	Người dùng lưu trữ và quản lý các công thức nấu ăn
UC005	Quản lý danh sách các loại thực phẩm	/food (POST) /food (GET) /food/{foodId} (GET) /food/{foodId} (PUT) /food/{foodId} (DELETE) /category (GET) /unit (GET) /admin/food (POST) /admin/food (GET) /admin/food/{foodId} (GET) /admin/food/{foodId} (PUT) /admin/food/{foodId} (DELETE)	Người dùng, Quản trị viên quản lý danh sách thực phẩm, danh mục và đơn vị
UC006	Quản lý nhóm	/group (POST) /group (GET) /group/add-member /group/remove-member /group/leave	Người dùng tạo/xóa nhóm, thêm/xóa thành viên trong nhóm
UC007	Thống kê và báo cáo	/reports/overview /reports/kitchen /reports/shopping /reports/meal /reports/full	Hệ thống tạo báo cáo về tủ lạnh, mua sắm, bữa ăn và thống kê tổng quan
UC008	Quản trị người dùng	/admin/user (GET) /admin/user/statistics /admin/user/{userId} (GET) /admin/user/{userId} (PUT) /admin/user/{userId}/toggle-active /admin/user/{userId} (DELETE)	Quản trị viên quản lý tài khoản người dùng, xem thống kê, cập nhật và xóa người dùng
UC009	Xác thực và phân quyền	/auth/register /auth/login /auth/verify-email /auth/resend-verification /auth/change-password	Người dùng đăng ký, đăng nhập, xác thực email và đổi mật khẩu
UC Bổ Sung			

Mã UC	Use Case	Danh Sách API	Mô Tả
UC10	Gợi ý công thức nấu ăn	/suggestions/recipes /suggestions/recipes/by-food	Hệ thống gợi ý công thức dựa trên thực phẩm có sẵn trong tủ lạnh
UC11	Quản lý thông báo	/notification (GET) /notification/{notificationId}/read /notification/read-all /notification/{notificationId} (DELETE)	Người dùng xem thông báo, đánh dấu đã đọc và xóa thông báo

Bảng 74: Danh sách API của hệ thống

4.2.2 Đặc tả chi tiết API

UC1 - Quản lý danh sách mua sắm **Mô tả chung:** UC1 quản lý toàn bộ quy trình mua sắm của nhóm, bao gồm: tạo danh sách mua sắm, thêm/sửa/xóa các món cần mua, giao việc mua sắm cho thành viên, theo dõi tiến độ mua sắm và đánh dấu hoàn thành. Hệ thống hỗ trợ nhiều danh sách song song với các trạng thái khác nhau (PENDING, IN_PROGRESS, COMPLETED).

API 1.1: Tạo Danh Sách Mua Sắm

Endpoint: POST /shopping/lists

Mô tả: API thực hiện việc tạo một danh sách mua sắm mới cho nhóm. Người dùng có thể tạo danh sách với tên, ghi chú, ngày mua dự kiến và giao cho một thành viên cụ thể trong nhóm để thực hiện việc mua sắm. Các món cần mua sẽ được thêm sau bằng API riêng.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Có	Tên danh sách mua sắm (1-100 ký tự)
assignedToUserId	Long	Có	ID người dùng được giao nhiệm vụ mua sắm (phải là thành viên nhóm)
note	String	Không	Ghi chú cho danh sách
shoppingDate	Date	Không	Ngày dự định đi mua (định dạng: YYYY-MM-DD)

Bảng 75: Tham số Request Body - API Tạo Danh Sách

Response Body (Success - 1000):

```

1 {
2   "code": "1000",

```



```
3  "message": "OK",
4  "data": {
5    "id": 123,
6    "name": "Weekend Shopping List",
7    "groupId": 10,
8    "assignedToUserId": 456,
9    "assignedToEmail": "member@example.com",
10   "assignedToName": "Nguyen Van A",
11   "note": "Buy in the morning",
12   "shoppingDate": "2026-01-05",
13   "status": "PENDING",
14   "items": [],
15   "totalItems": 0,
16   "purchasedItems": 0,
17   "createdBy": 789,
18   "createdAt": "2026-01-03T10:30:00Z",
19   "updatedAt": "2026-01-03T10:30:00Z"
20 }
21 }
```

Mã nguồn 1: Response thành công API 1.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.1.1	Tạo với đầy đủ thông tin: name, assignedToUserId, note, shoppingDate	1000 OK	Danh sách được tạo thành công với status = PENDING
1.1.2	Tạo không có shoppingDate (chỉ name và assignedToUserId)	1000 OK	shoppingDate = null, các trường khác hợp lệ
1.1.3	assignedToUserId không thuộc nhóm (ID = 999)	1003 Parameter value is invalid	assignedToUserId phải là thành viên nhóm
1.1.4	Tên rỗng (name = "")	1003 Parameter value is invalid	Tên không được để trống, frontend nên kiểm tra trước
1.1.5	Thiếu assignedToUserId	1002 Parameter is not enough	assignedToUserId là tham số bắt buộc
1.1.6	Tên vượt quá 100 ký tự	1003 Parameter value is invalid	Tên tối đa 100 ký tự
1.1.7	Không có token xác thực	9998 Token is invalid	Yêu cầu đăng nhập

Bảng 76: Test Cases - API 1.1 Tạo Danh Sách

API 1.2: Lấy Danh Sách Mua Sắm

Endpoint: GET /shopping/lists

Mô tả: API lấy tất cả danh sách mua sắm của nhóm hiện tại. Hỗ trợ lọc theo trạng thái (PENDING/IN_PROGRESS/COMPLETED) và tìm kiếm gần đúng theo tên danh sách.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
status	String	Không	Trạng thái danh sách: PENDING, IN_PROGRESS, COMPLETED
name	String	Không	Tên danh sách (tìm kiếm gần đúng, không phân biệt hoa thường)

Bảng 77: Query Parameters - API Lấy Danh Sách

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "lists": [
6       {
7         "id": 123,
8         "name": "Weekend Shopping List",
9         "groupId": 10,
10        "assignedToUserId": 456,
11        "assignedToEmail": "member@example.com",
12        "assignedToName": "Nguyen Van A",
13        "note": "Buy in the morning",
14        "shoppingDate": "2026-01-05",
15        "status": "PENDING",
16        "items": [],
17        "totalItems": 5,
18        "purchasedItems": 2,
19        "createdBy": 789,
20        "createdAt": "2026-01-03T10:30:00Z",
21        "updatedAt": "2026-01-03T10:30:00Z"
22      }
23    ],
24    "total": 10,
25    "pendingCount": 3,
26    "inProgressCount": 2,
27    "completedCount": 5
28  }
29 }
```

Mã nguồn 2: Response thành công API 1.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.2.1	Không có query parameters	1000 OK	Trả về tất cả danh sách mua sắm của nhóm

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.2.2	?status=PENDING	1000 OK	Chỉ trả về các danh sách chờ xử lý
1.2.3	?status=IN_PROGRESS	1000 OK	Chỉ trả về các danh sách đang thực hiện
1.2.4	?status=COMPLETED	1000 OK	Chỉ trả về các danh sách đã hoàn thành
1.2.5	?name=cuối tuần	1000 OK	Trả về danh sách có tên chứa "cuối tuần"
1.2.6	?status=PENDING&name=sắm	1000 OK	Kết hợp nhiều filter
1.2.7	Người dùng chưa có nhóm	1004 User is not validated	Người dùng chưa tham gia nhóm nào
1.2.8	Nhóm không có danh sách nào	1000 OK	data.lists = [], total = 0

Bảng 78: Test Cases - API 1.2 Lấy Danh Sách

API 1.3: Xem Chi Tiết Danh Sách Mua Sắm

Endpoint: GET /shopping/lists/{listId}

Mô tả: API lấy thông tin chi tiết một danh sách mua sắm cụ thể, bao gồm tất cả các món trong danh sách với đầy đủ thông tin về thực phẩm, đơn vị, số lượng và trạng thái mua.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
listId	Long	Có	ID của danh sách mua sắm

Bảng 79: Path Parameters - API Chi Tiết Danh Sách

Response Body (Success - 1000):

```
{
  "code": "1000",
  "message": "OK",
  "data": {
    "id": 123,
    "name": "Weekend Shopping List",
    "groupId": 10,
    "assignedToUserId": 456,
    "assignedToEmail": "member@example.com",
    "assignedToName": "Nguyen Van A",
    "note": "Buy in the morning",
    "shoppingDate": "2026-01-05",
    "status": "PENDING",
```

```
14  "items": [  
15    {  
16      "id": 1,  
17      "foodId": 100,  
18      "foodName": "Tomato",  
19      "categoryName": "Vegetables",  
20      "unitId": 1,  
21      "unitName": "kg",  
22      "quantity": 2.0,  
23      "actualQuantity": null,  
24      "isPurchased": false  
25    },  
26    {  
27      "id": 2,  
28      "foodId": 101,  
29      "foodName": "Beef",  
30      "categoryName": "Meat",  
31      "unitId": 1,  
32      "unitName": "kg",  
33      "quantity": 1.5,  
34      "actualQuantity": 1.5,  
35      "isPurchased": true  
36    }  
37  ],  
38  "totalItems": 2,  
39  "purchasedItems": 1,  
40  "createdBy": 789,  
41  "createdAt": "2026-01-03T10:30:00Z",  
42  "updatedAt": "2026-01-03T11:45:00Z"  
43 }  
44 }
```

Mã nguồn 3: Response thành công API 1.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.3.1	listId hợp lệ (thuộc nhóm của user)	1000 OK	Trả về đầy đủ thông tin danh sách và các món
1.3.2	listId không tồn tại (99999)	9994 No data or end of list data	Danh sách không tồn tại trong hệ thống
1.3.3	listId của nhóm khác	1009 Action has been done previously by this user	Không có quyền truy cập
1.3.4	listId không hợp lệ ("abc")	1003 Parameter value is invalid	listId phải là số nguyên dương
1.3.5	Danh sách rỗng (không có items)	1000 OK	data.items = [], totalItems = 0

Bảng 80: Test Cases - API 1.3 Chi Tiết Danh Sách

API 1.4: Lấy Danh Sách Được Giao Cho Tôi

Endpoint: GET /shopping/lists/my-assigned

Mô tả: API lấy các danh sách mua sắm được giao cho người dùng hiện tại để thực hiện việc mua sắm. Giúp người dùng dễ dàng xem nhiệm vụ của mình.

Request Headers:

- Authorization: Bearer {access_token}

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "lists": [
6       {
7         "id": 123,
8         "name": "Weekend Shopping List",
9         "status": "PENDING",
10        "createdBy": {
11          "id": 789,
12          "fullName": "Tran Van B"
13        },
14        "totalItems": 5,
15        "purchasedItems": 2,
16        "createdAt": "2026-01-03T10:30:00Z"
17      }
18    ]
19  }
20 }
```

Mã nguồn 4: Response thành công API 1.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.4.1	Người dùng có danh sách được giao	1000 OK	Trả về tất cả danh sách được giao cho user
1.4.2	Người dùng không có danh sách nào	1000 OK	data.lists = []
1.4.3	Chỉ hiển thị PENDING	1000 OK	Chỉ hiển thị danh sách chưa hoàn thành
1.4.4	Người dùng chưa tham gia nhóm	1004 User is not validated	Cần tham gia nhóm trước

Bảng 81: Test Cases - API 1.4 Danh Sách Được Giao

API 1.5: Cập Nhật Danh Sách Mua Sắm

Endpoint: PUT /shopping/lists/{listId}

Mô tả: API cập nhật thông tin danh sách mua sắm bao gồm tên, người được giao, ghi chú, ngày mua và trạng thái. Chỉ thành viên trong nhóm mới có quyền cập nhật.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
listId	Long	Có	ID của danh sách mua sắm

Bảng 82: Path Parameters - API Cập Nhật Danh Sách

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Không	Tên danh sách mới (1-100 ký tự)
assignedToUserId	Long	Không	ID người dùng mới được giao (phải là thành viên)
note	String	Không	Ghi chú mới
shoppingDate	Date	Không	Ngày mua sắm mới (YYYY-MM-DD)
status	String	Không	Trạng thái mới: PENDING, IN_PROGRESS, COMPLETED

Bảng 83: Request Body - API Cập Nhật Danh Sách

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 123,
6     "name": "Next Week Shopping",
7     "assignedToUserId": 456,
8     "assignedToName": "Nguyen Van A",
9     "note": "Buy in the afternoon",
10    "shoppingDate": "2026-01-10",
11    "status": "IN_PROGRESS",
12    "updatedAt": "2026-01-03T14:20:00Z"
13  }
14 }
```

Mã nguồn 5: Response thành công API 1.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.5.1	Cập nhật tên danh sách	1000 OK	Tên được cập nhật thành công
1.5.2	Thay đổi người được giao (assignedUserId = 456)	1000 OK	Phải là thành viên trong nhóm
1.5.3	Cập nhật trạng thái thành COMPLETED	1000 OK	Trạng thái được cập nhật
1.5.4	assignedUserId không thuộc nhóm (999)	1003 Parameter value is invalid	ID không hợp lệ
1.5.5	listId không tồn tại (99999)	9994 No data or end of list data	Danh sách không tồn tại
1.5.6	Cập nhật danh sách của nhóm khác	1009 Action has been done previously by this user	Không có quyền
1.5.7	Cập nhật với tên rỗng	1003 Parameter value is invalid	Tên không được rỗng

Bảng 84: Test Cases - API 1.5 Cập Nhật Danh Sách

API 1.6: Xóa Danh Sách Mua Sắm

Endpoint: DELETE /shopping/lists/{listId}

Mô tả: API xóa một danh sách mua sắm và tất cả các món trong danh sách đó. Thao tác này không thể hoàn tác, nên cần xác nhận từ người dùng trước khi thực hiện.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
listId	Long	Có	ID của danh sách mua sắm cần xóa

Bảng 85: Path Parameters - API Xóa Danh Sách

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 6: Response thành công API 1.6

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.6.1	listId hợp lệ	1000 OK	Danh sách và tất cả items bị xóa
1.6.2	listId không tồn tại (99999)	9994 No data or end of list data	Danh sách không tồn tại
1.6.3	listId của nhóm khác	1009 Action has been done previously by this user	Không có quyền xóa
1.6.4	Xóa danh sách đã COMPLETED	1000 OK	Vẫn có thể xóa, nên có xác nhận
1.6.5	listId không hợp lệ ("abc")	1003 Parameter value is invalid	listId phải là số

Bảng 86: Test Cases - API 1.6 Xóa Danh Sách

API 1.7: Hoàn Thành Danh Sách Mua Sắm

Endpoint: POST /shopping/lists/{listId}/complete

Mô tả: API đánh dấu danh sách mua sắm là đã hoàn thành. Thường được gọi sau khi người được giao đã mua xong tất cả các món hoặc kết thúc chuyến mua sắm.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
listId	Long	Có	ID của danh sách mua sắm

Bảng 87: Path Parameters - API Hoàn Thành Danh Sách

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 123,
6     "name": "Weekend Shopping List",
7     "status": "COMPLETED",
8     "completedAt": "2026-01-03T16:00:00Z"
9   }
10 }
```

Mã nguồn 7: Response thành công API 1.7

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.7.1	listId PENDING hợp lệ	1000 OK	Status chuyển sang COMPLETED
1.7.2	listId đã COMPLETED trước đó	1009 Action has been done previously by this user	Đã hoàn thành rồi
1.7.3	listId không tồn tại (99999)	9994 No data or end of list data	Danh sách không tồn tại
1.7.4	listId của nhóm khác	1009 Action has been done previously by this user	Không có quyền
1.7.5	Bất kỳ thành viên nào cũng có thể hoàn thành	1000 OK	Không giới hạn chỉ người được giao

Bảng 88: Test Cases - API 1.7 Hoàn Thành Danh Sách

API 1.8: Thêm Món Vào Danh Sách

Endpoint: POST /shopping/lists/{listId}/items

Mô tả: API thêm một hoặc nhiều món mới vào danh sách mua sắm đã có. Mỗi món bao gồm thông tin về thực phẩm, số lượng và đơn vị đo.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
listId	Long	Có	ID của danh sách mua sắm

Bảng 89: Path Parameters - API Thêm Món

Request Body:

```

1 {
2   "items": [
3     {
4       "foodId": 100,
5       "quantity": 1.5,
6       "unitId": 1
7     },
8     {
9       "foodId": 101,
10      "quantity": 2.0,
11      "unitId": 2
12    }
  ]
}
```

```
13 ]
14 }
```

Mã nguồn 8: Request Body API 1.8

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
items	Array	Có	Danh sách các món cần thêm
items[].foodId	Long	Có	ID thực phẩm (phải tồn tại)
items[].quantity	BigDecimal	Có	Số lượng (phải > 0)
items[].unitId	Long	Có	ID đơn vị đo (phải tồn tại)

Bảng 90: Request Body Parameters - API Thêm Món

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 123,
6     "name": "Weekend Shopping List",
7     "items": [
8       {
9         "id": 10,
10        "foodId": 100,
11        "foodName": "Tomato",
12        "categoryName": "Vegetables",
13        "unitId": 1,
14        "unitName": "kg",
15        "quantity": 1.5,
16        "actualQuantity": null,
17        "isPurchased": false
18      }
19    ]
20  }
21 }
```

Mã nguồn 9: Response thành công API 1.8

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.8.1	Thêm một món với thông tin hợp lệ	1000 OK	Món được thêm vào danh sách
1.8.2	Thêm nhiều món cùng lúc (array)	1000 OK	Tất cả món được thêm vào
1.8.3	quantity <= 0	1003 Parameter value is invalid	Số lượng phải > 0

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.8.4	foodId không tồn tại (99999)	9994 No data or end of list data	foodId không hợp lệ
1.8.5	listId không tồn tại (99999)	9994 No data or end of list data	Danh sách không tồn tại
1.8.6	Thêm vào danh sách đã COMPLETED	1009 Action has been done previously by this user	Không thể thêm vào DS hoàn thành
1.8.7	Thêm món trùng với món đã có	1000 OK	Tạo item mới hoặc cộng dồn (tùy logic)

Bảng 91: Test Cases - API 1.8 Thêm Món

API 1.9: Cập Nhật Món Trong Danh Sách

Endpoint: PUT /shopping/items/{itemId}

Mô tả: API cập nhật thông tin một món trong danh sách mua sắm, bao gồm số lượng và đơn vị đo. Không thể thay đổi loại thực phẩm, nếu muốn đổi món khác thì xóa và thêm mới.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
itemId	Long	Có	ID của món trong danh sách

Bảng 92: Path Parameters - API Cập Nhật Món

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
quantity	BigDecimal	Có	Số lượng mới (phải > 0)
unitId	Long	Có	ID đơn vị mới (phải tồn tại)

Bảng 93: Request Body - API Cập Nhật Món

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 10,
6     "foodId": 100,
7     "foodName": "Tomato",

```

```
8      "unitId": 1,
9      "unitName": "kg",
10     "quantity": 5.0,
11     "actualQuantity": null,
12     "isPurchased": false
13 }
14 }
```

Mã nguồn 10: Response thành công API 1.9

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.9.1	Cập nhật số lượng hợp lệ	1000 OK	Số lượng được cập nhật thành công
1.9.2	Thay đổi đơn vị (unitId = 3)	1000 OK	Đơn vị được thay đổi
1.9.3	quantity <= 0	1003 Parameter value is invalid	Số lượng phải > 0
1.9.4	unitId không tồn tại (99999)	9994 No data or end of list data	unitId không hợp lệ
1.9.5	itemId không tồn tại (99999)	9994 No data or end of list data	Món không tồn tại
1.9.6	Cập nhật món của nhóm khác	1009 Action has been done previously by this user	Không có quyền

Bảng 94: Test Cases - API 1.9 Cập Nhật Món

API 1.10: Xóa Món Khỏi Danh Sách

Endpoint: DELETE /shopping/items/{itemId}

Mô tả: API xóa một món khỏi danh sách mua sắm. Thao tác này không thể hoàn tác, nên cần xác nhận từ người dùng.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
itemId	Long	Có	ID của món cần xóa

Bảng 95: Path Parameters - API Xóa Món

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
```

```

3  "message": "OK",
4  "data": null
5  }

```

Mã nguồn 11: Response thành công API 1.10

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.10.1	itemId hợp lệ	1000 OK	Món bị xóa khỏi danh sách
1.10.2	itemId không tồn tại (99999)	9994 No data or end of list data	Món không tồn tại
1.10.3	Xóa món của nhóm khác	1009 Action has been done previously by this user	Không có quyền xóa
1.10.4	Xóa món đã purchased (true)	1000 OK	Vẫn có thể xóa
1.10.5	Xóa món khỏi DS đã COMPLETED	1009 Action has been done previously by this user	Không thể xóa

Bảng 96: Test Cases - API 1.10 Xóa Món

API 1.11: Đánh Dấu Món Đã Mua

Endpoint: POST /shopping/items/{itemId}/purchased

Mô tả: API đánh dấu một món trong danh sách là đã mua hoặc chưa mua. Người được giao đi mua sắm sẽ sử dụng API này khi mua được món hoặc muốn bỏ đánh dấu. API này thường được gọi nhiều lần trong quá trình mua sắm.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
itemId	Long	Có	ID của món cần đánh dấu

Bảng 97: Path Parameters - API Đánh Dấu Món

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
isPurchased	Boolean	Có	true = đã mua, false = chưa mua

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
---------	--------------	----------	-------

Bảng 98: Request Body - API Đánh Dấu Món

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 10,
6     "foodId": 100,
7     "foodName": "Tomato",
8     "unitId": 1,
9     "unitName": "kg",
10    "quantity": 2.0,
11    "actualQuantity": 2.0,
12    "isPurchased": true
13  }
14 }
```

Mã nguồn 12: Response thành công API 1.11

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
1.11.1	isPurchased = true	1000 OK	Món được đánh dấu đã mua
1.11.2	isPurchased = false (bỏ đánh dấu)	1000 OK	Món chuyển về chưa mua
1.11.3	itemId không tồn tại (99999)	9994 No data or end of list data	Món không tồn tại
1.11.4	Đánh dấu món của nhóm khác	1009 Action has been done previously by this user	Không có quyền
1.11.5	Đánh dấu món trong DS COMPLETED	1000 OK	Vẫn có thể thay đổi trạng thái
1.11.6	Thiếu tham số isPurchased	1002 Parameter is not enough	isPurchased là bắt buộc
1.11.7	Đánh dấu tất cả món là đã mua	1000 OK	Có thể tự động complete DS (tùy logic)

Bảng 99: Test Cases - API 1.11 Đánh Dấu Món

UC2 - Quản lý kế hoạch bữa ăn **Mô tả chung:** UC2 quản lý toàn bộ quy trình lập kế hoạch bữa ăn của nhóm, bao gồm: tạo kế hoạch cho các bữa ăn (sáng, trưa, tối), xem chi tiết kế hoạch, cập nhật và xóa kế hoạch. Hệ thống cho phép lên kế hoạch theo ngày, theo khoảng thời gian hoặc xem kế hoạch sắp tới. Mỗi kế hoạch bữa ăn được liên kết với một công thức nấu ăn cụ thể.

API 2.1: Tạo Kế Hoạch Bữa Ăn

Endpoint: POST /meal-plans

Mô tả: API thực hiện việc tạo một kế hoạch bữa ăn mới cho nhóm. Người dùng có thể lên kế hoạch cho các bữa sáng, trưa, tối, hoặc ăn vặt trong ngày, mỗi kế hoạch liên kết với một công thức nấu ăn cụ thể và số phần ăn.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
recipeId	Long	Có	ID công thức nấu ăn
mealDate	Date	Có	Ngày của bữa ăn (định dạng ISO: YYYY-MM-DD)
mealName	String	Có	Tên bữa ăn (ví dụ: "sáng", "trưa", "tối")
servings	Integer	Không	Số phần ăn (mặc định: 1, tối thiểu: 1)
note	String	Không	Ghi chú cho bữa ăn

Bảng 100: Tham số Request Body - API Tạo Kế Hoạch Bữa Ăn

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "recipeId": 5,
7     "recipeName": "Pho Bo",
8     "recipeDescription": "Traditional Vietnamese beef noodle soup",
9     "recipeImageUrl": "https://example.com/images/pho.jpg",
10    "mealDate": "2026-01-15",
11    "mealName": "sang",
12    "mealDisplayName": "Bua sang",
13    "servings": 2,
14    "note": "Bua sang dinh duong",
15    "groupId": 10,
16    "createdBy": 123,
17    "createdAt": "2026-01-03T10:30:00Z",
18    "updatedAt": "2026-01-03T10:30:00Z"
19  }
20 }
```

Mã nguồn 13: Response thanh cong API 2.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.1.1	Tạo với đầy đủ thông tin: recipeId=5, mealDate="2026-01-15", mealName="sáng", servings=2, note	1000 OK	Kế hoạch được tạo thành công với công thức đính kèm
2.1.2	Tạo không có servings (recipeId=3, mealDate="2026-01-15", mealName="trưa")	1000 OK	Kế hoạch được tạo với servings = 1 (mặc định)
2.1.3	Tạo với ngày trong quá khứ (mealDate="2025-12-01")	1000 OK	Cho phép tạo kế hoạch cho ngày trong quá khứ (để ghi nhận)
2.1.4	Thiếu mealName	1002 Parameter is not enough	mealName là tham số bắt buộc
2.1.5	Thiếu recipeId	1002 Parameter is not enough	recipeId là tham số bắt buộc
2.1.6	mealDate sai định dạng ("15/01/2026")	1003 Parameter value is invalid	Định dạng ngày không hợp lệ, phải là YYYY-MM-DD
2.1.7	recipeId không tồn tại (99999)	9994 No data or end of list data	recipeId không tồn tại trong hệ thống
2.1.8	Tạo kế hoạch trùng (cùng ngày và mealName)	1000 OK	Cho phép nhiều kế hoạch cùng ngày và mealName

Bảng 101: Test Cases - API 2.1 Tạo Kế Hoạch Bữa Ăn

API 2.2: Lấy Danh Sách Kế Hoạch Bữa Ăn

Endpoint: GET /meal-plans

Mô tả: API lấy tất cả kế hoạch bữa ăn của nhóm hiện tại, sắp xếp theo thời gian. Bao gồm cả kế hoạch trong quá khứ và tương lai.

Request Headers:

- Authorization: Bearer {access_token}

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": [
5     {
6       "id": 1,
7       "recipeId": 5,
8       "recipeName": "Pho Bo",
9       "recipeDescription": "Traditional Vietnamese beef noodle soup",
10      "recipeImageUrl": "https://example.com/images/pho.jpg",
```



```
11     "mealDate": "2026-01-15",
12     "mealName": "sang",
13     "mealDisplayName": "Bua sang",
14     "servings": 2,
15     "note": "Bua sang dinh duong",
16     "groupId": 10,
17     "createdBy": 123,
18     "createdAt": "2026-01-03T10:30:00Z",
19     "updatedAt": "2026-01-03T10:30:00Z"
20 },
21 {
22     "id": 2,
23     "recipeId": 8,
24     "recipeName": "Com Tam",
25     "recipeDescription": "Broken rice with grilled pork",
26     "recipeImageUrl": "https://example.com/images/comtam.jpg",
27     "mealDate": "2026-01-15",
28     "mealName": "trua",
29     "mealDisplayName": "Bua trua",
30     "servings": 3,
31     "note": null,
32     "groupId": 10,
33     "createdBy": 456,
34     "createdAt": "2026-01-03T11:00:00Z",
35     "updatedAt": "2026-01-03T11:00:00Z"
36 }
37 ]
38 }
```

Mã nguồn 14: Response thanh cong API 2.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.2.1	Lấy tất cả kế hoạch	1000 OK	Trả về tất cả kế hoạch bữa ăn của nhóm, sắp xếp theo mealDate tăng dần
2.2.2	Nhóm chưa có kế hoạch nào	1000 OK	data = [] (mảng rỗng)
2.2.3	Người dùng chưa tham gia nhóm	1004 User is not validated	Người dùng chưa tham gia nhóm nào
2.2.4	Token không hợp lệ	9998 Token is invalid	Yêu cầu đăng nhập lại
2.2.5	Kế hoạch bao gồm cả quá khứ và tương lai	1000 OK	Trả về tất cả kế hoạch không giới hạn thời gian

Bảng 102: Test Cases - API 2.2 Lấy Danh Sách Kế Hoạch

API 2.3: Xem Chi Tiết Kế Hoạch Bữa Ăn Endpoint: GET /meal-plans/{id}

Mô tả: API lấy thông tin chi tiết một kế hoạch bữa ăn cụ thể, bao gồm đầy đủ thông tin công thức và nguyên liệu.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
id	Long	Có	ID của kế hoạch bữa ăn

Bảng 103: Path Parameters - API Chi Tiết Kế Hoạch

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "recipeId": 5,
7     "recipeName": "Pho Bo",
8     "recipeDescription": "Traditional Vietnamese beef noodle soup",
9     "recipeImageUrl": "https://example.com/images/pho.jpg",
10    "mealDate": "2026-01-15",
11    "mealName": "sang",
12    "mealDisplayName": "Bua sang",
13    "servings": 2,
14    "note": "Bua sang dinh duong",
15    "groupId": 10,
16    "createdBy": 123,
17    "createdAt": "2026-01-03T10:30:00Z",
18    "updatedAt": "2026-01-03T10:30:00Z"
19  }
20 }
```

Mã nguồn 15: Response thanh cong API 2.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.3.1	id hợp lệ (id=1, thuộc nhóm của user)	1000 OK	Trả về đầy đủ thông tin kế hoạch và công thức
2.3.2	id không tồn tại (99999)	9994 No data or end of list data	Kế hoạch không tồn tại trong hệ thống
2.3.3	id của nhóm khác (id=50)	1009 Action has been done previously by this user	Không có quyền truy cập
2.3.4	id không hợp lệ ("abc")	1003 Parameter value is invalid	id phải là số nguyên dương

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.3.5	Xem chi tiết kế hoạch (có thể lấy thêm thông tin recipe từ API khác)	1000 OK	Response chỉ bao gồm recipeId, recipeName cơ bản

Bảng 104: Test Cases - API 2.3 Chi Tiết Kế Hoạch

API 2.4: Cập Nhật Kế Hoạch Bữa Ăn Endpoint: PUT /meal-plans/{id}

Mô tả: API cập nhật thông tin kế hoạch bữa ăn (ngày, loại bữa, công thức, số phần ăn, ghi chú). Tất cả các trường đều là tùy chọn, chỉ cập nhật các trường được gửi lên.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
id	Long	Có	ID của kế hoạch bữa ăn cần cập nhật

Bảng 105: Path Parameters - API Cập Nhật Kế Hoạch

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
recipeId	Long	Không	ID công thức nấu ăn mới
mealDate	Date	Không	Ngày của bữa ăn mới (định dạng ISO: YYYY-MM-DD)
mealName	String	Không	Tên bữa ăn mới
servings	Integer	Không	Số phần ăn mới (tối thiểu: 1)
note	String	Không	Ghi chú mới

Bảng 106: Tham số Request Body - API Cập Nhật Kế Hoạch

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "recipeId": 10,
7     "recipeName": "Bun Cha",
8     "recipeDescription": "Grilled pork with rice noodles",
9     "recipeImageUrl": "https://example.com/images/buncha.jpg",
10    "mealDate": "2026-01-20",
11    "mealName": "toi",
```

```
12     "mealDisplayName": "Bua toi",
13     "servings": 4,
14     "note": "Ghi chu moi",
15     "groupId": 10,
16     "createdBy": 123,
17     "createdAt": "2026-01-03T10:30:00Z",
18     "updatedAt": "2026-01-03T14:20:00Z"
19 }
20 }
```

Mã nguồn 16: Response thanh cong API 2.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.4.1	Cập nhật ngày bữa ăn (mealDate="2026-01-20", mealName="sáng", recipeId=5)	1000 OK	Ngày được cập nhật thành công
2.4.2	Thay đổi tên bữa ăn (mealName="tối")	1000 OK	Tên bữa ăn được thay đổi
2.4.3	Thay đổi công thức (recipeId=10)	1000 OK	Công thức được cập nhật (recipeId mới phải tồn tại)
2.4.4	Cập nhật số phần ăn (servings=4)	1000 OK	Số phần ăn được cập nhật
2.4.5	Cập nhật với servings không hợp lệ (servings=0)	1003 Parameter value is invalid	servings phải >= 1
2.4.6	Cập nhật với recipeId không tồn tại (recipeId=99999)	9994 No data or end of list data	recipeId không tồn tại trong hệ thống
2.4.7	Cập nhật kế hoạch không tồn tại (id=99999)	9994 No data or end of list data	Kế hoạch không tồn tại
2.4.8	Cập nhật kế hoạch của nhóm khác (id=50)	1009 Action has been done previously by this user	Không có quyền
2.4.9	Chỉ cập nhật note (note="Ghi chú mới")	1000 OK	Chỉ note được cập nhật, các trường khác giữ nguyên

Bảng 107: Test Cases - API 2.4 Cập Nhật Kế Hoạch

API 2.5: Xóa Kế Hoạch Bữa Ăn

Endpoint: DELETE /meal-plans/{id}

Mô tả: API xóa một kế hoạch bữa ăn khỏi lịch trình. Bất kỳ thành viên nào trong nhóm cũng có thể xóa kế hoạch.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
id	Long	Có	ID của kế hoạch bữa ăn cần xóa

Bảng 108: Path Parameters - API Xóa Kế Hoạch

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 17: Response thành công API 2.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.5.1	Xóa kế hoạch hợp lệ (id=1)	1000 OK	Kế hoạch bị xóa thành công
2.5.2	Xóa kế hoạch không tồn tại (id=99999)	9994 No data or end of list data	Kế hoạch không tồn tại
2.5.3	Xóa kế hoạch của nhóm khác (id=50)	1009 Action has been done previously by this user	Không có quyền
2.5.4	Xóa kế hoạch trong quá khứ (id=5)	1000 OK	Vẫn có thể xóa kế hoạch trong quá khứ
2.5.5	Xóa kế hoạch do thành viên khác tạo (id=8)	1000 OK	Bất kỳ thành viên nào cũng có thể xóa

Bảng 109: Test Cases - API 2.5 Xóa Kế Hoạch

API 2.6: Lấy Kế Hoạch Theo Ngày

Endpoint: GET /meal-plans/daily

Mô tả: API lấy tất cả các bữa ăn (sáng, trưa, tối) trong một ngày cụ thể, được nhóm theo loại bữa ăn.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
date	String	Có	Ngày cần xem (định dạng ISO: YYYY-MM-DD)

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
---------	--------------	----------	-------

Bảng 110: Query Parameters - API Lấy Kế Hoạch Theo Ngày

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "date": "2026-01-15",
6     "breakfast": [
7       {
8         "id": 1,
9         "recipeId": 5,
10        "recipeName": "Pho Bo",
11        "recipeImageUrl": "https://example.com/images/pho.jpg",
12        "mealName": "sang",
13        "servings": 2,
14        "note": "Bua sang dinh duong"
15      }
16    ],
17    "lunch": [
18      {
19        "id": 2,
20        "recipeId": 8,
21        "recipeName": "Com Tam",
22        "recipeImageUrl": "https://example.com/images/comtam.jpg",
23        "mealName": "trua",
24        "servings": 3,
25        "note": null
26      }
27    ],
28    "dinner": [
29      {
30        "id": 3,
31        "recipeId": 10,
32        "recipeName": "Bun Cha",
33        "recipeImageUrl": "https://example.com/images/buncha.jpg",
34        "mealName": "toi",
35        "servings": 4,
36        "note": "Bua toi gia dinh"
37      }
38    ]
39  }
40 }
```

Mã nguồn 18: Response thanh cong API 2.6

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.6.1	?date=2026-01-15 (ngày có đầy đủ bữa ăn)	1000 OK	Trả về tất cả bữa ăn trong ngày
2.6.2	?date=2026-02-01 (ngày chưa có bữa nào)	1000 OK	breakfast, lunch, dinner đều là mảng rỗng []
2.6.3	?date=2026-01-16 (chỉ có một vài bữa)	1000 OK	breakfast và dinner có dữ liệu, lunch là []
2.6.4	Thiếu tham số date	1002 Parameter is not enough	date là tham số bắt buộc
2.6.5	Định dạng date không hợp lệ (?date=15/01/2026)	1003 Parameter value is invalid	Định dạng không hợp lệ, phải là YYYY-MM-DD
2.6.6	?date=2026-01-03 (ngày hôm nay)	1000 OK	Trả về kế hoạch hôm nay
2.6.7	?date=2025-12-25 (ngày trong quá khứ)	1000 OK	Cho phép xem kế hoạch quá khứ

Bảng 111: Test Cases - API 2.6 Lấy Kế Hoạch Theo Ngày

API 2.7: Lấy Kế Hoạch Theo Khoảng Thời Gian

Endpoint: GET /meal-plans/range

Mô tả: API lấy tất cả kế hoạch bữa ăn trong một khoảng thời gian cụ thể (ví dụ: một tuần, một tháng). Hữu ích cho việc lập kế hoạch dài hạn.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
startDate	String	Có	Ngày bắt đầu (định dạng ISO: YYYY-MM-DD)
endDate	String	Có	Ngày kết thúc (định dạng ISO: YYYY-MM-DD)

Bảng 112: Query Parameters - API Lấy Kế Hoạch Theo Khoảng Thời Gian

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": [
5     {
6       "id": 1,
7       "recipeId": 5,
8       "recipeName": "Pho Bo",
```

```
9      "recipeDescription": "Traditional Vietnamese beef noodle soup",
10     "recipeImageUrl": "https://example.com/images/pho.jpg",
11     "mealDate": "2026-01-15",
12     "mealName": "sang",
13     "mealDisplayName": "Bua sang",
14     "servings": 2,
15     "note": "Bua sang dinh duong",
16     "groupId": 10,
17     "createdBy": 123,
18     "createdAt": "2026-01-03T10:30:00Z",
19     "updatedAt": "2026-01-03T10:30:00Z"
20   },
21   {
22     "id": 2,
23     "recipeId": 8,
24     "recipeName": "Com Tam",
25     "recipeDescription": "Broken rice with grilled pork",
26     "recipeImageUrl": "https://example.com/images/comtam.jpg",
27     "mealDate": "2026-01-16",
28     "mealName": "trua",
29     "mealDisplayName": "Bua trua",
30     "servings": 3,
31     "note": null,
32     "groupId": 10,
33     "createdBy": 456,
34     "createdAt": "2026-01-03T11:00:00Z",
35     "updatedAt": "2026-01-03T11:00:00Z"
36   }
37 ]
38 }
```

Mã nguồn 19: Response thanh cong API 2.7

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.7.1	?startDate=2026-01-13&endDate=2026-01-19 (một tuần)	1000 OK	Trả về tất cả kế hoạch trong tuần, sắp xếp theo ngày và mealName
2.7.2	?startDate=2026-01-01&endDate=2026-01-31 (một tháng)	1000 OK	Trả về tất cả kế hoạch trong tháng
2.7.3	Khoảng thời gian không có kế hoạch (?startDate=2026-06-01&endDate=2026-06-07)	1000 OK	data = [] (mảng rỗng)
2.7.4	Thiếu startDate hoặc endDate (?startDate=2026-01-01)	1002 Parameter is not enough	Cả hai tham số đều bắt buộc

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.7.5	endDate < startDate (?startDate=2026-01-20&endDate=2026-01-10)	1003 Parameter value is invalid	Khoảng thời gian không hợp lệ
2.7.6	Định dạng date không hợp lệ (?startDate=01/01/2026)	1003 Parameter value is invalid	Định dạng không hợp lệ
2.7.7	Khoảng thời gian quá dài (?startDate=2026-01-01&endDate=2026-12-31)	1000 OK hoặc 1003	Có thể giới hạn tối đa 90 ngày để tránh quá tải

Bảng 113: Test Cases - API 2.7 Lấy Kế Hoạch Theo Khoảng Thời Gian

API 2.8: Lấy Kế Hoạch Sắp Tới

Endpoint: GET /meal-plans/upcoming

Mô tả: API lấy các kế hoạch bữa ăn sắp tới trong X ngày tiếp theo, kể từ ngày hiện tại. Hữu ích cho việc xem nhanh kế hoạch gần nhất.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
days	Integer	Không	Số ngày kể từ hôm nay (mặc định: 7)

Bảng 114: Query Parameters - API Lấy Kế Hoạch Sắp Tới

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": [
5     {
6       "id": 1,
7       "recipeId": 5,
8       "recipeName": "Pho Bo",
9       "recipeDescription": "Traditional Vietnamese beef noodle soup",
10      "recipeImageUrl": "https://example.com/images/pho.jpg",
11      "mealDate": "2026-01-05",
12      "mealName": "sang",
13      "mealDisplayName": "Bua sang",
14      "servings": 2,
15      "note": "Bua sang dinh duong",
16      "groupId": 10,
17      "createdBy": 123,
```

```
18     "createdAt": "2026-01-03T10:30:00Z",
19     "updatedAt": "2026-01-03T10:30:00Z"
20 },
21 {
22     "id": 2,
23     "recipeId": 8,
24     "recipeName": "Com Tam",
25     "recipeDescription": "Broken rice with grilled pork",
26     "recipeImageUrl": "https://example.com/images/comtam.jpg",
27     "mealDate": "2026-01-06",
28     "mealName": "trua",
29     "mealDisplayName": "Bua trua",
30     "servings": 3,
31     "note": null,
32     "groupId": 10,
33     "createdBy": 456,
34     "createdAt": "2026-01-03T11:00:00Z",
35     "updatedAt": "2026-01-03T11:00:00Z"
36 }
37 ]
38 }
```

Mã nguồn 20: Response thành công API 2.8

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
2.8.1	Không có query parameter	1000 OK	Trả về kế hoạch từ hôm nay đến 7 ngày sau (mặc định)
2.8.2	?days=3	1000 OK	Trả về kế hoạch 3 ngày tới
2.8.3	?days=30	1000 OK	Trả về kế hoạch 1 tháng tới
2.8.4	Không có kế hoạch nào trong thời gian tới	1000 OK	data = [] (mảng rỗng)
2.8.5	days <= 0 (?days=0 hoặc ?days=-5)	1003 Parameter value is invalid	days phải > 0
2.8.6	days không phải số (?days=abc)	1003 Parameter value is invalid	days phải là số nguyên dương
2.8.7	days quá lớn (?days=1000)	1000 OK hoặc 1003	Có thể giới hạn days tối đa (ví dụ 365 ngày)

Bảng 115: Test Cases - API 2.8 Lấy Kế Hoạch Sắp Tới

UC3 - Quản lý thực phẩm trong bếp **Mô tả chung:** UC3 quản lý toàn bộ quy trình theo dõi thực phẩm trong bếp/kho của nhóm, bao gồm: thêm thực phẩm mới vào bếp, theo dõi số lượng và hạn sử dụng, cập nhật thông tin khi sử dụng, và xóa thực phẩm đã hết. Hệ thống tự động tính toán ngày hết hạn, cảnh báo thực phẩm sắp hết hạn và phân loại trạng thái (FRESH, EXPIRING_SOON, EXPIRED) để giúp nhóm quản lý hiệu quả và giảm lãng phí thực phẩm.

API 3.1: Thêm Thực Phẩm Vào Bếp

Endpoint: POST /kitchen

Mô tả: API thực hiện việc thêm thực phẩm mới vào bếp/kho của nhóm. Người dùng có thể ghi nhận số lượng, hạn sử dụng (tự động tính từ số ngày), và ghi chú. Hệ thống tự động tính ngày hết hạn và phân loại trạng thái.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID thực phẩm
quantity	BigDecimal	Có	Số lượng (phải lớn hơn 0)
unitId	Long	Có	ID đơn vị đo
useWithin	Integer	Không	Số ngày sử dụng (tối thiểu: 1). expiryDate = ngày hiện tại + useWithin
note	String	Không	Ghi chú (giới hạn 500 ký tự)

Bảng 116: Tham số Request Body - API Thêm Thực Phẩm Vào Bếp

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "foodId": 1,
7     "foodName": "Tomato",
8     "categoryName": "Vegetables",
9     "unitId": 1,
10    "unitName": "kg",
11    "foodImageUrl": "https://example.com/images/tomato.jpg",
12    "quantity": 2.5,
13    "useWithin": 30,
14    "expiryDate": "2026-02-02",
15    "daysUntilExpiry": 30,
16    "expiryStatus": "FRESH",
17    "note": "Moi mua hom nay",
18    "groupId": 10,
19    "createdAt": "2026-01-03T10:30:00Z",
20    "updatedAt": "2026-01-03T10:30:00Z"
21  }
22 }
```

Mã nguồn 21: Response thanh cong API 3.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.1.1	Thêm với đầy đủ thông tin: foodId=1, quantity=2.5, unitId=1, useWithin=30, note	1000 OK	Thực phẩm được thêm thành công, expiryDate tự động tính
3.1.2	Thêm không có useWithin (foodId=5, quantity=1, unitId=2)	1000 OK	useWithin = null, expiryDate = null (thực phẩm không hết hạn)
3.1.3	Thêm chỉ với thông tin bắt buộc (foodId=3, quantity=0.5, unitId=1)	1000 OK	useWithin và note có thể null
3.1.4	Thêm với quantity <= 0 (quantity=0)	1003 Parameter value is invalid	Số lượng phải lớn hơn 0
3.1.5	Thiếu quantity (chỉ có foodId, unitId)	1002 Parameter is not enough	quantity là tham số bắt buộc
3.1.6	foodId không tồn tại (foodId=99999)	9994 No data or end of list data	foodId không tồn tại trong hệ thống
3.1.7	unitId không tồn tại (unitId=99999)	9994 No data or end of list data	unitId không hợp lệ
3.1.8	useWithin không hợp lệ (useWithin=0)	1003 Parameter value is invalid	useWithin phải >= 1 nếu có giá trị
3.1.9	Thêm thực phẩm trùng (cùng foodId đã có)	1000 OK	Tạo item mới (cho phép trùng vì mỗi lần mua có hạn khác nhau)

Bảng 117: Test Cases - API 3.1 Thêm Thực Phẩm

API 3.2: Lấy Danh Sách Thực Phẩm Trong Bếp

Endpoint: GET /kitchen

Mô tả: API lấy danh sách tất cả thực phẩm trong bếp của nhóm. Hỗ trợ lọc theo thực phẩm cụ thể, tìm kiếm theo tên, và lọc theo ngày hết hạn. Dữ liệu được sắp xếp ưu tiên thực phẩm sắp hết hạn.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Không	Lọc theo ID thực phẩm cụ thể
foodName	String	Không	Tìm kiếm theo tên thực phẩm (không phân biệt hoa thường)

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
expiryBefore	String	Không	Lọc thực phẩm hết hạn trước ngày này (định dạng ISO: YYYY-MM-DD)

Bảng 118: Query Parameters - API Lấy Danh Sách Thực Phẩm

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "items": [
6       {
7         "id": 1,
8         "foodId": 1,
9         "foodName": "Tomato",
10        "categoryName": "Vegetables",
11        "unitId": 1,
12        "unitName": "kg",
13        "foodImageUrl": "https://example.com/images/tomato.jpg",
14        "quantity": 2.5,
15        "useWithin": 30,
16        "expiryDate": "2026-02-02",
17        "daysUntilExpiry": 30,
18        "expiryStatus": "FRESH",
19        "note": "Moi mua hom nay",
20        "groupId": 10,
21        "createdAt": "2026-01-03T10:30:00Z",
22        "updatedAt": "2026-01-03T10:30:00Z"
23      },
24      {
25        "id": 2,
26        "foodId": 5,
27        "foodName": "Beef",
28        "categoryName": "Meat",
29        "unitId": 1,
30        "unitName": "kg",
31        "foodImageUrl": "https://example.com/images/beef.jpg",
32        "quantity": 1.0,
33        "useWithin": 5,
34        "expiryDate": "2026-01-08",
35        "daysUntilExpiry": 5,
36        "expiryStatus": "EXPIRING_SOON",
37        "note": null,
38        "groupId": 10,
39        "createdAt": "2026-01-02T14:20:00Z",
40        "updatedAt": "2026-01-02T14:20:00Z"
41      }
42    ],
43    "total": 15,
```

```
44     "expiringCount": 3,  
45     "expiredCount": 1  
46 }  
47 }
```

Mã nguồn 22: Response thành công API 3.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.2.1	Không có query parameters	1000 OK	Trả về tất cả thực phẩm, sắp xếp theo expiryDate (gần nhất trước)
3.2.2	Bếp không có thực phẩm nào	1000 OK	data.items = [], total = 0
3.2.3	Lọc theo foodId cụ thể (?foodId=5)	1000 OK	Chỉ trả về các items có foodId = 5
3.2.4	Tìm kiếm theo tên (?foodName=cà chua)	1000 OK	Trả về items có tên chứa "cà chua"
3.2.5	Lọc hết hạn trước ngày cụ thể (?expiryBefore=2026-01-10)	1000 OK	Trả về items có expiryDate < 2026-01-10
3.2.6	Kết hợp nhiều filter (?foodName=thịt&expiryBefore=2026-02-01)	1000 OK	Trả về thịt sắp hết hạn trước ngày chỉ định
3.2.7	Người dùng chưa tham gia nhóm	1004 User is not validated	Cần tham gia nhóm trước
3.2.8	expiryBefore sai định dạng (?expiryBefore=01/01/2026)	1003 Parameter value is invalid	Định dạng phải là YYYY-MM-DD

Bảng 119: Test Cases - API 3.2 Lấy Danh Sách Thực Phẩm

API 3.3: Xem Chi Tiết Thực Phẩm Trong Bếp

Endpoint: GET /kitchen/{itemId}

Mô tả: API lấy thông tin chi tiết một thực phẩm cụ thể trong bếp, bao gồm đầy đủ thông tin về số lượng, hạn sử dụng, trạng thái và các thông tin liên quan.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
itemId	Long	Có	ID của item trong bếp

Bảng 120: Path Parameters - API Chi Tiết Thực Phẩm

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "foodId": 1,
7     "foodName": "Tomato",
8     "categoryName": "Vegetables",
9     "unitId": 1,
10    "unitName": "kg",
11    "foodImageUrl": "https://example.com/images/tomato.jpg",
12    "quantity": 2.5,
13    "useWithin": 30,
14    "expiryDate": "2026-02-02",
15    "daysUntilExpiry": 30,
16    "expiryStatus": "FRESH",
17    "note": "Moi mua hom nay",
18    "groupId": 10,
19    "createdAt": "2026-01-03T10:30:00Z",
20    "updatedAt": "2026-01-03T10:30:00Z"
21  }
22 }

```

Mã nguồn 23: Response thanh cong API 3.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.3.1	itemId hợp lệ (itemId=1, thuộc nhóm của user)	1000 OK	Trả về đầy đủ thông tin item
3.3.2	itemId không tồn tại (99999)	9994 No data or end of list data	Item không tồn tại trong hệ thống
3.3.3	itemId của nhóm khác (itemId=50)	1009 Action has been done previously by this user	Không có quyền truy cập
3.3.4	Xem chi tiết item đã hết hạn (itemId=10)	1000 OK	expiryStatus = EXPIRED, daysUntilExpiry âm
3.3.5	itemId không hợp lệ ("abc")	1003 Parameter value is invalid	itemId phải là số nguyên dương
3.3.6	Xem item không có expiryDate (itemId=5)	1000 OK	expiryDate = null, daysUntilExpiry = null, expiryStatus = null

Bảng 121: Test Cases - API 3.3 Chi Tiết Thực Phẩm

API 3.4: Lấy Thực Phẩm Sắp Hết Hạn

Endpoint: GET /kitchen/expiring

Mô tả: API lấy danh sách thực phẩm sắp hết hạn trong X ngày tới. Rất hữu ích để nhắc nhở người dùng sử dụng thực phẩm trước khi hết hạn, giảm lãng phí. API này được sử dụng cho tính năng thông báo hàng ngày.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
days	Integer	Không	Số ngày tính từ hôm nay (mặc định: 3)

Bảng 122: Query Parameters - API Lấy Thực Phẩm Sắp Hết Hạn

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": [
5     {
6       "id": 2,
7       "foodId": 5,
8       "foodName": "Beef",
9       "categoryName": "Meat",
10      "unitId": 1,
11      "unitName": "kg",
12      "foodImageUrl": "https://example.com/images/beef.jpg",
13      "quantity": 1.0,
14      "useWithin": 5,
15      "expiryDate": "2026-01-08",
16      "daysUntilExpiry": 5,
17      "expiryStatus": "EXPIRING_SOON",
18      "note": null,
19      "groupId": 10,
20      "createdAt": "2026-01-02T14:20:00Z",
21      "updatedAt": "2026-01-02T14:20:00Z"
22    },
23    {
24      "id": 7,
25      "foodId": 12,
26      "foodName": "Milk",
27      "categoryName": "Dairy",
28      "unitId": 3,
29      "unitName": "liter",
30      "foodImageUrl": "https://example.com/images/milk.jpg",
31      "quantity": 0.5,
32      "useWithin": 2,
33      "expiryDate": "2026-01-05",
34      "daysUntilExpiry": 2,
35      "expiryStatus": "EXPIRING_SOON",
36      "note": "Can su dung ngay",
```



```

37     "groupId": 10,
38     "createdAt": "2026-01-01T08:15:00Z",
39     "updatedAt": "2026-01-01T08:15:00Z"
40   }
41 ]
42 }
```

Mã nguồn 24: Response thành công API 3.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.4.1	Không có query parameter	1000 OK	Trả về items hết hạn trong 3 ngày tới (mặc định), bao gồm cả đã hết hạn
3.4.2	?days=7	1000 OK	Trả về items hết hạn trong 7 ngày tới
3.4.3	?days=1 (hôm nay)	1000 OK	Chỉ trả về items hết hạn trong ngày hôm nay
3.4.4	Không có thực phẩm nào sắp hết hạn	1000 OK	data = [] (mảng rỗng)
3.4.5	days <= 0 (?days=0 hoặc ?days=-5)	1003 Parameter value is invalid	days phải > 0
3.4.6	days không phải số (?days=abc)	1003 Parameter value is invalid	days phải là số nguyên dương
3.4.7	days quá lớn (?days=365)	1000 OK	Cho phép nhưng nên giới hạn tối đa 30 ngày

Bảng 123: Test Cases - API 3.4 Lấy Thực Phẩm Sắp Hết Hạn

API 3.5: Cập Nhật Thực Phẩm Trong Bếp

Endpoint: PUT /kitchen/{itemId}

Mô tả: API cập nhật thông tin thực phẩm trong bếp (số lượng, đơn vị, hạn sử dụng, ghi chú). Thường được sử dụng khi sử dụng một phần thực phẩm (giảm số lượng) hoặc cần điều chỉnh thông tin. Tất cả các trường đều là tùy chọn.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
itemId	Long	Có	ID của item trong bếp cần cập nhật

Bảng 124: Path Parameters - API Cập Nhật Thực Phẩm

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
quantity	BigDecimal	Không	Số lượng mới (phải > 0)
unitId	Long	Không	ID đơn vị mới
useWithin	Integer	Không	Số ngày sử dụng mới (tối thiểu: 1)
note	String	Không	Ghi chú mới
foodId	Long	Không	Đổi sang thực phẩm khác (ít dùng)

Bảng 125: Tham số Request Body - API Cập Nhật Thực Phẩm

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "foodId": 1,
7     "foodName": "Tomato",
8     "categoryName": "Vegetables",
9     "unitId": 2,
10    "unitName": "piece",
11    "foodImageUrl": "https://example.com/images/tomato.jpg",
12    "quantity": 1.5,
13    "useWithin": 5,
14    "expiryDate": "2026-01-08",
15    "daysUntilExpiry": 5,
16    "expiryStatus": "EXPIRING_SOON",
17    "note": "Đã dùng một nửa",
18    "groupId": 10,
19    "createdAt": "2026-01-03T10:30:00Z",
20    "updatedAt": "2026-01-03T15:45:00Z"
21  }
22 }
```

Mã nguồn 25: Response thành công API 3.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.5.1	Cập nhật số lượng (quantity=1.5, unitId=1)	1000 OK	Số lượng được cập nhật (dùng khi sử dụng thực phẩm)
3.5.2	Thay đổi useWithin (useWithin=5)	1000 OK	Hạn sử dụng được cập nhật, expiryDate tự động tính lại

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.5.3	Cập nhật đầy đủ (quantity=0.8, unitId=2, useWithin=5, note)	1000 OK	Tất cả thông tin được cập nhật
3.5.4	Cập nhật với quantity <= 0 (quantity=0)	1003 Parameter value is invalid	Số lượng phải > 0. Nếu hết nên xóa item
3.5.5	Cập nhật với unitId không tồn tại (unitId=99999)	9994 No data or end of list data	unitId không hợp lệ
3.5.6	Cập nhật item không tồn tại (itemId=99999)	9994 No data or end of list data	Item không tồn tại
3.5.7	Cập nhật item của nhóm khác (itemId=50)	1009 Action has been done previously by this user	Không có quyền
3.5.8	Chỉ cập nhật note (note="Ghi chú mới")	1000 OK	Chỉ note được cập nhật, các trường khác giữ nguyên
3.5.9	Thay đổi foodId (foodId=10)	1000 OK	Có thể chuyển item sang thực phẩm khác (ít dùng)

Bảng 126: Test Cases - API 3.5 Cập Nhật Thực Phẩm

API 3.6: Xóa Thực Phẩm Khỏi Bếp

Endpoint: DELETE /kitchen/{itemId}

Mô tả: API xóa một thực phẩm khỏi bếp. Thường dùng khi đã sử dụng hết thực phẩm hoặc thực phẩm đã hỏng/hết hạn. Bất kỳ thành viên nào trong nhóm cũng có thể xóa.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
itemId	Long	Có	ID của item cần xóa khỏi bếp

Bảng 127: Path Parameters - API Xóa Thực Phẩm

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 26: Response thành công API 3.6

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
3.6.1	Xóa item hợp lệ (itemId=1)	1000 OK	Item bị xóa khỏi bếp thành công
3.6.2	Xóa item không tồn tại (itemId=99999)	9994 No data or end of list data	Item không tồn tại
3.6.3	Xóa item của nhóm khác (itemId=50)	1009 Action has been done previously by this user	Không có quyền
3.6.4	Xóa item đã hết hạn (itemId=10)	1000 OK	Vẫn có thể xóa item đã hết hạn
3.6.5	Xóa item do thành viên khác thêm (itemId=8)	1000 OK	Bất kỳ thành viên nào cũng có thể xóa
3.6.6	Xóa với itemId không hợp lệ ("abc")	1003 Parameter value is invalid	itemId phải là số nguyên dương

Bảng 128: Test Cases - API 3.6 Xóa Thực Phẩm

UC4 - Quản lý công thức nấu ăn **Mô tả chung:** UC4 quản lý toàn bộ quy trình tạo và quản lý công thức nấu ăn của nhóm, bao gồm: tạo công thức mới với thông tin chi tiết và danh sách nguyên liệu, xem và tìm kiếm công thức, cập nhật thông tin công thức, thêm/sửa/xóa nguyên liệu trong công thức, và xóa công thức. Hệ thống hỗ trợ tải lên ảnh món ăn, nội dung HTML chi tiết và tự động tính toán khả năng thực hiện công thức dựa trên nguyên liệu có sẵn trong bếp.

API 4.1: Tạo Công Thức Nấu Ăn

Endpoint: POST /recipes

Mô tả: API thực hiện việc tạo một công thức nấu ăn mới cho nhóm. Người dùng có thể tạo công thức với thông tin chi tiết, tải lên ảnh món ăn và danh sách nguyên liệu. Hệ thống tự động tính toán số nguyên liệu khớp với thực phẩm có sẵn trong bếp.

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Có	Tên công thức (tối đa 200 ký tự)
description	String	Có	Mô tả ngắn gọn về món ăn
htmlContent	String	Có	Nội dung HTML của công thức (bao gồm hướng dẫn nấu ăn chi tiết)

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
image	File	Không	Ảnh món ăn (jpg, jpeg, png, gif. Tối đa 5MB)
ingredients	JSON Array	Không	Danh sách nguyên liệu (có thể thêm sau)
ingredients[].foodId	Long	Có	ID thực phẩm
ingredients[].quantity	BigDecimal	Có	Số lượng (phải > 0)
ingredients[].unitId	Long	Có	ID đơn vị đo
ingredients[].isMainIngredient	Boolean	Không	Là nguyên liệu chính hay không (mặc định: false)
ingredients[].note	String	Không	Ghi chú cho nguyên liệu

Bảng 129: Tham số Request Body - API Tạo Công Thức Nấu Ăn

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Pho bo Ha Noi",
7     "description": "Mon pho truyen thong cua Viet Nam",
8     "htmlContent": "<h2>Nguyen lieu</h2><ul><li>Xuong bo: 1kg</li>..."
9   },
10  "imageUrl": "https://example.com/images/pho.jpg",
11  "groupId": 10,
12  "isSystem": false,
13  "createdBy": 123,
14  "ingredients": [
15    {
16      "id": 1,
17      "foodId": 1,
18      "foodName": "Thit bo",
19      "foodImageUrl": "https://example.com/images/beef.jpg",
20      "quantity": 500,
21      "unitId": 1,
22      "unitName": "gram",
23      "isMainIngredient": true,
24      "note": "Thit bo tuoi",
25      "isAvailable": true
26    },
27    {
28      "id": 2,
29      "foodId": 2,
30      "foodName": "Banh pho",
31      "foodImageUrl": "https://example.com/images/noodles.jpg",
32      "quantity": 300,
33      "unitId": 1,
34      "unitName": "gram",
```

```

34         "isMainIngredient": true,
35         "note": "Banh pho tuoi",
36         "isAvailable": false
37     }
38 ],
39 "matchedIngredients": 1,
40 "totalIngredients": 2,
41 "createdAt": "2026-01-03T10:30:00Z",
42 "updatedAt": "2026-01-03T10:30:00Z"
43 }
44 }
```

Mã nguồn 27: Response thành công API 4.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.1.1	Tạo với đầy đủ thông tin: name, description, htmlContent, image, ingredients	1000 OK	Công thức được tạo thành công với ảnh và nguyên liệu
4.1.2	Tạo không có ảnh (name, description, htmlContent)	1000 OK	Công thức được tạo mà không có ảnh
4.1.3	Tạo không có nguyên liệu (ingredients=[])	1000 OK	Có thể thêm nguyên liệu sau
4.1.4	Thiếu description	1002 Parameter is not enough	description là tham số bắt buộc
4.1.5	Thiếu htmlContent	1002 Parameter is not enough	htmlContent là tham số bắt buộc
4.1.6	Tên quá dài (> 200 ký tự)	1003 Parameter value is invalid	Tên tối đa 200 ký tự
4.1.7	ingredients có quantity <= 0	1003 Parameter value is invalid	quantity phải > 0
4.1.8	ingredients có foodId không tồn tại (99999)	9994 No data or end of list data	foodId không tồn tại trong hệ thống
4.1.9	Ảnh không hợp lệ (upload .txt)	1003 Parameter value is invalid	File phải là ảnh (jpg, jpeg, png, gif)

Bảng 130: Test Cases - API 4.1 Tạo Công Thức

API 4.2: Lấy Danh Sách Công Thức

Endpoint: GET /recipes

Mô tả: API lấy danh sách tất cả công thức của nhóm, bao gồm cả công thức do thành viên tạo và công thức hệ thống. Hỗ trợ tìm kiếm theo tên công thức.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Không	Tìm kiếm theo tên công thức (gần đúng, không phân biệt hoa thường)

Bảng 131: Query Parameters - API Lấy Danh Sách Công Thức

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": [
5     {
6       "id": 1,
7       "name": "Pho bo Ha Noi",
8       "description": "Mon pho truyen thong cua Viet Nam",
9       "htmlContent": "<h2>Nguyen lieu</h2><ul><li>Xuong bo: 1kg</li>
10      >...",
11       "imageUrl": "https://example.com/images/pho.jpg",
12       "groupId": 10,
13       "isSystem": false,
14       "createdBy": 123,
15       "ingredients": [
16         {
17           "id": 1,
18           "foodId": 1,
19           "foodName": "Thit bo",
20           "foodImageUrl": "https://example.com/images/beef.jpg",
21           "quantity": 500,
22           "unitId": 1,
23           "unitName": "gram",
24           "isMainIngredient": true,
25           "note": "Thit bo tuoi",
26           "isAvailable": true
27         }
28       ],
29       "matchedIngredients": 1,
30       "totalIngredients": 2,
31       "createdAt": "2026-01-03T10:30:00Z",
32       "updatedAt": "2026-01-03T10:30:00Z"
33     }
34   ]
35 }
```

Mã nguồn 28: Response thanh cong API 4.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.2.1	Không có query parameters	1000 OK	Trả về tất cả công thức của nhóm
4.2.2	Nhóm chưa có công thức nào	1000 OK	data = [] (mảng rỗng)
4.2.3	Tìm kiếm theo tên (?name=phở)	1000 OK	Trả về các công thức có tên chứa "phở"
4.2.4	Tìm kiếm không có kết quả (?name=abcxyz123)	1000 OK	data = [] (mảng rỗng)
4.2.5	Người dùng chưa tham gia nhóm	1004 User is not validated	Cần tham gia nhóm trước
4.2.6	Token không hợp lệ	9998 Token is invalid	Yêu cầu đăng nhập lại

Bảng 132: Test Cases - API 4.2 Lấy Danh Sách Công Thức

API 4.3: Xem Chi Tiết Công Thức

Endpoint: GET /recipes/{recipeId}

Mô tả: API lấy thông tin chi tiết một công thức, bao gồm hướng dẫn nấu đầy đủ và danh sách nguyên liệu chi tiết với trạng thái có sẵn trong bếp.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
recipeId	Long	Có	ID của công thức nấu ăn

Bảng 133: Path Parameters - API Chi Tiết Công Thức

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Pho bo Ha Noi",
7     "description": "Mon pho truyền thống của Việt Nam",
8     "htmlContent": "<h2>Nguyên liệu</h2><ul><li>Xương bo: 1kg</li>..."
9   },
10  "imageUrl": "https://example.com/images/pho.jpg",
11  "groupId": 10,
12  "isSystem": false,
13  "createdBy": 123,
14  "ingredients": [
    {
      "id": 1,
      "name": "Xương bo",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 2,
      "name": "Thịt bò",
      "quantity": 2,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 3,
      "name": "Mì",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 4,
      "name": "Cà chua",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 5,
      "name": "Hành tím",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 6,
      "name": "Hành lá",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 7,
      "name": "Ớt",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 8,
      "name": "Dầu ăn",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 9,
      "name": "Nước mắm",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    },
    {
      "id": 10,
      "name": "Sốt tương",
      "quantity": 1,
      "unit": "kg",
      "status": "available"
    }
  ]
}
```



```
15         "id": 1,
16         "foodId": 1,
17         "foodName": "Thịt bò",
18         "foodImageUrl": "https://example.com/images/beef.jpg",
19         "quantity": 500,
20         "unitId": 1,
21         "unitName": "gram",
22         "isMainIngredient": true,
23         "note": "Thịt bò tươi",
24         "isAvailable": true
25     }
26 ],
27 "matchedIngredients": 1,
28 "totalIngredients": 2,
29 "createdAt": "2026-01-03T10:30:00Z",
30 "updatedAt": "2026-01-03T10:30:00Z"
31 }
32 }
```

Mã nguồn 29: Response thành công API 4.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.3.1	recipeId hợp lệ (recipeId=1, thuộc nhóm của user)	1000 OK	Trả về đầy đủ thông tin công thức và nguyên liệu
4.3.2	recipeId không tồn tại (99999)	9994 No data or end of list data	Công thức không tồn tại trong hệ thống
4.3.3	recipeId của nhóm khác, không public (recipeId=50)	1009 Action has been done previously by this user	Không có quyền truy cập
4.3.4	recipeId public của nhóm khác (recipeId=25)	1000 OK	Được phép xem công thức public
4.3.5	recipeId không hợp lệ ("abc")	1003 Parameter value is invalid	recipeId phải là số nguyên dương
4.3.6	Công thức không có nguyên liệu (recipeId=10)	1000 OK	ingredients = [], totalIngredients = 0

Bảng 134: Test Cases - API 4.3 Chi Tiết Công Thức

API 4.4: Cập Nhật Công Thức

Endpoint: PUT /recipes/{recipeId}

Mô tả: API cập nhật thông tin công thức nấu ăn (tên, mô tả, nội dung HTML, ảnh). Tất cả các trường đều là tùy chọn, chỉ cập nhật các trường được gửi lên. Không cập nhật nguyên liệu qua API này.

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
recipeId	Long	Có	ID của công thức cần cập nhật

Bảng 135: Path Parameters - API Cập Nhật Công Thức

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Không	Tên công thức mới
description	String	Không	Mô tả mới
htmlContent	String	Không	Nội dung HTML mới
image	File	Không	Ảnh mới (thay thế ảnh cũ)

Bảng 136: Tham số Request Body - API Cập Nhật Công Thức

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Pho bo Nam Dinh",
7     "description": "Mon pho dac san mien Bac",
8     "htmlContent": "<h2>Nguyen lieu</h2>...",
9     "imageUrl": "https://example.com/images/pho_new.jpg",
10    "groupId": 10,
11    "isSystem": false,
12    "createdBy": 123,
13    "ingredients": [
14      {
15        "id": 1,
16        "foodId": 1,
17        "foodName": "Thit bo",
18        "foodImageUrl": "https://example.com/images/beef.jpg",
19        "quantity": 500,
20        "unitId": 1,
21        "unitName": "gram",
22        "isMainIngredient": true,
23        "note": "Thit bo tuoi",
24        "isAvailable": true
25      }
26    ],
27    "matchedIngredients": 1,
28    "totalIngredients": 2,
```

```
29     "createdAt": "2026-01-03T10:30:00Z",
30     "updatedAt": "2026-01-03T15:20:00Z"
31 }
32 }
```

Mã nguồn 30: Response thành công API 4.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.4.1	Cập nhật tên (name="Phở bò Nam Định")	1000 OK	Tên được cập nhật thành công
4.4.2	Cập nhật ảnh (upload ảnh mới)	1000 OK	Ảnh cũ bị thay thế bởi ảnh mới
4.4.3	Cập nhật đầy đủ (name, description, htmlContent, image)	1000 OK	Tất cả thông tin được cập nhật
4.4.4	Cập nhật với tên rỗng (name="")	1003 Parameter value is invalid	Tên không được rỗng
4.4.5	Cập nhật công thức không tồn tại (recipeId=99999)	9994 No data or end of list data	Công thức không tồn tại
4.4.6	Cập nhật công thức của nhóm khác (recipeId=50)	1009 Action has been done previously by this user	Không có quyền
4.4.7	Cập nhật công thức do thành viên khác tạo (recipeId=8)	1000 OK	Bất kỳ thành viên nào cũng có thể cập nhật

Bảng 137: Test Cases - API 4.4 Cập Nhật Công Thức

API 4.5: Xóa Công Thức

Endpoint: DELETE /recipes/{recipeId}

Mô tả: API xóa một công thức nấu ăn và tất cả nguyên liệu liên quan. Các kế hoạch bữa ăn sử dụng công thức này sẽ có recipe = null.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
recipeId	Long	Có	ID của công thức cần xóa

Bảng 138: Path Parameters - API Xóa Công Thức

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }

```

Mã nguồn 31: Response thành công API 4.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.5.1	Xóa công thức hợp lệ (recipeId=1)	1000 OK	Công thức và tất cả ingredients bị xóa
4.5.2	Xóa công thức không tồn tại (recipeId=99999)	9994 No data or end of list data	Công thức không tồn tại
4.5.3	Xóa công thức của nhóm khác (recipeId=50)	1009 Action has been done previously by this user	Không có quyền
4.5.4	Xóa công thức đang dùng trong meal plan (recipeId=5)	1000 OK	Vẫn xóa được, meal plan sẽ có recipe = null
4.5.5	Xóa với recipeId không hợp lệ ("abc")	1003 Parameter value is invalid	recipeId phải là số nguyên dương

Bảng 139: Test Cases - API 4.5 Xóa Công Thức

API 4.6: Thêm Nguyên Liệu Vào Công Thức

Endpoint: POST /recipes/{recipeId}/ingredients

Mô tả: API thêm một hoặc nhiều nguyên liệu vào công thức đã có. Cho phép bổ sung nguyên liệu sau khi đã tạo công thức.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
recipeId	Long	Có	ID của công thức

Bảng 140: Path Parameters - API Thêm Nguyên Liệu

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID thực phẩm
quantity	BigDecimal	Có	Số lượng (phải > 0)
unitId	Long	Có	ID đơn vị đo
isMainIngredient	Boolean	Không	Là nguyên liệu chính hay không
note	String	Không	Ghi chú cho nguyên liệu

Bảng 141: Tham số Request Body - API Thêm Nguyên Liệu

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Pho bo Ha Noi",
7     "ingredients": [
8       {
9         "id": 1,
10        "foodId": 1,
11        "foodName": "Thit bo",
12        "foodImageUrl": "https://example.com/images/beef.jpg",
13        "quantity": 500,
14        "unitId": 1,
15        "unitName": "gram",
16        "isMainIngredient": true,
17        "note": "Thit bo tuoi",
18        "isAvailable": true
19      },
20      {
21        "id": 3,
22        "foodId": 5,
23        "foodName": "Hanh tay",
24        "foodImageUrl": "https://example.com/images/onion.jpg",
25        "quantity": 200,
26        "unitId": 1,
27        "unitName": "gram",
28        "isMainIngredient": false,
29        "note": "Thit bo than",
30        "isAvailable": false
31      }
32    ],
33    "matchedIngredients": 1,
34    "totalIngredients": 3
35  }
36 }
```

Mã nguồn 32: Response thành công API 4.6

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.6.1	Thêm với đầy đủ thông tin (foodId=5, quantity=200, unitId=1, note)	1000 OK	Nguyên liệu được thêm vào công thức
4.6.2	Thêm không có note (foodId=3, quantity=100, unitId=1)	1000 OK	note có thể null
4.6.3	Thêm với quantity <= 0 (quantity=0)	1003 Parameter value is invalid	Số lượng phải > 0
4.6.4	Thêm với foodId không tồn tại (foodId=99999)	9994 No data or end of list data	foodId không tồn tại
4.6.5	Thêm vào công thức không tồn tại (recipeId=99999)	9994 No data or end of list data	Công thức không tồn tại
4.6.6	Thêm nguyên liệu trùng (cùng foodId)	1000 OK	Cho phép trùng (có thể cần ở các bước khác nhau)
4.6.7	Thiếu quantity (chỉ có foodId, unitId)	1002 Parameter is not enough	quantity là tham số bắt buộc

Bảng 142: Test Cases - API 4.6 Thêm Nguyên Liệu

API 4.7: Cập Nhật Nguyên Liệu Trong Công Thức

Endpoint: PUT /recipes/ingredients/{ingredientId}

Mô tả: API cập nhật thông tin một nguyên liệu trong công thức (số lượng, đơn vị, ghi chú). Không cho phép thay đổi foodId.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
ingredientId	Long	Có	ID của nguyên liệu trong công thức

Bảng 143: Path Parameters - API Cập Nhật Nguyên Liệu

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
quantity	BigDecimal	Không	Số lượng mới (phải > 0)
unitId	Long	Không	ID đơn vị mới
isMainIngredient	Boolean	Không	Cập nhật trạng thái nguyên liệu chính

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
note	String	Không	Ghi chú mới

Bảng 144: Tham số Request Body - API Cập Nhật Nguyên Liệu

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "recipeId": 1,
7     "recipeName": "Pho bo Ha Noi",
8     "ingredients": [
9       {
10        "id": 1,
11        "foodId": 1,
12        "foodName": "Thit bo",
13        "foodImageUrl": "https://example.com/images/beef.jpg",
14        "quantity": 300,
15        "unitId": 1,
16        "unitName": "gram",
17        "isMainIngredient": true,
18        "note": "Cat lat mong",
19        "isAvailable": true
20      }
21    ]
22  }
23 }
```

Mã nguồn 33: Response thanh cong API 4.7

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.7.1	Cập nhật số lượng (quantity=300, unitId=1)	1000 OK	Số lượng được cập nhật
4.7.2	Thay đổi đơn vị (quantity=0.5, unitId=3)	1000 OK	Đơn vị được thay đổi
4.7.3	Cập nhật note (note="Cắt lát mỏng")	1000 OK	Note được cập nhật
4.7.4	Cập nhật với quantity <= 0 (quantity=-5)	1003 Parameter value is invalid	Số lượng phải > 0
4.7.5	Cập nhật với unitId không tồn tại (unitId=99999)	9994 No data or end of list data	unitId không hợp lệ
4.7.6	Cập nhật ingredient không tồn tại (ingredientId=99999)	9994 No data or end of list data	Ingredient không tồn tại

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.7.7	Cập nhật ingredient của công thức nhóm khác (ingredientId=50)	1009 Action has been done previously by this user	Không có quyền

Bảng 145: Test Cases - API 4.7 Cập Nhật Nguyên Liệu

API 4.8: Xóa Nguyên Liệu Khỏi Công Thức

Endpoint: DELETE /recipes/ingredients/{ingredientId}

Mô tả: API xóa một nguyên liệu khỏi công thức nấu ăn. Công thức vẫn tồn tại sau khi xóa nguyên liệu.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
ingredientId	Long	Có	ID của nguyên liệu cần xóa

Bảng 146: Path Parameters - API Xóa Nguyên Liệu

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 34: Response thành công API 4.8

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.8.1	Xóa nguyên liệu hợp lệ (ingredientId=1)	1000 OK	Nguyên liệu bị xóa khỏi công thức
4.8.2	Xóa nguyên liệu không tồn tại (ingredientId=99999)	9994 No data or end of list data	Ingredient không tồn tại
4.8.3	Xóa nguyên liệu của công thức nhóm khác (ingredientId=50)	1009 Action has been done previously by this user	Không có quyền
4.8.4	Xóa nguyên liệu cuối cùng (ingredientId=10)	1000 OK	Công thức vẫn tồn tại, chỉ không có nguyên liệu

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.8.5	Xóa với ingredientId không hợp lệ ("abc")	1003 Parameter value is invalid	ingredientId phải là số nguyên dương

Bảng 147: Test Cases - API 4.8 Xóa Nguyên Liệu

API 4.9: Tìm Công Thức Theo Nguyên Liệu

Endpoint: GET /recipes/by-ingredient/{foodId}

Mô tả: API tìm tất cả công thức có chứa một nguyên liệu cụ thể. Hữu ích khi muốn biết có thể nấu món gì với một loại thực phẩm có sẵn trong bếp. Rất hữu ích cho tính năng "Nấu gì với thực phẩm này?".

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm/nguyên liệu

Bảng 148: Path Parameters - API Tìm Công Thức Theo Nguyên Liệu

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "food": {
6       "id": 5,
7       "name": "Thịt bò",
8       "imageUrl": "https://example.com/images/beef.jpg"
9     },
10    "recipes": [
11      {
12        "id": 1,
13        "name": "Phở bò Hà Nội",
14        "description": "Món phở truyền thống của Việt Nam",
15        "imageUrl": "https://example.com/images/pho.jpg",
16        "htmlContent": "<h2>Nguyên liệu</h2>...",
17        "groupId": 10,
18        "isSystem": false,
19        "ingredientInRecipe": {
20          "quantity": 500,
21          "unitId": 1,
22          "unitName": "gram",
23          "note": "Thịt bò tươi"
24        },
25        "matchedIngredients": 3,
```

```
26     "totalIngredients": 5,
27     "createdBy": 123,
28     "createdAt": "2026-01-03T10:30:00Z"
29 },
30 {
31     "id": 15,
32     "name": "Bo luc lac",
33     "description": "Thit bo xao hanh tay",
34     "imageUrl": "https://example.com/images/bolc.jpg",
35     "htmlContent": "<h2>Nguyen lieu</h2>...",
36     "groupId": 10,
37     "isSystem": false,
38     "ingredientInRecipe": {
39         "quantity": 300,
40         "unitId": 1,
41         "unitName": "gram",
42         "note": "Thit bo than"
43     },
44     "matchedIngredients": 2,
45     "totalIngredients": 4,
46     "createdBy": 456,
47     "createdAt": "2025-12-20T08:15:00Z"
48 }
49 ]
50 }
51 }
```

Mã nguồn 35: Response thanh cong API 4.9

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
4.9.1	Tìm công thức có thực phẩm cụ thể (foodId=5)	1000 OK	Trả về tất cả công thức có thịt bò, bao gồm quantity và unit
4.9.2	Tìm với foodId không có trong công thức nào (foodId=99)	1000 OK	data.recipes = [] (mảng rỗng)
4.9.3	Tìm với foodId không tồn tại (foodId=99999)	9994 No data or end of list data	foodId không tồn tại trong hệ thống
4.9.4	Tìm công thức của nhiều nhóm (foodId=1)	1000 OK	Chỉ trả về công thức của nhóm hiện tại + công thức public
4.9.5	foodId không hợp lệ ("abc")	1003 Parameter value is invalid	foodId phải là số nguyên dương
4.9.6	Tìm công thức khi chưa có nhóm	1004 User is not validated	Cần tham gia nhóm trước

Bảng 149: Test Cases - API 4.9 Tìm Công Thức Theo Nguyên
Liệu

UC5 - Quản lý thực phẩm **Mô tả chung:** UC5 quản lý toàn bộ hệ thống thực phẩm của ứng dụng, bao gồm: quản lý thực phẩm cấp nhóm (thêm, xem, cập nhật, xóa thực phẩm tùy chỉnh của nhóm), quản lý danh mục và đơn vị đo lường (category & unit), và quản trị thực phẩm hệ thống (thực phẩm master data cho toàn bộ ứng dụng). Hệ thống phân biệt hai loại thực phẩm: thực phẩm hệ thống (isSystemFood = true, khả dụng cho tất cả nhóm) và thực phẩm nhóm (isSystemFood = false, chỉ khả dụng cho nhóm tạo ra).

4.2.3 Phần 1: API Quản Lý Thực Phẩm Nhóm

API 5.1: Thêm Thực Phẩm (Cấp Nhóm)

Endpoint: POST /food

Mô tả: API thực hiện việc thêm một loại thực phẩm mới vào danh sách thực phẩm của nhóm. Thực phẩm này chỉ khả dụng cho nhóm hiện tại và được đánh dấu isSystemFood = false.

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Có	Tên thực phẩm (1-100 ký tự)
categoryId	Long	Có	ID danh mục thực phẩm
image	File	Không	Ảnh thực phẩm (jpg, jpeg, png, gif)

Bảng 150: Tham số Request Body - API Thêm Thực Phẩm

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Rau muong VietGAP",
7     "categoryId": 3,
8     "categoryName": "Rau cu",
9     "imageUrl": "https://example.com/images/rau-muong.jpg",
10    "groupId": 10,
11    "isSystemFood": false,
12    "createdAt": "2026-01-03T10:30:00Z"
13  }
14 }
```

Mã nguồn 36: Response thành công API 5.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.1.1	name="Rau muống VietGAP", categoryId=3, image=[file]	1000 OK	Thực phẩm được thêm thành công với đầy đủ thông tin
5.1.2	name="Thịt gà ta", categoryId=5 (không có ảnh)	1000 OK	Thực phẩm được tạo mà không có ảnh, imageUrl = null
5.1.3	Không có field name	1002 Parameter is not enough	Thiếu tên thực phẩm
5.1.4	name=""(tên rỗng), categoryId=3	1003 Parameter value is invalid	Tên không được rỗng
5.1.5	name="Thực phẩm mới", categoryId=99999	9994 No data or end of list data	categoryId không tồn tại
5.1.6	Tên trùng với thực phẩm đã có trong nhóm	1000 OK	Cho phép trùng tên (khác nguồn gốc), nên có cảnh báo ở frontend
5.1.7	Upload file .pdf thay vì ảnh	1003 Parameter value is invalid	File phải là ảnh hợp lệ
5.1.8	Người dùng chưa tham gia nhóm	1004 User is not validated	Cần tham gia nhóm trước

Bảng 151: Test Cases - API 5.1 Thêm Thực Phẩm

API 5.2: Lấy Danh Sách Thực Phẩm (Cấp Nhóm)

Endpoint: GET /food

Mô tả: API lấy danh sách tất cả thực phẩm khả dụng cho nhóm, bao gồm thực phẩm do nhóm tạo và thực phẩm hệ thống. Hỗ trợ lọc theo danh mục và tìm kiếm theo tên.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
categoryId	Long	Không	Lọc theo danh mục
name	String	Không	Tìm kiếm theo tên (gần đúng, không phân biệt hoa thường)

Bảng 152: Query Parameters - API Lấy Danh Sách Thực Phẩm

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {

```

```
5  "foods": [  
6    {  
7      "id": 1,  
8      "name": "Rau muong VietGAP",  
9      "categoryId": 3,  
10     "categoryName": "Rau cu",  
11     "imageUrl": "https://example.com/images/rau-muong.jpg",  
12     "groupId": 10,  
13     "isSystemFood": false,  
14     "createdAt": "2026-01-03T10:30:00Z"  
15   },  
16   {  
17     "id": 100,  
18     "name": "Ca chua",  
19     "categoryId": 3,  
20     "categoryName": "Rau cu",  
21     "imageUrl": "https://example.com/images/ca-chua.jpg",  
22     "groupId": null,  
23     "isSystemFood": true,  
24     "createdAt": "2025-12-01T08:00:00Z"  
25   }  
26 ],  
27 "total": 2  
28 }  
29 }
```

Mã nguồn 37: Response thành công API 5.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.2.1	Không có query parameters	1000 OK	Trả về thực phẩm của nhóm + thực phẩm hệ thống
5.2.2	?categoryId=3	1000 OK	Chỉ trả về thực phẩm thuộc danh mục 3
5.2.3	?name=thịt	1000 OK	Trả về thực phẩm có tên chứa "thịt", không phân biệt hoa thường
5.2.4	?categoryId=5&name=bò	1000 OK	Kết hợp nhiều filter, trả về thịt bò
5.2.5	?name=abcxyz123	1000 OK	foods = [], total = 0 (không tìm thấy)
5.2.6	?categoryId=99999	1000 OK	foods = [], total = 0 (categoryId không tồn tại)
5.2.7	Người dùng chưa có nhóm	1004 User is not validated	Cần tham gia nhóm

Bảng 153: Test Cases - API 5.2 Lấy Danh Sách Thực Phẩm

API 5.3: Xem Chi Tiết Thực Phẩm (Cấp Nhóm)

Endpoint: GET /food/{foodId}

Mô tả: API lấy thông tin chi tiết một loại thực phẩm cụ thể. Người dùng có thể xem chi tiết thực phẩm của nhóm mình hoặc thực phẩm hệ thống.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm

Bảng 154: Path Parameters - API Xem Chi Tiết Thực Phẩm

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Rau muong VietGAP",
7     "categoryId": 3,
8     "categoryName": "Rau cu",
9     "imageUrl": "https://example.com/images/rau-muong.jpg",
10    "groupId": 10,
11    "isSystemFood": false,
12    "createdAt": "2026-01-03T10:30:00Z"
13  }
14 }
```

Mã nguồn 38: Response thành công API 5.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.3.1	foodId=1 (thực phẩm do nhóm tạo)	1000 OK	Trả về đầy đủ thông tin
5.3.2	foodId=100 (system food)	1000 OK	isSystemFood = true, groupId = null
5.3.3	foodId=99999 (không tồn tại)	9994 No data or end of list data	Thực phẩm không tồn tại
5.3.4	foodId=50 (của nhóm khác, không phải system food)	1009 Action has been done previously by this user	Không có quyền truy cập
5.3.5	foodId="abc"(không hợp lệ)	1003 Parameter value is invalid	foodId phải là số

TC	Đầu vào	Kết quả mong đợi	Ghi chú
----	---------	------------------	---------

Bảng 155: Test Cases - API 5.3 Xem Chi Tiết Thực Phẩm

API 5.4: Cập Nhật Thực Phẩm (Cấp Nhóm)

Endpoint: PUT /food/{foodId}

Mô tả: API cập nhật thông tin thực phẩm do nhóm tạo. Không thể cập nhật thực phẩm hệ thống. Cho phép cập nhật tên, danh mục và ảnh.

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm cần cập nhật

Bảng 156: Path Parameters - API Cập Nhật Thực Phẩm

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Không	Tên mới (1-100 ký tự)
categoryId	Long	Không	Danh mục mới
image	File	Không	Ảnh mới (jpg, jpeg, png, gif)

Bảng 157: Request Body - API Cập Nhật Thực Phẩm

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Rau muong huu co",
7     "categoryId": 4,
8     "categoryName": "Rau huu co",
9     "imageUrl": "https://example.com/images/rau-muong-new.jpg",
10    "groupId": 10,
11    "isSystemFood": false,
12    "createdAt": "2026-01-03T10:30:00Z"
13  }
14 }
```

Mã nguồn 39: Response thành công API 5.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.4.1	foodId=1, name="Rau muống hữu cơ"	1000 OK	Tên được cập nhật thành công
5.4.2	foodId=1, categoryId=4	1000 OK	Danh mục được thay đổi
5.4.3	foodId=1, image=[file mới]	1000 OK	Ảnh cũ bị thay thế
5.4.4	foodId=1, name=""	1003 Parameter value is invalid	Tên không được rỗng
5.4.5	foodId=1, categoryId=99999	9994 No data or end of list data	categoryId không tồn tại
5.4.6	foodId=99999	9994 No data or end of list data	Thực phẩm không tồn tại
5.4.7	foodId=100 (system food)	1009 Action has been done previously by this user	Không có quyền cập nhật system food
5.4.8	foodId=50 (của nhóm khác)	1009 Action has been done previously by this user	Không có quyền cập nhật thực phẩm nhóm khác

Bảng 158: Test Cases - API 5.4 Cập Nhật Thực Phẩm

API 5.5: Xóa Thực Phẩm (Cấp Nhóm)

Endpoint: DELETE /food/{foodId}

Mô tả: API xóa thực phẩm do nhóm tạo. Không thể xóa thực phẩm hệ thống hoặc thực phẩm đang được sử dụng trong kitchen, recipe hoặc shopping list.

Request Headers:

- Authorization: Bearer {access_token}

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm cần xóa

Bảng 159: Path Parameters - API Xóa Thực Phẩm

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 40: Response thành công API 5.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.5.1	foodId=1 (thực phẩm của nhóm, chưa dùng)	1000 OK	Thực phẩm bị xóa thành công
5.5.2	foodId=99999 (không tồn tại)	9994 No data or end of list data	Thực phẩm không tồn tại
5.5.3	foodId=100 (system food)	1009 Action has been done previously by this user	Không có quyền xóa system food
5.5.4	foodId=50 (của nhóm khác)	1009 Action has been done previously by this user	Không có quyền xóa thực phẩm nhóm khác
5.5.5	foodId=5 (có kitchen items sử dụng)	1009 Action has been done previously by this user	Không thể xóa vì đang được dùng trong kitchen
5.5.6	foodId=8 (có recipe ingredients sử dụng)	1009 Action has been done previously by this user	Không thể xóa vì đang được dùng trong recipe
5.5.7	foodId=10 (có shopping items sử dụng)	1009 Action has been done previously by this user	Không thể xóa vì đang được dùng trong shopping list

Bảng 160: Test Cases - API 5.5 Xóa Thực Phẩm

4.2.4 Phần 2: API Danh Mục và Đơn Vị

API 5.6: Lấy Danh Sách Danh Mục

Endpoint: GET /category

Mô tả: API lấy tất cả danh mục thực phẩm trong hệ thống (rau, củ, quả, thịt, cá, v.v.). API này không yêu cầu xác thực và có thể được cache vì ít thay đổi.

Request Headers:

- Authorization: Not required

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "categories": [
6       {
7         "id": 1,
```

```
8      "name": "Rau",
9      "description": "Cac loai rau cu qua",
10     "imageUrl": "https://example.com/images/rau.jpg"
11   },
12   {
13     "id": 2,
14     "name": "Thit",
15     "description": "Thit bo, heo, ga, vit",
16     "imageUrl": "https://example.com/images/thit.jpg"
17   },
18   {
19     "id": 3,
20     "name": "Ca",
21     "description": "Ca tuoi, ca bien",
22     "imageUrl": "https://example.com/images/ca.jpg"
23   }
24 ],
25 "total": 3
26 }
27 }
```

Mã nguồn 41: Response thanh cong API 5.6

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.6.1	Gọi API bình thường	1000 OK	Trả về tất cả danh mục với total
5.6.2	Danh sách rỗng (không nên xảy ra)	1000 OK	categories = [], total = 0
5.6.3	Gọi API không cần token	1000 OK	Vẫn trả về dữ liệu (API public)

Bảng 161: Test Cases - API 5.6 Lấy Danh Sách Danh Mục

API 5.7: Lấy Danh Sách Đơn Vị

Endpoint: GET /unit

Mô tả: API lấy tất cả đơn vị đo lường trong hệ thống (kg, gram, lít, ml, v.v.). API này không yêu cầu xác thực và có thể được cache.

Request Headers:

- Authorization: Not required

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "units": [
6       {
```

```

7      "id": 1,
8      "name": "Kilogram",
9      "abbreviation": "kg"
10     },
11     {
12         "id": 2,
13         "name": "Gram",
14         "abbreviation": "g"
15     },
16     {
17         "id": 3,
18         "name": "Lit",
19         "abbreviation": "l"
20     },
21     {
22         "id": 4,
23         "name": "Mililit",
24         "abbreviation": "ml"
25     }
26 ],
27 "total": 4
28 }
29 }
```

Mã nguồn 42: Response thành công API 5.7

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.7.1	Gọi API bình thường	1000 OK	Trả về tất cả đơn vị với total
5.7.2	Danh sách rỗng (không nên xảy ra)	1000 OK	units = [], total = 0
5.7.3	Gọi API không cần token	1000 OK	Vẫn trả về dữ liệu (API public)

Bảng 162: Test Cases - API 5.7 Lấy Danh Sách Đơn Vị

4.2.5 Phần 3: API Quản Trị Thực Phẩm Hệ Thống (Admin)

API 5.8: Tạo Thực Phẩm Hệ Thống (Admin)

Endpoint: POST /admin/food

Mô tả: API cho phép quản trị viên tạo thực phẩm master data cho toàn hệ thống. Thực phẩm này sẽ khả dụng cho tất cả các nhóm với isSystemFood = true và groupId = null. Chỉ user có role ADMIN mới được gọi API này.

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer {access_token} (ADMIN role required)

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Có	Tên thực phẩm (1-100 ký tự)
categoryId	Long	Có	ID danh mục
image	File	Không	Ảnh thực phẩm (jpg, jpeg, png, gif)

Bảng 163: Request Body - API Tạo Thực Phẩm Hệ Thống

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Tao thuc pham he thong thanh cong",
4   "data": {
5     "id": 100,
6     "name": "Ca chua",
7     "categoryId": 3,
8     "categoryName": "Rau cu",
9     "imageUrl": "https://example.com/images/ca-chua.jpg",
10    "groupId": null,
11    "isSystemFood": true,
12    "createdAt": "2026-01-03T10:30:00Z"
13  }
14 }
```

Mã nguồn 43: Response thanh cong API 5.8

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.8.1	Admin: name="Cà chua", categoryId=3, image=[file]	1000 OK	isSystemFood = true, groupId = null
5.8.2	Admin: name="Muối", categoryId=8 (không có ảnh)	1000 OK	imageUrl = null
5.8.3	User thường cố gọi API	1009 Action has been done previously by this user	Không có quyền (không phải ADMIN)
5.8.4	Admin: name=""	1003 Parameter value is invalid	Tên không được rỗng
5.8.5	Admin: categoryId=99999	9994 No data or end of list data	categoryId không tồn tại
5.8.6	Admin: tên trùng với thực phẩm hệ thống khác	1000 OK	Cho phép, nên có cảnh báo

Bảng 164: Test Cases - API 5.8 Tạo Thực Phẩm Hệ Thống

API 5.9: Lấy Danh Sách Thực Phẩm Hệ Thống (Admin)

Endpoint: GET /admin/food

Mô tả: API cho phép admin xem tất cả thực phẩm hệ thống để quản lý. Chỉ trả về thực phẩm có isSystemFood = true và groupId = null.

Request Headers:

- Authorization: Bearer {access_token} (ADMIN role required)

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
categoryId	Long	Không	Lọc theo danh mục
name	String	Không	Tìm kiếm theo tên (gần đúng)

Bảng 165: Query Parameters - API Lấy Danh Sách Thực Phẩm Hệ Thống

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "foods": [
6       {
7         "id": 100,
8         "name": "Ca chua",
9         "categoryId": 3,
10        "categoryName": "Rau cu",
11        "imageUrl": "https://example.com/images/ca-chua.jpg",
12        "groupId": null,
13        "isSystemFood": true,
14        "createdAt": "2026-01-03T10:30:00Z"
15      }
16    ],
17    "total": 1
18  }
19 }
```

Mã nguồn 44: Response thành công API 5.9

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.9.1	Admin: không có query parameters	1000 OK	Trả về tất cả system foods
5.9.2	Admin: ?categoryId=3	1000 OK	Chỉ trả về system foods thuộc danh mục 3
5.9.3	Admin: ?name=cà	1000 OK	Trả về system foods có tên chứa "cà"

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.9.4	User thường cố gọi API	1009 Action has been done previously by this user	Không có quyền
5.9.5	Admin: ?name=xyz123	1000 OK	foods = [], total = 0

Bảng 166: Test Cases - API 5.9 Lấy Danh Sách Thực Phẩm Hệ Thống

API 5.10: Xem Chi Tiết Thực Phẩm Hệ Thống (Admin)

Endpoint: GET /admin/food/{foodId}

Mô tả: API cho phép admin xem chi tiết thực phẩm hệ thống. Chỉ có thể xem thực phẩm có isSystemFood = true.

Request Headers:

- Authorization: Bearer {access_token} (ADMIN role required)

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm hệ thống

Bảng 167: Path Parameters - API Xem Chi Tiết Thực Phẩm Hệ Thống

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 100,
6     "name": "Ca chua",
7     "categoryId": 3,
8     "categoryName": "Rau cu",
9     "imageUrl": "https://example.com/images/ca-chua.jpg",
10    "groupId": null,
11    "isSystemFood": true,
12    "createdAt": "2026-01-03T10:30:00Z"
13  }
14 }
```

Mã nguồn 45: Response thành công API 5.10

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.10.1	Admin: foodId=100 (system food)	1000 OK	isSystemFood = true, groupId = null
5.10.2	Admin: foodId=99999	9994 No data or end of list data	Thực phẩm không tồn tại
5.10.3	User thường cố gọi API	1009 Action has been done previously by this user	Không có quyền
5.10.4	Admin: foodId=5 (group food, not system)	1009 Action has been done previously by this user	Không phải system food
5.10.5	Admin: foodId="abc"	1003 Parameter value is invalid	foodId phải là số

Bảng 168: Test Cases - API 5.10 Xem Chi Tiết Thực Phẩm Hệ Thống

API 5.11: Cập Nhật Thực Phẩm Hệ Thống (Admin)

Endpoint: PUT /admin/food/{foodId}

Mô tả: API cho phép admin cập nhật thông tin thực phẩm hệ thống. Cập nhật system food sẽ ảnh hưởng đến tất cả nhóm đang sử dụng.

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer {access_token} (ADMIN role required)

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm hệ thống

Bảng 169: Path Parameters - API Cập Nhật Thực Phẩm Hệ Thống

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Không	Tên mới (1-100 ký tự)
categoryId	Long	Không	Danh mục mới
image	File	Không	Ảnh mới (jpg, jpeg, png, gif)

Bảng 170: Request Body - API Cập Nhật Thực Phẩm Hệ Thống

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Cap nhap thuc pham he thong thanh cong",
4   "data": {
5     "id": 100,
6     "name": "Ca chua bi",
7     "categoryId": 5,
8     "categoryName": "Qua",
9     "imageUrl": "https://example.com/images/ca-chua-bi.jpg",
10    "groupId": null,
11    "isSystemFood": true,
12    "createdAt": "2026-01-03T10:30:00Z"
13  }
14 }
```

Mã nguồn 46: Response thành công API 5.11

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.11.1	Admin: name="Cà chua bi"	1000 OK	Tên được cập nhật
5.11.2	Admin: categoryId=5	1000 OK	Danh mục được cập nhật
5.11.3	Admin: image=[file mới]	1000 OK	Ảnh được thay thế
5.11.4	User thường cố cập nhật	1009 Action has been done previously by this user	Không có quyền
5.11.5	Admin: name=""	1003 Parameter value is invalid	Tên không được rỗng
5.11.6	Admin: foodId=99999	9994 No data or end of list data	Thực phẩm không tồn tại
5.11.7	Admin: foodId=5 (group food)	1009 Action has been done previously by this user	Không phải system food

Bảng 171: Test Cases - API 5.11 Cập Nhật Thực Phẩm Hệ Thống

API 5.12: Xóa Thực Phẩm Hệ Thống (Admin)

Endpoint: DELETE /admin/food/{foodId}

Mô tả: API cho phép admin xóa thực phẩm hệ thống. Chỉ xóa được nếu không có dữ liệu nào đang sử dụng trong toàn hệ thống. Xóa system food rất nguy hiểm và ảnh hưởng đến tất cả nhóm.

Request Headers:

- Authorization: Bearer {access_token} (ADMIN role required)

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
foodId	Long	Có	ID của thực phẩm hệ thống

Bảng 172: Path Parameters - API Xóa Thực Phẩm Hệ Thống

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Xoa thuc pham he thong thanh cong",
4   "data": null
5 }
```

Mã nguồn 47: Response thanh cong API 5.12

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
5.12.1	Admin: foodId=100 (system food, chưa dùng)	1000 OK	Thực phẩm bị xóa
5.12.2	Admin: foodId=99999	9994 No data or end of list data	Thực phẩm không tồn tại
5.12.3	User thường cố xóa	1009 Action has been done previously by this user	Không có quyền
5.12.4	Admin: foodId=10 (có dữ liệu sử dụng)	1009 Action has been done previously by this user	Không thể xóa vì đang được dùng
5.12.5	Admin: foodId=5 (group food, not system)	1009 Action has been done previously by this user	Không phải system food
5.12.6	Admin: foodId="abc"	1003 Parameter value is invalid	foodId phải là số

Bảng 173: Test Cases - API 5.12 Xóa Thực Phẩm Hệ Thống

UC6 - Quản lý nhóm **Mô tả chung:** UC6 quản lý toàn bộ quy trình tạo và quản lý nhóm trong ứng dụng, bao gồm: tạo nhóm mới, xem thông tin nhóm hiện tại, thêm thành viên vào nhóm, xóa thành viên khỏi nhóm, và rời khỏi nhóm. Hệ thống quy định mỗi người dùng chỉ có thể tham gia một nhóm tại một thời điểm. Người tạo nhóm tự động trở thành chủ nhóm (admin) và có quyền quản lý thành viên.

API 6.1: Tạo Nhóm

Endpoint: POST /group

Mô tả: API thực hiện việc tạo một nhóm mới. Người tạo nhóm tự động trở thành chủ nhóm (owner) và có quyền quản lý thành viên. Một người dùng chỉ có thể tham gia một nhóm tại một thời điểm.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Có	Tên nhóm (1-100 ký tự)

Bảng 174: Tham số Request Body - API Tạo Nhóm

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Gia đình Nguyen Van A",
7     "adminId": 123,
8     "adminName": "Nguyen Van A",
9     "members": [
10      {
11        "userId": 123,
12        "name": "Nguyen Van A",
13        "email": "admin@example.com",
14        "avatarUrl": "https://example.com/avatar.jpg",
15        "isAdmin": true,
16        "joinedAt": "2026-01-03T10:30:00Z"
17      }
18    ],
19     "createdAt": "2026-01-03T10:30:00Z"
20   }
21 }
```

Mã nguồn 48: Response thành công API 6.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.1.1	name="Gia đình Nguyễn Văn A"	1000 OK	Nhóm được tạo thành công, người tạo là admin
6.1.2	name="Nhóm bạn thân"	1000 OK	Nhóm được tạo thành công với tên hợp lệ

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.1.3	Thiếu field name	1002 Parameter is not enough	Thiếu tên nhóm
6.1.4	name=""(tên rỗng)	1003 Parameter value is invalid	Tên không được rỗng
6.1.5	name có > 100 ký tự	1003 Parameter value is invalid	Tên quá dài, tối đa 100 ký tự
6.1.6	User đã thuộc một nhóm khác	1009 Action has been done previously by this user	User phải rời nhóm cũ trước
6.1.7	Tên trùng với nhóm khác	1000 OK	Cho phép trùng tên (mỗi nhóm có id riêng)
6.1.8	User chưa xác thực email	9995 User is not validated	Cần xác thực email trước
6.1.9	Token không hợp lệ	9998 Token is invalid	Yêu cầu đăng nhập lại

Bảng 175: Test Cases - API 6.1 Tạo Nhóm

API 6.2: Xem Thông Tin Nhóm

Endpoint: GET /group

Mô tả: API lấy thông tin chi tiết nhóm mà người dùng hiện tại đang tham gia, bao gồm danh sách tất cả thành viên. API tự động lấy nhóm của user hiện tại, không cần tham số.

Request Headers:

- Authorization: Bearer {access_token}

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Gia đình Nguyen Van A",
7     "adminId": 123,
8     "adminName": "Nguyen Van A",
9     "members": [
10      {
11        "userId": 123,
12        "name": "Nguyen Van A",
13        "email": "admin@example.com",
14        "avatarUrl": "https://example.com/avatar.jpg",
15        "isAdmin": true,
16        "joinedAt": "2026-01-03T10:30:00Z"
17      },
18    ]
19  }
```

```
19     "userId": 456,  
20     "name": "Nguyen Van B",  
21     "email": "member@example.com",  
22     "avatarUrl": "https://example.com/avatar2.jpg",  
23     "isAdmin": false,  
24     "joinedAt": "2026-01-03T11:00:00Z"  
25   }  
26 ],  
27   "createdAt": "2026-01-03T10:30:00Z"  
28 }  
29 }
```

Mã nguồn 49: Response thành công API 6.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.2.1	User đang là member của một nhóm	1000 OK	Trả về đầy đủ thông tin nhóm và danh sách members
6.2.2	User chưa tham gia nhóm nào	1004 User is not validated	Chưa có nhóm, chuyển sang màn hình tạo/tham gia nhóm
6.2.3	Token không hợp lệ	9998 Token is invalid	Yêu cầu đăng nhập
6.2.4	Nhóm có nhiều thành viên (5+ members)	1000 OK	Trả về tất cả members, sắp xếp theo joinedAt
6.2.5	Nhóm chỉ có admin	1000 OK	members array chỉ có 1 phần tử (admin)

Bảng 176: Test Cases - API 6.2 Xem Thông Tin Nhóm

API 6.3: Thêm Thành Viên Vào Nhóm

Endpoint: POST /group/add-member

Mô tả: API cho phép chủ nhóm (owner) thêm thành viên mới vào nhóm bằng email. Người được thêm phải chưa thuộc nhóm nào khác và đã xác thực email.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
email	String	Có	Email của người dùng cần thêm vào nhóm

Bảng 177: Tham số Request Body - API Thêm Thành Viên

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "name": "Gia đình Nguyen Van A",
7     "adminId": 123,
8     "adminName": "Nguyen Van A",
9     "members": [
10      {
11        "userId": 123,
12        "name": "Nguyen Van A",
13        "email": "admin@example.com",
14        "avatarUrl": "https://example.com/avatar.jpg",
15        "isAdmin": true,
16        "joinedAt": "2026-01-03T10:30:00Z"
17      },
18      {
19        "userId": 456,
20        "name": "Nguyen Van B",
21        "email": "member@example.com",
22        "avatarUrl": "https://example.com/avatar2.jpg",
23        "isAdmin": false,
24        "joinedAt": "2026-01-03T12:00:00Z"
25      }
26    ],
27     "createdAt": "2026-01-03T10:30:00Z"
28   }
29 }
```

Mã nguồn 50: Response thành công API 6.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.3.1	email="member@example.com"(hợp lệ)	1000 OK	User được thêm vào nhóm, chỉ admin mới được thêm member
6.3.2	Thiếu field email	1002 Parameter is not enough	Thiếu email
6.3.3	email=""(rỗng)	1003 Parameter value is invalid	Email không hợp lệ
6.3.4	email="notexist@example.com"	1094 No data or end of list data	User không tồn tại trong hệ thống
6.3.5	email="invalid-email"(sai định dạng)	1003 Parameter value is invalid	Email không đúng định dạng
6.3.6	User không phải admin gọi API	1009 Action has been done previously by this user	Không có quyền, chỉ admin mới thêm được member

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.3.7	User đã thuộc nhóm khác	1009 Action has been done previously by this user	User phải rời nhóm cũ trước
6.3.8	User đã là member của nhóm này	1009 Action has been done previously by this user	Đã là member rồi
6.3.9	Admin cố thêm chính mình	1009 Action has been done previously by this user	Đã là member rồi
6.3.10	User chưa có nhóm cố thêm member	1004 User is not validated	Chưa có nhóm
6.3.11	User chưa xác thực email	9995 User is not validated	User cần xác thực email trước

Bảng 178: Test Cases - API 6.3 Thêm Thành Viên

API 6.4: Xóa Thành Viên Khỏi Nhóm

Endpoint: POST /group/remove-member

Mô tả: API cho phép chủ nhóm (owner) xóa thành viên ra khỏi nhóm. Không thể xóa chính mình (owner). Dữ liệu của member vẫn giữ nguyên trong nhóm.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
email	String	Có	Email của thành viên cần xóa

Bảng 179: Tham số Request Body - API Xóa Thành Viên

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 51: Response thành công API 6.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.4.1	email="member@example.com"(không hợp lệ)	1000 OK	Member bị xóa khỏi nhóm, dữ liệu vẫn giữ nguyên
6.4.2	Thiếu field email	1002 Parameter is not enough	Thiếu email
6.4.3	email="invalid-email"(không hợp lệ)	1003 Parameter value is invalid	Email không đúng định dạng
6.4.4	email="notexist@example.com"	1094 No data or end of list data	User không tồn tại
6.4.5	User không phải member của nhóm	1009 Action has been done previously by this user	Không phải member
6.4.6	Admin cố xóa chính mình	1009 Action has been done previously by this user	Không thể xóa admin
6.4.7	Thành viên thường cố xóa member khác	1009 Action has been done previously by this user	Không có quyền, chỉ admin mới xóa được
6.4.8	User chưa có nhóm cố xóa member	1004 User is not validated	Chưa có nhóm

Bảng 180: Test Cases - API 6.4 Xóa Thành Viên

API 6.5: Rời Khỏi Nhóm

Endpoint: POST /group/leave

Mô tả: API cho phép thành viên rời khỏi nhóm hiện tại. Owner không thể rời khỏi nhóm trừ khi transfer ownership hoặc xóa nhóm để tránh nhóm "mồ côi".

Request Headers:

- Authorization: Bearer {access_token}

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": null
5 }
```

Mã nguồn 52: Response thanh cong API 6.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
6.5.1	Member thường (không phải admin)	1000 OK	Member rời khỏi nhóm, có thể tạo/tham gia nhóm khác
6.5.2	Admin cố rời nhóm	1009 Action has been done previously by this user	Admin không thể rời, phải transfer ownership hoặc delete group
6.5.3	User chưa có nhóm	1004 User is not validated	Chưa có nhóm
6.5.4	Token không hợp lệ	9998 Token is invalid	Yêu cầu đăng nhập
6.5.5	Rời nhóm nhiều lần liên tiếp	1004 User is not validated (lần 2)	Lần 1: 1000 OK, Lần 2: đã rời rồi
6.5.6	Member cuối cùng (ngoài admin) rời nhóm	1000 OK	Nhóm còn lại admin, gửi notification cho admin

Bảng 181: Test Cases - API 6.5 Rời Khỏi Nhóm

UC7 - Báo cáo và thống kê **Mô tả chung:** UC7 cung cấp hệ thống báo cáo và thống kê toàn diện về hoạt động của nhóm, bao gồm: báo cáo tổng quan về tủ lạnh và phân loại thực phẩm, báo cáo chi tiết về xu hướng sử dụng tủ lạnh, thống kê mua sắm và chi tiêu, phân tích kế hoạch bữa ăn và công thức, cũng như báo cáo đầy đủ kết hợp tất cả các thống kê. Hệ thống hỗ trợ nhiều khoảng thời gian (tuần, tháng, quý, năm) và cung cấp dữ liệu trực quan cho việc ra quyết định.

API 7.1: Lấy Báo Cáo Tổng Quan

Endpoint: GET /reports/overview

Mô tả: API lấy thống kê tổng quan về tủ lạnh, phân loại thực phẩm theo danh mục, các món phổ biến nhất, và giá trị lãng phí. Hữu ích cho dashboard trang chủ.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
period	String	Không	Khoảng thời gian (week, month, quarter, year). Mặc định: month
topItemsLimit	Integer	Không	Số lượng top items trả về. Mặc định: 5

Bảng 182: Query Parameters - API Báo Cáo Tổng Quan

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
```



```

3  "message": "OK",
4  "data": {
5    "totalItems": 45,
6    "expiringItems": 8,
7    "expiredItems": 2,
8    "goodItems": 35,
9    "categoryStats": [
10   {
11     "categoryId": 3,
12     "name": "Rau cu",
13     "count": 15,
14     "percentage": 0.33
15   },
16   {
17     "categoryId": 5,
18     "name": "Thit",
19     "count": 12,
20     "percentage": 0.27
21   }
22 ],
23 "topItems": [
24   {
25     "foodId": 10,
26     "foodName": "Ca chua",
27     "count": 8,
28     "categoryId": 3,
29     "categoryName": "Rau cu"
30   }
31 ]
32 }
33 }
```

Mã nguồn 53: Response thành công API 7.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
7.1.1	Không có query parameter	1000 OK	Trả về thống kê tháng này (mặc định)
7.1.2	?period=week	1000 OK	Trả về thống kê 7 ngày gần nhất
7.1.3	?period=quarter	1000 OK	Trả về thống kê 3 tháng gần nhất
7.1.4	?period=year	1000 OK	Trả về thống kê năm hiện tại
7.1.5	?topItemsLimit=10	1000 OK	Trả về 10 món phổ biến nhất
7.1.6	?period=invalid	1003 Parameter value is invalid	period chỉ nhận: week, month, quarter, year
7.1.7	?topItemsLimit=-5	1003 Parameter value is invalid	topItemsLimit phải là số dương

TC	Đầu vào	Kết quả mong đợi	Ghi chú
7.1.8	Nhóm chưa có dữ liệu	1000 OK	Các giá trị = 0, mảng rỗng

Bảng 183: Test Cases - API 7.1 Báo Cáo Tổng Quan

API 7.2: Lấy Báo Cáo Tủ Lạnh

Endpoint: GET /reports/kitchen

Mô tả: API lấy thông kê chi tiết về tủ lạnh, bao gồm xu hướng theo tuần (items added/used/expired), phân tích theo danh mục, và breakdown trạng thái thực phẩm.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
startDate	String	Không	Ngày bắt đầu (YYYY-MM-DD). Mặc định: 7 ngày trước
endDate	String	Không	Ngày kết thúc (YYYY-MM-DD). Mặc định: hôm nay

Bảng 184: Query Parameters - API Báo Cáo Tủ Lạnh

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "statusBreakdown": {
6       "totalItems": 45,
7       "goodItems": 35,
8       "warningItems": 8,
9       "expiredItems": 2
10    },
11    "weeklyTrend": [
12      {
13        "date": "2026-01-01",
14        "dayOfWeek": "T5",
15        "itemsAdded": 5,
16        "itemsUsed": 3,
17        "itemsExpired": 1
18      },
19      {
20        "date": "2026-01-02",
21        "dayOfWeek": "T6",
22        "itemsAdded": 8,
23        "itemsUsed": 4,
24        "itemsExpired": 0
25      }
26    ]
27  }
28 }
```

```
25     }
26   ],
27   "categoryValues": [
28     {
29       "categoryId": 3,
30       "name": "Rau cu",
31       "itemCount": 15
32     },
33     {
34       "categoryId": 5,
35       "name": "Thit",
36       "itemCount": 12
37     }
38   ]
39 }
40 }
```

Mã nguồn 54: Response thành công API 7.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
7.2.1	Không có query parameter	1000 OK	Trả về xu hướng 7 ngày gần nhất
7.2.2	?startDate=2026-01-01&endDate=2026-01-15	1000 OK	Trả về xu hướng từ 01/01 đến 15/01
7.2.3	?startDate=2026-01-15&endDate=2026-01-01	1003 Parameter value is invalid	startDate phải trước endDate
7.2.4	?startDate=invalid-date	1003 Parameter value is invalid	Định dạng ngày không hợp lệ
7.2.5	Tủ lạnh rỗng	1000 OK	statusBreakdown.totalItems = 0

Bảng 185: Test Cases - API 7.2 Báo Cáo Tủ Lạnh

API 7.3: Lấy Báo Cáo Mua Sắm

Endpoint: GET /reports/shopping

Mô tả: API lấy thống kê về danh sách mua sắm, chi tiêu theo thời gian, và các món hay mua nhất. Hỗ trợ theo dõi xu hướng mua sắm và lập kế hoạch ngân sách.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
period	String	Không	Khoảng thời gian (week, month, quarter). Mặc định: month

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
frequentItemsLimit	Integer	Không	Số lượng frequent items. Mặc định: 5

Bảng 186: Query Parameters - API Báo Cáo Mua Sắm

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "listStats": {
6       "totalLists": 12,
7       "pendingLists": 2,
8       "inProgressLists": 1,
9       "completedLists": 9,
10      "totalItemsBought": 145
11    },
12    "weeklySpending": [
13      {
14        "date": "2026-01-01",
15        "dayOfWeek": "T5",
16        "itemCount": 15
17      },
18      {
19        "date": "2026-01-02",
20        "dayOfWeek": "T6",
21        "itemCount": 8
22      }
23    ],
24    "frequentItems": [
25      {
26        "foodId": 10,
27        "foodName": "Thịt bò",
28        "purchaseCount": 8,
29        "totalQuantity": 4.5,
30        "unitName": "kg"
31      },
32      {
33        "foodId": 15,
34        "foodName": "Cà chua",
35        "purchaseCount": 7,
36        "totalQuantity": 3.2,
37        "unitName": "kg"
38      }
39    ]
40  }
41 }
```

Mã nguồn 55: Response thành công API 7.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
7.3.1	?period=month	1000 OK	Trả về thống kê mua sắm tháng này
7.3.2	?period=week	1000 OK	Thống kê 7 ngày gần nhất
7.3.3	?period=quarter	1000 OK	Thống kê 3 tháng gần nhất
7.3.4	Chưa có danh sách mua sắm	1000 OK	listStats.totalLists = 0
7.3.5	?period=year	1003 Parameter value is invalid	period chỉ nhận: week, month, quarter
7.3.6	?frequentItemsLimit=-5	1003 Parameter value is invalid	Phải là số dương
7.3.7	?frequentItemsLimit=20	1000 OK	Trả về tối đa 20 món hay mua

Bảng 187: Test Cases - API 7.3 Báo Cáo Mua Sắm

API 7.4: Lấy Báo Cáo Bữa Ăn

Endpoint: GET /reports/meal

Mô tả: API lấy thống kê về kế hoạch bữa ăn, sử dụng công thức nấu ăn, xu hướng bữa ăn theo tuần, và các công thức phổ biến nhất. Giúp phân tích thói quen ăn uống của nhóm.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
period	String	Không	Khoảng thời gian (week, month, quarter). Mặc định: month
topRecipesLimit	Integer	Không	Số công thức phổ biến (1-10). Mặc định: 5

Bảng 188: Query Parameters - API Báo Cáo Bữa Ăn

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "mealStats": {
6       "totalMeals": 90,
7       "breakfasts": 30,
8       "lunches": 30,
9       "dinners": 30
10    },
11    "recipeUsage": {
```

```
12     "totalRecipesUsed": 45,
13     "uniqueRecipes": 15,
14     "usagePercentage": 0.5
15 },
16 "topRecipes": [
17   {
18     "recipeId": 1,
19     "recipeName": "Pho bo",
20     "imageUrl": "https://example.com/pho.jpg",
21     "usageCount": 8,
22     "lastUsedDate": "2026-01-02"
23   },
24   {
25     "recipeId": 2,
26     "recipeName": "Com chien",
27     "imageUrl": "https://example.com/com-chien.jpg",
28     "usageCount": 6,
29     "lastUsedDate": "2026-01-03"
30   }
31 ],
32 "weeklyMealTrend": [
33   {
34     "date": "2026-01-01",
35     "dayOfWeek": "T5",
36     "breakfastCount": 1,
37     "lunchCount": 1,
38     "dinnerCount": 1,
39     "totalMeals": 3
40   }
41 ]
42 }
43 }
```

Mã nguồn 56: Response thành công API 7.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
7.4.1	?period=month	1000 OK	Thống kê bữa ăn tháng này
7.4.2	?topRecipesLimit=10	1000 OK	Trả về 10 công thức phổ biến nhất
7.4.3	?period=week	1000 OK	Thống kê 7 ngày gần nhất
7.4.4	Chưa có kế hoạch bữa ăn	1000 OK	mealStats.totalMeals = 0
7.4.5	?topRecipesLimit=50	1003 Parameter value is invalid	Phải từ 1-10
7.4.6	?period=year	1003 Parameter value is invalid	period chỉ nhận: week, month, quarter
7.4.7	?topRecipesLimit=0	1003 Parameter value is invalid	Phải >= 1

TC	Đầu vào	Kết quả mong đợi	Ghi chú
----	---------	------------------	---------

Bảng 189: Test Cases - API 7.4 Báo Cáo Bữa Ăn

API 7.5: Lấy Báo Cáo Đầy Đủ

Endpoint: GET /reports/full

Mô tả: API lấy tất cả thống kê trong một lần gọi, kết hợp overview, kitchen, shopping, và meal reports. Hữu ích cho dashboard tổng quan nhưng có thể tốn thời gian xử lý hơn.

Request Headers:

- Authorization: Bearer {access_token}

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
period	String	Không	Khoảng thời gian (week, month, quarter). Mặc định: month

Bảng 190: Query Parameters - API Báo Cáo Đầy Đủ

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "overview": {
6       "totalItems": 45,
7       "expiringItems": 8,
8       "expiredItems": 2,
9       "goodItems": 35,
10      "categoryStats": [],
11      "topItems": []
12    },
13    "kitchen": {
14      "statusBreakdown": {
15        "totalItems": 45,
16        "goodItems": 35,
17        "warningItems": 8,
18        "expiredItems": 2
19      },
20      "weeklyTrend": [],
21      "categoryValues": []
22    },
23    "shopping": {
24      "listStats": {
25        "totalLists": 12,
26        "pendingLists": 2,
27        "inProgressLists": 1,
28        "completedLists": 9,

```

```

29     "totalItemsBought": 145
30 },
31     "weeklySpending": [],
32     "frequentItems": []
33 },
34     "meal": {
35         "mealStats": {
36             "totalMeals": 90,
37             "breakfasts": 30,
38             "lunches": 30,
39             "dinners": 30
40         },
41         "recipeUsage": {},
42         "topRecipes": [],
43         "weeklyMealTrend": []
44     },
45     "period": "month"
46 }
47 }

```

Mã nguồn 57: Response thành công API 7.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
7.5.1	?period=month	1000 OK	Trả về tất cả thống kê trong một response
7.5.2	?period=week	1000 OK	Báo cáo đầy đủ 7 ngày
7.5.3	?period=quarter	1000 OK	Báo cáo đầy đủ 3 tháng
7.5.4	?period=invalid	1003 Parameter value is invalid	period chỉ nhận: week, month, quarter
7.5.5	Nhóm mới không có dữ liệu	1000 OK	Các giá trị mặc định hoặc rỗng
7.5.6	Không có query parameter	1000 OK	Mặc định period=month

Bảng 191: Test Cases - API 7.5 Báo Cáo Đầy Đủ

UC8 - Quản trị người dùng (Admin) **Mô tả chung:** UC8 cung cấp các chức năng quản trị người dùng dành riêng cho admin, bao gồm: xem danh sách người dùng với các bộ lọc đa dạng, xem thống kê tổng quan về người dùng trong hệ thống, xem chi tiết thông tin từng người dùng, cập nhật thông tin và quyền hạn người dùng, bật/tắt trạng thái kích hoạt tài khoản, và xóa người dùng khỏi hệ thống. Tất cả các API trong UC này yêu cầu role ADMIN và có các cơ chế bảo vệ để ngăn admin tự xóa hoặc vô hiệu hóa chính mình.

API 8.1: Lấy Danh Sách Người Dùng

Endpoint: GET /admin/users

Mô tả: API lấy danh sách người dùng với khả năng lọc theo email, tên, vai trò và trạng thái kích hoạt. Bao gồm thống kê tổng số người dùng active, inactive, verified, unverified.

Request Headers:

- Authorization: Bearer {access_token}
- Role Required: ADMIN

Query Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
email	String	Không	Lọc theo email (tìm kiếm tương đối, không phân biệt hoa thường)
name	String	Không	Lọc theo tên (tìm kiếm tương đối)
role	String	Không	Lọc theo vai trò (USER, ADMIN)
isActive	Boolean	Không	Lọc theo trạng thái kích hoạt

Bảng 192: Query Parameters - API Lấy Danh Sách Người Dùng

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "users": [
6       {
7         "id": 1,
8         "email": "john@example.com",
9         "name": "John Doe",
10        "phone": "0123456789",
11        "avatarUrl": "https://example.com/avatar.jpg",
12        "role": "USER",
13        "isVerified": true,
14        "isActive": true,
15        "groupName": "Gia đình John",
16        "groupId": 10,
17        "createdAt": "2026-01-01T10:00:00Z",
18        "updatedAt": "2026-01-02T15:30:00Z"
19      },
20      {
21        "id": 2,
22        "email": "admin@example.com",
23        "name": "Admin User",
24        "phone": "0987654321",
25        "avatarUrl": "https://example.com/admin.jpg",
26        "role": "ADMIN",
27        "isVerified": true,
28        "isActive": true,
29        "groupName": null,
30        "groupId": null,
```

```
31         "createdAt": "2025-12-01T08:00:00Z",
32         "updatedAt": "2026-01-03T09:00:00Z"
33     }
34 ],
35 "total": 2,
36 "activeCount": 2,
37 "inactiveCount": 0,
38 "verifiedCount": 2,
39 "unverifiedCount": 0
40 }
41 }
```

Mã nguồn 58: Response thành công API 8.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.1.1	Không có query parameters	1000 OK	Danh sách tất cả người dùng với thống kê
8.1.2	?email=john@example.com	1000 OK	Người dùng có email chứa "john@example.com"
8.1.3	?name=John	1000 OK	Người dùng có tên chứa "John"
8.1.4	?role=ADMIN	1000 OK	Chỉ các admin
8.1.5	?isActive=true	1000 OK	Chỉ người dùng đang active
8.1.6	?role=USER&isActive=true	1000 OK	Kết hợp nhiều filter
8.1.7	User không phải admin	4030 Forbidden	Chỉ admin mới truy cập được
8.1.8	?role=INVALID	1003 Parameter value is invalid	role chỉ nhận USER hoặc ADMIN

Bảng 193: Test Cases - API 8.1 Lấy Danh Sách Người Dùng

API 8.2: Lấy Thống Kê Người Dùng

Endpoint: GET /admin/users/statistics

Mô tả: API lấy thống kê tổng quan về người dùng trong hệ thống. Hữu ích cho dashboard admin để hiển thị các số liệu tổng hợp.

Request Headers:

- Authorization: Bearer {access_token}
- Role Required: ADMIN

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "totalUsers": 150,
```

```
6      "activeUsers": 130,
7      "inactiveUsers": 20,
8      "verifiedUsers": 140,
9      "unverifiedUsers": 10
10   }
11 }
```

Mã nguồn 59: Response thành công API 8.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.2.1	Gọi API bình thường	1000 OK	Thống kê đầy đủ, totalUsers = activeUsers + inactiveUsers
8.2.2	User không phải admin	4030 Forbidden	Chỉ admin mới xem được thống kê
8.2.3	Hệ thống chưa có user nào	1000 OK	Tất cả giá trị = 0

Bảng 194: Test Cases - API 8.2 Lấy Thống Kê Người Dùng

API 8.3: Lấy Chi Tiết Người Dùng

Endpoint: GET /admin/users/{userId}

Mô tả: API lấy thông tin chi tiết của một người dùng cụ thể, bao gồm cả thông tin nhóm nếu user đang tham gia.

Request Headers:

- Authorization: Bearer {access_token}
- Role Required: ADMIN

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
userId	Long	Có	ID của người dùng cần xem

Bảng 195: Path Parameters - API Lấy Chi Tiết Người Dùng

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "OK",
4   "data": {
5     "id": 1,
6     "email": "john@example.com",
7     "name": "John Doe",
8     "phone": "0123456789",
9     "avatarUrl": "https://example.com/avatar.jpg",
10    "role": "USER",
```

```

11     "isVerified": true,
12     "isActive": true,
13     "groupName": "Gia đình John",
14     "groupId": 10,
15     "createdAt": "2026-01-01T10:00:00Z",
16     "updatedAt": "2026-01-02T15:30:00Z"
17 }
18 }

```

Mã nguồn 60: Response thành công API 8.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.3.1	userId=1 (hợp lệ)	1000 OK	Thông tin đầy đủ của user, groupName/groupId null nếu chưa có nhóm
8.3.2	userId=9999 (không tồn tại)	4040 Not Found	Không tìm thấy người dùng
8.3.3	userId=-1 (không hợp lệ)	1003 Parameter value is invalid	userId phải là số dương
8.3.4	User không phải admin	4030 Forbidden	Chỉ admin mới xem được
8.3.5	userId="abc"(không phải số)	1003 Parameter value is invalid	userId phải là số

Bảng 196: Test Cases - API 8.3 Lấy Chi Tiết Người Dùng

API 8.4: Cập Nhật Người Dùng

Endpoint: PUT /admin/users/{userId}

Mô tả: API cho phép admin cập nhật thông tin người dùng (tên, số điện thoại, vai trò, trạng thái). Tất cả các trường đều tùy chọn, chỉ gửi các trường cần cập nhật.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}
- Role Required: ADMIN

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
userId	Long	Có	ID của người dùng cần cập nhật

Bảng 197: Path Parameters - API Cập Nhật Người Dùng

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
name	String	Không	Tên mới
phone	String	Không	Số điện thoại mới
role	String	Không	Vai trò mới (USER, ADMIN)
isActive	Boolean	Không	Trạng thái kích hoạt mới
isVerified	Boolean	Không	Trạng thái xác thực email mới

Bảng 198: Request Body - API Cập Nhật Người Dùng

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Cap nhat nguoi dung thanh cong",
4   "data": {
5     "id": 1,
6     "email": "john@example.com",
7     "name": "John Updated",
8     "phone": "0987654321",
9     "avatarUrl": "https://example.com/avatar.jpg",
10    "role": "ADMIN",
11    "isVerified": true,
12    "isActive": true,
13    "groupName": "Gia dinh John",
14    "groupId": 10,
15    "createdAt": "2026-01-01T10:00:00Z",
16    "updatedAt": "2026-01-03T12:00:00Z"
17  }
18 }
```

Mã nguồn 61: Response thanh cong API 8.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.4.1	name="John Updated"	1000 OK	Tên được cập nhật
8.4.2	role="ADMIN"	1000 OK	Role được nâng lên ADMIN
8.4.3	name, phone, isVerified cùng lúc	1000 OK	Tất cả trường được cập nhật
8.4.4	isActive=false	1000 OK	Tài khoản bị vô hiệu hóa
8.4.5	userId=9999 (không tồn tại)	4040 Not Found	User không tồn tại
8.4.6	role="SUPER_ADMIN"	1003 Parameter value is invalid	Chỉ chấp nhận USER hoặc ADMIN
8.4.7	Admin tự cập nhật quyền mình	4000 Bad Request	Admin không thể tự giáng cấp
8.4.8	Request body rỗng {}	1000 OK	Không có thay đổi

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.4.9	User không phải admin	4030 Forbidden	Chỉ admin mới cập nhật được

Bảng 199: Test Cases - API 8.4 Cập Nhật Người Dùng

API 8.5: Bật/Tắt Trạng Thái Người Dùng

Endpoint: PATCH /admin/users/{userId}/toggle-status

Mô tả: API để bật/tắt trạng thái kích hoạt của người dùng (toggle isActive). Nếu đang active → inactive, nếu đang inactive → active. Admin không thể tắt chính mình.

Request Headers:

- Authorization: Bearer {access_token}
- Role Required: ADMIN

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
userId	Long	Có	ID của người dùng cần toggle status

Bảng 200: Path Parameters - API Bật/Tắt Trạng Thái

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Kích hoạt tài khoản thành công",
4   "data": {
5     "id": 1,
6     "email": "john@example.com",
7     "name": "John Doe",
8     "phone": "0123456789",
9     "avatarUrl": "https://example.com/avatar.jpg",
10    "role": "USER",
11    "isVerified": true,
12    "isActive": true,
13    "groupName": "Gia đình John",
14    "groupId": 10,
15    "createdAt": "2026-01-01T10:00:00Z",
16    "updatedAt": "2026-01-03T12:30:00Z"
17  }
18 }
```

Mã nguồn 62: Response thành công API 8.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.5.1	User isActive=false	1000 OK	isActive=true, message "Kích hoạt tài khoản thành công"
8.5.2	User isActive=true	1000 OK	isActive=false, message "Vô hiệu hóa tài khoản thành công"
8.5.3	userId=9999 (không tồn tại)	4040 Not Found	User không tồn tại
8.5.4	userId = current admin's ID	4000 Bad Request	Không thể vô hiệu hóa chính mình
8.5.5	User không phải admin	4030 Forbidden	Chỉ admin mới toggle được
8.5.6	Toggle nhiều lần liên tiếp	1000 OK	Trạng thái thay đổi mỗi lần

Bảng 201: Test Cases - API 8.5 Bật/Tắt Trạng Thái

API 8.6: Xóa Người Dùng

Endpoint: DELETE /admin/users/{userId}

Mô tả: API để xóa người dùng khỏi hệ thống. Xóa vĩnh viễn và không thể khôi phục. Admin không thể xóa chính mình và phải giữ ít nhất 1 admin trong hệ thống.

Request Headers:

- Authorization: Bearer {access_token}
- Role Required: ADMIN

Path Parameters:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
userId	Long	Có	ID của người dùng cần xóa

Bảng 202: Path Parameters - API Xóa Người Dùng

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "Xoa nguoi dung thanh cong",
4   "data": null
5 }
```

Mã nguồn 63: Response thanh công API 8.6

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
8.6.1	userId=5 (user thường)	1000 OK	User bị xóa vĩnh viễn, có thể xóa dữ liệu liên quan
8.6.2	userId=9999 (không tồn tại)	4040 Not Found	User không tồn tại
8.6.3	userId = current admin's ID	4000 Bad Request	Không thể xóa chính mình
8.6.4	Xóa admin cuối cùng	4000 Bad Request	Phải có ít nhất 1 admin
8.6.5	User không phải admin	4030 Forbidden	Chỉ admin mới xóa được
8.6.6	Xóa user đang trong nhóm	1000 OK	User bị xóa, dữ liệu nhóm vẫn giữ

Bảng 203: Test Cases - API 8.6 Xóa Người Dùng

UC9 - Xác thực và phân quyền **Mô tả chung:** UC9 quản lý toàn bộ quy trình xác thực và phân quyền trong hệ thống, bao gồm: đăng ký tài khoản mới với gửi email xác thực, đăng nhập và cấp phát token, xác thực email bằng mã verification code, gửi lại mã xác thực khi cần thiết, và đổi mật khẩu cho người dùng đã đăng nhập. Hệ thống sử dụng JWT token cho authentication, yêu cầu xác thực email trước khi sử dụng đầy đủ tính năng, và có các cơ chế bảo vệ như rate limiting chống brute force và spam.

API 9.1: Đăng Ký

Endpoint: POST /auth/register

Mô tả: API cho phép người dùng mới tạo tài khoản trong hệ thống. Sau khi đăng ký thành công, hệ thống gửi email xác thực chứa mã 6 số đến địa chỉ email đã đăng ký và tự động đăng nhập (trả về access token).

Request Headers:

- Content-Type: application/json

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
email	String	Có	Email đăng ký (phải đúng định dạng và chưa tồn tại)
password	String	Có	Mật khẩu (tối thiểu 6 ký tự)
name	String	Có	Họ và tên đầy đủ (tối thiểu 2 ký tự)
phone	String	Không	Số điện thoại (10-11 số, bắt đầu bằng 0)

Bảng 204: Request Body - API Đăng Ký

Response Body (Success - 1000):


```
1 {
2   "code": "1000",
3   "message": "Dang ky thanh cong",
4   "data": {
5     "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
6     "tokenType": "Bearer",
7     "expiresIn": 86400,
8     "user": {
9       "id": 1,
10      "email": "newuser@example.com",
11      "name": "Nguyen Van A",
12      "phone": "0912345678",
13      "avatarUrl": null,
14      "role": "USER",
15      "isVerified": false,
16      "isActive": true,
17      "createdAt": "2026-01-03T10:00:00Z"
18    }
19  }
20 }
```

Mã nguồn 64: Response thanh cong API 9.1

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
9.1.1	email, password, name, phone đầy đủ	1000 OK	Tài khoản được tạo, trả về token, HTTP 201, isVerified=false
9.1.2	Email đã tồn tại	4000 Bad Request	Email đã được sử dụng
9.1.3	Thiếu email	1002 Parameter is not enough	Email là trường bắt buộc
9.1.4	Thiếu password	1002 Parameter is not enough	Password là trường bắt buộc
9.1.5	Thiếu name	1002 Parameter is not enough	Tên là trường bắt buộc
9.1.6	email="invalid-email"	1003 Parameter value is invalid	Email không đúng định dạng
9.1.7	password="123"(< 6 ký tự)	1003 Parameter value is invalid	Password tối thiểu 6 ký tự
9.1.8	phone="123"(không hợp lệ)	1003 Parameter value is invalid	Số điện thoại phải 10-11 số, bắt đầu bằng 0
9.1.9	name="A"(quá ngắn)	1003 Parameter value is invalid	Tên tối thiểu 2 ký tự
9.1.10	Không có phone (optional)	1000 OK	Số điện thoại là trường tùy chọn

TC	Đầu vào	Kết quả mong đợi	Ghi chú
----	---------	---------------------	---------

Bảng 205: Test Cases - API 9.1 Đăng Ký

API 9.2: Đăng Nhập

Endpoint: POST /auth/login

Mô tả: API xác thực người dùng và trả về access token để sử dụng cho các API requests tiếp theo. Token có thời hạn và cần lưu vào secure storage.

Request Headers:

- Content-Type: application/json

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
email	String	Có	Email đăng nhập
password	String	Có	Mật khẩu

Bảng 206: Request Body - API Đăng Nhập

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Đăng nhập thành công",
4   "data": {
5     "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
6     "tokenType": "Bearer",
7     "expiresIn": 86400,
8     "user": {
9       "id": 1,
10      "email": "user@example.com",
11      "name": "Nguyen Van A",
12      "phone": "0912345678",
13      "avatarUrl": "https://example.com/avatar.jpg",
14      "role": "USER",
15      "isVerified": true,
16      "isActive": true,
17      "createdAt": "2026-01-01T10:00:00Z"
18    }
19  }
20 }
```

Mã nguồn 65: Response thành công API 9.2

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
9.2.1	email, password hợp lệ	1000 OK	Trả về user info và token, tokenType="Bearer"
9.2.2	Thiếu email	1002 Parameter is not enough	Email là trường bắt buộc
9.2.3	Thiếu password	1002 Parameter is not enough	Password là trường bắt buộc
9.2.4	Email không tồn tại	9994 No data or end of list data	Tài khoản không tồn tại
9.2.5	Password sai	1003 Parameter value is invalid	Mật khẩu không chính xác
9.2.6	Email chưa xác thực	9995 User is not validated	Cần xác thực email trước, chuyển đến màn hình verify
9.2.7	Tài khoản bị khóa (isActive=false)	1009 Action has been done previously by this user	Tài khoản bị khóa
9.2.8	Đăng nhập sai 5 lần liên tiếp	1009 Action has been done previously by this user	Tài khoản tạm khóa 15 phút, có rate limiting

Bảng 207: Test Cases - API 9.2 Đăng Nhập

API 9.3: Xác Thực Email

Endpoint: POST /verify-email

Mô tả: API xác thực email của người dùng bằng mã verification code (6 số) được gửi qua email sau khi đăng ký. User phải đã đăng nhập (có token) mới verify được.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
code	String	Có	Mã xác thực (6 ký tự số)

Bảng 208: Request Body - API Xác Thực Email

Response Body (Success - 1000):

```

1 {
2   "code": "1000",
3   "message": "Xác thực email thành công",
4   "data": {

```

```
5      "user": {
6        "id": 1,
7        "email": "user@example.com",
8        "name": "Nguyen Van A",
9        "isVerified": true
10     }
11 }
12 }
```

Mã nguồn 66: Response thành công API 9.3

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
9.3.1	code="123456"(đúng)	1000 OK	Email được xác thực, isVerified=true
9.3.2	Thiếu code	1002 Parameter is not enough	Thiếu mã xác thực
9.3.3	code="999999"(sai)	1003 Parameter value is invalid	Mã xác thực không chính xác
9.3.4	Mã đã hết hạn (> 30 phút)	1003 Parameter value is invalid	Mã đã hết hạn, cần gửi lại
9.3.5	Email đã verify rồi	1009 Action has been done previously by this user	Đã xác thực trước đó
9.3.6	Nhập sai 5 lần liên tiếp	1009 Action has been done previously by this user	Mã bị vô hiệu, cần gửi lại
9.3.7	Không có token	4010 Unauthorized	Cần đăng nhập trước

Bảng 209: Test Cases - API 9.3 Xác Thực Email

API 9.4: Gửi Lại Mã Xác Thực

Endpoint: POST /auth/resent-verification

Mô tả: API gửi lại mã xác thực email khi mã cũ đã hết hạn hoặc người dùng không nhận được email. Lấy email từ token, có rate limit tối đa 3 lần/giờ.

Request Headers:

- Authorization: Bearer {access_token}

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Gui lai ma xac thuc thanh cong",
4   "data": {
```

```
5      "email": "user@example.com",
6      "sentAt": "2026-01-03T10:30:00Z",
7      "expiresIn": 1800
8    }
9  }
```

Mã nguồn 67: Response thành công API 9.4

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
9.4.1	Gọi API với token hợp lệ	1000 OK	Mã mới được gửi, mã cũ vô hiệu, expiresIn=1800 (30 phút)
9.4.2	Email đã xác thực	1009 Action has been done previously by this user	Email đã verify rồi
9.4.3	Gửi lần thứ 4 trong 1 giờ	1009 Action has been done previously by this user	Vượt quá giới hạn 3 lần/giờ
9.4.4	Gửi lại trong vòng 1 phút	1009 Action has been done previously by this user	Vui lòng đợi, chống spam
9.4.5	Không có token	4010 Unauthorized	Cần đăng nhập
9.4.6	Token hết hạn	4010 Unauthorized	Token không hợp lệ

Bảng 210: Test Cases - API 9.4 Gửi Lại Mã Xác Thực

API 9.5: Đổi Mật Khẩu

Endpoint: POST /auth/change-password

Mô tả: API cho phép người dùng đã đăng nhập đổi mật khẩu của mình. Cần xác thực mật khẩu cũ và mật khẩu mới phải khớp với xác nhận.

Request Headers:

- Content-Type: application/json
- Authorization: Bearer {access_token}

Request Body:

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
oldPassword	String	Có	Mật khẩu hiện tại
newPassword	String	Có	Mật khẩu mới (tối thiểu 6 ký tự)

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
confirmPassword	String	Có	Xác nhận mật khẩu mới (phải trùng newPassword)

Bảng 211: Request Body - API Đổi Mật Khẩu

Response Body (Success - 1000):

```
1 {
2   "code": "1000",
3   "message": "Doi mat khau thanh cong",
4   "data": null
5 }
```

Mã nguồn 68: Response thanh cong API 9.5

Các Test Case:

TC	Đầu vào	Kết quả mong đợi	Ghi chú
9.5.1	oldPassword, newPassword, confirmPassword hợp lệ	1000 OK	Password được đổi, token không bị revoke
9.5.2	Thiếu oldPassword	1002 Parameter is not enough	Thiếu password cũ
9.5.3	oldPassword sai	1003 Parameter value is invalid	Password cũ không chính xác
9.5.4	newPassword="123"(< 6 ký tự)	1003 Parameter value is invalid	Password mới tối thiểu 6 ký tự
9.5.5	confirmPassword không khớp newPassword	1003 Parameter value is invalid	Password xác nhận không khớp
9.5.6	Thiếu newPassword	1002 Parameter is not enough	Thiếu password mới
9.5.7	Thiếu confirmPassword	1002 Parameter is not enough	Thiếu xác nhận password
9.5.8	Không có token	4010 Unauthorized	Cần đăng nhập
9.5.9	Token hết hạn	4010 Unauthorized	Token không hợp lệ

Bảng 212: Test Cases - API 9.5 Đổi Mật Khẩu

UC bổ sung - Gợi Ý Công Thức Nấu Ăn Mô tả chung:

Use case này cung cấp các API để hỗ trợ gợi ý công thức nấu ăn thông minh dựa trên nguyên liệu có sẵn trong tủ lạnh của nhóm. Hệ thống tự động phân tích độ phù hợp giữa công thức và nguyên liệu hiện có, giúp người dùng tối ưu hóa việc sử dụng thực phẩm và giảm lãng phí. Bao gồm tính năng gợi ý tổng quát dựa trên tất cả nguyên liệu và gợi ý theo từng loại thực phẩm cụ thể.

API 10.1: Gợi Ý Công Thức Tổng Quát

Endpoint: GET /suggestions/recipes

Mô tả: API gợi ý các công thức nấu ăn dựa trên tất cả nguyên liệu có sẵn trong tủ lạnh của nhóm. Hệ thống tự động tính toán độ phù hợp giữa công thức và nguyên liệu, sắp xếp theo mức độ khớp từ cao đến thấp.

Request Header:

Tham số	Mô tả
Method	GET
Authorization	Bearer {access_token} (bắt buộc)

Query Parameters:

Không có (hệ thống tự động lấy tất cả nguyên liệu từ kitchen của group)

Response Body:

```

1 {
2   "code": "string",
3   "message": "string",
4   "data": [
5     {
6       "id": "long",
7       "name": "string",
8       "description": "string",
9       "htmlContent": "string",
10      "imageUrl": "string",
11      "groupId": "long",
12      "isSystem": "boolean",
13      "createdBy": "long",
14      "ingredients": [
15        {
16          "id": "long",
17          "foodId": "long",
18          "foodName": "string",
19          "quantity": "decimal",
20          "unit": "string"
21        }
22      ],
23      "createdAt": "string",
24      "updatedAt": "string",
25      "matchedIngredients": "int",
26      "totalIngredients": "int"
27    }
28  ]
29 }
```

Response Fields:

Trường	Kiểu	Mô tả
id	long	ID của công thức

Trường	Kiểu	Mô tả
name	string	Tên công thức
description	string	Mô tả công thức
htmlContent	string	Nội dung hướng dẫn chi tiết (HTML format)
imageUrl	string	URL ảnh minh họa
groupId	long	ID nhóm (null nếu isSystem = true)
isSystem	boolean	Công thức hệ thống hay tự tạo
createdBy	long	ID người tạo
ingredients	array	Danh sách nguyên liệu cần thiết
createdAt	string	Thời gian tạo
updatedAt	string	Thời gian cập nhật
matchedIngredients	int	Số nguyên liệu đã có trong kitchen
totalIngredients	int	Tổng số nguyên liệu cần cho công thức

Các Test Case:

STT	Mô tả	Kết quả mong đợi	Ghi chú
1	Gợi ý thành công	1000 OK	Trả về danh sách recipes phù hợp. Tự động lấy tất cả kitchen items của group. Sắp xếp theo matchedIngredients/totalIngredients cao. Bao gồm cả system recipes và group recipes
2	User chưa có nhóm	4000 Bad Request	Cần tham gia nhóm trước khi sử dụng tính năng gợi ý
3	Kitchen trống	1000 OK với data = []	Frontend hiển thị empty state "Hãy thêm nguyên liệu vào tủ lạnh"
4	Không có recipe phù hợp	1000 OK với data = []	Có items nhưng không có recipe nào match với nguyên liệu hiện có
5	Token không hợp lệ	9998 Token is invalid	Cần đăng nhập để sử dụng tính năng

API 10.2: Gợi Ý Công Thức Theo Thực Phẩm

Endpoint: GET /suggestions/recipes/by-food

Mô tả: API gợi ý các công thức nấu ăn có sử dụng một loại thực phẩm cụ thể. Hữu ích khi người dùng muốn tìm món ăn từ một nguyên liệu đang có sẵn hoặc muốn sử dụng nguyên liệu trước khi hết hạn.

Request Header:

Tham số	Mô tả
Method	GET
Authorization	Bearer {access_token} (bắt buộc)

Query Parameters:

Tham số	Kiểu	Bắt buộc	Mô tả
foodId	long	Có	ID của thực phẩm cần tìm công thức

Response Body:

```

1 {
2   "code": "string",
3   "message": "string",
4   "data": [
5     {
6       "id": "long",
7       "name": "string",
8       "description": "string",
9       "htmlContent": "string",
10      "imageUrl": "string",
11      "groupId": "long",
12      "isSystem": "boolean",
13      "createdBy": "long",
14      "ingredients": [
15        {
16          "id": "long",
17          "foodId": "long",
18          "foodName": "string",
19          "quantity": "decimal",
20          "unit": "string"
21        }
22      ],
23      "createdAt": "string",
24      "updatedAt": "string",
25      "matchedIngredients": "int",
26      "totalIngredients": "int"
27    }
28  ]
29 }
```

Response Fields: Giống API 10.1

Các Test Case:

STT	Mô tả	Kết quả mong đợi	Ghi chú
1	Tìm công thức theo thực phẩm thành công	1000 OK	Input: ?foodId=5. Trả về recipes có sử dụng food này. Tìm trong cả system recipes và group recipes. Mỗi recipe đảm bảo có foodId này trong ingredients
2	Thiếu foodId	1002 Parameter is not enough	foodId là tham số bắt buộc
3	foodId không tồn tại	4040 Not Found	Input: ?foodId=99999. Không tìm thấy food với ID này
4	foodId không hợp lệ	1003 Parameter value is invalid	Input: ?foodId=-1 hoặc foodId không phải số
5	Không có recipe nào sử dụng food này	1000 OK với data = []	Gợi ý user tạo recipe mới với food này
6	User chưa có nhóm	4000 Bad Request	Cần tham gia nhóm trước
7	Token không hợp lệ	9998 Token is invalid	Cần đăng nhập

UC bổ sung - Quản Lý Thông Báo Mô tả chung:

Use case này cung cấp hệ thống thông báo toàn diện cho người dùng, giúp cập nhật các sự kiện quan trọng như thực phẩm sắp hết hạn, thành viên mới tham gia nhóm, công thức được chia sẻ, lời nhắc bữa ăn, và nhiệm vụ mua sắm được giao. Hệ thống hỗ trợ quản lý trạng thái đọc/chưa đọc, xóa thông báo, và đánh dấu hàng loạt, giúp người dùng kiểm soát hiệu quả các thông tin cập nhật.

API 11.1: Lấy Danh Sách Thông Báo

Endpoint: GET /notification

Mô tả: API lấy toàn bộ danh sách thông báo của người dùng hiện tại, bao gồm thống kê số thông báo chưa đọc. Thông báo được sắp xếp theo thời gian tạo giảm dần (mới nhất trước).

Request Header:

Tham số	Mô tả
Method	GET
Authorization	Bearer {access_token} (bắt buộc)

Query Parameters:

Không có query parameters. Backend tự động trả về tất cả notifications của user, sắp xếp theo createdAt DESC (mới nhất trước).

Response Body:

```

1 {
2   "code": "string",
3   "message": "string",
4   "data": {
5     "notifications": [
6       {
7         "id": "long",
8         "title": "string",
9         "message": "string",
10        "type": "string",
11        "referenceType": "string",
12        "referenceId": "long",
13        "isRead": "boolean",
14        "createdAt": "timestamp"
15      }
16    ],
17    "unreadCount": "int",
18    "total": "int"
19  }
20 }
```

Response Fields:

Trường	Kiểu	Mô tả
Data Object		
notifications	array	Danh sách thông báo
unreadCount	int	Số thông báo chưa đọc
total	int	Tổng số thông báo
Notification Object		
id	long	ID của thông báo
title	string	Tiêu đề thông báo
message	string	Nội dung thông báo
type	string	Loại thông báo (EXPIRY_ALERT, MEMBER_ADDED, RECIPE_SHARED, MEAL_REMINDER, SHOPPING_ASSIGNED, SYSTEM)
referenceType	string	Loại entity liên quan (FOOD, GROUP, RECIPE, MEAL, SHOPPING_LIST)
referenceId	long	ID của entity liên quan
isRead	boolean	Trạng thái đã đọc
createdAt	timestamp	Thời điểm tạo thông báo (format: YYYY-MM-DDTHH:mm:ss)

Các Test Case:

STT	Mô tả	Kết quả mong đợi	Ghi chú
1	Lấy danh sách thông báo thành công	1000 OK	Trả về toàn bộ notifications với unreadCount và total. Sắp xếp theo createdAt DESC. Bao gồm cả đã đọc và chưa đọc. Không có phân trang
2	User chưa có thông báo nào	1000 OK	User mới tạo tài khoản. Trả về notifications = [], unreadCount = 0, total = 0
3	Token không hợp lệ	9998 Token is invalid	Cần đăng nhập

API 11.2: Đánh Dấu Thông Báo Đã Đọc

Endpoint: POST /notification/{notificationId}/read

Mô tả: API đánh dấu một thông báo cụ thể là đã đọc. Cập nhật trường isRead = true và giảm unreadCount.

Request Header:

Tham số	Mô tả
Method	POST
Authorization	Bearer {access_token} (bắt buộc)

Path Parameters:

Tham số	Kiểu	Bắt buộc	Mô tả
notificationId	long	Có	ID của thông báo cần đánh dấu đã đọc

Response Body:

```

1 {
2   "code": "string",
3   "message": "string",
4   "data": null
5 }
```

Các Test Case:

STT	Mô tả	Kết quả mong đợi	Ghi chú
1	Đánh dấu đã đọc thành công	1000 OK	Input: POST /notification/123/read. isRead được set = true. Update unread count. Response trả về data = null
2	Đánh dấu notification không tồn tại	9994 No data or end of list data	Input: POST /notification/99999/read
3	Đánh dấu notification của user khác	1009 Action has been done previously by this user	notificationId thuộc về user khác. Không có quyền
4	Đánh dấu notification đã được đọc trước đó	1000 OK	Notification đã isRead = true. Vẫn thành công (Idempotent operation)
5	notificationId không hợp lệ	1003 Parameter value is invalid	Input: POST /notification/abc/read. ID phải là số
6	Token không hợp lệ	9998 Token is invalid	Cần đăng nhập

API 11.3: Đánh Dấu Tất Cả Thông Báo Đã Đọc

Endpoint: POST /notification/read-all

Mô tả: API đánh dấu tất cả thông báo của user là đã đọc. Hữu ích khi user muốn clear tất cả notifications chưa đọc cùng lúc.

Request Header:

Tham số	Mô tả
Method	POST
Authorization	Bearer {access_token} (bắt buộc)

Response Body:

```
1 {
2   "code": "string",
3   "message": "string",
4   "data": null
5 }
```

Các Test Case:

STT	Mô tả	Kết quả mong đợi	Ghi chú
1	Đánh dấu tất cả thành công	1000 OK	Input: POST /notification/read-all. Tất cả notifications được đánh dấu đã đọc. Chỉ update notifications chưa đọc (isRead = false)
2	Không có notification chưa đọc	1000 OK	Tất cả notifications đã isRead = true. Vẫn thành công
3	User chưa có notification nào	1000 OK	User mới, chưa có notifications
4	Token không hợp lệ	9998 Token is invalid	Cần đăng nhập
5	Gọi nhiều lần liên tiếp	1000 OK	Gọi 2 lần liên tiếp. Cả 2 lần đều thành công (Idempotent operation)

API 11.4: Xóa Thông Báo

Endpoint: DELETE /notification/{notificationId}

Mô tả: API xóa một thông báo cụ thể khỏi danh sách của user. Thao tác này không thể phục hồi.

Request Header:

Tham số	Mô tả
Method	DELETE
Authorization	Bearer {access_token} (bắt buộc)

Path Parameters:

Tham số	Kiểu	Bắt buộc	Mô tả
notificationId	long	Có	ID của thông báo cần xóa

Response Body:

```

1 {
2   "code": "string",
3   "message": "string",
4   "data": null
5 }
```

Các Test Case:

STT	Mô tả	Kết quả mong đợi	Ghi chú
1	Xóa thông báo thành công	1000 OK	Input: DELETE /notification/123. Notification bị xóa. Update unread count nếu notification chưa đọc. Không thể phục hồi sau khi xóa
2	Xóa notification không tồn tại	9994 No data or end of list data	Input: DELETE /notification/99999
3	Xóa notification của user khác	1009 Action has been done previously by this user	notificationId thuộc user khác. Không có quyền
4	Xóa notification đã xóa trước đó	9994 No data or end of list data	Notification đã bị xóa. Không tồn tại
5	notificationId không hợp lệ	1003 Parameter value is invalid	Input: DELETE /notification/abc. ID phải là số
6	Token không hợp lệ	9998 Token is invalid	Cần đăng nhập

5 Xây dựng ứng dụng đa nền tảng

5.1 Kiến trúc ứng dụng

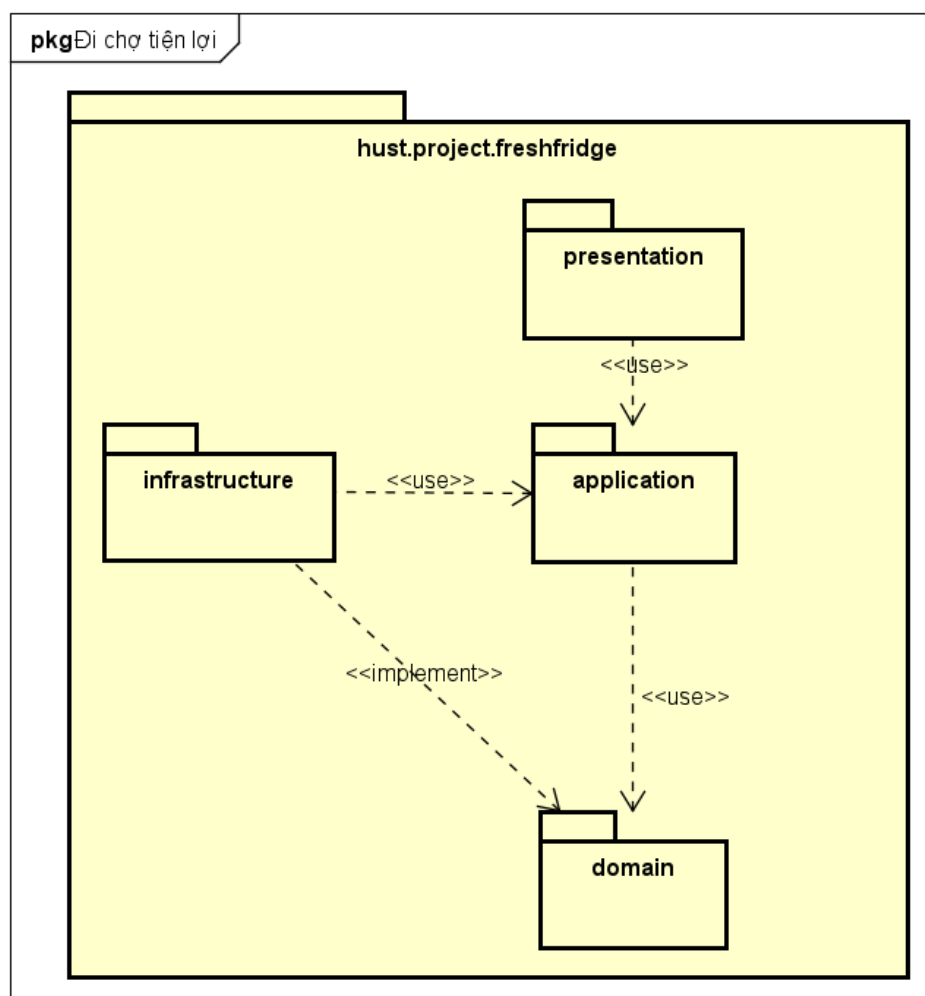
5.1.1 Lựa chọn kiến trúc phần mềm và giải thích sơ bộ

Ứng dụng “Đi chợ tiện lợi” được xây dựng theo mô hình **Client-Server Architecture** kết hợp với **RESTful API**, trong đó:

- **Client (Frontend):** Ứng dụng di động đa nền tảng phát triển bằng React Native + Expo.
- **Server (Backend):** Hệ thống API server xây dựng bằng Spring Boot (Java).
- **Database:** PostgreSQL để lưu trữ dữ liệu.

Kiến trúc Backend: Clean Architecture

Phía Backend được thiết kế theo **Clean Architecture** với nguyên tắc **Dependency Inversion** - các lớp bên trong không phụ thuộc vào các lớp bên ngoài.



Hình 49: Sơ đồ phân lớp các gói (Package Diagram) theo Clean Architecture

Vai trò của gói domain (Tầng lõi)

Đây là thành phần quan trọng nhất, chứa các thực thể nghiệp vụ (Entities) và các giao diện kho lưu trữ (Repository Interfaces). Gói này tập trung các quy tắc nghiệp vụ cốt lõi, được thiết kế độc lập hoàn toàn với framework hay công cụ bên thứ ba, giúp lõi ứng dụng ổn định trước các thay đổi công nghệ.

Vai trò của gói application (Tầng nghiệp vụ)

Gói này chứa các kịch bản sử dụng (Use Cases) điều phối logic ứng dụng. Nó đóng vai trò cầu nối, tiếp nhận yêu cầu từ tầng Presentation để thực thi các chức năng nghiệp vụ cụ thể thông qua việc tương tác với các Entity và Interface tại tầng Domain.

Vai trò của gói presentation (Tầng giao tiếp)

Đảm nhận nhiệm vụ xử lý tương tác với người dùng qua các API Controller. Các thành phần tại đây tiếp nhận Request HTTP, giải mã dữ liệu, chuyển giao cho tầng Application xử lý và trả về Response phù hợp cho ứng dụng Client.

Vai trò của gói infrastructure (Tầng hạ tầng)

Chứa các cài đặt kỹ thuật chi tiết hỗ trợ hệ thống, bao gồm triển khai thực tế các Repository (JPA), cấu hình cơ sở dữ liệu PostgreSQL, thiết lập bảo mật (Security) và các bộ chuyển đổi dữ liệu (Mappers).

Quy tắc thiết kế và sự phụ thuộc

Hệ thống tuân thủ nghiêm ngặt nguyên tắc phụ thuộc một chiều hướng vào tầng *domain*. Các gói được phân tầng rõ ràng: tầng *presentation* không được phép truy cập trực tiếp *domain* mà phải thông qua *application*. Đồng thời, việc áp dụng Dependency Inversion giúp tầng lõi không biết về sự tồn tại của các Framework bên ngoài, đảm bảo tính độc lập và khả năng kiểm thử (Testable) tối đa.

Kiến trúc Frontend: Component-Based Architecture

Phía Frontend sử dụng **Component-Based Architecture** kết hợp với **React Query** cho server state và **Zustand** cho client state:

- **Component-Based:** Giao diện được chia thành các component độc lập, tái sử dụng.
- **Unidirectional Data Flow:** Luồng dữ liệu một chiều với State Management.

5.1.2 Các cải tiến so với lý thuyết

1. **Tách biệt JPA Entity và Domain Entity:** Domain Entity là pure Java object, JPA Model chứa các annotations. Mapper class chịu trách nhiệm chuyển đổi qua lại, giúp lõi nghiệp vụ hoàn toàn sạch (clean).
2. **Use Case Pattern:** Thay vì một Service class khổng lồ, mỗi UseCase tập trung vào một nhóm chức năng cụ thể, giúp tuân thủ nguyên tắc Single Responsibility.
3. **Repository Pattern với Interface Segregation:** Domain layer định nghĩa interface, Infrastructure layer triển khai thực tế. Điều này cho phép dễ dàng thay đổi công nghệ lưu trữ (ví dụ từ SQL sang NoSQL) mà không sửa đổi logic nghiệp vụ.
4. **React Query cho Server State:** Thay vì quản lý mọi thứ trong Global State (như Redux), ứng dụng dùng React Query để tối ưu hóa việc caching, đồng bộ dữ liệu và giảm thiểu số lượng API call không cần thiết.

5.2 Tổ chức thư mục của dự án

Cấu trúc tệp và thư mục của một dự án đóng vai trò quyết định đến mức độ dễ dàng trong việc điều hướng, bảo trì và mở rộng hệ thống. Một tổ chức thư mục hợp lý giúp các nhà phát triển nhanh chóng tìm thấy các thành phần nghiệp vụ hoặc kỹ thuật tương ứng mà không làm xáo trộn các lớp kiến trúc.

5.2.1 Cấu trúc mã nguồn Backend (Spring Boot)

Dựa trên mô hình Clean Architecture, mã nguồn Backend được tổ chức phân lớp nghiêm ngặt để tách biệt logic nghiệp vụ khỏi các chi tiết hạ tầng.

```
src/main/java/hust/project/freshfridge/
|-- FreshfridgeApplication.java    # Điểm khởi chạy ứng dụng
|-- domain/                        # LAYER 1: Nghiệp vụ cốt lõi
|   |-- constant/                  # Hằng số nghiệp vụ
|   |-- entity/                    # Domain Entities (Pure Java Objects)
|   |-- exception/                 # Ngoại lệ cấp nghiệp vụ
|   |-- repository/                # Repository Interfaces (Ports)
|   |-- service/                   # Domain Services
|-- application/                   # LAYER 2: Logic ứng dụng
|   |-- dto/                       # Data Transfer Objects (Request/Response)
|   |-- scheduler/                 # Các tác vụ lập lịch tu động
|   |-- usecase/                   # Use Cases / Application Services
|-- infrastructure/                # LAYER 3: Các thành phần bên ngoài
|   |-- config/                    # Cấu hình Spring Boot
|   |-- persistence/               # Cài đặt lưu trữ (JPA, Models, Mappers)
|   |-- security/                  # Cấu hình bảo mật (JWT, Auth)
|   |-- service/                   # Dịch vụ hạ tầng kỹ thuật
|-- presentation/                  # LAYER 4: Tầng giao tiếp (API)
|   |-- controller/                # REST Controllers
|   |-- exception/                 # Global Exception Handlers

src/main/resources/
|-- application.yaml                # Cấu hình tham số môi trường
|-- db/migration/                  # Quản lý phiên bản DB (Flyway)
```

Phân tích ưu và nhược điểm (Backend)

- **Ưu điểm:** Tách biệt trách nhiệm rõ ràng (Separation of Concerns), giúp bảo trì dễ dàng khi cần thay đổi cơ sở dữ liệu hoặc framework mà không ảnh hưởng đến lõi nghiệp vụ. Cấu trúc này cũng tối ưu cho việc viết Unit Test cho tầng *domain*.
- **Nhược điểm:** Độ phức tạp ban đầu cao và phát sinh nhiều mã nguồn bổ trợ (Boilerplate code) như các Mapper và DTO để chuyển đổi dữ liệu giữa các lớp.

5.2.2 Cấu trúc mã nguồn Frontend (React Native)

Mã nguồn Frontend được tổ chức theo hướng mô-đun hóa và quản lý theo tính năng (feature-based), giúp tối ưu hóa việc tái sử dụng thành phần giao diện.

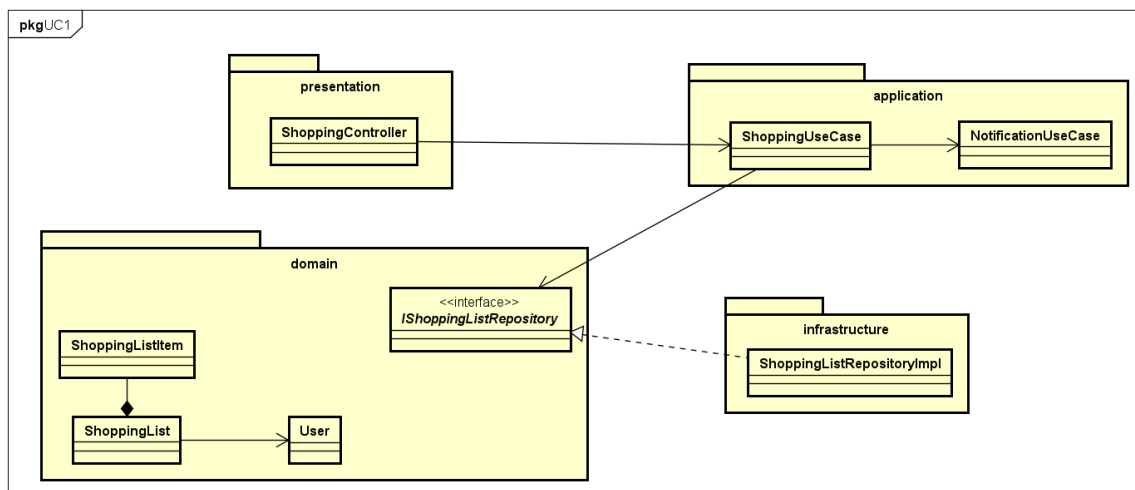
```
frontend/
|-- App.tsx                # Điểm khởi đầu (Root Component)
|-- app.json               # Cấu hình Expo/React Native
|-- src/                   # Mã nguồn chính
|   |-- api/               # Tầng giao tiếp mạng (Axios client, Endpoints)
|   |-- components/        # UI Components tái sử dụng (Common, Form)
|   |-- constants/         # Hằng số toàn cục
|   |-- hooks/             # Custom React Hooks
|   |-- screens/           # Các màn hình theo tính năng (Auth, Food, Home...)
|   |-- services/          # Business logic services (Auth, Storage)
|   |-- store/             # Quản lý trạng thái (Zustand/Redux)
|   |-- theme/             # Design system (Colors, Typography)
|   |-- types/             # TypeScript definitions
|   |-- utils/             # Hàm tiện ích dùng chung
|   |-- validation/        # Quy tắc kiểm tra dữ liệu (Zod/Yup)
```

Phân tích ưu và nhược điểm (Frontend)

- **Ưu điểm:** Cách tổ chức theo *screens* và *components* giúp nhà phát triển dễ dàng định vị mã nguồn theo chức năng hiển thị. Việc tách riêng tầng *api* và *store* giúp quản lý luồng dữ liệu tập trung, tránh tình trạng logic nghiệp vụ bị trộn lẫn vào giao diện.
- **Nhược điểm:** Cấu trúc thư mục có phân cấp sâu, đòi hỏi việc thiết lập Path Aliases (đường dẫn tắt) trong cấu hình TypeScript để tránh các đường dẫn import quá dài gây khó đọc.

5.3 Thiết kế chi tiết các gói

5.3.1 Thiết kế chi tiết cho UC1: Quản lý danh sách mua sắm



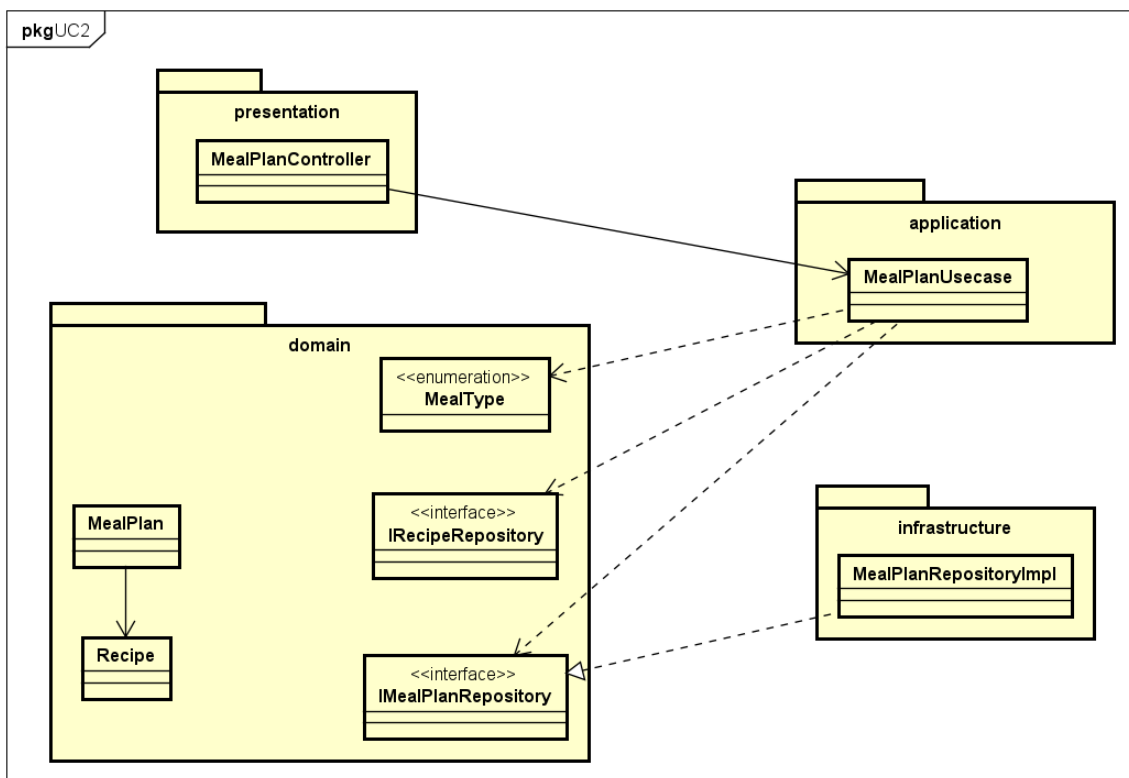
Hình 50: Biểu đồ gói chi tiết cho UC1: Quản lý danh sách mua sắm

Mô tả các mối quan hệ nghiệp vụ

- **Điều phối nghiệp vụ:** ShoppingController liên kết với ShoppingUseCase thông qua quan hệ *Association* để xử lý các yêu cầu tạo, cập nhật và xóa danh sách mua sắm từ người dùng.

- **Nghịch đảo phụ thuộc (DIP):** Tầng nghiệp vụ (ShoppingUseCase) tương tác với dữ liệu thông qua giao diện IShoppingListRepository. Việc thực thi chi tiết tại lớp ShoppingListRepositoryImpl thuộc tầng Infrastructure giúp hệ thống dễ dàng bảo trì và thay đổi công nghệ lưu trữ mà không làm ảnh hưởng đến logic lõi.
- **Quản lý mặt hàng:** Mỗi quan hệ *Composition* giữa ShoppingList và ShoppingListItem khẳng định tính toàn vẹn của dữ liệu; các mặt hàng mua sắm là thành phần không thể tách rời của một danh sách cụ thể.
- **Phân quyền và giao việc:** Quan hệ *Association* từ ShoppingList tới thực thể User mô tả chức năng gán trách nhiệm đi chợ cho một thành viên cụ thể trong nhóm, đáp ứng yêu cầu theo dõi tiến độ mua sắm của cá nhân.

5.3.2 Thiết kế chi tiết cho UC2: Quản lý dự định bữa ăn



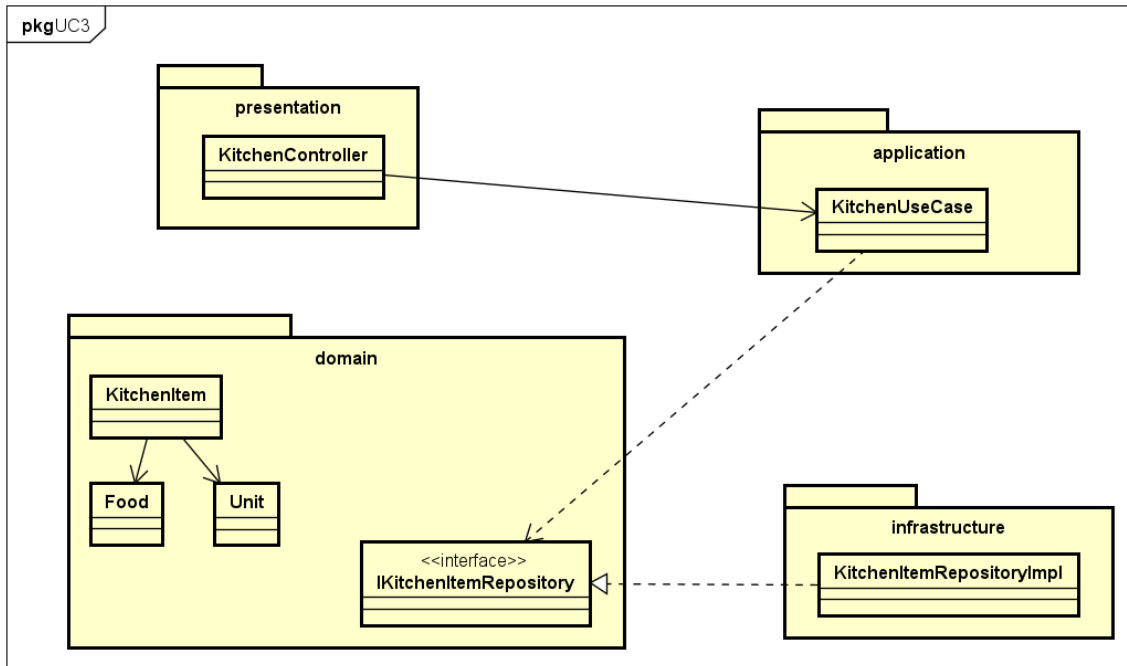
Hình 51: Biểu đồ gói chi tiết cho UC2: Quản lý dự định bữa ăn

Giải thích thiết kế và các mối quan hệ

- **Tầng Presentation:** MealPlanController tiếp nhận các yêu cầu HTTP để tạo, cập nhật hoặc lấy kế hoạch bữa ăn theo ngày/khoảng thời gian. Lớp này có quan hệ *Association* tới MealPlanUsecase.
- **Điều phối nghiệp vụ:** MealPlanUsecase đóng vai trò trung tâm, điều phối logic kiểm tra quyền truy cập công thức và tính hợp lệ của thời gian bữa ăn. Nó phụ thuộc (*Dependency*) vào các interface repository để tương tác với dữ liệu.
- **Cấu trúc thực thể:** Thực thể MealPlan lưu trữ thông tin về ngày ăn, loại bữa ăn (thông qua MealType) và liên kết tới một Recipe cụ thể. Mối quan hệ giữa MealPlan và Recipe đảm bảo mỗi kế hoạch bữa ăn luôn gắn liền với một hướng dẫn nấu ăn cụ thể.

- **Tính linh hoạt của hạ tầng:** Bằng việc sử dụng quan hệ *Realization* giữa *MealPlanRepositoryImpl* và *IMealPlanRepository*, hệ thống tách biệt hoàn toàn logic lưu trữ khỏi logic nghiệp vụ, cho phép thay đổi phương thức lưu trữ mà không ảnh hưởng đến các ca sử dụng của người dùng.

5.3.3 Thiết kế chi tiết cho UC3: Quản lý thực phẩm trong bếp

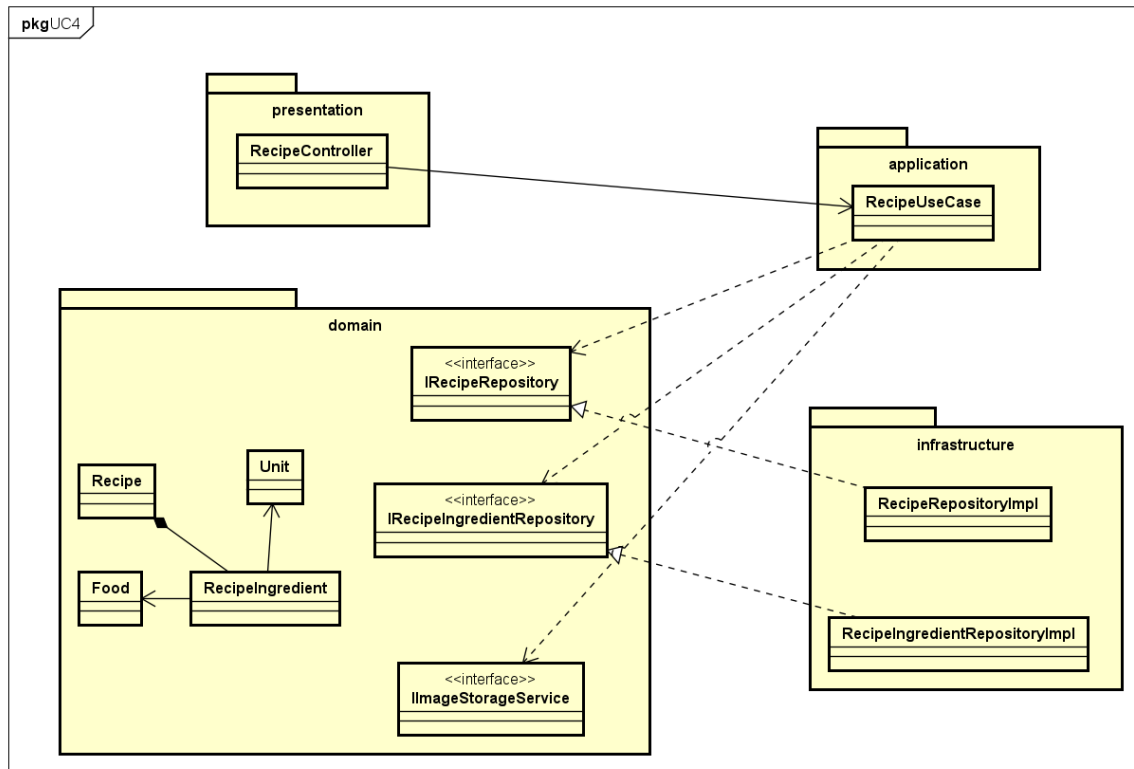


Hình 52: Biểu đồ gói chi tiết cho UC3: Quản lý thực phẩm trong bếp

Giải thích thiết kế và quan hệ giữa các lớp

- **Tầng Presentation và Application:** *KitchenController* nắm giữ một thực thể của *KitchenUseCase* qua quan hệ *Association*. Mọi yêu cầu như thêm thực phẩm vào tủ lạnh hoặc cập nhật lượng tồn kho đều được chuyển giao cho Use Case xử lý.
- **Tương tác dữ liệu nghiệp vụ:** *KitchenUseCase* phụ thuộc (*Dependency*) vào giao diện *IKitchenItemRepository*. Việc định nghĩa interface tại tầng Domain và triển khai tại tầng Infrastructure (*KitchenItemRepositoryImpl*) giúp hệ thống tuân thủ nguyên tắc Nghịch đảo phụ thuộc (DIP).
- **Cấu trúc thông tin thực phẩm:** Thực thể *KitchenItem* là đối tượng trung tâm quản lý các thông tin như ngày hết hạn và số lượng tồn thực tế. Nó có quan hệ *Association* tới *Food* để lấy thông tin định danh thực phẩm và tới *Unit* để xác định đơn vị đo lường.
- **Tính mở rộng:** Thiết kế này cho phép hệ thống dễ dàng tích hợp thêm các tính năng tự động như *ExpiryCheckScheduler* để quét các *KitchenItem* sắp hết hạn và gửi thông báo cho người dùng mà không cần thay đổi cấu trúc cốt lõi.

5.3.4 Thiết kế chi tiết cho UC4: Quản lý công thức nấu ăn

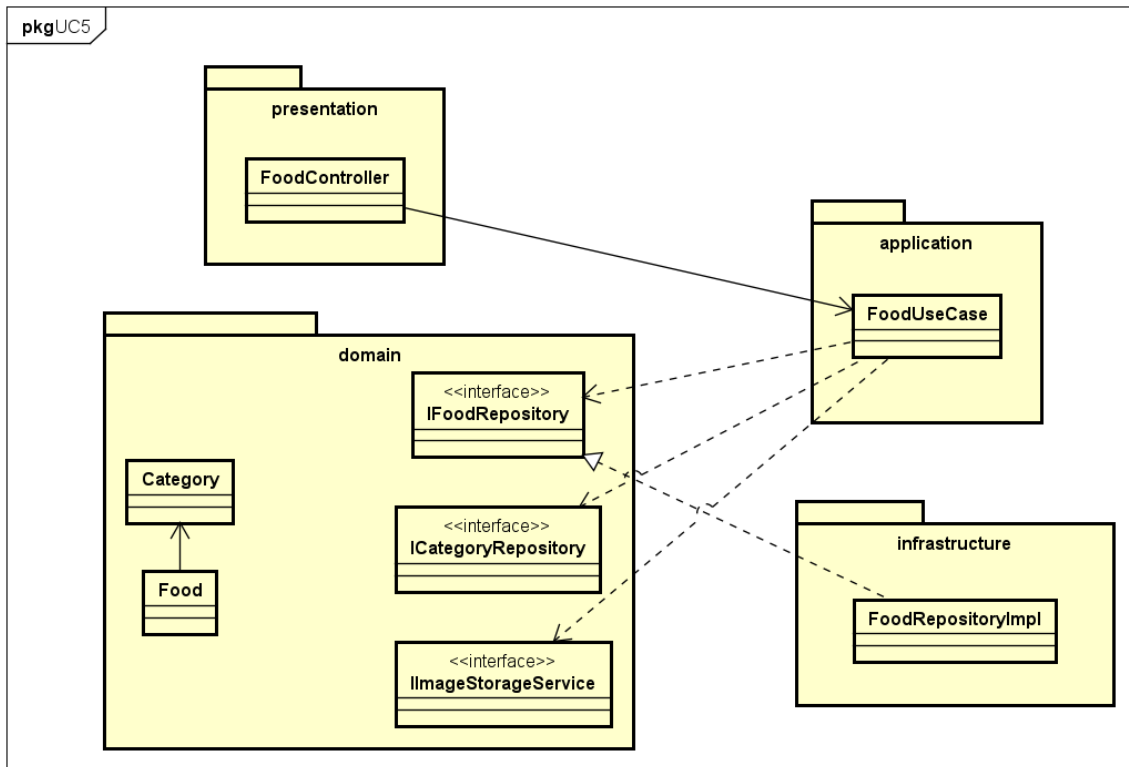


Hình 53: Biểu đồ gói chi tiết cho UC4: Quản lý công thức nấu ăn

Giải thích thiết kế và các mối quan hệ

- **Tầng Presentation và Application:** RecipeController đóng vai trò điều phối các yêu cầu HTTP, liên kết với RecipeUseCase qua quan hệ *Association*. Tầng nghiệp vụ chịu trách nhiệm kiểm tra quyền sở hữu công thức (theo nhóm hoặc hệ thống) trước khi thực hiện các thao tác chỉnh sửa.
- **Cấu trúc dữ liệu thành phần:** Mối quan hệ *Composition* giữa Recipe và RecipeIngredient thể hiện tính ràng buộc chặt chẽ; các nguyên liệu là bộ phận không thể tách rời của một công thức. Mỗi nguyên liệu (RecipeIngredient) lại kết hợp với Food và Unit để xác định rõ loại thực phẩm và định lượng cần thiết.
- **Tính linh hoạt và Đóng gói:** Hệ thống sử dụng các giao diện repository (IRecipeRepository) tại tầng Domain để che giấu chi tiết cài đặt hạ tầng. Quan hệ *Realization* từ các lớp Impl tại tầng Infrastructure giúp đảm bảo tầng nghiệp vụ không bị ảnh hưởng khi thay đổi công nghệ lưu trữ.
- **Dịch vụ hỗ trợ:** RecipeUseCase sử dụng IImageStorageService qua quan hệ *Dependency* để quản lý các tệp tin hình ảnh một cách độc lập, giúp giao diện người dùng trở nên trực quan hơn.

5.3.5 Thiết kế chi tiết cho UC5: Quản lý danh sách thực phẩm

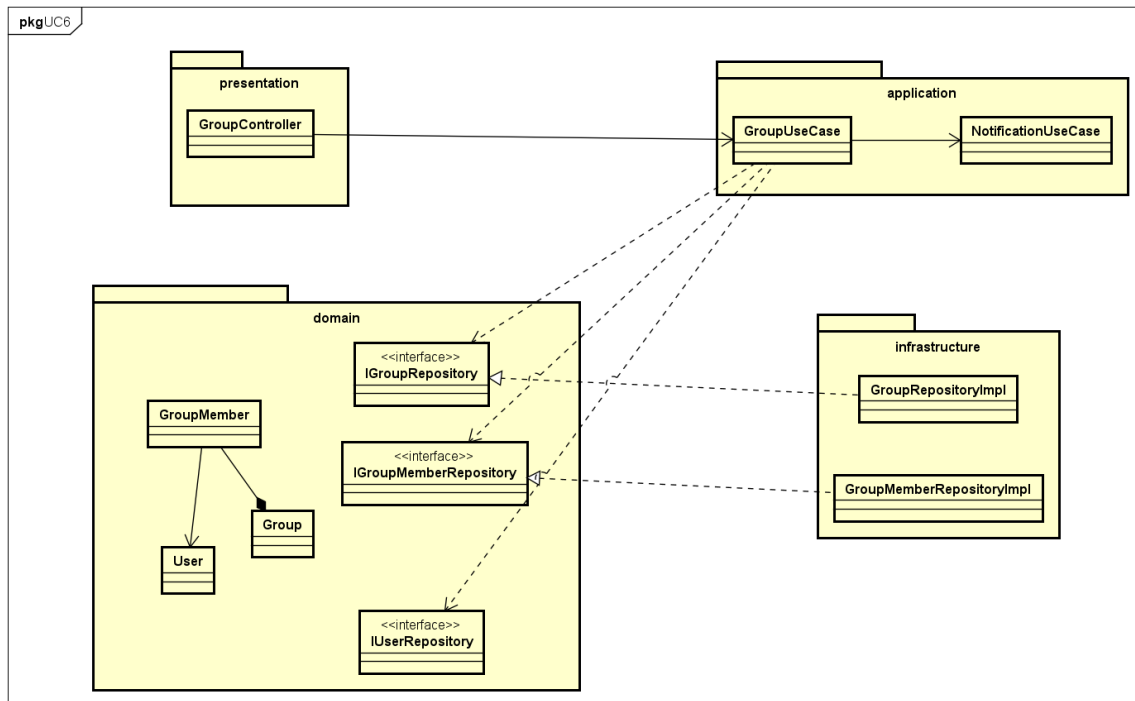


Hình 54: Biểu đồ gói chi tiết cho UC5: Quản lý danh sách thực phẩm

Giải thích thiết kế và quan hệ giữa các lớp

- **Điều phối giao tiếp:** FoodController tại tầng Presentation tiếp nhận các yêu cầu quản lý thực phẩm và liên kết với FoodUseCase qua quan hệ *Association*. Tầng này đảm bảo dữ liệu đầu vào được kiểm tra hợp lệ trước khi đi sâu vào logic nghiệp vụ.
- **Logic nghiệp vụ và Trừu tượng hóa:** FoodUseCase đóng vai trò điều phối chính, tương tác với các giao diện repository thông qua quan hệ *Dependency*. Việc phụ thuộc vào IFoodRepository và ICategoryRepository giúp tầng nghiệp vụ không bị ràng buộc vào các chi tiết lưu trữ dữ liệu cụ thể.
- **Cấu trúc thực thể và Phân loại:** Thực thể Food giữ vai trò trung tâm lưu trữ thông tin về tên, mô tả và hình ảnh. Quan hệ *Association* từ Food tới Category cho phép hệ thống quản lý thực phẩm theo từng nhóm danh mục khoa học, hỗ trợ việc tìm kiếm và lọc dữ liệu hiệu quả.
- **Cài đặt hạ tầng và Dịch vụ ngoại vi:** Lớp FoodRepositoryImpl thực hiện giao diện IFoodRepository (quan hệ *Realization*), kết nối trực tiếp với cơ sở dữ liệu. Ngoài ra, FoodUseCase còn sử dụng IImageStorageService để quản lý các tệp tin hình ảnh một cách chuyên biệt, tách biệt khỏi luồng dữ liệu nghiệp vụ thông thường.

5.3.6 Thiết kế chi tiết cho UC6: Quản lý nhóm

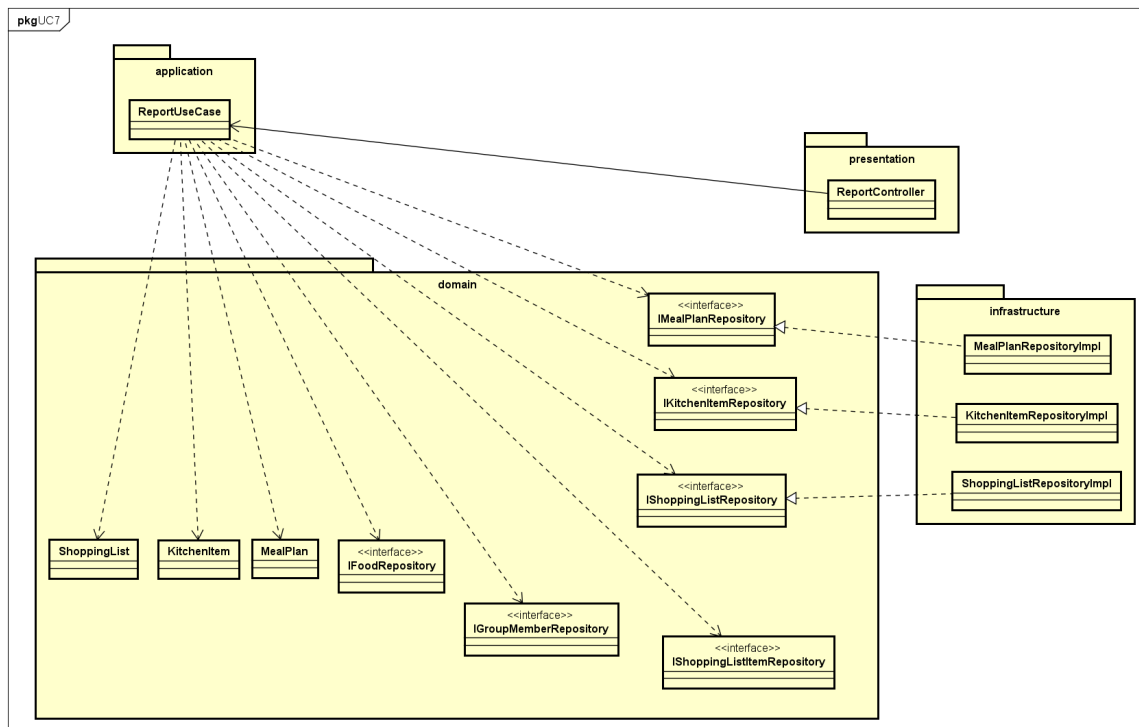


Hình 55: Biểu đồ gói chi tiết cho UC6: Quản lý nhóm

Giải thích thiết kế và quan hệ giữa các lớp

- **Tầng Presentation và Application:** GroupController đóng vai trò là điểm tiếp nhận các yêu cầu như tạo nhóm (*createGroup*), thêm thành viên (*addMember*) hoặc rời nhóm (*leaveGroup*), sau đó ủy thác cho GroupUseCase thông qua quan hệ *Association*.
- **Logic nghiệp vụ tập trung:** GroupUseCase điều phối toàn bộ luồng nghiệp vụ của nhóm. Lớp này sử dụng quan hệ *Dependency* tới các giao diện repository như IGroupRepository và IGroupMemberRepository để đảm bảo tính độc lập của tầng Domain theo kiến trúc Clean Architecture.
- **Cấu trúc ràng buộc thành viên:** Mỗi quan hệ *Composition* từ Group tới GroupMember thể hiện rằng danh sách thành viên là một phần hữu cơ của nhóm; khi nhóm bị giải tán bởi quản trị viên, các bản ghi thành viên liên quan sẽ tự động bị xóa bỏ để bảo đảm tính toàn vẹn dữ liệu.
- **Tương tác hệ thống:** GroupUseCase kết hợp với NotificationUseCase (quan hệ *Association*) để tự động gửi thông báo thời gian thực khi có biến động về nhân sự trong nhóm, ví dụ như gửi lời mời (*GROUP_INVITE*) hoặc thông báo thành viên mới gia nhập.
- **Thực thi hạ tầng:** Các lớp Impl tại tầng Infrastructure chịu trách nhiệm hiện thực hóa các giao diện trừu tượng (*Realization*), kết nối logic ứng dụng với hệ quản trị cơ sở dữ liệu mà không làm lộ các chi tiết cài đặt kỹ thuật lên tầng nghiệp vụ.

5.3.7 Thiết kế chi tiết cho UC7: Thống kê và báo cáo

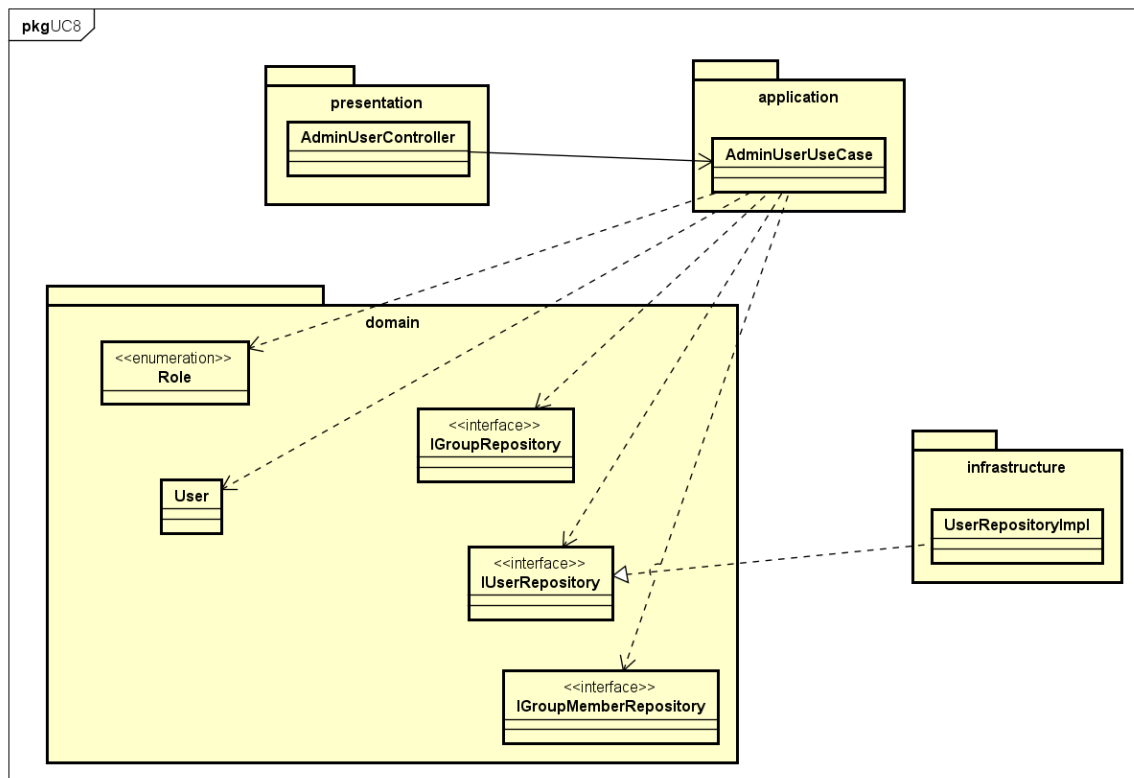


Hình 56: Biểu đồ gói chi tiết cho UC7: Thống kê và báo cáo

Giải thích thiết kế và quan hệ giữa các lớp

- **Trung tâm điều phối dữ liệu:** ReportController liên kết với ReportUseCase qua quan hệ *Association*. Tầng này chịu trách nhiệm cung cấp các điểm cuối (endpoints) để người dùng truy xuất các loại báo cáo khác nhau như báo cáo tổng quan (*Overview*), báo cáo kho bếp (*Kitchen*) và báo cáo mua sắm (*Shopping*).
- **Phụ thuộc đa nguồn:** ReportUseCase thể hiện quan hệ *Dependency* tới hàng loạt các giao diện kho lưu trữ (IKitchenItemRepository, IShoppingListRepository, v.v.). Điều này cho thấy tính chất của module báo cáo là tầng xử lý dữ liệu cấp cao, tổng hợp thông tin từ tất cả các mảng nghiệp vụ trong hệ thống.
- **Xử lý nghiệp vụ thống kê:** Tầng nghiệp vụ sử dụng các đối tượng dữ liệu như KitchenItem và ShoppingList để phân tích các chỉ số về hiệu quả đi chợ và mức độ lãng phí thực phẩm.
- **Tính đóng gói và bảo trì:** Việc tương tác hoàn toàn qua các interface giúp ReportUseCase không bị ảnh hưởng bởi cấu trúc bảng trong cơ sở dữ liệu. Quan hệ *Realization* từ tầng Infrastructure đảm bảo rằng logic thống kê luôn được tách biệt khỏi các truy vấn SQL phức tạp phía dưới.

5.3.8 Thiết kế chi tiết cho UC8: Quản trị người dùng



Hình 57: Biểu đồ gói chi tiết cho UC8: Quản trị người dùng

Giải thích thiết kế và quan hệ giữa các lớp

- **Điều phối quản trị:** AdminUserController tại tầng Presentation tiếp nhận các yêu cầu đặc quyền từ Quản trị viên, liên kết với AdminUserUseCase qua quan hệ *Association*. Tầng này được bảo vệ nghiêm ngặt bởi cơ chế phân quyền (*Role-based Access Control*).
- **Logic nghiệp vụ và Xử lý ràng buộc:** AdminUserUseCase chịu trách nhiệm thực hiện các quy tắc như ngăn cấm Admin tự vô hiệu hóa tài khoản chính mình. Lớp này sử dụng quan hệ *Dependency* tới IUserRepository để truy vấn và IGroupRepository để xử lý dọn dẹp dữ liệu nhóm (xóa nhóm nếu người dùng bị xóa là chủ nhóm).
- **Thực thể và Phân quyền:** Thực thể User đóng vai trò trung tâm lưu trữ trạng thái tài khoản. Quan hệ phụ thuộc tới Enum Role cho phép quản trị viên thay đổi vai trò giữa người dùng thông thường và quản trị viên một cách an toàn.
- **Thống kê và Báo cáo:** Bên trong AdminUserUseCase định nghĩa cấu trúc nội UserStatistics để tổng hợp các chỉ số về số lượng tài khoản đang hoạt động, đã xác thực hoặc đang bị khóa, hỗ trợ Admin trong việc nắm bắt sức khỏe hệ thống.
- **Tách biệt hạ tầng:** Việc hiện thực hóa giao diện qua UserRepositoryImpl (quan hệ *Realization*) giúp logic quản trị người dùng không bị phụ thuộc vào các chi tiết kỹ thuật của PostgreSQL hay Spring Data JPA.

5.4 Thiết kế chi tiết lớp

Phần này trình bày chi tiết về các lớp chính trong hệ thống theo mô hình CRC (Class-Responsibility-Collaboration), giúp làm rõ vai trò, trách nhiệm và mối quan hệ giữa các thành phần.

5.4.1 Các Lớp Entity (Domain Layer)

User (Người dùng)

Class Name	User
Responsibility	- Lưu trữ thông tin người dùng hệ thống - Quản lý xác thực và phân quyền - Theo dõi trạng thái tài khoản
Collaborators	- Group (là admin hoặc thành viên) - GroupMember (quan hệ membership) - Notification (nhận thông báo) - KitchenItem, Recipe, ShoppingList (người tạo)

Bảng 230: CRC Card - User

Thuộc tính	Kiểu dữ liệu	Mô tả
id	Long	ID định danh duy nhất của người dùng
email	String	Địa chỉ email (dùng để đăng nhập)
password	String	Mật khẩu đã được mã hóa
name	String	Họ tên người dùng
phone	String	Số điện thoại
avatarUrl	String	URL ảnh đại diện
role	String	Vai trò: "USER" hoặc "ADMIN"
isVerified	Boolean	Trạng thái xác minh email
isActive	Boolean	Trạng thái kích hoạt tài khoản
fcmToken	String	Token Firebase Cloud Messaging
createdAt	LocalDateTime	Thời điểm tạo tài khoản
updatedAt	LocalDateTime	Thời điểm cập nhật cuối

Bảng 231: Thuộc tính - User

Group (Nhóm gia đình)

Class Name	Group
Responsibility	- Đại diện cho một nhóm gia đình - Quản lý admin của nhóm - Làm đơn vị chia sẻ dữ liệu (thực phẩm, công thức, danh sách mua sắm)
Collaborators	- User (admin của nhóm) - GroupMember (danh sách thành viên) - KitchenItem, Food, Recipe, ShoppingList, MealPlan

Bảng 232: CRC Card - Group

Thuộc tính	Kiểu dữ liệu	Mô tả
id	Long	ID định danh duy nhất của nhóm
name	String	Tên nhóm
adminId	Long	ID của người quản trị nhóm
createdAt	LocalDateTime	Thời điểm tạo nhóm
updatedAt	LocalDateTime	Thời điểm cập nhật cuối

KitchenItem (Thực phẩm trong bếp)

Class Name	KitchenItem
Responsibility	- Quản lý thực phẩm trong bếp của nhóm - Theo dõi số lượng, hạn sử dụng - Cảnh báo thực phẩm sắp hết hạn
Collaborators	- Food (thực phẩm) - Group (nhóm sở hữu) - Unit (đơn vị đo) - User (người tạo)

Bảng 233: CRC Card - KitchenItem

Thuộc tính	Kiểu dữ liệu	Mô tả
id	Long	ID định danh
foodId	Long	ID thực phẩm
groupId	Long	ID nhóm
unitId	Long	ID đơn vị đo
quantity	BigDecimal	Số lượng
useWithin	Integer	Số ngày sử dụng
expiryDate	LocalDate	Ngày hết hạn
note	String	Ghi chú
createdBy	Long	ID người thêm
createdAt	LocalDateTime	Thời điểm tạo
updatedAt	LocalDateTime	Thời điểm cập nhật cuối

Bảng 234: Thuộc tính - KitchenItem

Recipe (Công thức nấu ăn)

Class Name	Recipe
Responsibility	- Lưu trữ công thức nấu ăn - Quản lý thông tin chi tiết (thời gian, độ khó, khẩu phần) - Hỗ trợ công thức hệ thống và công thức riêng của nhóm
Collaborators	- RecipeIngredient (nguyên liệu) - Group (nhóm sở hữu) - MealPlan (kế hoạch bữa ăn) - User (người tạo)

Bảng 235: CRC Card - Recipe

Thuộc tính	Kiểu dữ liệu	Mô tả
id	Long	ID định danh
name	String	Tên công thức
description	String	Mô tả ngắn
htmlContent	String	Nội dung HTML chi tiết
imageUrl	String	URL hình ảnh
groupId	Long	ID nhóm (null nếu là công thức hệ thống)
isSystem	Boolean	Đánh dấu công thức hệ thống
createdBy	Long	ID người tạo
difficulty	String	Độ khó: "EASY", "MEDIUM", "HARD"
prepTime	Integer	Thời gian chuẩn bị (phút)
cookTime	Integer	Thời gian nấu (phút)
servings	Integer	Số khẩu phần
tags	List<String>	Danh sách tags
createdAt	LocalDateTime	Thời điểm tạo
updatedAt	LocalDateTime	Thời điểm cập nhật cuối

Bảng 236: Thuộc tính - Recipe

ShoppingList (Danh sách mua sắm)

Class Name	ShoppingList
Responsibility	- Quản lý danh sách mua sắm của nhóm - Giao việc cho thành viên - Theo dõi trạng thái hoàn thành
Collaborators	- Group (nhóm) - User (người được giao, người tạo) - ShoppingListItem (các mục cần mua)

Bảng 237: CRC Card - ShoppingList

Thuộc tính	Kiểu dữ liệu	Mô tả
id	Long	ID định danh
name	String	Tên danh sách
groupId	Long	ID nhóm
assignedToUserId	Long	ID người được giao
note	String	Ghi chú
shoppingDate	LocalDate	Ngày mua sắm dự kiến
status	String	Trạng thái: "PENDING", "IN_PROGRESS", "COMPLETED"
createdBy	Long	ID người tạo
createdAt	LocalDateTime	Thời điểm tạo
updatedAt	LocalDateTime	Thời điểm cập nhật cuối

Bảng 238: Thuộc tính - ShoppingList

5.4.2 Các Lớp Repository (Domain Layer)

IUserRepository

Class Name	IUserRepository
Responsibility	- Định nghĩa interface truy cập dữ liệu User - CRUD operations cho User - Tìm kiếm và lọc người dùng
Collaborators	- User (entity)

Bảng 239: CRC Card - IUserRepository

Phương thức	Kiểu trả về	Mô tả
save(User user)	User	Lưu/cập nhật người dùng
findById(Long id)	Optional<User>	Tìm theo ID
findByEmail(String email)	Optional<User>	Tìm theo email
existsByEmail(String)	boolean	Kiểm tra email tồn tại
count()	long	Đếm tổng số người dùng
deleteById(Long id)	void	Xóa người dùng

Bảng 240: Phương thức - IUserRepository

IKitchenItemRepository

Class Name	IKitchenItemRepository
Responsibility	- Định nghĩa interface truy cập dữ liệu KitchenItem - Tìm kiếm thực phẩm sắp hết hạn
Collaborators	- KitchenItem (entity)

Bảng 241: CRC Card - IKitchenItemRepository

Phương thức	Kiểu trả về	Mô tả
save(KitchenItem)	KitchenItem	Lưu/cập nhật
findById(Long id)	Optional<KitchenItem>	Tìm theo ID
findByGroupId(Long)	List<KitchenItem>	Lấy theo nhóm
findExpiringItems(...)	List<KitchenItem>	Tìm thực phẩm sắp hết hạn
deleteById(Long id)	void	Xóa

Bảng 242: Phương thức - IKitchenItemRepository

IRecipeRepository

Class Name	IRecipeRepository
Responsibility	- Định nghĩa interface truy cập dữ liệu Recipe - Tìm kiếm công thức hệ thống và của nhóm
Collaborators	- Recipe (entity)

Bảng 243: CRC Card - IRecipeRepository

Phương thức	Kiểu trả về	Mô tả
save(Recipe recipe)	Recipe	Lưu/cập nhật
findById(Long id)	Optional<Recipe>	Tìm theo ID
findByGroupId(Long)	List<Recipe>	Lấy theo nhóm
findSystemRecipes()	List<Recipe>	Lấy công thức hệ thống
searchByName(...)	List<Recipe>	Tìm theo tên
deleteById(Long id)	void	Xóa

Bảng 244: Phương thức - IRecipeRepository

5.4.3 Các Lớp UseCase (Application Layer)

UserUseCase

Class Name	UserUseCase
Responsibility	- Xử lý nghiệp vụ đăng ký, đăng nhập - Quản lý profile người dùng - Xác thực email, đổi mật khẩu
Collaborators	- IUserRepository - PasswordEncoder - JwtTokenProvider - EmailService - IImageStorageService

Bảng 245: CRC Card - UserUseCase

Phương thức	Kiểu trả về	Mô tả
register(RegisterRequest)	AuthResponse	Đăng ký tài khoản mới
login(LoginRequest)	AuthResponse	Đăng nhập
verifyEmail(...)	void	Xác thực email
changePassword(...)	void	Đổi mật khẩu
getProfile(Long userId)	UserResponse	Lấy thông tin profile
updateProfile(...)	UserResponse	Cập nhật profile

Bảng 246: Phương thức chính - UserUseCase

GroupUseCase

Class Name	GroupUseCase
Responsibility	- Xử lý nghiệp vụ tạo và quản lý nhóm - Quản lý thành viên nhóm - Gửi thông báo khi có thay đổi
Collaborators	- IGroupRepository - IGroupMemberRepository - IUserRepository - NotificationUseCase

Bảng 247: CRC Card - GroupUseCase

Phương thức	Kiểu trả về	Mô tả
createGroup(...)	GroupResponse	Tạo nhóm mới
getGroup(Long userId)	GroupResponse	Lấy thông tin nhóm
getGroupMembers(...)	List<MemberResponse>	Lấy danh sách thành viên
addMember(...)	GroupResponse	Thêm thành viên
removeMember(...)	void	Xóa thành viên
leaveGroup(Long userId)	void	Rời khỏi nhóm

Bảng 248: Phương thức chính - GroupUseCase

KitchenUseCase

Class Name	KitchenUseCase
Responsibility	- Quản lý thực phẩm trong bếp - Theo dõi hạn sử dụng - Lọc và tìm kiếm thực phẩm
Collaborators	- IKitchenItemRepository - IFoodRepository - ICategoryRepository - IUnitRepository - IGroupMemberRepository

Bảng 249: CRC Card - KitchenUseCase

Phương thức	Kiểu trả về	Mô tả
createKitchenItem(...)	KitchenItemResponse	Thêm thực phẩm vào bếp
updateKitchenItem(...)	KitchenItemResponse	Cập nhật thực phẩm
deleteKitchenItem(...)	void	Xóa thực phẩm
getKitchenItems(...)	List<KitchenItemResponse>	Lấy danh sách với bộ lọc
getExpiringItems(...)	List<KitchenItemResponse>	Lấy thực phẩm sắp hết hạn

Bảng 250: Phương thức chính - KitchenUseCase

RecipeUseCase

Class Name	RecipeUseCase
Responsibility	- Quản lý công thức nấu ăn - Quản lý nguyên liệu công thức - Phân quyền truy cập công thức
Collaborators	- IRecipeRepository - IRecipeIngredientRepository - IFoodRepository - IGroupMemberRepository - IImageStorageService

Bảng 251: CRC Card - RecipeUseCase

Phương thức	Kiểu trả về	Mô tả
createRecipe(...)	RecipeResponse	Tạo công thức mới
updateRecipe(...)	RecipeResponse	Cập nhật công thức
deleteRecipe(...)	void	Xóa công thức
getRecipes(...)	List<RecipeResponse>	Lấy danh sách công thức
findRecipesByIngredient(...)	List<RecipeResponse>	Tìm công thức theo nguyên liệu
addIngredient(...)	RecipeResponse	Thêm nguyên liệu

Bảng 252: Phương thức chính - RecipeUseCase

ShoppingUseCase

Class Name	ShoppingUseCase
Responsibility	- Quản lý danh sách mua sắm - Quản lý các mục trong danh sách - Đánh dấu đã mua và thêm vào bếp
Collaborators	- IShoppingListRepository - IShoppingListItemRepository - IFoodRepository - IUserRepository - NotificationUseCase

Bảng 253: CRC Card - ShoppingUseCase

Phương thức	Kiểu trả về	Mô tả
createShoppingList(...)	ShoppingListResponse	Tạo danh sách mới
updateShoppingList(...)	ShoppingListResponse	Cập nhật danh sách
deleteShoppingList(...)	void	Xóa danh sách
addItems(...)	ShoppingListResponse	Thêm mục vào danh sách
markItemPurchased(...)	ShoppingItemResponse	Đánh dấu đã mua

Bảng 254: Phương thức chính - ShoppingUseCase

MealPlanUseCase

Class Name	MealPlanUseCase
Responsibility	- Lập kế hoạch bữa ăn - Quản lý lịch ăn theo ngày/tuần - Lấy kế hoạch theo khoảng thời gian
Collaborators	- IMealPlanRepository - IRecipeRepository - IGroupMemberRepository - IGroupRepository

Bảng 255: CRC Card - MealPlanUseCase

Phương thức	Kiểu trả về	Mô tả
createMealPlan(...)	MealPlanResponse	Tạo kế hoạch bữa ăn
updateMealPlan(...)	MealPlanResponse	Cập nhật kế hoạch
getMealPlans(...)	List<MealPlanResponse>	Lấy tất cả kế hoạch của nhóm
getMealPlansByDateRange(...)	List<MealPlanResponse>	Lấy theo khoảng ngày
getDailyMealPlan(...)	DailyMealPlanResponse	Lấy kế hoạch trong ngày

Bảng 256: Phương thức chính - MealPlanUseCase

ReportUseCase

Class Name	ReportUseCase
Responsibility	- Tổng hợp dữ liệu thống kê - Tạo báo cáo tổng quan, kho bếp, mua sắm - Phân tích xu hướng sử dụng
Collaborators	- IKitchenItemRepository - IShoppingListRepository - IMealPlanRepository - IRecipeRepository

Bảng 257: CRC Card - ReportUseCase

Phương thức	Kiểu trả về	Mô tả
getOverviewReport(...)	OverviewReportResponse	Lấy báo cáo tổng quan
getKitchenReport(...)	KitchenReportResponse	Báo cáo kho bếp
getShoppingReport(...)	ShoppingReportResponse	Báo cáo mua sắm
getWasteAnalysis(...)	WasteAnalysisResponse	Phân tích lãng phí

Bảng 258: Phương thức chính - ReportUseCase

AdminUserUseCase

Class Name	AdminUserUseCase
Responsibility	- Quản trị người dùng hệ thống - Vô hiệu hóa/kích hoạt tài khoản - Thống kê người dùng
Collaborators	- IUserRepository - IGroupRepository

Bảng 259: CRC Card - AdminUserUseCase

Phương thức	Kiểu trả về	Mô tả
getAllUsers(...)	List<UserResponse>	Lấy danh sách người dùng
getUserStatistics()	UserStatistics	Lấy thống kê người dùng
toggleUserStatus(...)	void	Bật/tắt trạng thái tài khoản
changeUserRole(...)	void	Thay đổi vai trò người dùng
deleteUser(...)	void	Xóa người dùng

Bảng 260: Phương thức chính - AdminUserUseCase

5.4.4 Các Lớp Controller (Presentation Layer)

AuthController

Class Name	AuthController
Responsibility	- Xử lý HTTP requests cho xác thực - Định nghĩa các endpoints đăng ký, đăng nhập - Validate và chuyển đổi request/response
Collaborators	- UserUseCase

Bảng 261: CRC Card - AuthController

Endpoint	Method	Mô tả
/api/auth/register	POST	Đăng ký tài khoản mới
/api/auth/login	POST	Đăng nhập hệ thống
/api/auth/verify-email	POST	Xác thực email
/api/auth/resend-verification	POST	Gửi lại mã xác thực

Bảng 262: Các endpoint - AuthController

GroupController

Class Name	GroupController
Responsibility	- Xử lý HTTP requests cho quản lý nhóm - Định nghĩa các endpoints quản lý thành viên - Validate và chuyển đổi request/response
Collaborators	- GroupUseCase - UserPrincipal (current user)

Bảng 263: CRC Card - GroupController

Endpoint	Method	Mô tả
/api/groups	POST	Tạo nhóm mới
/api/groups/my-group	GET	Lấy thông tin nhóm hiện tại
/api/groups/members	GET	Lấy danh sách thành viên
/api/groups/members	POST	Thêm thành viên
/api/groups/members/{id}	DELETE	Xóa thành viên
/api/groups/leave	POST	Rời khỏi nhóm

Bảng 264: Các endpoint - GroupController

KitchenController

Class Name	KitchenController
Responsibility	- Xử lý HTTP requests cho quản lý thực phẩm trong bếp - Validate và chuyển đổi request/response - Định nghĩa các endpoints RESTful
Collaborators	- KitchenUseCase - UserPrincipal (current user)

Bảng 265: CRC Card - KitchenController

Endpoint	Method	Mô tả
/api/kitchen	POST	Thêm thực phẩm vào bếp
/api/kitchen	GET	Lấy danh sách thực phẩm
/api/kitchen/{id}	GET	Lấy chi tiết thực phẩm
/api/kitchen/{id}	PUT	Cập nhật thực phẩm
/api/kitchen/{id}	DELETE	Xóa thực phẩm
/api/kitchen/expiring	GET	Lấy thực phẩm sắp hết hạn

Bảng 266: Các endpoint - KitchenController

RecipeController

Class Name	RecipeController
Responsibility	- Xử lý HTTP requests cho quản lý công thức - Validate và chuyển đổi request/response - Định nghĩa các endpoints RESTful
Collaborators	- RecipeUseCase - UserPrincipal (current user)

Bảng 267: CRC Card - RecipeController

Endpoint	Method	Mô tả
/api/recipes	POST	Tạo công thức mới
/api/recipes	GET	Lấy danh sách công thức
/api/recipes/{id}	GET	Lấy chi tiết công thức
/api/recipes/{id}	PUT	Cập nhật công thức
/api/recipes/{id}	DELETE	Xóa công thức
/api/recipes/{id}/ingredients	POST	Thêm nguyên liệu

Bảng 268: Các endpoint - RecipeController

ShoppingController

Class Name	ShoppingController
Responsibility	- Xử lý HTTP requests cho quản lý danh sách mua sắm - Validate và chuyển đổi request/response - Định nghĩa các endpoints RESTful
Collaborators	- ShoppingUseCase - UserPrincipal (current user)

Bảng 269: CRC Card - ShoppingController

Endpoint	Method	Mô tả
/api/shopping-lists	POST	Tạo danh sách mua sắm
/api/shopping-lists	GET	Lấy danh sách mua sắm
/api/shopping-lists/{id}	GET	Lấy chi tiết
/api/shopping-lists/{id}	PUT	Cập nhật danh sách
/api/shopping-lists/{id}	DELETE	Xóa danh sách
/api/shopping-lists/{id}/items	POST	Thêm mục mua sắm

Bảng 270: Các endpoint - ShoppingController

MealPlanController

Class Name	MealPlanController
Responsibility	- Xử lý HTTP requests cho quản lý kế hoạch bữa ăn - Validate và chuyển đổi request/response - Định nghĩa các endpoints RESTful
Collaborators	- MealPlanUseCase - UserPrincipal (current user)

Bảng 271: CRC Card - MealPlanController

Endpoint	Method	Mô tả
/api/meal-plans	POST	Tạo kế hoạch bữa ăn
/api/meal-plans	GET	Lấy danh sách kế hoạch
/api/meal-plans/{id}	GET	Lấy chi tiết
/api/meal-plans/{id}	PUT	Cập nhật kế hoạch
/api/meal-plans/{id}	DELETE	Xóa kế hoạch
/api/meal-plans/daily	GET	Lấy kế hoạch trong ngày

Bảng 272: Các endpoint - MealPlanController

ReportController

Class Name	ReportController
Responsibility	- Xử lý HTTP requests cho báo cáo và thống kê - Validate tham số thời gian - Định nghĩa các endpoints RESTful
Collaborators	- ReportUseCase - UserPrincipal (current user)

Bảng 273: CRC Card - ReportController

Endpoint	Method	Mô tả
/api/reports/overview	GET	Lấy báo cáo tổng quan
/api/reports/kitchen	GET	Lấy báo cáo kho bếp
/api/reports/shopping	GET	Lấy báo cáo mua sắm
/api/reports/waste	GET	Phân tích lãng phí

Bảng 274: Các endpoint - ReportController

AdminUserController

Class Name	AdminUserController
Responsibility	- Xử lý HTTP requests cho quản trị người dùng - Validate quyền Admin - Định nghĩa các endpoints RESTful
Collaborators	- AdminUserUseCase - UserPrincipal (current user)

Bảng 275: CRC Card - AdminUserController

Endpoint	Method	Mô tả
/api/admin/users	GET	Lấy danh sách người dùng
/api/admin/users/statistics	GET	Lấy thống kê người dùng
/api/admin/users/{id}/toggle-status	PUT	Bật/tắt trạng thái
/api/admin/users/{id}/role	PUT	Thay đổi vai trò
/api/admin/users/{id}	DELETE	Xóa người dùng

Bảng 276: Các endpoint - AdminUserController

5.5 Các giải pháp khác đã xây dựng trong chương trình

Trong quá trình phát triển ứng dụng, bên cạnh các tính năng cốt lõi, nhóm đã triển khai một loạt các giải pháp kỹ thuật hỗ trợ nhằm tối ưu hóa hiệu suất, tăng cường tính bảo mật và nâng cao trải nghiệm người dùng. Phần này trình bày chi tiết về cách thức xử lý giao tiếp hệ thống, quản lý dữ liệu và tận dụng các dịch vụ nền tảng của thiết bị di động.

5.5.1 Cơ chế giao tiếp với Back-end và xử lý truy vấn API

Trong môi trường ứng dụng di động, việc quản lý các luồng dữ liệu thông qua HTTP Request đòi hỏi sự tập trung để xử lý các vấn đề về xác thực và đồng bộ hóa. Hệ thống đã triển khai giải pháp dựa trên Axios và React Query:

- **Axios Interceptors:** Thiết lập một instance Axios tập trung để quản lý logic giao tiếp.
 - *Request Interceptor:* Tự động trích xuất JWT Token từ kho lưu trữ bảo mật và đính kèm vào header **Authorization** cho mọi yêu cầu.
 - *Response Interceptor:* Xử lý lỗi hệ thống tập trung. Đặc biệt, cơ chế tự động làm mới mã xác thực (Auto-refresh Token) được kích hoạt khi nhận mã lỗi 401, giúp duy trì phiên làm việc của người dùng mà không cần đăng nhập lại.
- **Quản lý trạng thái Server với React Query:** Thay vì quản lý dữ liệu API bằng các biến state thông thường, dự án sử dụng `@tanstack/react-query` để tối ưu hóa hiệu suất thông qua:
 - Cơ chế *Caching* dữ liệu giúp giảm thiểu số lượng Request không cần thiết.
 - *Optimistic Updates* để cập nhật giao diện ngay lập tức trước khi nhận phản hồi từ Server, tạo cảm giác mượt mà cho người dùng.
 - Tự động thực hiện truy vấn lại (Background Refetching) khi thiết bị kết nối mạng trở lại.
- **Đảm bảo an toàn kiểu dữ liệu (Type-safety):** Toàn bộ các Endpoint được định nghĩa dưới dạng module hóa trong TypeScript, áp dụng Generic Types cho cả Request và Response, giúp hạn chế tối đa lỗi Runtime.

5.5.2 Chiến lược quản lý trạng thái ứng dụng (State Management)

Ứng dụng sở hữu cấu trúc dữ liệu phức tạp bao gồm quản lý nhóm, kho thực phẩm và lịch ăn uống. Tác giả đã áp dụng mô hình quản lý trạng thái phân tầng:

- **Trạng thái phía Client (Zustand):** Sử dụng thư viện Zustand với ưu điểm nhẹ và hiệu suất cao hơn so với Redux. Các trạng thái về phiên đăng nhập (*Auth State*), bộ lọc dữ liệu (*Filters*) và tùy chỉnh người dùng được tổ chức thành các Slice riêng biệt.
- **Trạng thái phía Server (React Query):** Đóng vai trò là nguồn dữ liệu tin cậy (Single Source of Truth) cho các thông tin từ cơ sở dữ liệu.
- **Cơ chế duy trì trạng thái (Persistence):** Tích hợp Middleware `persist` của Zustand kết hợp với `AsyncStorage` để duy trì các trạng thái quan trọng ngay cả khi ứng dụng bị đóng hoặc khởi động lại thiết bị.

5.5.3 Giải pháp lưu trữ dữ liệu cục bộ và bảo mật

Để đáp ứng các yêu cầu khác nhau về bảo mật và hiệu suất, hệ thống lưu trữ phía Client được thiết kế theo mô hình hai tầng:

Giải pháp này đảm bảo tuân thủ các tiêu chuẩn bảo mật cho ứng dụng di động (như OWASP Mobile Top 10) bằng cách mã hóa các thông tin nhạy cảm ở cấp độ phần cứng (Keychain trên iOS và EncryptedSharedPreferences trên Android).

Loại lưu trữ	Công nghệ sử dụng	Dữ liệu quản lý
Secure Storage	Expo Secure Store (AES-256)	JWT Tokens, Credentials.
Standard Storage	AsyncStorage	User Profile, Settings, Caches.

Bảng 277: Phân tầng lưu trữ dữ liệu cục bộ

5.5.4 Hệ thống đa ngôn ngữ và chuẩn hóa hiển thị

Mặc dù mục tiêu hiện tại là thị trường Việt Nam, kiến trúc ứng dụng vẫn được thiết kế để sẵn sàng mở rộng đa ngôn ngữ (Internationalization) thông qua giải pháp *Constants-based Mapping*:

- **Ánh xạ Enum:** Chuyển đổi các giá trị Enum từ Backend (thường là tiếng Anh như VEGETABLES, FRIDGE) sang ngôn ngữ hiển thị tương ứng thông qua một lớp Utility tập trung.
- **Quản lý thông báo tập trung:** Toàn bộ các thông điệp lỗi và thông báo thành công được định nghĩa trong các hằng số, giúp đảm bảo tính nhất quán về thuật ngữ trên toàn bộ giao diện người dùng.

5.5.5 Tích hợp dịch vụ nền tảng (Native Features)

Thông qua Expo SDK, ứng dụng đã khai thác tối đa các tính năng đặc trưng của thiết bị di động để nâng cao trải nghiệm người dùng:

- **Push Notifications:** Triển khai thông báo đẩy nhằm nhắc nhở người dùng về thực phẩm sắp hết hạn và lịch trình bữa ăn. Hệ thống hỗ trợ xử lý thông báo cả khi ứng dụng đang chạy nền hoặc đã đóng hoàn toàn.
- **Xử lý hình ảnh (Camera & Image Picker):** Cho phép người dùng cập nhật ảnh thực phẩm và công thức nấu ăn. Hình ảnh trước khi tải lên được xử lý nén (Compression) và định lại kích cỡ (Resizing) để tiết kiệm băng thông và không gian lưu trữ của Server.
- **Tối ưu hóa hiển thị hình ảnh:** Sử dụng thư viện expo-image hỗ trợ các định dạng thể hệ mới như WebP và AVIF, kết hợp với cơ chế hiển thị *Blurhash* giúp cải thiện tốc độ tải trang và chỉ số trải nghiệm người dùng (UX).

Kết luận chung về các giải pháp kỹ thuật:

Các giải pháp nêu trên đã tạo ra một nền tảng kỹ thuật vững chắc cho ứng dụng Fresh Fridge. Việc kết hợp giữa các công nghệ hiện đại như React Query, Zustand và Expo SDK không chỉ giải quyết được bài toán về hiệu suất và bảo mật ở thời điểm hiện tại mà còn đảm bảo khả năng mở rộng và bảo trì hệ thống trong tương lai.

6 Kiểm thử chương trình

6.1 Kiểm thử các chức năng đã thực hiện

Phần này mô tả chi tiết quá trình kiểm thử các chức năng cốt lõi của hệ thống “Đi chợ tiện lợi” nhằm đảm bảo phần mềm hoạt động đúng yêu cầu thiết kế và xử lý tốt các logic nghiệp vụ đặc thù.

Kỹ thuật kiểm thử sử dụng:

- **Kiểm thử hộp đen (Black-box Testing):** Tập trung vào đầu vào và đầu ra dựa trên mô tả Use Case.
- **Phân vùng tương đương (Equivalence Partitioning):** Chia nhóm dữ liệu hợp lệ (đúng định dạng, logic đúng) và không hợp lệ (sai định dạng, logic sai).
- **Phân tích giá trị biên (Boundary Value Analysis):** Kiểm tra số lượng (0, số âm, số cực lớn), độ dài mật khẩu, ngày hết hạn (quá khứ/tương lai).

6.1.1 Kiểm thử cho chức năng: Đăng ký tài khoản (UC-09.1)

Mô tả chức năng: Đăng ký tài khoản mới cho người dùng.

Lưu ý: Mật khẩu tối thiểu 8 ký tự, có chữ hoa và số. Email và SĐT là duy nhất.

A. Thiết kế Test Case chi tiết

Mục	Nội dung chi tiết
Test No.	TC_AUTH_01
Current status	☒ Passed ☐ Failed ☐ Pending
Title	Đăng ký tài khoản thành công với dữ liệu hợp lệ
Description	Đảm bảo người dùng tạo được tài khoản khi nhập đúng định dạng và đủ độ mạnh mật khẩu.
Approach	Kiểm thử luồng cơ bản (Happy Path).

Bảng 278: Thông tin Test Case TC_AUTH_01

Các bước thực hiện:

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
1	Mở màn hình Đăng ký	Truy cập chức năng	Form đăng ký hiển thị đầy đủ các trường	✓
2	Nhập "Họ tên": Nguyễn Văn A	Nhập dữ liệu text	Dữ liệu được ghi nhận	✓
3	Nhập "Email": nva@test.com (chưa tồn tại)	Nhập khóa định danh	Dữ liệu được ghi nhận	✓
4	Nhập "SĐT": 0987654321	Nhập thông tin liên hệ	Dữ liệu được ghi nhận	✓

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
5	Nhập "Mật khẩu": Abc12345 (có hoa, có số, 8 ký tự)	Nhập bảo mật	Dữ liệu được ghi nhận, hiển thị độ mạnh pass	✓
6	Nhập "Xác nhận mật khẩu": Abc12345	Kiểm tra khớp	Không báo lỗi không khớp	✓
7	Nhấn "Đăng ký"	Gửi yêu cầu	Hệ thống gửi mã OTP về email/SMS, chuyển sang màn hình nhập OTP	✓

Bảng 279: Các bước thực hiện Test Case TC_AUTH_01

Nhận xét: Chức năng hoạt động đúng luồng với dữ liệu chuẩn.

B. Bảng tổng hợp kết quả kiểm thử

STT	Input	Output	Exception	KQ
1	Để trống bất kỳ trường bắt buộc nào (Tên, Email, Pass...)	Button Đăng ký bị disable hoặc báo đỏ trường thiếu	Đã xử lý (Validation)	PASS
2	Nhập Email đã tồn tại trong DB	Hiển thị thông báo "Email đã được sử dụng"	Đã xử lý (Check Duplicate)	PASS
3	Mật khẩu: "password" (không hoa, không số)	Hiển thị lỗi "Mật khẩu cần chữ hoa và số"	Đã xử lý (Regex check)	PASS
4	Mật khẩu xác nhận không khớp	Hiển thị lỗi "Mật khẩu không khớp"	Đã xử lý (Validation)	PASS
5	Nhập mã OTP sai quá 3 lần	Tài khoản tạm khóa hoặc yêu cầu gửi lại mã sau X phút	Logic bảo mật	PASS

Bảng 280: Bảng tổng hợp kết quả kiểm thử chức năng Đăng ký tài khoản

6.1.2 Kiểm thử cho chức năng: Theo dõi danh sách mua sắm (UC-01.3)

Mô tả chức năng: Tick chọn đã mua hàng khi đi chợ.

Lưu ý logic nghiệp vụ đặc thù: Nếu số lượng thực tế < số lượng dự định → Đánh dấu đã mua phần đó VÀ tạo thêm 1 mục mới cho phần còn thiếu.

A. Thiết kế Test Case chi tiết

Mục	Nội dung chi tiết
Test No.	TC_SHOP_01
Current status	☒ Passed ☐ Failed ☐ Pending
Title	Kiểm tra logic tách mục khi mua thiếu số lượng (Partial Purchase)
Description	Đảm bảo hệ thống tự động tạo mục mới khi người dùng không mua đủ số lượng yêu cầu.
Approach	Kiểm thử logic nghiệp vụ (Business Logic Testing).

Bảng 281: Thông tin Test Case TC_SHOP_01

Các bước thực hiện:

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
1	Mở danh sách mua sắm có mục: "Thịt bò", SL dự định: 5kg	Chuẩn bị dữ liệu	Mục hiển thị trạng thái "Chưa mua"	✓
2	Tick vào ô checkbox của "Thịt bò"	Xác nhận hành động	Hộp thoại nhập "Số lượng thực tế" hiện ra	✓
3	Nhập số lượng: 3 (nhỏ hơn 5)	Nhập tình huống thiếu	Nút xác nhận sáng lên	✓
4	Nhấn "Xác nhận"	Gửi dữ liệu	1. Mục "Thịt bò 3kg" chuyển trạng thái "Đã mua". 2. Xuất hiện mục mới "Thịt bò 2kg" trạng thái "Chưa mua".	✓
5	Kiểm tra thông báo	Kiểm tra đồng bộ	Thành viên khác trong nhóm nhận thông báo cập nhật list	✓

Bảng 282: Các bước thực hiện Test Case TC_SHOP_01

Nhận xét: Đây là tính năng quan trọng để tránh quên đồ khi đi chợ.

B. Bảng tổng hợp kết quả kiểm thử

STT	Input	Output	Exception	KQ
1	Nhập số lượng thực tế = Số lượng dự định	Mục chuyển sang "Đã mua", không tạo mục mới	Logic chuẩn	PASS
2	Nhập số lượng thực tế > Số lượng dự định	Mục chuyển sang "Đã mua" với số lượng mới	Logic chuẩn (Over-buying)	PASS
3	Nhập số lượng thực tế = 0 hoặc số âm	Báo lỗi "Số lượng không hợp lệ"	Validation	PASS

STT	Input	Output	Exception	KQ
4	Nhấn "Hoàn thành đi chợ" khi vẫn còn mục chưa tick	Hiển thị cảnh báo "Vẫn còn đồ chưa mua. Bạn chắc chắn chứ?"	UX/Confirm Dialog	PASS

Bảng 283: Bảng tổng hợp kết quả kiểm thử chức năng Theo dõi danh sách mua sắm

6.1.3 Kiểm thử cho chức năng: Quản lý thực phẩm trong bếp (UC-03.1 & UC-03.4)

Mô tả chức năng: Thêm thực phẩm vào bếp và logic nhắc hạn sử dụng.
Lưu ý: Yêu cầu nhắc nhở trước 3 ngày.

A. Thiết kế Test Case chi tiết

Mục	Nội dung chi tiết
Test No.	TC_KITCHEN_01
Current status	☒ Passed ☐ Failed ☐ Pending
Title	Thêm thực phẩm với ngày hết hạn gần và kiểm tra cảnh báo
Description	Kiểm tra việc thêm thực phẩm và hệ thống nhận diện trạng thái "Sắp hết hạn".
Approach	Kiểm thử biên thời gian (Time boundary testing).

Bảng 284: Thông tin Test Case TC_KITCHEN_01

Các bước thực hiện:

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
1	Vào màn hình "Bếp của tôi" → Chọn "Thêm thực phẩm"	Truy cập chức năng	Form thêm thực phẩm hiển thị	✓
2	Chọn thực phẩm: "Sữa tươi", SL: 1, Đơn vị: Hộp	Nhập thông tin cơ bản	Dữ liệu hợp lệ	✓
3	Chọn "Ngày hết hạn": [Ngày hiện tại + 2 ngày]	Nhập điều kiện biên (nhỏ hơn 3 ngày)	Ngày được chọn	✓
4	Nhấn "Lưu"	Lưu trữ	Thực phẩm được thêm vào danh sách	✓
5	Quan sát danh sách thực phẩm vừa thêm	Kiểm tra hiển thị	Item "Sữa tươi" có nhãn cảnh báo màu vàng/đỏ (Sắp hết hạn)	✓

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
------	-----------	----------	-------------------	----

Bảng 285: Test Case TC_KITCHEN_01 - Các bước thực hiện

Nhận xét: Hệ thống cần tính toán đúng số ngày còn lại để gắn nhãn warning.

B. Bảng tổng hợp kết quả kiểm thử

STT	Input	Output	Exception	KQ
1	Thêm thực phẩm đã có sẵn trong bếp (trùng tên)	Hiển thị thông báo hỏi: "Cập nhật số lượng hay tạo mới?" hoặc chuyển sang màn hình Edit	Xử lý trùng lặp (Duplicate)	PASS
2	Chọn ngày hết hạn là ngày trong quá khứ	Hiển thị cảnh báo "Thực phẩm đã hết hạn?" nhưng vẫn cho lưu → Gắn nhãn "Hết hạn" ngay lập tức	Logic nghiệp vụ	PASS
3	Xóa thực phẩm đang có trong một Công thức nấu ăn	Hệ thống vẫn cho xóa khỏi Bếp (nhưng không xóa khỏi CSDL Công thức)	Ràng buộc quan hệ	PASS

Bảng 286: Bảng tổng hợp kết quả kiểm thử chức năng Quản lý thực phẩm trong bếp

6.1.4 Kiểm thử cho chức năng: Tạo nhóm và Thêm thành viên (UC-06.1 & UC-06.2)

Mô tả chức năng: Tạo nhóm gia đình mới và mời thành viên tham gia.

Lưu ý: Người tạo sẽ là Trưởng nhóm. Mời thành viên cần thông qua email/SĐT và hệ thống phải tìm thấy tài khoản đó trước khi gửi lời mời.

A. Thiết kế Test Case chi tiết

Mục	Nội dung chi tiết
Test No.	TC_GROUP_01
Current status	☒ Passed ☐ Failed ☐ Pending
Title	Tạo nhóm mới và mời thành viên thành công
Description	Đảm bảo quy trình tạo nhóm hoạt động và quyền trưởng nhóm được gán đúng, cơ chế mời thành viên hoạt động chính xác.
Approach	Kiểm thử luồng nghiệp vụ (Business Flow).

Bảng 287: Thông tin Test Case TC_GROUP_01

Các bước thực hiện:

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
1	Tại màn hình Nhóm, chọn "Tạo nhóm mới"	Truy cập chức năng	Form tạo nhóm hiển thị	✓
2	Nhập tên nhóm: "Gia đình hạnh phúc"	Nhập thông tin	Dữ liệu hợp lệ	✓
3	Nhấn "Tạo nhóm"	Xác nhận tạo	Nhóm được tạo, User hiện tại hiển thị là "Trưởng nhóm" (Admin)	✓
4	Chọn chức năng "Thêm thành viên"	Mở rộng nhóm	Popup tìm kiếm thành viên hiện ra	✓
5	Nhập SĐT của User B (đã có tài khoản)	Tìm user	Hệ thống hiển thị tên và avatar của User B	✓
6	Nhấn "Gửi lời mời"	Gửi request	Thông báo "Đã gửi lời mời" xuất hiện	✓
7	(Trên thiết bị User B) Kiểm tra thông báo	Kiểm tra phía người nhận	User B nhận được Notification mời tham gia nhóm	✓

Bảng 288: Các bước thực hiện Test Case TC_GROUP_01

Nhận xét: Chức năng chia sẻ dữ liệu phụ thuộc hoàn toàn vào việc nhóm có được tạo thành công hay không.

B. Bảng tổng hợp kết quả kiểm thử

STT	Input	Output	Exception	KQ
1	Để trống tên nhóm khi tạo	Button "Tạo" bị vô hiệu hóa hoặc báo lỗi đỏ	Validation (Required field)	PASS
2	Mời một SĐT/Email chưa đăng ký App	Hệ thống báo lỗi "Không tìm thấy người dùng"	Xử lý dữ liệu không tồn tại	PASS
3	Mời một thành viên đã ở trong nhóm rồi	Hệ thống báo "Thành viên này đã tham gia nhóm"	Logic trùng lặp	PASS
4	User thường (không phải trưởng nhóm) cố xóa thành viên khác	Nút xóa bị ẩn hoặc báo "Không có quyền"	Kiểm tra phân quyền (RBAC)	PASS

Bảng 289: Bảng tổng hợp kết quả kiểm thử chức năng Tạo nhóm và Thêm thành viên

6.1.5 Kiểm thử cho chức năng: Quản lý Công thức nấu ăn (UC-04.1 & UC-04.3)

Mô tả chức năng: Tạo công thức nấu ăn mới (Cá nhân hoặc Chung).

Lưu ý: Bắt buộc phải có ít nhất 1 nguyên liệu chính.

A. Thiết kế Test Case chi tiết

Mục	Nội dung chi tiết
Test No.	TC_RECIPES_01
Current status	☒ Passed ☐ Failed ☐ Pending
Title	Tạo công thức nấu ăn cá nhân mới
Description	Kiểm tra khả năng tạo công thức với đầy đủ nguyên liệu chính và phụ.
Approach	Kiểm thử hộp đen (Black-box).

Bảng 290: Thông tin Test Case TC_RECIPES_01

Các bước thực hiện:

Bước	Hành động	Mục đích	Kết quả mong muốn	OK
1	Vào màn hình "Công thức", chọn "Tạo công thức mới"	Truy cập chức năng	Form nhập liệu hiển thị	✓
2	Nhập Tên món: "Canh chua cá lóc"	Nhập thông tin	Text được ghi nhận	✓
3	Phần Nguyên liệu chính: Chọn "Cá lóc", SL: 1, Đơn vị: Con	Nhập nguyên liệu bắt buộc	Dữ liệu được ghi nhận	✓
4	Phần Nguyên liệu phụ: Chọn "Cà chua", SL: 2, Đơn vị: Quả	Nhập nguyên liệu phụ	Dữ liệu được ghi nhận	✓
5	Nhấn "Lưu công thức"	Lưu trữ	Thông báo "Tạo công thức thành công"	✓
6	Tìm kiếm lại món "Canh chua cá lóc"	Kiểm tra hiển thị	Món ăn hiển thị đúng trong danh sách công thức của nhóm	✓

Bảng 291: Các bước thực hiện Test Case TC_RECIPES_01

Nhận xét: Cần đảm bảo công thức cá nhân/nhóm thì các nhóm khác không nhìn thấy.

B. Bảng tổng hợp kết quả kiểm thử

STT	Input	Output	Exception	KQ
1	Nhập tên món nhưng không nhập nguyên liệu nào	Báo lỗi "Cần ít nhất 1 nguyên liệu chính"	Validate nghiệp vụ	PASS
2	Nhập số lượng nguyên liệu là số âm (-5 gram)	Báo lỗi "Số lượng phải lớn hơn 0"	Validate dữ liệu số	PASS
3	User thường cố gắng chỉnh sửa "Công thức chung" (do Admin tạo)	Nút Sửa/Xóa bị ẩn hoặc báo "Không có quyền"	Phân quyền dữ liệu	PASS

Bảng 292: Bảng tổng hợp kết quả kiểm thử chức năng Quản lý Công thức nấu ăn

6.1.6 Kiểm thử cho chức năng: Lên thực đơn (UC-02.1)

Mô tả chức năng: Lên kế hoạch bữa ăn cho các ngày trong tuần.

Lưu ý: Một bữa có thể có nhiều món.

Bảng tổng hợp kết quả kiểm thử

STT	Input	Output	Exception	KQ
1	Thêm 3 món ăn khác nhau vào cùng một bữa "Tối Thứ 2"	Hệ thống ghi nhận cả 3 món cho bữa tối đó	Logic Multi-item	PASS
2	Thêm món ăn vào ngày trong quá khứ	Hệ thống vẫn cho phép (để lưu ký ức/thống kê) hoặc cảnh báo nhẹ	Logic thời gian	PASS
3	Xóa một món khỏi thực đơn đã lên lịch	Món ăn biến mất khỏi lịch, không ảnh hưởng các món khác	Thao tác Delete	PASS

Bảng 293: Bảng tổng hợp kết quả kiểm thử chức năng Lên thực đơn

6.2 Kiểm thử yêu cầu phi chức năng

Phần này đánh giá các khía cạnh hiệu năng, bảo mật và tính khả dụng.

Tiêu chí	Nội dung đánh giá	Trạng thái
Hiệu năng	- Thời gian phản hồi thao tác tick chọn mua hàng < 1 giây. - Tìm kiếm thực phẩm trả về kết quả < 1 giây. - Đồng bộ dữ liệu khi user khác thêm món < 5 giây (Realtime).	Tốt
Offline Mode	- Ngắt mạng internet, mở App → Vẫn xem được Danh sách mua sắm đã lưu. - Tick chọn "Đã mua" khi offline → Lưu local. - Bật lại mạng → Dữ liệu tự động đồng bộ lên server.	Tốt
Tính khả dụng	- Giao diện tiếng Việt chuẩn, không lỗi font. - Các nút bấm trên màn hình nhỏ (iPhone SE/Android cũ) không bị chồng chéo.	Tốt
Bảo mật	- Dữ liệu password trong Database phải được mã hóa (không thấy plain text). - User A không thể xem danh sách mua sắm của Group B (Test qua API).	Ổn định

Bảng 294: Bảng đánh giá yêu cầu phi chức năng

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trong khuôn khổ học phần **IT4788 - Phát triển ứng dụng đa nền tảng**, dự án xây dựng ứng dụng **FreshFridge - “Đi chợ tiện lợi”** đã hoàn thành các mục tiêu đề ra về việc tạo ra một giải pháp quản lý thực phẩm và mua sắm thông minh. Dưới đây là những tổng kết chi tiết về quá trình thực hiện và lộ trình cải tiến hệ thống trong tương lai.

1. Kết luận

1.1. Những kết quả đã đạt được

Dự án đã hiện thực hóa thành công một hệ thống quản lý tích hợp với các kết quả cụ thể sau:

- **Về chức năng:** Hoàn thiện các module cốt lõi bao gồm: quản lý kho thực phẩm (theo dõi vị trí, số lượng, hạn sử dụng), danh sách mua sắm nhóm, kế hoạch bữa ăn và hệ thống thông báo đẩy (Push Notifications) nhắc nhở thông minh.
- **Về kỹ thuật:** Xây dựng hệ thống ổn định dựa trên kiến trúc **Clean Architecture** (Back-end Spring Boot) và kiến trúc **Component-based** (Front-end React Native + Expo). Hệ thống đảm bảo tính nhất quán dữ liệu thông qua PostgreSQL và khả năng đồng bộ thời gian thực với React Query.
- **Về bảo mật và lưu trữ:** Triển khai cơ chế xác thực JWT, mã hóa dữ liệu nhạy cảm trên thiết bị (Secure Store) và tối ưu hóa hình ảnh qua Cloudinary.

1.2. Ưu điểm và hạn chế

- **Ưu điểm:** Ứng dụng có giao diện thân thiện, tính thực tiễn cao trong việc giải quyết bài toán lãng phí thực phẩm. Cấu trúc mã nguồn được tổ chức chặt chẽ, sử dụng TypeScript giúp giảm thiểu lỗi runtime và dễ dàng bảo trì.
- **Hạn chế:** Hệ thống chưa triển khai cơ chế Caching (Redis) để tối ưu hóa hiệu năng API. Ngoài ra, việc thiếu kiểm thử tự động (E2E Testing) và các tính năng thông minh dựa trên AI/ML là những điểm cần được khắc phục trong các phiên bản tiếp theo.

2. Hướng phát triển

Dựa trên những hạn chế đã nêu, lộ trình phát triển của FreshFridge được định hướng theo ba giai đoạn:

2.1. Ngắn hạn: Tối ưu hóa và Bảo mật

- Triển khai **Redis Caching** và tối ưu hóa truy vấn cơ sở dữ liệu để nâng cao tốc độ phản hồi.
- Tăng tỷ lệ bao phủ kiểm thử (Unit Test & Integration Test) lên trên 80%.
- Bổ sung cơ chế bảo mật nâng cao như **Refresh Token Rotation** và **Rate Limiting** để bảo vệ tài nguyên hệ thống.

2.2. Trung hạn: Tính năng thông minh và Đa ngữ

- Tích hợp **Barcode Scanner** và công nghệ **OCR** để hỗ trợ người dùng nhập liệu thực phẩm từ hóa đơn hoặc mã vạch sản phẩm.
- Xây dựng mô hình **AI Suggestion** để đề xuất món ăn dựa trên nguyên liệu thực tế và thói quen dinh dưỡng của người dùng.
- Hoàn thiện hệ thống đa ngôn ngữ (Internationalization) để tiếp cận người dùng quốc tế.

2.3. Dài hạn: Hệ sinh thái IoT và Cộng đồng

- Kết nối với các thiết bị **Smart Fridge** và các thiết bị gia dụng IoT để tự động hóa quy trình kiểm kê thực phẩm.
- Phát triển mạng xã hội chia sẻ công thức nấu ăn và các giải pháp quản lý chi tiêu thực phẩm dành cho doanh nghiệp nhỏ (nhà hàng, canteen).
- Triển khai hệ thống trên hạ tầng đám mây chuyên nghiệp với quy trình **CI/CD** và giám sát tập trung (ELK Stack).

3. Tổng kết cuối cùng

Ứng dụng FreshFridge không chỉ là một sản phẩm kỹ thuật mà còn là giải pháp thực tế hướng tới lối sống hiện đại và bền vững. Với nền tảng kiến trúc hiện có, hệ thống hoàn toàn đủ tiềm năng để mở rộng thành một hệ sinh thái quản lý gia đình thông minh, mang lại giá trị thiết thực cho cộng đồng người dùng.

Tài liệu tham khảo

- [1] *Bài giảng môn Phát triển ứng dụng đa nền tảng*, Đại học Bách Khoa Hà Nội.
- [2] Spring Projects, “Spring Boot Documentation”, *Spring.io*.
<https://docs.spring.io/spring-boot/index.html>.
- [3] Meta Platforms, Inc., “Learn React Native”, *reactnative.dev*.
<https://reactnative.dev/>.

Phụ lục

1. Mã nguồn dự án (Github Repository):
<https://github.com/QuanTuanHuy/FreshFridge>
2. Tài liệu báo cáo, Slide và Demo:
https://drive.google.com/drive/folders/1L41P_nNdRefjLXJ4Ub-gBc_QNaXTdn0p